

Compiler Design Preparation

Generated 9/27/2026, 10:20:46 PM • 63 messages • Source: Shared ChatGPT link
<https://chatgpt.com/share/6ab94953-7128-83e8-b005-531ec09a94a3>

USER

Compiler design

ASSISTANT

I'll set up **Compiler Design** in the same GATE-focused format as your Computer Networks, DBMS, and Operating Systems preparation: complete syllabus coverage, note-ready explanations, shortcuts, PYQ traps, and one topic at a time so you can keep replying **NEXT**.

ASSISTANT

COMPILER DESIGN — GATE 2027 COMPLETE PREPARATION

We'll do **Compiler Design** exactly like your **Computer Networks, DBMS, and Operating Systems** preparation: no dependence on long YouTubelectures, complete concept teaching, note-ready material, shortcuts/mnemonics, solved examples, GATE traps, and PYQ-oriented practice.

I checked the **official GATE 2027 CSE syllabus released by IIT Madras**. Compiler Design is **Section 7**, and the official syllabus is:

Lexical analysis, parsing, syntax-directed translation. Runtime environments. Intermediate code generation. Local optimisation, Data flow analyses: constant propagation, liveness analysis, common sub expression elimination.

Because GATE 2027 states that syllabi have been revised, we'll use this **2027 syllabus**, not an older coaching syllabus.

Our Complete Compiler Design Roadmap

We'll break that small official paragraph into everything you actually need to solve GATE questions.

PART A — FOUNDATION

1. What is a compiler?
2. Compiler vs interpreter
3. Language processing system
4. Phases of a compiler
5. Front end vs back end
6. Analysis phase vs synthesis phase
7. Symbol table
8. Error handling
9. Pass vs phase

PART B — LEXICAL ANALYSIS

- 10. Lexical analyzer
- 11. Characters !' lexemes !' tokens
- 12. Token, pattern and lexeme
- 13. Token representation
- 14. Role of lexical analyzer
- 15. Lexical errors
- 16. Input buffering
- 17. Regular expressions in lexical analysis
- 18. Finite automata and lexical analysis
- 19. NFA !' DFA idea
- 20. Lexical analyzer generators / LEX concepts
- 21. GATE token-counting problems

PART C — PARSING / SYNTAX ANALYSIS

- 22. Parser basics
- 23. Context-free grammar revision
- 24. Parse trees
- 25. Derivations
- 26. Leftmost/rightmost derivation
- 27. Ambiguous grammar
- 28. Removing ambiguity
- 29. Left recursion
- 30. Eliminating immediate left recursion
- 31. Eliminating indirect left recursion
- 32. Left factoring

PART D — TOP-DOWN PARSING

- 33. Recursive-descent parsing
- 34. Predictive parsing
- 35. FIRST
- 36. FOLLOW
- 37. Nullable variables
- 38. FIRST/FOLLOW calculations
- 39. LL(1) grammar
- 40. LL(1) parsing table
- 41. String parsing using LL(1)
- 42. LL(1) conflicts
- 43. Properties and GATE tricks

PART E — BOTTOM-UP PARSING

- 44. Bottom-up parsing idea
- 45. Shift-reduce parsing
- 46. Handles
- 47. Handle pruning

- 48. Shift/reduce conflict
- 49. Reduce/reduce conflict
- 50. Viable prefixes

PART F — LR PARSING

- 51. LR parser architecture
- 52. LR(0) items
- 53. Closure
- 54. GOTO
- 55. Canonical collection
- 56. LR(0) automaton
- 57. LR(0) parsing table
- 58. SLR(1)
- 59. LR(1)
- 60. LALR(1)
- 61. LR(0) vs SLR vs LALR vs CLR
- 62. Grammar-class relationships
- 63. Number-of-state questions
- 64. Conflict questions

PART G — SYNTAX-DIRECTED TRANSLATION

- 65. Syntax-directed definition
- 66. Attributes
- 67. Synthesized attributes
- 68. Inherited attributes
- 69. Attribute dependency
- 70. Dependency graphs
- 71. S-attributed definitions
- 72. L-attributed definitions
- 73. Syntax-directed translation schemes
- 74. Evaluation order
- 75. GATE attribute problems

PART H — RUNTIME ENVIRONMENTS

- 76. Runtime storage organization
- 77. Static allocation
- 78. Stack allocation
- 79. Heap allocation
- 80. Activation records
- 81. Activation trees
- 82. Control link
- 83. Access/static link
- 84. Return address
- 85. Parameters

- 86. Local variables
- 87. Temporaries
- 88. Recursion
- 89. Parameter passing
- 90. Scope and binding
- 91. Static vs dynamic scoping

PART I — INTERMEDIATE CODE GENERATION

- 92. Intermediate representations
- 93. Syntax trees
- 94. DAG
- 95. Three-address code
- 96. Quadruples
- 97. Triples
- 98. Indirect triples
- 99. Translation of arithmetic expressions
- 100. Boolean expressions
- 101. Control-flow statements
- 102. Backpatching
- 103. GATE TAC-count questions

PART J — BASIC BLOCKS & CONTROL FLOW

- 104. Leaders
- 105. Basic blocks
- 106. Finding number of basic blocks
- 107. Flow graphs
- 108. Predecessor/successor
- 109. Entry/exit
- 110. DAG representation of basic blocks

PART K — LOCAL OPTIMIZATION

- 111. Compiler optimization basics
- 112. Machine-independent vs dependent
- 113. Local vs global optimization
- 114. Constant folding
- 115. Algebraic simplification
- 116. Strength reduction
- 117. Copy propagation
- 118. Dead-code elimination
- 119. Common-subexpression elimination
- 120. Loop-related basics
- 121. Peephole-style concepts where useful

PART L — DATA-FLOW ANALYSIS

- 122. What is data-flow analysis?

- 123IN and OUT sets
- 124GEN and KILL sets
- 125Forward analysis
- 126Backward analysis
- 127May vs must analysis
- 128Data-flow equations
- 129Fixed-point computation

PART M — CONSTANT PROPAGATION

- 130Constant propagation
- 131Forward-flow nature
- 132Meet operation intuition
- 133CFG problems

PART N — LIVENESS ANALYSIS

- 134Live variables
- 135USE/DEF sets
- 136IN/OUT equations
- 137Backward analysis
- 138Iterative solution
- 139GATE numerical/set questions

PART O — COMMON SUBEXPRESSION ELIMINATION

- 140Available expressions
- 141GEN/KILL
- 142Forward analysis
- 143Common-subexpression elimination using data-flow information
- 144CFG-based questions

FINAL

- 145Complete formula sheet
- 146All important grammar relations
- 147FIRST/FOLLOW shortcuts
- 148LR-family comparison sheet
- 149Data-flow equation sheet
- 150Common GATE traps
- 151Mixed GATE-level questions
- 152Rapid-revision notes
- 153Final Compiler Design checklist

So we will **not leave a syllabus word unexplained.**

PART 1 — INTRODUCTION TO COMPILER DESIGN

1. What is a Compiler?

Suppose you write:

```
c
int a = 10;
int b = 20;
int c = a + b;
```

The computer's processor cannot directly understand C code.

It ultimately needs **machine-level instructions**.

A compiler translates a program written in one language into an equivalent program in another language.

Simple definition

Compiler = Source Program !' Target Program translator

Usually:

$$\boxed{\text{High-Level Language}} \rightarrow \text{Machine/Assembly Code}$$

Example:

```
TEXT
C Program
    !"
Compiler
    !"
Machine/Assembly Code
```

2. Compiler's Main Job

A compiler has two major responsibilities:

1. Translation

Convert source code into target code.

2. Error detection

Detect errors in the source program.

For example:

```
c
int x = ;
```

The compiler should recognize that this statement is syntactically incorrect.

3. Language Processing System

A very important conceptual picture is:

```
TEXT
Source Program
    !"
```

```
Preprocessor
    !"
Modified Source Program
    !"
Compiler
    !"
Assembly Program
    !"
Assembler
    !"
Relocatable Machine Code
    !"
Linker + Libraries
    !"
Executable
    !"
Loader
    !"
Memory
```

Know what every stage does.

Preprocessor

Handles things such as:

```
C
#include <stdio.h>
#define MAX 100
```

It produces a modified source program.

Compiler

Converts:

```
TEXT
High-level language
    !"
Assembly / Intermediate / Target code
```

Assembler

Converts:

```
TEXT
Assembly code
    !"
Machine code
```

Linker

Combines:

- object files
- library routines
- external functions

Example:

Your program calls:

```
c
printf()
```

The actual implementation may come from a library.

The **linker connects it**.

Loader

Loads the executable program into **main memory** so that execution can begin.

Memory Trick

Remember:

P C A L L

TEXT

```
P !' P r e p r o c e s s o r
C !' C o m p i l e r
A !' A s s e m b l e r
L !' L i n k e r
L !' L o a d e r
```

Think:

“PC ALL”

or simply:

Source !' P-C-A-L-L !' Execution

4. Compiler vs Interpreter

This distinction is basic but useful.

Compiler

Translates a substantial portion/program before execution.


```
Source
    !"
Compiler
    !"
Target Program
    !"
Execution
```

Processes and executes source instructions through the interpreter without normally producing an independent target executable in the same way.

```
Source Program      !"  
Interpreter        !"  
Execution
```

Compiler	Interpreter
Produces translated target program	Directly interprets program operations
Compilation precedes later execution	Translation/execution are closely coupled
Errors can be reported during compilation	Errors often surface while interpreting execution
Compiled target can run independently	Interpreter is normally required

```
SOURCE PROGRAM
! "
% % % % % % % % % % % % % % % % % % % % % % % % % %
%   1 .   L e x i c a l   A n a l y s i s   %
% % % % % % % % % % % % % % % % % % % % % % % % % %
! "

TOKENS
! "
% % % % % % % % % % % % % % % % % % % % % % % % % %
%   2 .   S y n t a x   A n a l y s i s   %
% % % % % % % % % % % % % % % % % % % % % % % % % %
! "

PARSE TREE
! "
% % % % % % % % % % % % % % % % % % % % % % % % % %
%   3 .   S e m a n t i c   A n a l y s i s   %
```

```

% % % % % % % % % % % % % % % % % % % % %
! "
ANNOTATED TREE
! "
% % % % % % % % % % % % % % % % % % % % %
% 4 . I n t e r m e d i a t e   C o d e   G e n . %
% % % % % % % % % % % % % % % % % % % % %
! "
INTERMEDIATE CODE
! "
% % % % % % % % % % % % % % % % % % % % %
% 5 . C o d e   O p t i m i z a t i o n %
% % % % % % % % % % % % % % % % % % % % %
! "
OPTIMIZED CODE
! "
% % % % % % % % % % % % % % % % % % % % %
% 6 . C o d e   G e n e r a t i o n %
% % % % % % % % % % % % % % % % % % % % %
! "
TARGET CODE

```

Two supporting components interact with several phases:

TEXT

Symbol Table
Error Handler

6. Shortcut for the Six Phases

Take first letters:

L S S I O C

- Lexical analysis
- Syntax analysis
- Semantic analysis
- Intermediate code generation
- Optimization
- Code generation

Mnemonic:

Lazy Students Study In Our College

TEXT

L !' L e x i c a l
S !' S y n t a x
S !' S e m a n t i c
I !' I n t e r m e d i a t e C o d e
O !' O p t i m i z a t i o n
C !' C o d e G e n e r a t i o n

Remember **LSSIOC**.

7. Understand the Entire Compiler With ONE Example

Consider:

```
C
position = initial + rate * 60;
```

We'll send this through the compiler.

PHASE 1 — Lexical Analysis

The lexical analyzer breaks the character stream into **tokens**.

Input:

```
TEXT
position = initial + rate * 60 ;
```

Possible tokens:

```
TEXT
<id, position>
<=>
<id, initial>
<+>
<id, rate>
<*>
<num, 60>
<;>
```

Conceptually:

```
TEXT
C h a r a c t e r s ! '   T o k e n s
```

The lexical analyzer is also called the:

Scanner

PHASE 2 — Syntax Analysis

The parser checks whether tokens follow the grammar.

For:

```
TEXT
initial + rate * 60
```

we may get a tree such as:

```
TEXT
      +
     / \
```

```
initial  *  
        / \  
rate    60
```

Because multiplication has higher precedence.

Conceptually:

```
TEXT  
T o k e n s ! '   P a r s e / S y n t a x   S t r u c t u r e
```

Syntax analyzer =

Parser

PHASE 3 — Semantic Analysis

Now the compiler checks **meaning-related constraints**.

For example:

```
C  
int x;  
x = "hello";
```

The syntax may look structurally valid.

But the types may be incompatible.

Semantic analysis checks things such as:

- types
- declarations
- scope
- compatible operands
- function arguments

PHASE 4 — Intermediate Code Generation

Compiler creates an intermediate representation.

For:

```
TEXT  
position = initial + rate * 60
```

Three-address style code:

```
TEXT  
t1 = rate * 60  
t2 = initial + t1  
position = t2
```

t1 and t2 are temporaries.

PHASE 5 — Code Optimization

Compiler attempts to improve code while preserving its meaning.

Example:

TEXT

```
x = 2 * 3
```

can become:

TEXT

```
x = 6
```

This is an example of:

Constant folding

Another example:

TEXT

```
x = y + 0
```

becomes:

TEXT

```
x = y
```

PHASE 6 — Code Generation

Finally, intermediate representation is translated toward machine/assembly instructions.

Conceptually:

TEXT

```
LOAD ...  
MUL ...  
ADD ...  
STORE ...
```

The exact instructions depend on the target architecture.

8. Front End vs Back End

Another highly useful classification.

Front End

Concerned mainly with understanding/analyzing the source program.

Includes approximately:

TEXT

Lexical Analysis
Syntax Analysis
Semantic Analysis
Intermediate representation generation

Think:

Front end understands the program.

Back End

Concerned mainly with improving the representation and producing target code.

TEXT

Optimization
Code Generation

Think:

Back end builds the target program.

9. Analysis Phase vs Synthesis Phase

Analysis

Break the source program down and determine its structure/meaning.

TEXT

Source Program
! "
Internal Representation

Synthesis

Use the internal representation to construct target code.

TEXT

Internal Representation
! "
Target Program

Memory:

Analysis = Understand

Synthesis = Construct

10. Symbol Table

The compiler needs information about identifiers.

For example:

```
c
int age;
float salary;
```

A conceptual symbol table might contain:

Name	Type
age	int
salary	float

In real compilers it may also store:

- type
- scope
- memory location
- parameter information
- return type
- dimensions
- linkage information

Shortcut

Think:

Symbol Table = Compiler's directory of names

11. Error Handler

Errors may be found during different compiler phases.

Example:

```
TEXT
Lexical error
Syntax error
Semantic error
```

Lexical example

Illegal/invalid character sequence.

Syntax example

```
C
x = a + ;
```

Grammar structure is invalid.

Semantic example

```
C
int x;
x = "Gate";
```

Type rules may be violated.

12. Phase vs Pass

This is commonly confused.

Phase

A **logical step** in compilation.

Examples:

- lexical analysis
- parsing
- optimization

Pass

One complete processing traversal over a representation/program during compilation.

A compiler may combine several logical phases in a single pass.

Therefore:

$\boxed{\text{Phase} \neq \text{Pass}}$

Very important.

13. One-Line Phase Revision

Memorize this table:

Phase	Input	Main Output/Job
Lexical	Characters	Tokens
Syntax	Tokens	Parse/syntax structure
Semantic	Syntax structure	Meaning/type checks
Intermediate Code	Analyzed representation	IR / three-address code
Optimization	IR	Improved IR

Phase	Input	Main Output/Job
Code Generation	IR	Target code

The most important first two:

$\boxed{\text{Characters} \xrightarrow{\text{Lexical}} \text{Tokens}}$

$\boxed{\text{Tokens} \xrightarrow{\text{Parser}} \text{Syntax Structure}}$

GATE TRAP 1

Who produces tokens?

' Lexical analyzer

Not parser.

GATE TRAP 2

Who consumes tokens?

' Parser / syntax analyzer

GATE TRAP 3

Scanner = ?

' Lexical analyzer.

GATE TRAP 4

Parser = ?

' Syntax analyzer.

GATE TRAP 5

TEXT

```
int x;
x = "abc";
```

Assuming the language does not permit that assignment, the important problem is generally:

' **Semantic/type error**

not lexical.

GATE TRAP 6

Are compiler phases and passes identical?

'LNo.

A pass is implementation-oriented; a phase is a logical compilation function.

30-SECOND REVISION

TEXT

Compiler:

S o u r c e ! ' T a r g e t

Language processing:

P r e p r o c e s s o r ! ' C o m p i l e r ! ' A s s e m b l e r ! ' L i n k e r ! ' L o a d e r
Mnemonic = P-C-A-L-L

Compiler phases:

Lexical

Syntax

Semantic

Intermediate Code

Optimization

Code Generation

Mnemonic:

L a z y S t u d e n t s S t u d y I n O u r C o l l e g e

Lexical:

C h a r a c t e r s ! ' T o k e n s
Scanner

Syntax:

T o k e n s ! ' P a r s e S t r u c t u r e
Parser

Semantic:

Meaning / type checking

Symbol Table:

Information about identifiers

P h a s e " ` P a s s

NOTEBOOK SHORTCUT

Put this boxed diagram on the **first page of Compiler Design**:

TEXT

SOURCE

!"

L E X I C A L ! ' T o k e n s

!"

S Y N T A X ! ' P a r s e s t r u c t u r e

!"

S E M A N T I C ! ' T y p e / m e a n i n g c h e c k s

!"

I N T E R M E D I A T E ! ' T A C / I R

!"

O P T I M I Z A T I O N ! ' B e t t e r I R

Master mnemonic

L S S I O C

Lazy Students Study In Our College

That single mnemonic gives you the complete compiler pipeline.

Next

PART 2 — LEXICAL ANALYSIS

We will go deeply into:

token vs lexeme vs pattern !' token representation !' lexical analyzer working !' symbol-table interaction !' lexical errors !' regular expressions/finite automata connection !' GATE token-counting questions and traps.

Reply **NEXT**.

USER

Compiler Design Lexical analysis, parsing, syntax-directed translation. Runtime environments. Intermediate code generation. Local optimisation, Data flow analyses: constant propagation, liveness analysis, common sub expression elimination

This are the complete syllabus copy of gate So i have no flexibility to wach some hours of youtube video i want you to take time and give complete syllabus in detail in which you take your own time and give complete information related to gate and all the stuff give me a perfect prompt ... I keep changing the syllabus i also include different syllabus of gate So that i can make notes and give me some shortcut to remember concepts

ASSISTANT

Yes. Since you will keep changing the **GATE subject and syllabus**, the best solution is one reusable **master prompt** where you only replace the subject name and paste the official syllabus.

Use this in a fresh chat for every GATE subject.

:::writing{variant="document" id="58314"}

GATE COMPLETE SUBJECT TEACHER — MASTER PROMPT

I am preparing for **GATE** and I have limited time.

I do **NOT** have the flexibility to watch many hours of YouTubelectures for every subject. I want to learn the subject mainly from this chat and make my own handwritten/typed notes from your explanations.

Act as my:

- GATE subject expert
- personal teacher
- concept explainer

- notes maker
 - PYQ-oriented mentor
 - problem-solving trainer
 - revision coach
 - memory-trick creator
-

MY CURRENT SUBJECT

Subject: Compiler Design

OFFICIAL GATE SYLLABUS

Compiler Design:

Lexical analysis, parsing, syntax-directed translation. Runtime environments. Intermediate code generation. Local optimisation, Data flow analyses: constant propagation, liveness analysis, common sub expression elimination.

MAIN GOAL

Convert the above official syllabus into a **complete GATE preparation course inside this chat**.

Do not merely explain the syllabus headings.

Expand every syllabus point into all the concepts, subtopics, formulas, algorithms, properties, special cases, numerical/problem types, and prerequisite ideas that can reasonably be tested in GATE.

My goal is that after completing your course and solving PYQs, I should not need to depend on long video lectures for theory.

RULE 1 — FIRST EXPAND THE COMPLETE SYLLABUS

Before teaching, convert the syllabus into a detailed numbered roadmap.

For example, if the syllabus says:

Lexical analysis

expand it into relevant concepts such as:

- role of lexical analyzer
 - token
 - lexeme
 - pattern
 - token representation
 - regular expressions relevant to lexical analysis
 - finite automata connection
 - lexical errors
 - input buffering if relevant for GATE
 - token-counting questions
-

- interaction with symbol table
- common GATE traps

Do the same for **every syllabus phrase**.

The roadmap should be detailed enough that I can verify that no important topic has been skipped.

RULE 2 — TEACH ONE LOGICAL CHUNK AT A TIME

Do NOT dump the entire subject in one huge answer.

Divide the complete course into numbered parts such as:

PART 1 — Foundations **PART 2 — Lexical Analysis** **PART 3 — Context-Free Grammars and Parsing Basics**
PART 4 — FIRST and FOLLOW etc.

Teach **one manageable logical chunk per response**.

At the end, stop and write:

Reply NEXT for Part X.

When I type **NEXT**, continue exactly from where you stopped.

Do not restart the subject and do not repeat previous material unnecessarily.

RULE 3 — ASSUME I NEED THE CONCEPT FROM THE FOUNDATION

For every new concept:

1. Tell me **what it is**.
2. Tell me **why we need it**.
3. Explain the intuition in simple language.
4. Give the formal definition.
5. Give a small example.
6. Then move to the GATE-level interpretation.
7. Then explain the common question patterns.

Do not jump directly into formulas without building intuition.

RULE 4 — MAKE THE EXPLANATION NOTE-FRIENDLY

Write explanations so that I can directly convert them into GATE notes.

Use:

- clear headings
 - short paragraphs
 - bullets
 - tables
 - formulas
 - diagrams using text/ASCII
 - step-by-step algorithms
-

- worked examples
- comparison tables

Avoid unnecessary storytelling or very long theory that has little GATE value.

Whenever useful, explicitly mark:

WRITE THIS IN NOTES

and give me the concise form that is worth memorizing.

RULE 5 — DISTINGUISH THREE LEVELS

Whenever useful, separate information into:

CONCEPT

What I must understand.

REMEMBER

Facts/formulas/properties that must be memorized.

GATE USE

How GATE can ask a question from it.

This is very important.

RULE 6 — GIVE MEMORY TRICKS

For concepts that involve:

- sequences
- properties
- formulas
- classifications
- parsing families
- compiler phases
- data-flow directions
- relationships
- algorithms

create simple mnemonics or memory shortcuts.

Do not create complicated mnemonics that are harder than the concept itself.

Use formats like:

Memory Trick: Shortcut: 1-line recall: Exam memory:

RULE 7 — EXPLAIN FORMULAS COMPLETELY

Whenever a formula is introduced:

1. Write the formula clearly.
-

2. Explain every variable/symbol.
3. Explain when to use it.
4. Give a small example.
5. Mention important special cases.
6. Mention common mistakes.

Never give a formula without explaining its meaning.

RULE 8 — SOLVE NUMERICALS STEP BY STEP

Compiler Design contains many problem-solving areas.

For every important question type:

- show the question pattern
- identify the concept
- solve step by step
- show the fastest valid GATE approach
- mention traps
- give a shortcut if one exists

For parsing/grammar problems, never skip intermediate steps that are necessary to understand the solution.

RULE 9 — GIVE SPECIAL ATTENTION TO HIGH-VALUE COMPILER DESIGN AREAS

Cover these thoroughly whenever they are relevant to the syllabus:

Lexical Analysis

- token
- lexeme
- pattern
- lexical analyzer
- scanner
- regular expressions
- finite automata connection
- token counting
- lexical errors
- symbol-table interaction

Parsing Foundations

- grammar terminology needed for parsing
- derivation
- leftmost derivation
- rightmost derivation
- parse tree

- ambiguity
- left recursion
- left factoring
- nullable variables
- FIRST
- FOLLOW

Top-Down Parsing

- recursive descent
- predictive parsing
- LL(1)
- LL(1) conditions
- parsing-table construction
- conflicts
- string parsing

Bottom-Up Parsing

- shift-reduce parsing
- handle
- handle pruning
- viable prefix
- shift/reduce conflict
- reduce/reduce conflict

LR Family

Where required for GATE, explain:

- LR(0) items
- augmented grammar
- closure
- GOTO
- canonical collection
- LR(0)
- SLR(1)
- CLR/LR(1)
- LALR(1)
- ACTION/GOTO table
- conflicts
- relationships between grammar classes

Give a clear comparison among:

LL(1), LR(0), SLR(1), LALR(1), CLR(1)

and teach me how to identify which class a grammar belongs to.

Syntax-Directed Translation

- syntax-directed definitions
- attributes
- synthesized attributes
- inherited attributes
- dependency graphs
- evaluation order
- S-attributed definitions
- L-attributed definitions
- syntax-directed translation schemes
- relevant GATE problems

Runtime Environments

Cover relevant concepts such as:

- runtime storage organization
- static allocation
- stack allocation
- heap allocation
- activation records
- activation trees
- parameters
- return value/address
- local variables
- temporaries
- control/dynamic links
- access/static links
- recursion
- scope issues where relevant
- parameter passing where relevant

Intermediate Code Generation

- intermediate representations
- syntax tree
- DAG where relevant
- three-address code
- temporaries
- quadruples
- triples
- indirect triples
- arithmetic expressions
- Boolean/control-flow translation where relevant
- backpatching if relevant to GATE

Basic Blocks and Flow Graphs

If needed for optimization/data-flow analysis, teach:

- leaders
- basic blocks
- identifying basic blocks
- control-flow graphs
- predecessors
- successors
- entry/exit nodes

Local Optimisation

Explain relevant optimizations such as:

- constant folding
- algebraic simplification
- strength reduction
- copy propagation
- dead-code elimination
- common-subexpression elimination
- local optimization using DAG where relevant

Clearly distinguish similar optimization techniques.

Data-Flow Analysis

Build the foundation properly:

- control-flow graph
- IN and OUT
- GEN and KILL
- forward analysis
- backward analysis
- may analysis
- must analysis
- iterative/fixed-point idea

Then thoroughly cover the syllabus topics:

Constant Propagation

- what information flows
- direction
- equations/meet intuition where required
- CFG problems

Liveness Analysis

- live/dead variable
- USE/DEF

- IN/OUT equations
- backward nature
- iterative calculation
- solved GATE-style examples

Common Subexpression Elimination

- available expressions
- GEN/KILL
- IN/OUT
- forward nature
- how data-flow information enables elimination
- solved examples

RULE 10 — ALWAYS EXPLAIN RELATIONSHIPS

GATE frequently tests relationships and properties.

For example, when teaching parsing classes, explicitly explain relationships such as whether one grammar class is a subset/superset of another.

Do not simply give a diagram.

Explain:

- why the relationship exists
- what implication follows
- what does NOT follow
- common GATE traps

RULE 11 — INCLUDE GATE TRAPS

After every important topic add:

GATE TRAPS

Include statements/options that students commonly confuse.

For example:

- token vs lexeme
- phase vs pass
- left recursion vs ambiguity
- LL vs LR
- shift/reduce vs reduce/reduce conflict
- synthesized vs inherited attribute
- static link vs dynamic/control link
- constant folding vs constant propagation
- dead code vs unreachable code
- liveness direction
- available expressions direction

Explain why the wrong statement is wrong.

RULE 12 — INCLUDE PYQ-STYLE THINKING

You do not need to reproduce copyrighted PYQs word-for-word.

Instead:

- identify recurring GATE question patterns
- create equivalent GATE-style questions
- explain how to recognize the pattern
- solve representative problems
- tell me what concept the examiner is testing

Label these sections:

GATE PATTERN GATE-STYLE EXAMPLE FAST SOLUTION

RULE 13 — ADD MINI PRACTICE

At the end of each major topic give approximately:

- 2 easy questions
- 2 medium GATE-level questions
- 1 tricky conceptual question

Do not immediately reveal the answers if self-testing would be more useful.

Put solutions after a separator so I can attempt them first.

RULE 14 — GIVE A 30-SECOND REVISION

At the end of every logical part provide:

30-SECOND REVISION

It should contain only the most important:

- definitions
- formulas
- relationships
- shortcuts
- traps

that I should recall before solving questions.

RULE 15 — BUILD A RUNNING FORMULA/FACT SHEET

Whenever we encounter an important formula, condition or relationship, keep adding it mentally to a final revision sheet.

At the end of the complete subject, give me:

COMPILER DESIGN FINAL REVISION SHEET

containing:

- all important formulas
- FIRST/FOLLOW rules
- LL(1) conditions
- LR-family relationships
- important compiler-phase facts
- syntax-directed translation properties
- runtime-environment facts
- intermediate-code representations
- optimization distinctions
- data-flow directions
- GEN/KILL and IN/OUT equations
- common GATE traps
- memory tricks

It should be usable for last-day revision.

RULE 16 — CREATE A CONFUSION TABLE

At the end of the subject create one large comparison section for commonly confused concepts such as:

- token vs lexeme vs pattern
 - compiler vs interpreter
 - phase vs pass
 - parse tree vs syntax tree
 - left recursion vs left factoring
 - LL vs LR
 - LR(0) vs SLR vs LALR vs CLR
 - synthesized vs inherited attributes
 - S-attributed vs L-attributed
 - quadruple vs triple vs indirect triple
 - static vs stack vs heap allocation
 - control link vs access link
 - constant folding vs constant propagation
 - common-subexpression elimination vs copy propagation
 - live variables vs available expressions
 - forward vs backward data-flow analysis
-

RULE 17 — USE SIMPLE LANGUAGE BUT DO NOT OVERSIMPLIFY

Assume I am a GATE aspirant, not an expert compiler engineer.

Use simple language first, but teach enough depth to solve difficult GATE questions.

If a term is technically important, use the correct technical terminology and explain it simply instead of removing it.

RULE 18 — DO NOT SKIP PREREQUISITES

If a syllabus topic depends on a prerequisite, briefly teach that prerequisite.

For example, if LR parsing requires understanding:

- context-free grammar
- derivation
- augmented grammar
- items

teach those first.

However, do not wander into material that has little relevance to GATE.

RULE 19 — PRIORITIZE GATE

Whenever deciding how much detail to teach, prioritize:

1. Concepts directly present in the official syllabus
2. Concepts repeatedly needed to solve GATE questions
3. Prerequisites needed for those concepts
4. Common traps and properties

Avoid excessive implementation/compiler-construction details unless they help with GATE.

RULE 20 — NEVER ASSUME I UNDERSTOOD A PREVIOUS STEP

For difficult topics such as:

- FIRST/FOLLOW
- LL(1)
- closure/GOTO
- LR parsing tables
- syntax-directed translation
- activation records
- three-address code
- basic blocks
- liveness analysis
- constant propagation
- available expressions/common-subexpression elimination

show the procedure in a clear sequence.

Use:

Step 1 !' Step 2 !' Step 3 !' ...

RULE 21 — TEACH SHORTCUTS ONLY AFTER THE CORRECT METHOD

First teach the conceptually correct/general method.

Then provide:

GATE Shortcut

if a valid faster technique exists.

Do not teach shortcuts that only work in special cases without clearly stating the limitation.

RULE 22 — PERIODIC REVISION

After approximately every 4–6 major parts, provide a short cumulative revision:

- previous concepts
- formulas
- relationships
- 5–10 mixed questions

This should prevent me from forgetting earlier material.

RULE 23 — TRACK THE ROADMAP

At the beginning of each new response after I type NEXT, show only a tiny progress indicator such as:

Completed: Parts 1–5 | Current: Part 6 | Remaining: Parts 7–20

Do not repeat the entire roadmap every time.

RULE 24 — FINAL SUBJECT COMPLETION CHECK

When the complete course is finished, return to the official syllabus line by line and create a checklist:

' Lexical analysis ' Parsing ' Syntax-directed translation ' Runtime environments ' Intermediate code generation '
Local optimisation ' Constant propagation ' Liveness analysis ' Common subexpression elimination

Under each item, list exactly which concepts we covered.

This is to verify that **nothing in the official GATE syllabus was missed**.

RULE 25 — FINAL EXAM TRAINING

After completing theory, give me a final practice sequence:

LEVEL 1 — Concept Check

Basic questions.

LEVEL 2 — Standard GATE

Normal GATE-level problems.

LEVEL 3 — Traps

Questions designed around confusing properties.

LEVEL 4 — Mixed Problems

Questions combining multiple Compiler Design concepts.

LEVEL 5 — Rapid Revision Test

A short test covering the whole subject.

For every solution explain not only the answer, but also the fastest reasoning.

RESPONSE STYLE

Use:

- simple English
- clean formatting
- formulas
- tables
- diagrams
- examples
- memory tricks
- GATE traps
- concise note-ready summaries

Avoid:

- unnecessarily huge paragraphs
- motivational filler
- repeatedly telling me to watch videos/books
- skipping calculations
- assuming knowledge
- teaching irrelevant postgraduate compiler theory

If something is important for GATE, teach it properly even if it requires multiple messages.

START NOW

First:

1. Analyze the exact official syllabus I pasted.
2. Expand it into a **complete numbered Compiler Design roadmap**.
3. Explain which portions are especially important/problem-oriented for GATE.
4. Create the overall course structure.
5. Then immediately begin **PART 1** from the foundations.

After completing Part 1, stop.

I will type:

NEXT

and you will continue with Part 2.

Continue this process until the **entire official syllabus has been completed, revised and tested.** ...

The most useful part of this prompt is that **next time you do not need to rewrite it.** Change only these two sections:

Subject: Operating Systems / DBMS / Algorithms / TOC / COA / etc. **Official GATE Syllabus:**
paste that subject's exact syllabus

Everything else can remain the same. For Compiler Design specifically, this prompt will force the teaching to cover the full path from **lexical analysis !' parsing !' syntax-directed translation !' runtime environments !' intermediate code !' optimization !' data-flow analysis**, rather than giving you only surface-level notes.

USER

GATE COMPLETE SUBJECT TEACHER — MASTER PROMPT

I am preparing for **GATE** and I have limited time.

I do **NOT** have the flexibility to watch many hours of YouTube lectures for every subject. I want to learn the subject mainly from this chat and make my own handwritten/typed notes from your explanations.

Act as my:

- GATE subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor
- problem-solving trainer
- revision coach
- memory-trick creator

MY CURRENT SUBJECT

Subject: Compiler Design

OFFICIAL GATE SYLLABUS

Compiler Design:

Lexical analysis, parsing, syntax-directed translation. Runtime environments. Intermediate code generation. Local optimisation, Data flow analyses: constant propagation, liveness analysis, common sub expression elimination.

MAIN GOAL

Convert the above official syllabus into a **complete GATE preparation course inside this chat.**

Do not merely explain the syllabus headings.

Expand every syllabus point into all the concepts, subtopics, formulas, algorithms, properties, special cases, numerical/problem types, and prerequisite ideas that can reasonably be tested in GATE.

My goal is that after completing your course and solving PYQs, I should not need to depend on long video lectures

for theory.

RULE 1 — FIRST EXPAND THE COMPLETE SYLLABUS

Before teaching, convert the syllabus into a detailed numbered roadmap.

For example, if the syllabus says:

Lexical analysis

expand it into relevant concepts such as:

- role of lexical analyzer
- token
- lexeme
- pattern
- token representation
- regular expressions relevant to lexical analysis
- finite automata connection
- lexical errors
- input buffering if relevant for GATE
- token-counting questions
- interaction with symbol table
- common GATE traps

Do the same for **every syllabus phrase**.

The roadmap should be detailed enough that I can verify that no important topic has been skipped.

RULE 2 — TEACH ONE LOGICAL CHUNK AT A TIME

Do NOT dump the entire subject in one huge answer.

Divide the complete course into numbered parts such as:

**PART 1 — Foundations PART 2 — Lexical Analysis PART 3 — Context-Free Grammars and Parsing Basics
PART 4 — FIRST and FOLLOW** etc.

Teach **one manageable logical chunk per response**.

At the end, stop and write:

Reply NEXT for Part X.

When I type **NEXT**, continue exactly from where you stopped.

Do not restart the subject and do not repeat previous material unnecessarily.

RULE 3 — ASSUME I NEED THE CONCEPT FROM THE FOUNDATION

For every new concept:

1. Tell me **what it is**.
2. Tell me **why we need it**.

3. Explain the intuition in simple language.
4. Give the formal definition.
5. Give a small example.
6. Then move to the GATE-level interpretation.
7. Then explain the common question patterns.

Do not jump directly into formulas without building intuition.

RULE 4 — MAKE THE EXPLANATION NOTE-FRIENDLY

Write explanations so that I can directly convert them into GATE notes.

Use:

- clear headings
- short paragraphs
- bullets
- tables
- formulas
- diagrams using text/ASCII
- step-by-step algorithms
- worked examples
- comparison tables

Avoid unnecessary storytelling or very long theory that has little GATE value.

Whenever useful, explicitly mark:

WRITE THIS IN NOTES

and give me the concise form that is worth memorizing.

RULE 5 — DISTINGUISH THREE LEVELS

Whenever useful, separate information into:

CONCEPT

What I must understand.

REMEMBER

Facts/formulas/properties that must be memorized.

GATE USE

How GATE can ask a question from it.

This is very important.

RULE 6 — GIVE MEMORY TRICKS

For concepts that involve:

- sequences
- properties
- formulas
- classifications
- parsing families
- compiler phases
- data-flow directions
- relationships
- algorithms

create simple mnemonics or memory shortcuts.

Do not create complicated mnemonics that are harder than the concept itself.

Use formats like:

Memory Trick: Shortcut: 1-line recall: Exam memory:

RULE 7 — EXPLAIN FORMULAS COMPLETELY

Whenever a formula is introduced:

1. Write the formula clearly.
2. Explain every variable/symbol.
3. Explain when to use it.
4. Give a small example.
5. Mention important special cases.
6. Mention common mistakes.

Never give a formula without explaining its meaning.

RULE 8 — SOLVE NUMERICALS STEP BY STEP

Compiler Design contains many problem-solving areas.

For every important question type:

- show the question pattern
- identify the concept
- solve step by step
- show the fastest valid GATE approach
- mention traps
- give a shortcut if one exists

For parsing/grammar problems, never skip intermediate steps that are necessary to understand the solution.

RULE 9 — GIVE SPECIAL ATTENTION TO HIGH-VALUE COMPILER DESIGN AREAS

Cover these thoroughly whenever they are relevant to the syllabus:

Lexical Analysis

- token
- lexeme
- pattern
- lexical analyzer
- scanner
- regular expressions
- finite automata connection
- token counting
- lexical errors
- symbol-table interaction

Parsing Foundations

- grammar terminology needed for parsing
- derivation
- leftmost derivation
- rightmost derivation
- parse tree
- ambiguity
- left recursion
- left factoring
- nullable variables
- FIRST
- FOLLOW

Top-Down Parsing

- recursive descent
- predictive parsing
- LL(1)
- LL(1) conditions
- parsing-table construction
- conflicts
- string parsing

Bottom-Up Parsing

- shift-reduce parsing
- handle
- handle pruning
- viable prefix
- shift/reduce conflict
- reduce/reduce conflict

LR Family

Where required for GATE, explain:

- LR(0) items
- augmented grammar
- closure
- GOTO
- canonical collection
- LR(0)
- SLR(1)
- CLR/LR(1)
- LALR(1)
- ACTION/GOTO table
- conflicts
- relationships between grammar classes

Give a clear comparison among:

LL(1), LR(0), SLR(1), LALR(1), CLR(1)

and teach me how to identify which class a grammar belongs to.

Syntax-Directed Translation

- syntax-directed definitions
- attributes
- synthesized attributes
- inherited attributes
- dependency graphs
- evaluation order
- S-attributed definitions
- L-attributed definitions
- syntax-directed translation schemes
- relevant GATE problems

Runtime Environments

Cover relevant concepts such as:

- runtime storage organization
- static allocation
- stack allocation
- heap allocation
- activation records
- activation trees
- parameters
- return value/address
- local variables

- temporaries
- control/dynamic links
- access/static links
- recursion
- scope issues where relevant
- parameter passing where relevant

Intermediate Code Generation

- intermediate representations
- syntax tree
- DAG where relevant
- three-address code
- temporaries
- quadruples
- triples
- indirect triples
- arithmetic expressions
- Boolean/control-flow translation where relevant
- backpatching if relevant to GATE

Basic Blocks and Flow Graphs

If needed for optimization/data-flow analysis, teach:

- leaders
- basic blocks
- identifying basic blocks
- control-flow graphs
- predecessors
- successors
- entry/exit nodes

Local Optimisation

Explain relevant optimizations such as:

- constant folding
- algebraic simplification
- strength reduction
- copy propagation
- dead-code elimination
- common-subexpression elimination
- local optimization using DAG where relevant

Clearly distinguish similar optimization techniques.

Data-Flow Analysis

Build the foundation properly:

- control-flow graph
- IN and OUT
- GEN and KILL
- forward analysis
- backward analysis
- may analysis
- must analysis
- iterative/fixed-point idea

Then thoroughly cover the syllabus topics:

Constant Propagation

- what information flows
- direction
- equations/meet intuition where required
- CFG problems

Liveness Analysis

- live/dead variable
- USE/DEF
- IN/OUT equations
- backward nature
- iterative calculation
- solved GATE-style examples

Common Subexpression Elimination

- available expressions
- GEN/KILL
- IN/OUT
- forward nature
- how data-flow information enables elimination
- solved examples

RULE 10 — ALWAYS EXPLAIN RELATIONSHIPS

GATE frequently tests relationships and properties.

For example, when teaching parsing classes, explicitly explain relationships such as whether one grammar class is a subset/superset of another.

Do not simply give a diagram.

Explain:

- why the relationship exists
- what implication follows
- what does NOT follow
- common GATE traps

RULE 11 — INCLUDE GATE TRAPS

After every important topic add:

GATE TRAPS

Include statements/options that students commonly confuse.

For example:

- token vs lexeme
- phase vs pass
- left recursion vs ambiguity
- LL vs LR
- shift/reduce vs reduce/reduce conflict
- synthesized vs inherited attribute
- static link vs dynamic/control link
- constant folding vs constant propagation
- dead code vs unreachable code
- liveness direction
- available expressions direction

Explain why the wrong statement is wrong.

RULE 12 — INCLUDE PYQ-STYLE THINKING

You do not need to reproduce copyrighted PYQs word-for-word.

Instead:

- identify recurring GATE question patterns
- create equivalent GATE-style questions
- explain how to recognize the pattern
- solve representative problems
- tell me what concept the examiner is testing

Label these sections:

GATE PATTERN GATE-STYLE EXAMPLE FAST SOLUTION

RULE 13 — ADD MINI PRACTICE

At the end of each major topic give approximately:

- 2 easy questions
- 2 medium GATE-level questions
- 1 tricky conceptual question

Do not immediately reveal the answers if self-testing would be more useful.

Put solutions after a separator so I can attempt them first.

RULE 14 — GIVE A 30-SECOND REVISION

At the end of every logical part provide:

30-SECOND REVISION

It should contain only the most important:

- definitions
- formulas
- relationships
- shortcuts
- traps

that I should recall before solving questions.

RULE 15 — BUILD A RUNNING FORMULA/FACT SHEET

Whenever we encounter an important formula, condition or relationship, keep adding it mentally to a final revision sheet.

At the end of the complete subject, give me:

COMPILER DESIGN FINAL REVISION SHEET

containing:

- all important formulas
- FIRST/FOLLOW rules
- LL(1) conditions
- LR-family relationships
- important compiler-phase facts
- syntax-directed translation properties
- runtime-environment facts
- intermediate-code representations
- optimization distinctions
- data-flow directions
- GEN/KILL and IN/OUT equations
- common GATE traps
- memory tricks

It should be usable for last-day revision.

RULE 16 — CREATE A CONFUSION TABLE

At the end of the subject create one large comparison section for commonly confused concepts such as:

- token vs lexeme vs pattern
- compiler vs interpreter

- phase vs pass
- parse tree vs syntax tree
- left recursion vs left factoring
- LL vs LR
- LR(0) vs SLR vs LALR vs CLR
- synthesized vs inherited attributes
- S-attributed vs L-attributed
- quadruple vs triple vs indirect triple
- static vs stack vs heap allocation
- control link vs access link
- constant folding vs constant propagation
- common-subexpression elimination vs copy propagation
- live variables vs available expressions
- forward vs backward data-flow analysis

RULE 17 — USE SIMPLE LANGUAGE BUT DO NOT OVERSIMPLIFY

Assume I am a GATE aspirant, not an expert compiler engineer.

Use simple language first, but teach enough depth to solve difficult GATE questions.

If a term is technically important, use the correct technical terminology and explain it simply instead of removing it.

RULE 18 — DO NOT SKIP PREREQUISITES

If a syllabus topic depends on a prerequisite, briefly teach that prerequisite.

For example, if LR parsing requires understanding:

- context-free grammar
- derivation
- augmented grammar
- items

teach those first.

However, do not wander into material that has little relevance to GATE.

RULE 19 — PRIORITIZE GATE

Whenever deciding how much detail to teach, prioritize:

1. Concepts directly present in the official syllabus
2. Concepts repeatedly needed to solve GATE questions
3. Prerequisites needed for those concepts
4. Common traps and properties

Avoid excessive implementation/compiler-construction details unless they help with GATE.

RULE 20 — NEVER ASSUME I UNDERSTOOD A PREVIOUS STEP

For difficult topics such as:

- FIRST/FOLLOW
- LL(1)
- closure/GOTO
- LR parsing tables
- syntax-directed translation
- activation records
- three-address code
- basic blocks
- liveness analysis
- constant propagation
- available expressions/common-subexpression elimination

show the procedure in a clear sequence.

Use:

Step 1 !' Step 2 !' Step 3 !' ...

RULE 21 — TEACH SHORTCUTS ONLY AFTER THE CORRECT METHOD

First teach the conceptually correct/general method.

Then provide:

GATE Shortcut

if a valid faster technique exists.

Do not teach shortcuts that only work in special cases without clearly stating the limitation.

RULE 22 — PERIODIC REVISION

After approximately every 4–6 major parts, provide a short cumulative revision:

- previous concepts
- formulas
- relationships
- 5–10 mixed questions

This should prevent me from forgetting earlier material.

RULE 23 — TRACK THE ROADMAP

At the beginning of each new response after I type NEXT, show only a tiny progress indicator such as:

Completed: Parts 1–5 | Current: Part 6 | Remaining: Parts 7–20

Do not repeat the entire roadmap every time.

RULE 24 — FINAL SUBJECT COMPLETION CHECK

When the complete course is finished, return to the official syllabus line by line and create a checklist:

' Lexical analysis ' Parsing ' Syntax-directed translation ' Runtime environments ' Intermediate code generation '
Local optimisation ' Constant propagation ' Liveness analysis ' Common subexpression elimination

Under each item, list exactly which concepts we covered.

This is to verify that **nothing in the official GATE syllabus was missed**.

RULE 25 — FINAL EXAM TRAINING

After completing theory, give me a final practice sequence:

LEVEL 1 — Concept Check

Basic questions.

LEVEL 2 — Standard GATE

Normal GATE-level problems.

LEVEL 3 — Traps

Questions designed around confusing properties.

LEVEL 4 — Mixed Problems

Questions combining multiple Compiler Design concepts.

LEVEL 5 — Rapid Revision Test

A short test covering the whole subject.

For every solution explain not only the answer, but also the fastest reasoning.

RESPONSE STYLE

Use:

- simple English
- clean formatting
- formulas
- tables
- diagrams
- examples
- memory tricks
- GATE traps
- concise note-ready summaries

Avoid:

- unnecessarily huge paragraphs
- motivational filler

- repeatedly telling me to watch videos/books
- skipping calculations
- assuming knowledge
- teaching irrelevant postgraduate compiler theory

If something is important for GATE, teach it properly even if it requires multiple messages.

START NOW

First:

1. Analyze the exact official syllabus I pasted.
2. Expand it into a **complete numbered Compiler Design roadmap**.
3. Explain which portions are especially important/problem-oriented for GATE.
4. Create the overall course structure.
5. Then immediately begin **PART 1** from the foundations.

After completing Part 1, stop.

I will type:

NEXT

and you will continue with Part 2.

Continue this process until the **entire official syllabus has been completed, revised and tested**.

ASSISTANT

I'll use exactly this syllabus as the boundary and build the course around GATE-relevant depth only. I'll first map the full subject into parts, flag the most problem-heavy areas, then teach **Part 1** and stop there so your notes stay manageable.

ASSISTANT

Compiler Design — Complete GATE Course

I will treat **the syllabus you pasted as the official boundary**. Anything outside it will be taught only when it is a prerequisite for solving GATE questions.

1. COMPLETE COMPILER DESIGN ROADMAP

MODULE A — FOUNDATION

Part 1 — Compiler Fundamentals

1. What is a compiler?
2. Source language and target language
3. Compiler vs interpreter
4. Language-processing system
5. Preprocessor

- 6. Compiler
 - 7. Assembler
 - 8. Linker
 - 9. Loader
 - 10. Phases of a compiler
 - 11. Analysis phase vs synthesis phase
 - 12. Front end vs back end
 - 13. Symbol table
 - 14. Error handling
 - 15. Phase vs pass
 - 16. One-pass vs multipass idea
-

MODULE B — LEXICAL ANALYSIS

Part 2 — Lexical Analysis Fundamentals

- 17. Role of lexical analyzer
- 18. Character stream
- 19. Token
- 20. Lexeme
- 21. Pattern
- 22. Token attributes
- 23. Token representation
- 24. Scanner
- 25. Interaction with parser
- 26. Interaction with symbol table
- 27. Ignoring whitespace/comments
- 28. Lexical errors
- 29. Token-counting problems

Part 3 — Regular Expressions and Finite Automata for Lexical Analysis

- 30. Why lexical analysis uses regular languages
- 31. Regular expressions revision
- 32. Regular definitions
- 33. Recognition of identifiers/numbers/operators
- 34. Finite automata connection
- 35. NFA vs DFA relevance
- 36. RE !' NFA !' DFA idea
- 37. Longest-prefix / maximal-munch principle
- 38. Reserved words vs identifiers
- 39. Lexical ambiguities
- 40. GATE-style recognition problems

Part 4 — Input Buffering and Lexical Analyzer Design

- 41. Why buffering is required

- 42. Buffer-pair idea
 - 43. Sentinel concept
 - 44. Lookahead
 - 45. Retract
 - 46. Lexical analyzer generators — only relevant concepts
-

MODULE C — PARSING FOUNDATIONS

Part 5 — CFG and Parsing Basics

- 47. Parser
- 48. Role of syntax analyzer
- 49. Context-free grammar essentials
- 50. Terminals
- 51. Non-terminals
- 52. Productions
- 53. Start symbol
- 54. Sentential form
- 55. Sentence
- 56. Derivation
- 57. Leftmost derivation
- 58. Rightmost derivation
- 59. Parse tree
- 60. Relation between derivations and parse trees

Part 6 — Ambiguity, Left Recursion and Left Factoring

- 61. Ambiguous grammar
 - 62. Ambiguous string
 - 63. Multiple parse trees
 - 64. Operator precedence
 - 65. Associativity
 - 66. Removing ambiguity where possible
 - 67. Left recursion
 - 68. Immediate left recursion
 - 69. Eliminating immediate left recursion
 - 70. Indirect left recursion
 - 71. Left factoring
 - 72. Difference: left recursion vs left factoring
-

MODULE D — TOP-DOWN PARSING

Part 7 — FIRST, FOLLOW and NULLABLE

- 73. Nullable non-terminal
 - 74. FIRST of terminal
-

- 75. FIRST of non-terminal
- 76. FIRST of sequence
- 77. μ handling
- 78. FOLLOW
- 79. \$ rule
- 80. Complete FIRST rules
- 81. Complete FOLLOW rules
- 82. Step-by-step computation
- 83. GATE shortcuts and traps

Part 8 — Recursive Descent and Predictive Parsing

- 84. Top-down parsing
- 85. Recursive-descent parser
- 86. Backtracking
- 87. Predictive parsing
- 88. Why left recursion is a problem
- 89. Why left factoring helps
- 90. Predictive parser structure

Part 9 — LL(1) Parsing

- 91. Meaning of LL(1)
- 92. LL(1) grammar
- 93. LL(1) conditions
- 94. SELECT/PREDICT sets
- 95. Parsing-table construction
- 96. Table conflicts
- 97. Parsing an input string
- 98. Stack operations
- 99. Acceptance/error
- 100. Identifying whether a grammar is LL(1)

MODULE E — BOTTOM-UP PARSING

Part 10 — Shift-Reduce Parsing

- 101. Bottom-up parsing
- 102. Reverse rightmost derivation
- 103. Shift operation
- 104. Reduce operation
- 105. Handle
- 106. Handle pruning
- 107. Viable prefix
- 108. Shift-reduce parser
- 109. Shift/reduce conflict
- 110. Reduce/reduce conflict

MODULE F — LR PARSING

Part 11 — LR(0) Items, Closure and GOTO

- 111 What LR means
- 112 Augmented grammar
- 113 LR item
- 114 Dot notation
- 115 Kernel/non-kernel intuition
- 116 Closure
- 117 GOTO
- 118 Canonical collection of LR(0) items
- 119 LR automaton

Part 12 — LR(0) Parsing

- 120 ACTION table
- 121 GOTO table
- 122 Shift entries
- 123 Reduce entries
- 124 Accept entry
- 125 LR(0) conflicts
- 126 Identifying LR(0) grammars
- 127 Parsing strings with an LR table

Part 13 — SLR(1)

- 128 Why SLR improves LR(0)
- 129 FOLLOW sets in SLR
- 130 SLR reduce rule
- 131 Constructing SLR table
- 132 Detecting SLR conflicts
- 133 LR(0) vs SLR

Part 14 — CLR(1) / Canonical LR(1)

- 134 LR(1) item
- 135 Lookahead
- 136 LR(1) closure
- 137 GOTO
- 138 Canonical LR(1) states
- 139 CLR table idea
- 140 Why CLR is more powerful
- 141 State-count implications

Part 15 — LALR(1)

- 142 Motivation for LALR
- 143 Same LR(0) core

- 144 Merging CLR states
- 145 Combining lookaheads
- 146 New conflicts caused by merging
- 147 LALR table
- 148 LALR vs CLR

Part 16 — Parser Family Relationships

- 149 LL(1)
- 150 LR(0)
- 151 SLR(1)
- 152 LALR(1)
- 153 CLR(1)
- 154 Relative expressive power
- 155 Inclusion relationships
- 156 Which implications are valid
- 157 Which reverse implications are invalid
- 158 Conflict-based GATE questions

MODULE G — SYNTAX-DIRECTED TRANSLATION

Part 17 — Syntax-Directed Definitions

- 159 Syntax-directed translation
- 160 Syntax-directed definition
- 161 Attributes
- 162 Synthesized attributes
- 163 Inherited attributes
- 164 Semantic rules
- 165 Annotated parse tree
- 166 Dependency graph
- 167 Attribute-evaluation order

Part 18 — S-Attributed and L-Attributed Definitions

- 168 S-attributed definitions
- 169 Bottom-up evaluation
- 170 L-attributed definitions
- 171 Left-to-right restriction
- 172 Synthesized vs inherited
- 173 S-attributed vs L-attributed
- 174 Syntax-directed translation schemes
- 175 GATE attribute problems

MODULE H — RUNTIME ENVIRONMENTS

Part 19 — Runtime Storage and Activation Records

- 176 Runtime environment
- 177 Storage organization
- 178 Code area
- 179 Static/global area
- 180 Stack
- 181 Heap
- 182 Static allocation
- 183 Stack allocation
- 184 Heap allocation
- 185 Procedure activation
- 186 Activation tree
- 187 Activation record / stack frame
- 188 Parameters
- 189 Return value
- 190 Return address
- 191 Local variables
- 192 Temporaries
- 193 Saved machine status

Part 20 — Links, Scope, Recursion and Parameter Passing

- 194 Control/dynamic link
- 195 Access/static link
- 196 Difference between the two
- 197 Static scope
- 198 Nested procedures
- 199 Recursion
- 200 Why stack allocation supports recursion
- 201 Parameter passing basics
- 202 Call by value/reference where relevant

MODULE I — INTERMEDIATE CODE GENERATION

Part 21 — Intermediate Representations and Three-Address Code

- 203 Why intermediate code is needed
- 204 Intermediate representation
- 205 Syntax tree
- 206 Abstract syntax tree
- 207 DAG
- 208 Three-address code
- 209 Assignment statements
- 210 Unary operations
- 211 Binary operations

- 212 Conditional jumps
- 213 Unconditional jumps
- 214 Procedure-call forms
- 215 Temporaries

Part 22 — Quadruples, Triples and Indirect Triples

- 216 Quadruple representation
- 217 op, arg1, arg2, result
- 218 Triples
- 219 Statement-number references
- 220 Indirect triples
- 221 Comparison among the three
- 222 Expression-to-TAC questions
- 223 Minimum temporary questions
- 224 Number-of-instruction questions

Part 23 — Boolean Expressions and Control Flow

- 225 Boolean expression translation
- 226 Short-circuit evaluation
- 227 Relational expressions
- 228 Conditional jumps
- 229 if
- 230 if-else
- 231 while
- 232 Boolean control flow
- 233 Backpatching basics
- 234 truelist
- 235 falselist
- 236 nextlist

MODULE J — BASIC BLOCKS & LOCAL OPTIMISATION

Part 24 — Basic Blocks and Flow Graphs

- 237 Basic block
- 238 Leader
- 239 Rules for identifying leaders
- 240 Constructing basic blocks
- 241 Control-flow graph
- 242 Predecessor
- 243 Successor
- 244 Entry/exit
- 245 GATE block-count questions

Part 25 — Local Optimisation

- 246 What optimization means
 - 247 Semantics preservation
 - 248 Local vs global optimization
 - 249 Constant folding
 - 250 Algebraic simplification
 - 251 Strength reduction
 - 252 Copy propagation
 - 253 Dead-code elimination
 - 254 Common-subexpression elimination
 - 255 DAG of a basic block
 - 256 Identifying common subexpressions
 - 257 Eliminating redundant computation
-

MODULE K — DATA-FLOW ANALYSIS

Part 26 — Data-Flow Analysis Foundations

- 258 Why data-flow analysis is needed
- 259 CFG-based reasoning
- 260 Program point
- 261 Data-flow fact
- 262 IN set
- 263 OUT set
- 264 GEN set
- 265 KILL set
- 266 Transfer function
- 267 Meet operator
- 268 Forward analysis
- 269 Backward analysis
- 270 May analysis
- 271 Must analysis
- 272 Fixed-point iteration

Part 27 — Constant Propagation

- 273 Constant value information
- 274 UNDEF / constant / NAC-style intuition
- 275 Forward direction
- 276 Transfer function
- 277 Merge/meet
- 278 CFG calculation
- 279 Difference from constant folding
- 280 GATE problems

Part 28 — Liveness Analysis

- 281 Live variable

282Dead variable

283USE

284DEF

285Backward analysis

286Liveness equations

$$\bigcup_{S \in \text{succ}(B)} \text{IN}[S]$$

$$\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] - \text{DEF}[B])$$

287Iterative solution

288Statement-level liveness

289Basic-block-level liveness

290Register-allocation relevance

291GATE problems

Part 29 — Common Subexpression Elimination / Available Expressions

292Common expression

293Available expression

294GEN

295KILL

296Forward analysis

297Must-analysis idea

298Intersection at joins

299IN/OUT equations

300Using availability to eliminate recomputation

301GATE problems

MODULE L — FINAL GATE REVISION

Part 30 — Complete Compiler Design Revision

302Compiler phases

303Lexical-analysis summary

304FIRST/FOLLOW sheet

305LL(1) conditions

306LR parser relationships

307Syntax-directed translation sheet

308Runtime-environment sheet

309TAC sheet

310Optimization confusion table

311Data-flow direction table

312Important equations

313GATE traps

314Full syllabus checklist

Part 31 — Final Mixed Practice

- 315 Concept check
 - 316 Standard GATE questions
 - 317 Parser traps
 - 318 Numerical/set problems
 - 319 Mixed compiler questions
 - 320 Rapid-revision test
-

2. MOST PROBLEM-ORIENTED AREAS

These deserve especially careful practice.

~~0=1hr~~ Part A — Must Master

- FIRST and FOLLOW
- LL(1) checking and parsing table
- shift-reduce concepts
- LR(0) items
- closure and GOTO
- LR(0), SLR, LALR and CLR relationships
- parser conflicts
- synthesized/inherited attributes
- three-address code
- quadruples/triples
- basic blocks
- data-flow equations
- liveness analysis
- available expressions/common-subexpression elimination

These involve procedures, tables, sets or multiple reasoning steps.

~~0=1hr~~ Part B — Conceptually Very Important

- token/lexeme/pattern
- ambiguity
- left recursion
- left factoring
- activation records
- static vs dynamic links
- constant folding vs propagation
- local optimizations

~~0=1hr~~ Part C — Foundation

- compiler phases
- front end/back end
- compiler/interpreter
- assembler/linker/loader
- phase vs pass

Foundation questions are usually easier, but these concepts support the entire subject.

PART 1 — COMPILER FUNDAMENTALS

1. What Is a Compiler?

CONCEPT

You may write:

```
C
int a = 10;
int b = 20;
int c = a + b;
```

A CPU does not directly execute this C source code.

We need software that translates the source program into a form suitable for execution.

That software is the **compiler**.

Formal idea

A compiler is a program that translates a program written in a **source language** into an equivalent program in a **target language**, while reporting errors found in the source program.

```
TEXT
Source Program
      !"
  COMPILER
      !"
Target Program
```

Usually:

```
TEXT
High-Level Language
      !"
  Compiler
      !"
Assembly / Machine-oriented code
```

WRITE THIS IN NOTES

Compiler: A translator that converts a source program from a source language into an equivalent target program and reports errors encountered during translation.

2. Compiler vs Interpreter

An interpreter also allows high-level programs to execute, but the model is different.

Compiler

TEXT

```
Source Program
      ! "
  Compiler
      ! "
Target Program
      ! "
  Execution
```

The translation result can later be executed.

Interpreter

TEXT

```
Source Program
      ! "
  Interpreter
      ! "
  Execution/result
```

The interpreter directly participates in executing the source program.

Comparison

Compiler	Interpreter
Produces target representation/program	Executes by interpreting source operations
Compilation occurs before later execution	Translation and execution are closely connected
Target program can normally be reused	Interpreter generally participates each time
Compiler reports errors while translating	Errors may be encountered during interpretation

GATE TRAP

Do not blindly memorize:

"Every compiler translates the whole program at once."

Modern systems can combine compilation, interpretation and JIT compilation.

For GATE fundamentals, understand the **conceptual distinction**, not an absolute implementation rule.

3. Complete Language-Processing System

Compilation is not the only step between source code and execution.

A common model is:

TEXT

```
Source Program
      ! "
  Preprocessor
      ! "
```

```

Modified Source
! "
Compiler
! "
Assembly Program
! "
Assembler
! "
Relocatable/Object Code
! "
Linker
! "
Executable
! "
Loader
! "
Memory !' Execution

```

Memory Trick

P C A L L

Preprocessor Compiler Assembler Linker Loader

Think:

Program makes a P-C-A-L-L before execution.

4. Preprocessor

A preprocessor processes source-level directives before normal compilation.

Example:

```

C
#include <stdio.h>
#define MAX 100

```

It may perform tasks such as:

- macro expansion
- file inclusion
- conditional compilation

Conceptually:

```

TEXT
Source
! "
Preprocessor
! "
Modified Source

```

5. Assembler

The assembler translates **assembly language** into **machine/object code**.

TEXT

Assembly

!"

Assembler

!"

Object/Machine Code

REMEMBER

Compiler: high-level !' lower-level/target representation **Assembler:** assembly !' machine/object code

6. Linker

Suppose your program calls a library routine.

C

```
printf("Hello");
```

Your object file may contain a reference to `printf`, while the implementation exists elsewhere.

The **linker** combines:

- object modules
- library routines
- external references

and produces an executable or another linked program image.

1-line Recall

Linker = joins separately translated pieces.

7. Loader

The **loader** places the executable program into memory and prepares it for execution.

Memory Trick

Linker links. Loader loads.

Do not confuse them.

8. Phases of a Compiler

This is the first major diagram of Compiler Design.

TEXT

SOURCE PROGRAM

```
!"
% % % % % % % % % % % % % % % % % % % %
%   L e x i c a l   A n a l y s i s   %
% % % % % % % % % % % % % % % % % % % %
!"
```

TOKENS

```
!"
% % % % % % % % % % % % % % % % % % % %
%   S y n t a x   A n a l y s i s   %
% % % % % % % % % % % % % % % % % % % %
!"
```

SYNTAX STRUCTURE

```
!"
% % % % % % % % % % % % % % % % % % % %
%   S e m a n t i c   A n a l y s i s   %
% % % % % % % % % % % % % % % % % % % %
!"
```

ANNOTATED STRUCTURE

```
!"
% % % % % % % % % % % % % % % % % % % %
%   I n t e r m e d i a t e   C o d e   G e n .   %
% % % % % % % % % % % % % % % % % % % %
!"
```

INTERMEDIATE CODE

```
!"
% % % % % % % % % % % % % % % % % % % %
%   C o d e   O p t i m i z a t i o n %
% % % % % % % % % % % % % % % % % % % %
!"
```

OPTIMIZED CODE

```
!"
% % % % % % % % % % % % % % % % % % % %
%   C o d e   G e n e r a t i o n   %
% % % % % % % % % % % % % % % % % % % %
!"
```

TARGET CODE

Supporting structures include:

TEXT

Symbol Table
Error Handling

These may interact with several compiler phases.

9. Memory Trick for Compiler Phases

The six classic phases are:

1. Lexical analysis
2. Syntax analysis
3. Semantic analysis
4. Intermediate code generation
5. Optimization
6. Code generation

Mnemonic:

L S S I O C

Lazy Students Study In Our College

TEXT

```
L !'  L e x i c a l
S !'  S y n t a x
S !'  S e m a n t i c
I !'  I n t e r m e d i a t e
O !'  O p t i m i z a t i o n
C !'  C o d e   g e n e r a t i o n
```

WRITE THIS IN NOTES

$$[\boxed{L \rightarrow S \rightarrow S \rightarrow I \rightarrow O \rightarrow C}]$$

10. Understand All Six Phases Using One Example

Consider:

TEXT

```
a = b + c * 2
```

Phase 1 — Lexical Analysis

Lexical analysis groups characters into **tokens**.

Possible token sequence:

TEXT

```
<id,a>
=
<id,b>
+
<id,c>
*
<num,2>
```

Conceptually:

$$[\boxed{\text{Characters}} \rightarrow \text{Tokens}]$$

Lexical analyzer is also called a:

Scanner

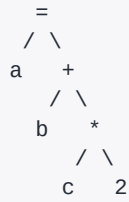
We will study this deeply in Part 2.

Phase 2 — Syntax Analysis

The syntax analyzer checks grammatical structure.

It recognizes something equivalent to:

TEXT



The multiplication is grouped before addition.

Conceptually:

$\boxed{\text{Tokens}} \rightarrow \text{Syntax/Parse Structure}$

Syntax analyzer is also called:

Parser

Phase 3 — Semantic Analysis

Now the compiler checks whether the program makes **semantic sense according to language rules**.

Examples:

- Was the variable declared?
- Are operand types compatible?
- Are function arguments correct?
- Is an identifier being used correctly?

Example:

```
c
int x;
x = "hello";
```

In a language where this assignment is illegal, the problem is semantic/type-related.

Important

A program can be:

' lexically valid ' syntactically valid 'Lsemantically invalid

11. Lexical vs Syntax vs Semantic

This distinction is extremely important.

Lexical

Concerned with forming tokens.

Example:

TEXT

C h a r a c t e r s ! ' i d e n t i f i e r s , n u m b e r s , o p e r a t o r s

Syntax

Concerned with grammatical structure.

Example:

C

```
x = a + ;
```

The expression structure is incomplete.

Semantic

Concerned with meaning/type/declaration constraints.

Example:

C

```
int x;  
x = "abc";
```

Memory Trick

Think:

L !' Words

S !' Sentence structure

S !' Meaning

Similar to English:

TEXT

L e t t e r s ! ' W o r d s ! ' G r a m m a r ! ' M e a n i n g

Compiler:

TEXT

C h a r a c t e r s ! ' T o k e n s ! ' S y n t a x ! ' S e m a n t i c s

12. Intermediate Code Generation

After understanding the source program, a compiler can generate an intermediate representation.

For:

TEXT

```
a = b + c * 2
```

possible three-address code:

TEXT

```
t1 = c * 2
t2 = b + t1
a = t2
```

where `t1`, `t2` are temporary variables.

Why intermediate code?

It helps separate:

TEXT

```
Source-language-dependent processing
      from
Target-machine-dependent processing
```

We'll study this deeply later.

13. Code Optimization

Optimization tries to improve code while preserving program meaning.

Example:

TEXT

```
x = 2 * 10
```

can become:

TEXT

```
x = 20
```

This is **constant folding**.

Another example:

TEXT

```
x = y + 0
```

can be simplified to:

TEXT

```
x = y
```

Important

Optimization should preserve the observable semantics required by the language.

It is not simply "change anything to make code shorter."

14. Code Generation

Code generation converts the intermediate representation toward the target-machine code.

Conceptually:

```
TEXT
Intermediate Code
      !"
Code Generator
      !"
Assembly/Machine Instructions
```

Possible machine-oriented instructions:

```
TEXT
LOAD ...
MUL ...
ADD ...
STORE ...
```

Exact instructions depend on the target architecture.

15. Symbol Table

Consider:

```
C
int count;
float salary;
```

The compiler needs information about these identifiers.

A conceptual symbol table:

Identifier	Type
count	int
salary	float

A real symbol table may store:

- identifier name
- type
- scope
- storage information
- parameter information
- return type
- dimensions
- attributes

Memory Trick

Symbol Table = compiler's directory of identifiers.

It is used by multiple phases.

16. Error Handling

Different errors can arise at different stages.

Lexical Error

Problem recognizing valid tokens.

Syntax Error

Tokens do not form a valid grammatical structure.

Example:

```
C
x = a + ;
```

Semantic Error

The structure may be grammatically legal but violates meaning/type/declaration rules.

Example:

```
C
int x;
x = "abc";
```

assuming incompatible types.

17. Analysis Phase vs Synthesis Phase

A compiler is sometimes conceptually divided into:

Analysis

Understand the source program and construct an internal representation.

```
TEXT
Source
! "
Analysis
! "
Intermediate Representation
```

Synthesis

Use that representation to create the target program.

TEXT

```
Intermediate Representation
      |
      |! "
      |
Synthesis
      |
      |! "
      |
Target Code
```

Memory Trick

Analysis = Understand

Synthesis = Build

18. Front End vs Back End

Front End

Mostly concerned with source-program analysis.

Typical tasks:

- lexical analysis
- syntax analysis
- semantic analysis
- creation of an intermediate representation

It is strongly connected to the **source language**.

Back End

Mostly concerned with:

- optimization
- target code generation

It is strongly connected to the **target machine/architecture**.

Exam Memory

TEXT

```
Front End !' Understand source
Back End !' Produce/improve target code
```

The exact engineering boundary can vary between compiler implementations, so think conceptually rather than treating every phase split as universal.

19. Phase vs Pass

A common GATE confusion.

Phase

A **logical function** performed by the compiler.

Examples:

- lexical analysis
- syntax analysis
- optimization

Pass

A traversal/processing of the program or intermediate representation.

A single pass may perform multiple logical phases.

Therefore:

$\boxed{\text{Phase}} \neq \text{Pass}$

Example

A compiler could combine:

```
TEXT
Lexical Analysis
+
Parsing
+
some Semantic Analysis
```

during one processing pass.

20. One-Pass vs Multi-Pass Compiler

One-pass compiler

Processes its relevant representation essentially once.

Potential benefit:

- less compilation overhead
- simpler in suitable languages

But may have less information available for sophisticated analysis/optimization.

Multi-pass compiler

Processes one or more representations multiple times.

Potential advantages:

- more analysis opportunities

- stronger optimization
- easier separation of compiler components

GATE USE

Typical conceptual question:

Does six compiler phases imply six passes?

Answer

'L No.

There is no necessary one-to-one relation.

21. Important Input/Output of Compiler Phases

WRITE THIS IN NOTES

Phase	Main Input	Main Job/Output
Lexical	Characters	Tokens
Syntax	Tokens	Syntax/parse structure
Semantic	Syntax structure	Checked/annotated structure
Intermediate code	Analyzed representation	IR/TAC
Optimization	IR	Improved IR
Code generation	IR	Target code

22. MASTER COMPILER FLOW

Write this once in your notebook:

```

TEXT
SOURCE PROGRAM
    ! "
LEXICAL
    ! "
TOKENS
    ! "
SYNTAX
    ! "
PARSE/SYNTAX STRUCTURE
    ! "
SEMANTIC
    ! "
ANNOTATED STRUCTURE
    ! "
INTERMEDIATE CODE
    ! "
OPTIMIZATION
    ! "
  
```

CODE GENERATION
! "
TARGET PROGRAM

Side components:

TEXT
SYMBOL TABLE
ERROR HANDLING

GATE TRAPS

Trap 1

Lexical analyzer produces a parse tree.

'L False.

It produces **tokens**.

Trap 2

Parser converts characters directly into tokens.

'L False.

Lexical analyzer forms tokens; parser consumes them.

Trap 3

Scanner = parser.

'L False.

TEXT
Scanner = Lexical analyzer
Parser = Syntax analyzer

Trap 4

Assembler converts high-level C directly to machine code.

'L Generally false in the standard compilation model.

Assembler handles:

$\boxed{\text{Assembly}} \rightarrow \text{Object/Machine Code}$

Trap 5

Linker and loader perform the same task.

'LFalse.

TEXT

Linker !' combines / resolves program modules and libraries
Loader !' places executable into memory for execution.

Trap 6

Compiler phase = compiler pass.

'LFalse.

A phase is logical.

A pass is implementation/traversal-oriented.

Trap 7

All semantic errors must be syntax errors.

'LFalse.

A statement can follow grammar correctly yet violate type or declaration rules.

GATE PATTERN1 — Identify the Phase

Suppose a compiler detects that:

```
c
int a;
a = "GATE";
```

violates type rules.

Which phase is mainly responsible?

FAST SOLUTION

Type compatibility !' **Semantic analysis**

' Answer: **Semantic analyzer**

GATE PATTERN2 — Input/Output

Question:

Which compiler phase converts a stream of characters into a stream of tokens?

FAST SOLUTION

Characters !' Tokens

' **Lexical analysis**

GATE PATTERN3 — Phase vs Pass

Statement:

A compiler having six phases necessarily requires six passes.

'L False.

Multiple phases may operate in the same pass.

MINI PRACTICE — PART 1

Try these before checking the answers.

Q1 — Easy

Which component converts assembly language into machine/object code?

A. Parser B. Assembler C. Linker D. Loader

Q2 — Easy

What is the output consumed by the parser?

A. Characters B. Tokens C. Machine instructions D. Object modules

Q3 — Medium

Which is most directly a semantic-analysis task?

A. Grouping characters into identifiers B. Determining whether parentheses satisfy grammar C. Checking whether operands have compatible types D. Generating assembly instructions

Q4 — Medium

A compiler has eight logical phases. Which statement is necessarily true?

A. It requires exactly eight passes B. It requires at least eight passes C. Some phases must use separate executable programs D. None of the above follows merely from the number of phases

Q5 — Tricky

Consider:

```
c
x = y + ;
```

Assuming every individual character can be tokenized correctly, the first major problem is primarily:

A. Lexical B. Syntax C. Optimization D. Linking

SOLUTIONS

Q1

' B — Assembler

\[Assembly \rightarrow Machine/Object\ Code \]

Q2

' B — Tokens

Lexical analyzer produces tokens; parser consumes them.

Q3

' C — Checking operand type compatibility

That concerns program meaning/type rules.

Q4

' D

Number of logical phases does **not** determine number of passes.

Q5

' B — Syntax error

The individual tokens are valid, but:

TEXT

y + ;

does not form a valid expression according to the grammar.

30-SECOND REVISION

Pipeline

P C A L L

TEXT

Preprocessor
Compiler
Assembler
Linker
Loader

Compiler Phases

L S S I O C

Lazy Students Study In Our College

TEXT

Lexical
Syntax
Semantic
Intermediate Code
Optimization
Code Generation

Most Important Mapping

$\boxed{\text{Characters} \rightarrow \text{Lexical} \rightarrow \text{Tokens}}$

$\boxed{\text{Tokens} \rightarrow \text{Parser} \rightarrow \text{Syntax Structure}}$

TEXT

Semantic !' type / meaning checks
Intermediate !' IR / TAC
Optimization !' improve IR
Code Generation !' target code

Scanner vs Parser

TEXT

Scanner = Lexical Analyzer
Parser = Syntax Analyzer

Analysis vs Synthesis

TEXT

Analysis = Understand
Synthesis = Construct target

Final trap

$\boxed{\text{Phase} \neq \text{Pass}}$

Completed: Part 1/31 | Next: Part 2 — Lexical Analysis Fundamentals

Reply **NEXT** for **Part 2 — Token, Lexeme, Pattern, Token Attributes, Scanner Working, Symbol Table Interaction, Lexical Errors and GATE Token-Counting Problems.**

USER

next

Compiler Design — Part 2: Lexical Analysis Fundamentals

Completed: Part 1/31 | Current: Part 2 | Remaining: Parts 3–31

This part is extremely important because many later compiler concepts assume that you clearly understand:

$\boxed{\text{Token vs Lexeme vs Pattern}}$

That distinction is a classic GATE testing area.

1. What Is Lexical Analysis?

Consider this source statement:

```
c
sum = a + 25;
```

To us, this looks like one statement.

But initially, the compiler receives a **sequence of characters**:

```
TEXT
s u m   =   a   +   2 5 ;
```

The first phase of the compiler groups these characters into meaningful units.

```
TEXT
Characters
    ! "
Lexical Analyzer
    ! "
Tokens
```

CONCEPT

Lexical analysis is the first phase of the compiler.

Its main job is:

Read the source program as a character stream and group characters into meaningful sequences called lexemes, each represented by a token.

WRITE THIS IN NOTES

$\boxed{\text{Character Stream} \rightarrow \text{Lexical Analyzer} \rightarrow \text{Token Stream}}$

Lexical analyzer is also called:

$\boxed{\text{Scanner}}$

2. Why Do We Need a Lexical Analyzer?

Why not give every character directly to the parser?

Suppose the parser received:

```
TEXT
i n t   c o u n t   =   1 0 0 ;
```

It would have to deal with individual characters, whitespace, comments, identifiers, numbers, etc.

Instead, the lexical analyzer converts this into something like:

```
TEXT
<int>
<id, count>
<=>
<num, 100>
<;>
```

The parser now sees meaningful grammatical symbols.

So:

```
TEXT
Source characters
      !"
Lexical Analyzer
      !"
Simplified token stream
      !"
Parser
```

Main advantage

It separates two problems:

```
TEXT
L e x i c a l   A n a l y z e r   ! '   " W h a t   a r e   t h e   w o r d s ? "
P a r s e r                               ! '   " I s   t h e   s e n t e n c e   g r a m m a t i c a l l y   c o r r e
```

3. Three Most Important Terms

You must distinguish:

1. Token
2. Lexeme
3. Pattern

Let's build them from an example.

Consider:

```
c
total = price + 50;
```

4. Lexeme

CONCEPT

A **lexeme** is the actual sequence of characters in the source program that matches the pattern of a token.

In:

```
C
total = price + 50;
```

the lexemes are approximately:

```
TEXT
total
=
price
+
50
;
```

So:

A lexeme is the actual text occurring in the program.

Examples

For:

```
C
count = value + 100;
```

lexemes include:

- `count`
- `=`
- `value`
- `+`
- `100`
- `;`

WRITE THIS IN NOTES

Lexeme: Actual sequence of source-program characters corresponding to a token.

5. Token

CONCEPT

A **token** is a category/class representing a set of lexemes having the same lexical role.

Examples:

```
TEXT
IDENTIFIER
NUMBER
PLUS
MINUS
IF
WHILE
```

Suppose the source contains:

```
C
age = marks + total;
```

`age`, `marks`, and `total` are three different lexemes.

But all belong to the same token category:

```
TEXT
IDENTIFIER
```

Therefore:

```
TEXT
age      !'  I D E N T I F I E R
m a r k s      t o k e n
total
```

6. Pattern

CONCEPT

A **pattern** describes the rule/form that determines which lexemes belong to a token.

Example identifier rule:

```
TEXT
letter followed by zero or more letters/digits
```

A regular-expression-style pattern might be:

```
[ letter(letter|digit)^* ]
```

Then:

```
TEXT
x
count
marks10
Gate2027
```

may all satisfy that identifier pattern.

So:

Pattern = rule describing valid lexemes for a token.

7. Token vs Lexeme vs Pattern — Master Table

WRITE THIS IN NOTES

Term	Meaning	Example	
Token	Category/name	ID	
Lexeme	Actual characters	count	
Pattern	Rule describing lexemes	`letter(letter\`	digit)*`

Another example:

Token	Lexemes	Pattern
NUMBER	10, 357, 9082	one or more digits
ID	x, sum, a12	letter followed by letters/digits
RELOP	<, <=, == etc.	language-defined relational operators

Memory Trick

T-L-P

Think:

Token = Type Lexeme = Literal text Pattern = Property/rule

Or even shorter:

TEXT

T O K E N !' C a t e g o r y
L E X E M E !' A c t u a l w o r d
P A T T E R N !' R u l e

8. Example — Identify Token, Lexeme and Pattern

Source:

```
c
salary = basic + 5000;
```

For salary:

TEXT

```
Lexeme = salary
Token = ID
Pattern = letter(letter | digit)*
```

For 5000 :

TEXT

```
Lexeme = 5000
Token = NUM
Pattern = digit+
```

For + :

TEXT

```
Lexeme = +
Token = PLUS
Pattern = "+"
```

GATE TRAP 1

Are x, y, count, sum different tokens?

Not necessarily.

They are different **lexemes** but may all have the same token:

`\boxed{ID}`

GATE TRAP 2

Is identifier a lexeme?

Usually, in compiler terminology:

TEXT

```
identifier !' token category
count !' lexeme
```

Do not reverse them.

9. Token Representation

A token is often represented conceptually as:

`\boxed{\langle token\_name, attribute\_value \rangle}`

Example:

TEXT

```
<id, pointer-to-symbol-table-entry>
```

or

```
TEXT
<num, 25>
```

Not every token requires an attribute.

For example:

```
TEXT
<PLUS>
<SEMICOLON>
```

may be sufficient.

10. Why Does a Token Need an Attribute?

Suppose the source is:

```
C
total = amount + count;
```

The parser needs to know that all three are identifiers.

So the token category can be:

```
TEXT
ID
```

But later compiler phases must distinguish:

```
TEXT
total
amount
count
```

Hence an attribute can point to information about the actual identifier.

Conceptually:

```
TEXT
<ID, ptr1>
<=>
<ID, ptr2>
<+>
<ID, ptr3>
<;>
```

where:

```
TEXT
ptr1 !' symbol-table entry for total
ptr2 !' symbol-table entry for amount
ptr3 !' symbol-table entry for count
```

11. Symbol Table Interaction

Recall from Part 1:

Symbol table stores information about identifiers.

Suppose:

```
C
int count;
float rate;
```

Possible conceptual table:

Name	Type	Scope
count	int	local/global
rate	float	local/global

When the lexical analyzer recognizes an identifier, it may create/find a corresponding symbol-table entry and pass a reference as the token attribute.

Conceptually:

TEXT

Source:

```
count = rate + 10;
```

Symbol Table

```

% % % % % % % % % % % % %
< I D , p 1 > % % % !' %   c o u n t   %
% % % % % % % % % % % % %$
< I D , p 2 > % % % !' %   r a t e   %
% % % % % % % % % % % % %
```

Exact responsibilities differ between compiler implementations, but this conceptual relationship is important.

12. Categories of Tokens

Common token categories include:

1. Keywords

TEXT

```
if
else
while
int
float
return
```

2. Identifiers

TEXT

```
count
sum
x
student1
```

3. Constants/Literals

TEXT

```
10
3.14
'a'
"hello"
```

4. Operators

TEXT

```
+
-
*
/
=
==
<=
&&
```

5. Separators/Punctuators

TEXT

```
;  
,  
(  
)  
{  
}  
[  
]
```

Memory Trick

K I L O S

TEXT

```
K !'  K e y w o r d s
I !'  I d e n t i f i e r s
L !'  L i t e r a l s
O !'  O p e r a t o r s
S !'  S e p a r a t o r s
```

Think:

Compiler scans KILOS of tokens.

13. Keyword vs Identifier

Suppose the lexical rule for identifiers permits:

```
TEXT
letter(letter|digit)*
```

Then the word:

```
TEXT
while
```

also matches the general shape of an identifier.

But `while` is a reserved keyword.

Therefore the lexer must distinguish:

```
TEXT
while !'  WHILE  token
while1 !'  ID    token
```

assuming `while1` is a legal identifier.

We'll discuss this more with lexical ambiguity and regular expressions in Part 3.

14. What Does the Lexical Analyzer Do?

Main responsibilities include:

1. Reads source characters

```
TEXT
characters
```

2. Groups them into lexemes

```
TEXT
count
+
25
```

3. Produces tokens

```
TEXT
<ID>
<PLUS>
<NUM>
```

4. Ignores irrelevant lexical material where appropriate

Such as:

- spaces
- tabs
- newlines in languages where they are not syntactically significant
- comments

5. Helps maintain identifier information

Through interaction with the symbol table.

6. Detects certain lexical errors

Such as invalid character sequences.

15. Whitespace and Comments

Consider:

```
C
x      =      y + 10;
```

and:

```
C
x=y+10;
```

In C-like syntax, the token sequence is essentially the same:

```
TEXT
ID = ID + NUM ;
```

Whitespace is generally used to separate tokens but usually does not itself become a parser token.

Similarly:

```
C
x = y; /* update x */
```

The comment is typically removed/ignored by lexical processing.

Important Qualification

Do not memorize:

"Whitespace is always ignored in every programming language."

That is false.

Some languages make whitespace or indentation syntactically meaningful.

For conventional GATE compiler questions, use the lexical rules given or implied by the language.

18. Lexical Error

A **lexical error** occurs when the scanner cannot form a valid token according to the lexical rules.

Example, suppose identifiers may contain letters/digits only and the language has no @ operator:

```
C
a = b @ c;
```

Then @ may cause a lexical error.

Another possible example is an incorrectly formed literal such as an unterminated string, depending on the language.

```
C
name = "GATE;
```

The lexer may be unable to recognize a valid string literal.

19. Lexical vs Syntax Error

Very important.

Consider:

```
C
x = y + ;
```

Every piece can be individually recognized:

```
TEXT
ID
=
ID
+
;
```

So lexical analysis succeeds.

But the token sequence violates grammar.

Therefore:

$\boxed{\text{Syntax Error}}$

Consider:

```
TEXT
x = y @ z;
```

Assume @ belongs to no valid token in the language.

Then:

$\boxed{\text{Lexical Error}}$

Quick Test

Ask:

Can the input characters be divided into valid tokens?

If **no** !' possibly lexical error.

If **yes**, but token arrangement violates grammar !' syntax error.

20. Token Counting — Very Important GATE Pattern

GATE may give a source statement/program fragment and ask:

How many tokens are present?

You must count **lexical units**, not individual characters.

Example 1

```
c
int a = 10;
```

Tokens:

```
TEXT
int
a
=
10
;
```

Number of tokens:

$\boxed{5}$

Notice:

10 contains two characters but is **one numeric token**.

Example 2

```
c
a = b + c;
```

Tokens:

```
TEXT
a
=
```

```
b
+
c
;
```

Total:

$\boxed{6}$

Example 3

```
c
if(a>=10)
```

Assuming `>=` is one relational operator token:

```
TEXT
if
(
a
>=
10
)
```

Total:

$\boxed{6}$

GATE TRAP

Do not count:

```
TEXT
> =
```

as two tokens if the language defines `>=` as a single operator.

21. Maximal Munch Intuition

Suppose the input contains:

```
TEXT
>=
```

and the language defines both:

```
TEXT
>
>=
```

as valid operators.

Normally, the lexer chooses the **longest valid prefix**:

TEXT

>=

rather than:

TEXT

>

=

This principle is commonly called:

Maximal Munch / Longest Match

We'll study it properly in Part 3.

For token-counting questions, it can matter immediately.

Example 4 — Increment Operator

C

```
i++;
```

Assuming `++` is a single token:

TEXT

i

++

;

Total:

$\boxed{3}$

not 4.

Example 5

C

```
a+++b
```

This is a famous lexical-thinking pattern.

Under C-like maximal munch:

TEXT

a

++

+

b

So lexical grouping is:

TEXT

```
a ++ + b
```

not necessarily:

TEXT

```
a + ++ b
```

The lexer chooses the longest valid token beginning at each point.

Whether the resulting token sequence is semantically/syntactically useful is a separate issue.

22. String Literal Token Counting

Consider:

C

```
printf("Hello GATE");
```

A string literal is normally treated as **one token**, despite containing multiple characters and a space.

Possible tokenization:

TEXT

```
printf  
(  
"Hello GATE"  
)  
;  
;
```

Depending on lexical classification, `printf` is an identifier and the quoted text is a string literal.

Total here:

$\boxed{5}$

23. Comments and Token Counting

Consider:

C

```
a = b; // update a
```

Typically the comment does not appear in the token stream delivered to the parser.

Tokens:

TEXT

```
a  
=  
b  
;  
;
```

Total:

$\boxed{4}$

24. Multi-Character Identifiers

```
c
studentMarks = totalMarks;
```

Do not count each character.

Tokens:

```
TEXT
studentMarks
=
totalMarks
;
```

Total:

$\boxed{4}$

GATE Shortcut for Token Counting

Scan left to right and classify each **complete lexical unit**:

```
TEXT
Keyword | Identifier | Literal | Operator | Separator
```

Use:

KILOS

Then apply:

Longest valid token first

for competing operators such as:

```
TEXT
++
--
>=
<=
==
!=
&&
||
```

25. Token Count vs Number of Distinct Token Types

These are not the same.

Example:

```
c
x = y + z;
```

Individual token occurrences:

```
TEXT
ID
=
ID
+
ID
;
```

Number of token occurrences:

\[6 \]

But token categories/types represented are:

```
TEXT
ID
ASSIGN
PLUS
SEMICOLON
```

Only 4 distinct token categories.

Read the wording of a question carefully.

26. Token Class vs Token Instance

Consider:

```
c
a = b + c;
```

There are three occurrences of identifier tokens:

```
TEXT
a
b
c
```

All belong to token class:

```
TEXT
ID
```

Thus:

TEXT

```
Token      class      !'      ID
Token      instances  !'      a ,    b ,    c
```

27. Lexical Analysis and Regular Languages

Why can lexical analyzers be implemented efficiently?

Because many token classes can be described using **regular expressions** and recognized by **finite automata**.

Example identifier:

```
\[ letter(letter|digit)^* \]
```

Conceptual pipeline:

TEXT

```
Token Pattern
      !"
Regular Expression
      !"
Finite Automaton
      !"
Lexical Analyzer
```

This connects Compiler Design with Theory of Computation.

We will develop this carefully in **Part 3**.

28. What the Lexical Analyzer Does NOT Usually Do

Do not assign every compiler job to the lexer.

It generally does **not** decide:

Whether an expression obeys CFG structure

```
c
a + * b
```

!' parser/syntax issue.

Whether two operand types are compatible

```
c
int x;
x = "abc";
```

!' semantic issue.

Whether an expression should be optimized

!' optimizer.

Which machine register should finally be used

!' back-end/code-generation concern.

GATE TRAPS

Trap 1 — Token vs Lexeme

`count` is typically:

' lexeme

`ID` is:

' token category

Trap 2 — Multiple Identifiers

TEXT

x, y, total

These are not necessarily three token **types**.

They are three lexemes belonging to one token class:

$\backslash$ ID $\backslash$

Trap 3 — Number of Characters "Number of Tokens

TEXT

12345

can be:

$\backslash$ $\boxed{1}$ token $\backslash$

Trap 4 — Multi-character operators

TEXT

<=

>=

==

++

may each be a single token.

Trap 5 — Spaces

Whitespace generally does not become ordinary parser tokens in C-like languages.

Trap 6 — Comments

Comments are normally discarded by the lexical analyzer before parsing.

Trap 7 — Invalid arrangement vs invalid token

```
c
a = + * b;
```

If every symbol is a valid token, the problem is primarily **syntax**, not lexical.

GATE PATTERN1 — Token/Lexeme Identification

Consider:

```
c
sum = value + 10;
```

For `value`:

- `value` ! lexeme
- `ID` ! token
- identifier rule ! pattern

FAST SOLUTION

Remember:

$\boxed{\text{Actual text = Lexeme}}$

GATE PATTERN2 — Token Counting

Question:

How many tokens occur in:

```
c
if(a<=10) a++;
```

Assuming standard C-like tokenization:

```
TEXT
if
(
a
<=
10
)
a
++
;
```

Count:

$\boxed{9}$

GATE PATTERN3 — Lexical vs Syntax

Input:

```
c
x = y + ;
```

All individual units are valid tokens.

Therefore:

' lexical analysis can recognize them.

But grammar is invalid.

Syntax error

GATE PATTERN4 — Longest Match

Suppose both `>` and `>=` are valid operators.

Input:

```
TEXT
a>=b
```

Lexer returns:

```
TEXT
ID  >=  ID
```

not:

```
TEXT
ID  >  =  ID
```

because of longest-match behavior.

WRITE THIS IN NOTES — PART 2 SUMMARY TABLE

Concept	Meaning
Scanner	Lexical analyzer
Lexeme	Actual character sequence
Token	Category of lexeme
Pattern	Rule defining token's lexemes

Concept	Meaning
Token attribute	Extra information associated with token
Lexer's input	Characters
Lexer's output	Tokens
Maximal munch	Choose longest valid token
Lexical error	Characters cannot form legal token
Syntax error	Legal tokens arranged illegally

MINI PRACTICE — PART 2

Try without looking at the solutions.

Q1 — Easy

For the source lexeme `count123`, suppose it satisfies the identifier rule.

Which is most likely its token?

A. NUM B. ID C. COUNT D. PATTERN

Q2 — Easy

In:

```
c
x = 123;
```

how many tokens are present?

Assume standard C-like lexical rules.

Q3 — Medium

In:

```
c
a<=b
```

assuming `<=` is a valid single operator, how many tokens are produced?

A. 3 B. 4 C. 5 D. 6

Q4 — Medium

Which statement is correct?

A. A token is the actual character sequence in the source. B. A pattern is the actual character sequence in the source. C. A lexeme is an actual character sequence matching a token pattern. D. Token and lexeme always mean exactly the same thing.

Q5 — Tricky

Suppose these operators are valid:

TEXT

+
++

How is:

TEXT

a+++b

typically tokenized using maximal munch?

A. a + ++ b B. a ++ + b C. a + + + b D. Lexer must always report an error

SOLUTIONS

Q1

' B — ID

count123 = lexeme.

ID = token category.

Q2

Tokens:

TEXT

x
=
123
;

Therefore:

$\boxed{4}$

Q3

TEXT

a
<=
b

Therefore:

' A — 3

Q4

' C

A lexeme is an actual character sequence matching the pattern of a token.

Q5

' B

Using longest match:

TEXT

a
++
+
b

Therefore:

$\boxed{a\ ++\ +\ b}$

30-SECOND REVISION

Main flow

$\boxed{\text{Characters} \rightarrow \text{Scanner} \rightarrow \text{Tokens}}$

Token–Lexeme–Pattern

TEXT

Token	!	Category
Lexeme	!	Actual text
Pattern	!	Rule

Example:

TEXT

Token = ID
Lexeme = count123
Pattern = letter(letter|digit)*

Token representation

$\boxed{\langle \text{token name, attribute} \rangle}$

Example:

TEXT

<ID, symbol-table-pointer>

Token groups

KILOS

TEXT

```
K !'  K e y w o r d
I !'  I d e n t i f i e r
L !'  L i t e r a l
O !'  O p e r a t o r
S !'  S e p a r a t o r
```

Error shortcut

TEXT

```
Cannot form valid token
! "
Lexical error

Valid tokens but invalid arrangement
! "
Syntax error
```

Longest Match

Choose the longest valid token beginning at the current input position.

Classic trap

TEXT

```
age, count, total
```

= three **lexemes**, but all may belong to one token category:

```
\[ \boxed{ID} \]
```

Completed: Parts 1–2/31 | Next: Part 3 — Regular Expressions and Finite Automata for Lexical Analysis

Reply **NEXT** for **Part 3**, where we'll connect lexical analysis to **regular expressions, identifiers/numbers, NFA/DFA, longest-prefix matching, reserved words, and GATE recognition problems.**

USER

NEXT

ASSISTANT

Compiler Design — Part 3: Regular Expressions & Finite Automata in Lexical Analysis

Completed: Parts 1–2/31 | Current: Part 3 | Remaining: Parts 4–31

In Part 2, we learned:

```
\[ \boxed{\text{Characters}} \rightarrow \text{Lexical Analyzer} \rightarrow \text{Tokens} \]
```

Now the important question is:

How does the lexical analyzer know what an identifier, number, keyword, operator, etc. looks like?

The answer is primarily:

$\boxed{\text{Regular Expressions + Finite Automata}}$

1. Why Regular Expressions Are Used in Lexical Analysis

Consider these valid identifiers:

```
TEXT
x
count
total20
Gate2027
```

We do not want to manually list every possible identifier.

Instead, we describe their **pattern**.

For example:

```
TEXT
letter followed by zero or more letters/digits
```

Using regular-expression notation:

$\boxed{\text{letter}(\text{letter}|\text{digit})^*}$

This one rule describes potentially infinitely many identifiers.

CONCEPT

Lexical structures such as:

- identifiers
- integers
- floating-point numbers
- operators
- keywords
- whitespace

can generally be described by **regular languages**.

Regular languages can be:

1. specified using **regular expressions**
2. recognized using **finite automata**

Hence:

```
TEXT
TOKEN
! "
```

Pattern
!"
Regular Expression
!"
Finite Automaton
!"
Recognition by Lexer

WRITE THIS IN NOTES

Regular expressions specify token patterns; finite automata recognize the lexemes matching those patterns.

Memory:

RE describes, FA recognizes.

2. Quick Regular Expression Revision

You have Theory of Computation separately, so we only need the compiler-relevant portion here.

Suppose:

$\Sigma = \{a, b\}$

The most important regular-expression operations are:

1. Union

$a|b$

means:

TEXT
a OR b

Language:

$\{a, b\}$

2. Concatenation

ab

means:

a followed by b

Language:

$\{ab\}$

3. Kleene Star

$\text{\texttt{[a^*]}}$

means zero or more occurrences of **a**.

Language:

$\text{\texttt{[\epsilon,a,aa,aaa,\ldots]}}$

Important:

$\text{\texttt{[\boxed{\epsilon\text{in } L(a^*)}]}}$

because zero repetitions are allowed.

4. Positive Closure

$\text{\texttt{[a^+]}}$

means one or more occurrences.

$\text{\texttt{[\{a,aa,aaa,\ldots\}]}}$

Therefore:

$\text{\texttt{[\boxed{a^+=aa^*}]}}$

5. Optional

Sometimes notation:

$\text{\texttt{[a?]}}$

means zero or one occurrence:

$\text{\texttt{[\epsilon,a]}}$

3. * vs + — Important

This frequently causes mistakes.

Kleene Star

$\text{\texttt{[r^*]}}$

means:

$\text{\texttt{[0,1,2,3,\ldots]}}$

occurrences.

Therefore it includes $\text{\texttt{[\epsilon]}}$.

Positive Closure

$\text{\texttt{[r^+]}}$

means:

$\text{\texttt{[1,2,3,\ldots]}}$

occurrences.

Does not include ϵ , unless r itself can somehow generate ϵ .

Memory Trick

Star can start with zero. Plus needs at least one.

4. Regular Definition

Complex token patterns are easier to write using named definitions.

Suppose:

TEXT

```
digit    !'  0 | 1 | 2 | . . . | 9
letter !'  a | b | . . . | z | A | B | . . . | Z
```

Then:

TEXT

```
id !'  letter ( letter | digit ) *
```

This is called a **regular definition** style.

Instead of writing an enormous regular expression repeatedly, we assign names to useful components.

WRITE THIS IN NOTES

TEXT

```
digit    !'  0 | 1 | . . . | 9
letter !'  a | b | . . . | z | A | . . . | Z
id !'  letter ( letter | digit ) *
```

5. Identifier Pattern

Consider:

$\boxed{\text{letter}(\text{letter}|\text{digit})^*}$

What does it mean?

First position

Must be:

TEXT

letter

Remaining positions

Can contain:

TEXT

letter OR digit

zero or more times.

Therefore:

String	Valid?
x	'
abc	'
a123	'
x5y7	'
123x	'L
7	'L

assuming this exact identifier rule.

GATE Shortcut

For:

$\backslash [\text{letter}(\text{letter}|\text{digit})^* \backslash]$

ask only:

Does it begin with a letter?

If yes, every remaining character must be a letter/digit.

6. Integer Pattern

Suppose:

$\backslash [\text{digit}=0|1|\cdots|9 \backslash]$

An unsigned integer can be described as:

$\backslash [\boxed{\text{digit}^+} \backslash]$

or equivalently:

$\backslash [\boxed{\text{digit}\backslash \text{digit}^*} \backslash]$

Examples:

TEXT

0

7

```
25
12345
```

All are one **NUM** token.

7. Signed Number: Important Lexer Detail

You might think a negative integer must be:

```
\[ -?digit^+ \]
```

But whether `-` belongs to the number token depends on the language's lexical specification.

For example:

```
c
x = -10;
```

could conceptually be tokenized as:

```
TEXT
ID ASSIGN MINUS NUM SEMICOLON
```

where:

```
TEXT
-
```

is a separate operator.

This allows parsing to decide whether `-` is unary minus or subtraction.

GATE TRAP

Do not automatically assume:

```
TEXT
-10
```

is always one lexical token.

Use the lexical specification given in the question.

8. Floating-Point Pattern — Intuition

A simplified floating number might have the form:

```
TEXT
digits . digits
```

Example:

TEXT

12.75

A simplified RE:

`[ digit^+\.digit^+ ]`

Here `.` is intended as the actual decimal point.

Examples:

TEXT

3.14
10.50
0.25

A real programming language may allow much more:

TEXT

5.
.25
3.2e10
6E-4

But do **not** memorize one universal floating-point RE.

GATE will specify enough information when detailed token rules matter.

9. Keywords as Token Patterns

Keywords include:

TEXT

if
else
while
for
return

A keyword has essentially a fixed-string pattern.

For example:

TEXT

I F !' i f
W H I L E !' w h i l e

But notice something interesting.

The identifier pattern:

`[ letter(letter|digit)^* ]`

also matches:

TEXT

```
if
while
return
```

So how does the lexer distinguish a keyword from an identifier?

This is important.

10. Keyword vs Identifier Problem

Consider:

```
c
while
```

It satisfies:

```
TEXT
letter(letter|digit)*
```

Therefore, it could look like an `ID`.

But the language reserves it as a keyword.

So the lexer must recognize:

```
TEXT
w h i l e ! '   W H I L E
```

rather than:

```
TEXT
w h i l e ! '   I D
```

11. Two Common Ways to Handle Keywords

Conceptually, either:

Method 1 — Separate keyword rules

Specify keywords explicitly and give them appropriate priority.

```
TEXT
w h i l e ! '   W H I L E
i f           ! '   I F
...
```

Then have the general identifier rule.

Method 2 — Recognize as identifier and check reserved-word table

Lexer recognizes:

```
TEXT
while
```

as identifier-shaped.

Then it checks a table:

```
TEXT
Is "while" reserved?
```

Yes!

```
TEXT
WHILE
```

Otherwise !'

```
TEXT
ID
```

GATE TRAP

`while1` is generally not split as:

```
TEXT
while
1
```

if the identifier rule permits:

```
TEXT
while1
```

as one identifier.

Why?

Because the lexer normally uses the **longest valid prefix**.

So:

```
TEXT
while    !'   keyword
while1  !'   identifier
```

12. Operators and Regular Patterns

Programming languages contain operators such as:

TEXT

+
-
*
/
=
==
<
<=
>
>=

Some share prefixes.

For example:

TEXT

<
<=

and:

TEXT

=
==

This creates a recognition issue.

Suppose input is:

TEXT

<=

After reading `<`, should the scanner stop?

No.

It may read one more character and discover:

TEXT

<=

which is a longer valid operator.

This leads us to one of the most important lexical rules.

13. Maximal Munch / Longest-Match Rule

CONCEPT

When more than one token can match the beginning of the remaining input, the scanner generally chooses the **longest prefix that forms a valid token**.

Example

Valid tokens include:

TEXT

>
>=

Input:

TEXT

>=

Scanner prefers:

TEXT

>=

over:

TEXT

>

because it is longer.

WRITE THIS IN NOTES

Maximal munch: At the current position, choose the longest prefix of the remaining input that matches a token pattern.

Memory:

Lexer is greedy for valid characters.

14. Classic Example — `a+++b`

Suppose:

TEXT

+
++

are both valid operators.

Input:

TEXT

`a+++b`

Start:

TEXT

a

!' ID.

Remaining:

TEXT

+++b

Both `+` and `++` match the prefix.

Longest is:

TEXT

++

Take it.

Remaining:

TEXT

+b

Take:

TEXT

+

then:

TEXT

b

Thus:

`\[ \boxed{a\quad ++\quad +\quad b} \]`

15. Longest Match Does NOT Mean Longest String Anywhere

This is a subtle trap.

The lexer chooses:

longest valid prefix starting at the current input position

It does not search arbitrarily far into the string.

Example

Input:

TEXT

abc+xyz

It does not combine `abcxyz`.

It starts at `a`, recognizes longest valid identifier:

TEXT

abc

Stops before `+`.

Then recognizes:

TEXT

+

Then:

TEXT

xyz

16. What if Two Token Rules Match the Same Longest Lexeme?

Suppose:

TEXT

if

matches:

1. keyword rule `IF`
2. general identifier rule `ID`

Both match exactly two characters.

Then the lexical specification needs a **priority rule**.

In lexer-generator systems, one common convention is conceptually:

1. longest match first
2. if same length, use rule priority/order

But exact conventions depend on the lexical-analyzer tool/specification.

GATE Lesson

Two separate issues:

TEXT

1. Longest match
2. Priority when lengths tie

Do not confuse them.

17. Regular Expression !' Finite Automaton

The regular expression is a **description**.

The lexer needs an actual recognition mechanism.

For example:

`\[ digit^+ \]`

describes positive-length digit strings.

A finite automaton can recognize them.

Conceptually:

```
TEXT
      digit
      % % % % % % % % % % % % % %
      ! "
(q0) --digit--> ((q1))
```

where:

- `\(q_0\)` = start state
- `\(q_1\)` = accepting state
- every additional digit loops on `\(q_1\)`

So:

```
TEXT
5      !'   a c c e p t
4 2    !'   a c c e p t
8 2 9 3 1 4 !' a c c e p t
a b c    !'  r e j e c t
```

18. DFA for an Identifier

Pattern:

`\[ letter(letter|digit)^* \]`

Conceptual DFA:

```
TEXT
      letter/digit
      % % % % % % % % % % % % % %
      %
      ! "
(q0) --letter--> ((q1))
```

At `q0`:

- letter !' `q1`
- digit !' reject/trap

At `q1`:

- letter !' q1
- digit !' q1
- invalid identifier character !' token ends / appropriate action

Therefore:

TEXT

```
a b c 2 0 !' a c c e p t e d
x !' a c c e p t e d
2 0 a b c !' r e j e c t e d a s a n i d e n t i f i e r
```

19. Why DFA Is Very Suitable for Lexical Analysis

A DFA works by:

1. maintaining one current state
2. reading input left to right
3. changing state for each character
4. deciding whether a valid token pattern has been matched

This makes scanning efficient.

For each input symbol, the lexer performs essentially a state transition.

20. NFA vs DFA in Lexical Analysis

You should know the conceptual pipeline:

$\boxed{\text{Regular Expression}} \rightarrow \text{NFA} \rightarrow \text{DFA}$

Often:

1. token specifications are written as regular expressions
2. REs can be converted to NFAs
3. NFAs can be converted to DFAs
4. DFA is used for efficient scanning

A DFA may also be minimized.

Conceptually:

TEXT

Token REs
!"

NFA
!"

DFA
!"

Optional minimization
!"

Scanner

REMEMBER

Regular-expression and finite-automaton expressive power:

$\boxed{\text{RE} \equiv \text{NFA} \equiv \text{DFA}}$

They describe/recognize exactly the **regular languages**.

This is also a Theory of Computation fact.

21. Do We Need NFA or DFA at Runtime?

Conceptually, generated lexical analyzers commonly implement deterministic recognition.

Why prefer deterministic processing?

Because with a DFA:

$\boxed{\text{Current State} + \text{Input Character} \rightarrow \text{Exactly One Next State}}$

No need to explore multiple possible paths.

22. Multiple Token Patterns Can Be Combined

Suppose the lexer must recognize:

```
TEXT
ID
NUM
PLUS
MINUS
```

Instead of running four completely separate scanners, their recognition rules can conceptually be combined into one finite automaton.

Accepting states can indicate **which token has been recognized**.

Example conceptual idea:

```
TEXT
      letter      % % % % % % !'   I D   s t a t e
    /
S T A R T % % %
    \
      digit      % % % % % % !'   N U M   s t a t e
```

Other transitions recognize operators.

23. Accepting State and Token Type

Different accepting states can correspond to different tokens.

Conceptually:

TEXT

```
q 1 ! ' I D
q 2 ! ' N U M B E R
q 3 ! ' < =
q 4 ! ' <
```

When scanning reaches the appropriate accepting state and determines the token ends, the lexer returns that token category.

24. Lookahead — Why It Is Needed

Suppose the input begins:

TEXT

<

But tokens include:

TEXT

<

<=

After reading <, the lexer cannot always immediately decide.

It may inspect the next character.

If next character is:

TEXT

=

recognize:

TEXT

<=

Otherwise:

TEXT

<

This checking of upcoming character(s) is called **lookahead**.

25. Example of Lookahead

Input:

TEXT

a < b

After scanning <, look at next character:

TEXT
space

So token is:

TEXT
<

But input:

TEXT
a <= b

after <, next character is =.

Therefore:

TEXT
<=

is recognized.

26. Retract — Intuition

Sometimes the scanner reads one character beyond the current token to know that the token has ended.

Example:

TEXT
abc+

Suppose it is reading an identifier.

It reads:

TEXT
a
b
c

All valid identifier characters.

Then sees:

TEXT
+

Now it knows the identifier ended before +.

Conceptually it may need to leave/retract + so it can become the beginning of the next token.

We'll study input buffering and retract properly in **Part 4**.

27. Important Boundary: Lexical Analysis Uses Regular Languages

Lexical analysis is generally appropriate for patterns that can be represented by regular languages.

But many programming-language structures require more power.

Example:

TEXT

```
balanced nested parentheses
```

Arbitrarily nested structure such as:

TEXT

```
((()))
```

requires remembering unbounded nesting depth.

A plain finite automaton cannot, in general, recognize arbitrarily balanced parentheses.

That is the job of **parsing/context-free grammar**, not ordinary lexical analysis.

WRITE THIS IN NOTES

TEXT

```
Lexical structure !' Regular Expressions / Finite Automata  
Syntactic structure !' Context-Free Grammars / Parsers
```

This distinction is extremely important.

28. Why Not Use Regular Expressions for the Whole Programming Language?

Consider:

```
c  
if (a) {  
    while (b) {  
        ...  
    }  
}
```

The compiler must verify nested structures:

TEXT

```
( )  
{ }
```

with arbitrary nesting.

Finite automata have only finite state memory.

Parsing needs something capable of handling recursive structure.

Therefore:

```
TEXT
LEXER
Regular Languages
! "
TOKEN STREAM
! "
PARSER
Context-Free Grammar
```

Memory Trick

R before C

```
TEXT
R e g u l a r ! ' L e x i c a l
C o n t e x t - F r e e ! ' S y n t a x
```

Or:

Lexer recognizes words; Parser recognizes recursive structure.

29. Example — Complete Lexical Recognition

Suppose token rules are:

```
\[ ID=letter(letter|digit)^* \]
```

```
\[ NUM=digit^+ \]
```

and operators/separators include:

```
TEXT
=
+
;
```

Input:

```
C
sum2 = value + 25;
```

Step 1

Starting at s :

```
TEXT
sum2
```

matches longest identifier.

Return:

TEXT

<ID, sum2>

Step 2

Ignore whitespace.

Recognize:

TEXT

=

Return:

TEXT

<ASSIGN>

Step 3

Recognize:

TEXT

value

Return:

TEXT

<ID, value>

Step 4

Recognize:

TEXT

+

Return:

TEXT

<PLUS>

Step 5

Recognize:

TEXT

25

Return:

TEXT

<NUM, 25>

Step 6

Recognize:

TEXT

;

Return:

TEXT

<SEMICOLON>

Final token stream:

TEXT

ID ASSIGN ID PLUS NUM SEMICOLON

30. GATE Recognition Pattern

Suppose:

$\backslash [\text{ID} = \text{letter}(\text{letter}|\text{digit})^* \backslash]$

Determine which are identifiers:

TEXT

1. gate2027
2. 2027gate
3. x
4. a1b2
5. 7

Check

gate2027

Starts with letter !' '

2027gate

Starts with digit !' 'L

x

letter, followed by zero extra characters !' '

a1b2

starts letter, remaining letter/digits !' '

7

starts digit !' 'L

Therefore:

$\backslash \boxed{1,3,4} \backslash$

31. GATE Pattern — Find Longest Prefix

Suppose tokens include:

```
TEXT
=
==
===
```

and all are valid.

Input starts:

```
TEXT
===x
```

Using maximal munch:

```
TEXT
===
```

is selected first.

Not:

```
TEXT
==
=
```

because `===` is the longest valid prefix.

32. What if the Longest Prefix Eventually Fails?

This is a useful deeper idea.

Suppose a scanner keeps reading while there is some possible transition.

It may need to remember the **last accepting state**.

If later characters cause failure, it can return the lexeme corresponding to the most recent valid accepting position.

Conceptually:

```
TEXT
Read forward
    !"
Remember last valid complete token
    !"
If extension fails
    !"
Return longest previously valid token
```

This implements longest-match behavior.

You need the intuition, not lexer-generator implementation details.

33. Reserved Words and Symbol Table

Consider:

```
TEXT
if
count
while
sum
```

A reserved-word table could conceptually contain:

Lexeme	Token
if	IF
while	WHILE
return	RETURN

When an identifier-shaped lexeme is recognized:

```
TEXT
if found in reserved table
    !"
return keyword token

otherwise
    !"
return ID
```

34. Keyword Prefix Trap

Consider:

```
TEXT
ifelse
```

Suppose identifiers can contain arbitrary letters.

Do not automatically tokenize:

```
TEXT
if
else
```

Because the longest identifier-shaped lexeme is:

```
TEXT
ifelse
```

Thus typically:

$\boxed{\text{ID}(\text{ifelse})}$

unless the language defines different rules.

Similarly:

```
TEXT
while1
```

normally becomes:

$\boxed{\text{ID}(\text{while1})}$

rather than:

```
TEXT
WHILE NUM
```

35. Lexical Specification Priority

Suppose both patterns match:

```
TEXT
if
```

Pattern A:

```
TEXT
I F ! ' " i f "
```

Pattern B:

```
TEXT
I D ! ' l e t t e r ( l e t t e r | d i g i t ) *
```

They match the same longest lexeme.

Now the lexer requires priority.

A typical specification gives the explicit keyword priority over general identifier recognition.

Remember

$\boxed{\text{Longest match first; tie resolved by token-rule priority/specification}}$

Do not assume a particular tool's exact priority rules unless stated.

36. Regular Expression Precedence

When regular expressions appear in GATE, remember normal precedence:

$\boxed{\text{ ;> ; concatenation ;> ; } }$

That means:

$\boxed{ab^*c}$

is interpreted as:

$\backslash [(a(b^*))|c \backslash]$

not some arbitrary grouping.

Memory Trick

Star sticks tightest, then join, then OR.

37. Example of RE Precedence

Regular expression:

$\backslash [ab^* \backslash]$

means:

TEXT

a followed by zero or more b's

Language:

TEXT

a
ab
abb
abbb
...

It does **not** mean:

$\backslash [(ab)^* \backslash]$

which would produce:

TEXT

μ
ab
abab
ababab
...

Parentheses matter.

GATE TRAPS

Trap 1 — RE vs FA

Regular expression **recognizes** tokens.

Not quite.

Better terminology:

TEXT

RE !' specifies / describes a regular language
FA !' recognizes that language

Trap 2 — *

$\backslash[a^* \backslash]$

does **not** mean one or more.

It means:

$\backslash[\boxed{0\text{ or more}} \backslash]$

Therefore:

$\backslash[\epsilon \text{ in } L(a^*) \backslash]$

Trap 3 — +

$\backslash[a^+ \backslash]$

means:

$\backslash[\boxed{1\text{ or more}} \backslash]$

Trap 4 — Keyword prefix

TEXT

while1

is not normally:

TEXT

WHILE NUM

If `while1` is a legal identifier, maximal munch gives one ID token.

Trap 5 — Longest Match

Longest match means:

longest valid prefix at the current position

not longest token appearing somewhere later.

Trap 6 — Lexer vs Parser

Balanced recursive nesting is not generally a regular-language task.

TEXT

```
L e x e r ! '   r e g u l a r  
P a r s e r ! '   c o n t e x t - f r e e
```

Trap 7 — DFA vs NFA Power

A DFA is **not more expressive** than an NFA.

$\boxed{\text{DFA} \equiv \text{NFA} \equiv \text{RE}}$

They recognize exactly regular languages.

Trap 8 — Negative numbers

Do not universally assume:

TEXT

-10

is one token.

It may be:

TEXT

MINUS NUM

depending on lexical rules.

GATE PATTERN1 — Identify a Valid Lexeme

Given:

$\text{ID} = \text{letter}(\text{letter}|\text{digit})^*$

Which is valid?

A. 123a B. a123 C. 1a2 D. 2

FAST SOLUTION

First character must be letter.

' B — `a123`'

GATE PATTERN2 — * and +

Which includes ϵ ?

A. (a^+) B. (a^*)

' B

Because star permits zero repetitions.

GATE PATTERN3 — Maximal Munch

Valid tokens:

```
TEXT
<
<=
=
```

Input:

```
TEXT
a<=b
```

Tokenization:

```
TEXT
ID
<=
ID
```

not:

```
TEXT
ID
<
=
ID
```

GATE PATTERN4 — Lexical vs Parsing Capability

Which mechanism is naturally suited for arbitrary balanced parentheses?

A. Regular expression B. DFA C. Context-free grammar/parser D. Lexical token table

' C

Arbitrary nested structure requires more than finite-state recognition.

GATE PATTERN5 — Keyword vs ID

Rules:

```
TEXT
I F !' i f
I D !' l e t t e r ( l e t t e r | d i g i t ) *
```

Input:

TEXT

if2

Assuming longest match:

TEXT

if2

matches ID with length 3.

if matches keyword with length 2.

Therefore:

$\boxed{ID(if2)}$

MINI PRACTICE — PART 3

Try these first.

Q1 — Easy

Given:

$[digit=0|1|\cdots|9]$

Which RE denotes one or more digits?

A. $(digit^*)$ B. $(digit^+)$ C. $(digit|\epsilon)$ D. $(digit?)$

Q2 — Easy

For:

$ID=letter(letter|digit)^*$

which string is invalid?

A. x B. x10 C. gate2027 D. 2027gate

Q3 — Medium

Suppose valid operators are:

TEXT

+

++

Using maximal munch, tokenize:

TEXT

x+++++y

Q4 — Medium

Which statement is correct?

A. DFA recognizes strictly more languages than NFA. B. NFA recognizes strictly more languages than DFA. C. DFA, NFA and regular expressions have equivalent expressive power. D. Regular expressions recognize all context-free languages.

Q5 — Tricky

Suppose:

TEXT

```
F O R ! ' " f o r "  
I D ! ' l e t t e r ( l e t t e r | d i g i t ) *  
N U M ! ' d i g i t +
```

Tokenize:

TEXT

for10

using longest-match behavior.

A. FOR NUM B. ID C. FOR ID D. lexical error

SOLUTIONS

Q1

' B

`\[ digit^+ \]`

means one or more digits.

Q2

' D — `2027gate`

The first character must be a letter.

Q3

Input:

TEXT

x+++++y

First:

TEXT

x ! ' I D

Remaining:

TEXT

+++++y

Longest operator repeatedly:

TEXT

++
++
+

Then:

TEXT

y

Therefore:

$\boxed{\text{ID} \quad ++ \quad ++ \quad + \quad \text{ID}}$

Q4

' C

$\boxed{\text{RE} \equiv \text{NFA} \equiv \text{DFA}}$

for regular languages.

Q5

for matches keyword length 3.

But:

TEXT

for10

matches the ID pattern with length 5.

Longest match wins.

' B — ID

WRITE THIS IN NOTES — MASTER SUMMARY

Lexical specification

$\boxed{\text{Token Patterns} \rightarrow \text{Regular Expressions}}$

Recognition

$\boxed{\text{Regular Expression} \rightarrow \text{NFA} \rightarrow \text{DFA} \rightarrow \text{Scanner}}$

Equivalence

$\boxed{RE \equiv NFA \equiv DFA}$

Identifier example

$\boxed{\text{letter}(\text{letter}|\text{digit})^*}$

Number example

$\boxed{\text{digit}^+}$

Closure

TEXT

r * ! ' zero or more
r + ! ' one or more

Longest match

Choose the longest valid prefix starting at the current input position.

Language levels

TEXT

Lexical Analysis
! "
Regular Languages
RE / Finite Automata

Parsing
! "
Context-Free Languages
CFG / Parser

30-SECOND REVISION

RE describes; FA recognizes

$\boxed{RE \equiv NFA \equiv DFA}$

Typical identifier

$\boxed{\text{letter}(\text{letter}|\text{digit})^*}$

Typical unsigned integer

$\boxed{\text{digit}^+}$

Closure

$\backslash[r^*=0\backslash\text{text}\{\text{ or more}\} \backslash]$

$\backslash[r^+=1\backslash\text{text}\{\text{ or more}\} \backslash]$

Maximal munch

$\backslash[\backslash\text{boxed}\{\backslash\text{text}\{\text{Longest valid prefix}\}\} \backslash]$

Example:

TEXT

a+++b

becomes:

TEXT

a ++ + b

Keyword trap

TEXT

w h i l e ! ' W H I L E
w h i l e 1 ! ' I D

assuming normal identifier rules.

Main compiler distinction

$\backslash[\backslash\text{boxed}\{\backslash\text{text}\{\text{Regular !' Lexer}\}\} \backslash]$

$\backslash[\backslash\text{boxed}\{\backslash\text{text}\{\text{Context-Free !' Parser}\}\} \backslash]$

Precedence of RE operations

$\backslash[\backslash\text{boxed}\{ * > \text{concatenation} > | \} \backslash]$

Completed: Parts 1–3/31 | Next: Part 4 — Input Buffering & Lexical Analyzer Design

Reply **NEXT** for **Part 4**, where we'll cover **why buffering is needed, buffer pairs, sentinels, lexemeBegin/forward pointers, lookahead, retract, and the exact GATE-style pointer/buffering concepts without unnecessary implementation detail.**

USER

Next

ASSISTANT

Compiler Design — Part 4: Input Buffering & Lexical Analyzer Design

Completed: Parts 1–3/31 | Current: Part 4 | Remaining: Parts 5–31

In Part 3, we learned how token patterns are described using **regular expressions** and recognized using **finite**

automata.

Now we answer a practical question:

The source file may contain thousands or millions of characters. How does the lexical analyzer read them efficiently while supporting lookahead and retract?

The answer is **input buffering**.

1. Why Is Input Buffering Needed?

Suppose the source program is:

```
c
total = price + quantity * 10;
```

The lexical analyzer reads characters one after another:

```
TEXT
t !' o !' t !' a !' l !' s p a c e !' = !' . . .
```

A naive implementation could request **one character from the file for every character scanned**.

That is inefficient because accessing an input file repeatedly is much slower than accessing memory.

So instead:

```
TEXT
Source File
! "
Read a block of characters
! "
    Buffer in memory
    ! "
Lexer reads characters from buffer
```

CONCEPT

Input buffering means reading a block of source characters into memory and allowing the lexical analyzer to scan those characters from memory.

Why?

Mainly to reduce expensive input operations.

WRITE THIS IN NOTES

Input buffering: Source characters are read in blocks into memory so that lexical analysis can scan them efficiently instead of performing an I/O operation for every character.

2. Why Does the Lexer Sometimes Need Lookahead?

Consider these operators:

TEXT

<
<=

Suppose the lexer reads:

TEXT

<

Can it immediately return the `<` token?

Not necessarily.

It must inspect the next character.

Case 1

TEXT

< x

Next character is not `=`.

Token:

TEXT

<

Case 2

TEXT

<=

Next character is `=`.

Token:

TEXT

<=

So the lexer may have to read ahead before determining where the current token ends.

This is called:

`\[ \boxed{\text{Lookahead}} \]`

3. Lookahead

CONCEPT

Lookahead means examining one or more upcoming characters to determine which token should be recognized.

Example:

```
TEXT
=  
==
```

When the lexer reads the first `=`:

```
TEXT
=
```

it looks at the next character.

If next character is `=`:

```
TEXT
==
```

return equality operator.

Otherwise:

```
TEXT
=
```

return assignment operator.

1-line recall

Lookahead = inspect future input before committing to a token.

4. Why Is Retract Needed?

Consider:

```
TEXT
count+
```

Suppose the identifier pattern is:

`\[ letter(letter|digit)^* \]`

The lexer reads:

```
TEXT
c
co
cou
coun
count
```

Every character still belongs to the identifier.

Then it reads:

TEXT

+

Now the lexer realizes:

+ does not belong to this identifier.

Therefore the lexeme should be:

TEXT

count

and **+** must be processed as the next token.

Conceptually:

TEXT

count+

!'

lexer read one character too far

It must move back or otherwise preserve the extra character.

This is called **retract**.

WRITE THIS IN NOTES

Retract: Move the scanning position backward when one or more characters were read beyond the end of the current lexeme.

Memory trick:

TEXT

L o o k a h e a d ! ' L o o k f o r w a r d
R e t r a c t ! ' M o v e b a c k w a r d

5. Two Important Pointers

A classic buffering scheme uses two pointers:

1. **lexemeBegin**

Points to the beginning of the current lexeme.

2. **forward**

Moves forward while searching for the end of the lexeme.

Suppose:

TEXT

```
count = 10;
```

While recognizing `count` :

TEXT

```
c o u n t   =  
!'          !'  
lexemeBegin forward
```

After determining the lexeme:

TEXT

```
count
```

the lexer can return:

TEXT

```
<ID, ...>
```

Then `lexemeBegin` advances to the beginning of the next token.

WRITE THIS IN NOTES

TEXT

```
lexemeBegin !' beginning of current lexeme  
forward    !' scans ahead to find lexeme end
```

Memory Trick

Begin stays; Forward explores.

6. Example Using the Two Pointers

Input:

C

```
sum = a + 25;
```

Initially:

TEXT

```
s u m   =   a   +   2 5 ;  
!'      '  
lexemeBegin  
!'      '  
forward
```

`forward` advances:

TEXT

```
S u m
! '      ! '
LB  F
```

It sees the following space.

Now:

TEXT

```
sum
```

has been recognized as an identifier.

Return:

TEXT

```
<ID, sum>
```

Then move past irrelevant whitespace and begin recognizing the next token.

7. Why One Buffer Can Be Inconvenient

Imagine a fixed-size buffer containing:

TEXT

```
| a b c d e f g h |
```

Suppose the current token starts near the end of this buffer but continues into the next block.

For example:

TEXT

```
Current block:
... identif
```

```
Next block:
ier123 ...
```

The lexeme:

TEXT

```
identifier123
```

crosses the buffer boundary.

The scanner therefore needs a convenient way to continue scanning without treating every boundary as a special complicated case.

A classic solution is the:

Two-Buffer Scheme

10. Sentinel Technique

There is still a problem.

Without a special mechanism, after reading every character the scanner might need to ask:

Have I reached the end of this buffer half?

That means frequent boundary checking.

A classic optimization is to place a special marker called a:

`\boxed{\text{Sentinel}}`

at the end of each buffer half.

Conceptually:

TEXT

Buffer 1

% % % % % % % % % % % % % % % %
% c h a r s . . . S E N T I N E L %
% % % % % % % % % % % % % % % %

Buffer 2

% % % % % % % % % % % % % % % %
% c h a r s . . . S E N T I N E L %
% % % % % % % % % % % % % % % %

The sentinel is traditionally represented using an end-marker such as `EOF`.

11. What Does the Sentinel Do?

When `forward` encounters the sentinel, the scanner knows:

- it has reached a buffer boundary, or
- possibly the true end of the source file.

Then it can refill/switch buffers as required.

Without sentinels, boundary testing may need to occur for each scanned character.

With sentinels, normal characters can generally be processed without a separate buffer-boundary check every time.

WRITE THIS IN NOTES

Sentinel: Special marker placed at the end of a buffer half to simplify detection of the buffer boundary and reduce repeated boundary checks.

12. Sentinel Is NOT Necessarily End of Source File

This is an important trap.

Suppose:

TEXT

Buffer 1

[a b c d EOF]

The EOF marker at the end of this half may simply mean:

end of this buffer half; load/continue with the other half.

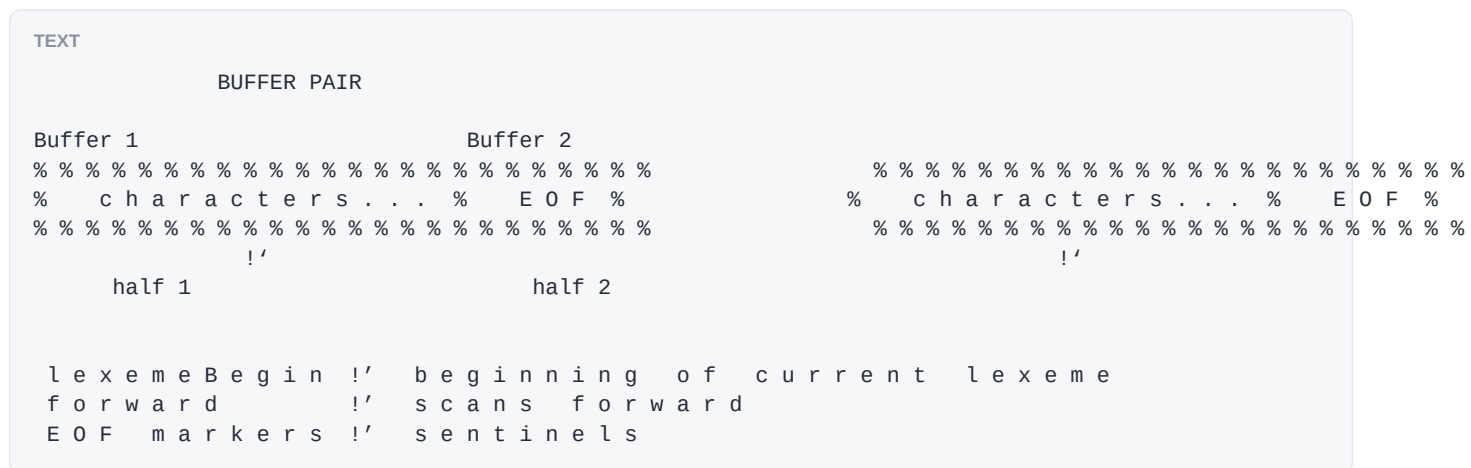
It does **not** automatically mean the physical source file has ended.

The implementation determines whether the encountered marker corresponds to:

1. buffer boundary, or
2. actual end of input.

13. Classic Two-Buffer Picture

For notes, draw:



14. Basic Working of the Scheme

Suppose **forward** scans through Buffer 1.

Step 1

Read ordinary characters.

Step 2

Reach sentinel at end of Buffer 1.

Step 3

Continue with/refill Buffer 2 as appropriate.

Step 4

Scan Buffer 2.

Step 5

When Buffer 2 is exhausted, refill Buffer 1 with the next block.

Conceptually:

TEXT

B u f f e r 1 ! ' B u f f e r 2 ! ' r e f i l l B u f f e r 1 ! ' r e f i l l B u f f e r 2

until actual end of source.

15. Lexeme Crossing the Boundary

Suppose:

TEXT

Buffer 1: totalVa
Buffer 2: lue + x

The lexeme:

TEXT

totalValue

starts in Buffer 1 and ends in Buffer 2.

The scheme must preserve the portion beginning at `lexemeBegin`.

TEXT

Buffer 1 Buffer 2
... totalVa | lue + x ...
 ! ' ! '
 LB forward

Lexeme = totalValue

This is why buffer design and pointers matter.

16. What Happens After a Token Is Recognized?

Suppose:

TEXT

abc+

While scanning the identifier:

TEXT

lexemeBegin
!"
a b c +
 ! '
 forward

+ proves that the identifier has ended.

So:

Step 1

Current lexeme:

```
TEXT
abc
```

Step 2

Return:

```
TEXT
<ID, abc>
```

Step 3

Ensure `+` remains available for the next token.

Step 4

Move `lexemeBegin` appropriately.

17. Lookahead vs Retract

These two are related but not identical.

Concept	Meaning
Lookahead	Read/inspect future character(s) to decide token
Retract	Move back after reading beyond current lexeme

Example

Input:

```
TEXT
<
```

with operators `<` and `<=`.

Reading the character after `<` is **lookahead**.

If that character is not `=` and was consumed, returning to it is **retract**.

Memory Trick

Lookahead asks "what comes next?"

Retract says "I went too far."

18. Maximal Munch and Buffering

Recall from Part 3:

Lexer normally selects the longest valid prefix.

Consider valid patterns:

```
TEXT
>
>=
>>
>>=
```

Input:

```
TEXT
>>=X
```

To choose the longest token, the lexer may have to continue scanning several characters.

Hence:

$\boxed{\text{Maximal munch requires lookahead capability}}$

and buffering makes lookahead efficient.

19. Example — Longest Valid Prefix

Suppose valid tokens include:

```
TEXT
=
==
```

Input:

```
TEXT
==X
```

Step 1

Read first `=`.

Valid token exists, but don't necessarily stop.

Step 2

Read second `=`.

Now:

```
TEXT
==
```

is also valid and longer.

Step 3

Read/inspect `x`.

`==x` is not an operator.

Therefore longest complete token seen was:

```
TEXT
==
```

Return:

```
TEXT
EQ
```

Then process `x` as the next lexeme.

20. Last Accepting State — Useful Intuition

A lexical analyzer using a DFA may pass through both accepting and non-accepting states while attempting to find the longest match.

It can conceptually remember:

the most recent accepting state and its input position.

If continuation eventually fails:

```
TEXT
scan farther
    !"
failure
    !"
return to last accepting position
```

This lets it recognize the **longest valid lexeme**.

21. Example of Why This Matters

Imagine token patterns where:

```
TEXT
a
abcd
```

are valid tokens, but intermediate prefix `ab` by itself isn't a complete token.

Input:

TEXT

abcx

The scanner might:

TEXT

```
a           !'  a c c e p t i n g
a b         !'  s t i l l   p o t e n t i a l l y   u s e f u l
a b c       !'  s t i l l   p o t e n t i a l l y   u s e f u l
a b c x     !'  f a i l u r e
```

If no longer token is accepted after `a`, it may need to fall back to the last accepting position corresponding to `a`.

The exact DFA depends on the token patterns, but the key concept is:

`\[ \boxed{\text{Longest match may require remembering last acceptance}} \]`

22. Input Buffering Is an Efficiency Technique

Do not confuse **input buffering** with lexical recognition itself.

Regular expressions/automata answer:

What token does this sequence represent?

Buffering answers:

How can we read the characters efficiently?

Therefore:

TEXT

Token recognition

!' R E / D F A

Efficient input access

!' B u f f e r i n g

GATE TRAP 1 — `lexemeBegin` vs `forward`

`forward` identifies where the lexeme starts.

'LFalse.

Usually:

TEXT

```
l e x e m e B e g i n !'  b e g i n n i n g
f o r w a r d         !'  s c a n s   a h e a d
```

GATE TRAP 2 — Why Buffer?

The main reason for buffering is to change the grammar.

'LFalse.

Buffering improves **input/scanning efficiency**.

GATE TRAP 3 — Sentinel

A sentinel is an ordinary source-program character.

'LFalse.

It is a special marker used by the buffering mechanism.

GATE TRAP 4 — Every Sentinel Means Physical EOF

'LFalse.

A buffer-half sentinel can mark a buffer boundary; additional logic distinguishes it from actual end-of-input.

GATE TRAP 5 — Lookahead vs Retract

They are not synonyms.

TEXT

```
L o o k a h e a d ! '   i n s p e c t   f u r t h e r
R e t r a c t       ! '   b a c k   u p
```

GATE TRAP 6 — Buffer Boundary Ends Token

'LFalse.

A lexeme may span across two buffer halves.

Example:

TEXT

```
Buffer 1      Buffer 2
identi | fier
```

still forms:

TEXT

```
identifier
```

GATE TRAP 7 — Maximal Munch

The scanner should stop as soon as it reaches **any** accepting state.

'L Not necessarily.

There may be a longer valid token.

Example:

TEXT

<
<=

After reading <, it may need to continue.

23. GATE-STYLE EXAMPLE 1

A lexical analyzer uses two pointers:

TEXT

lexemeBegin
forward

Which statement is correct?

- A. Both always move together. B. lexemeBegin marks the beginning of a lexeme while forward searches for its end.
C. forward marks the beginning and never changes. D. Both are used by the parser rather than lexer.

FAST SOLUTION

Names tell us:

TEXT

lexemeBegin = beginning
forward = scanning pointer

' B

24. GATE-STYLE EXAMPLE 2

Why are sentinels used in a two-buffer lexical input scheme?

- A. To identify semantic errors B. To eliminate all DFA states C. To simplify buffer-end detection D. To eliminate the parser

' C

25. GATE-STYLE EXAMPLE 3

Suppose valid operators are:

TEXT

<
<=

After reading <, why might the lexer need to inspect the next character?

' To determine whether the token is:

TEXT
<

or:

TEXT
<=

This is:

$\boxed{\text{Lookahead}}$

26. GATE-STYLE EXAMPLE 4

Input:

TEXT
abc+

The lexer recognizes identifiers matching:

letter^+

It reads **+** while determining where the identifier ends.

What must conceptually happen?

The recognized lexeme is:

TEXT
abc

and **+** must remain for the next token.

Therefore the scanner may require:

$\boxed{\text{Retract}}$

or an equivalent mechanism.

27. Common Buffering Terms — Final Table

WRITE THIS IN NOTES

Term	Meaning
Buffer	Memory area holding a block of source characters
Two-buffer scheme	Uses two alternating input halves
Sentinel	Marker placed at end of each half
lexemeBegin	Beginning of current lexeme

Term	Meaning
forward	Scans ahead through input
Lookahead	Inspect future character(s)
Retract	Move back after overscanning
Maximal munch	Select longest valid lexeme

MINI PRACTICE — PART 4

Try these before checking the solutions.

Q1 — Easy

The main purpose of input buffering in lexical analysis is:

A. Grammar simplification B. Reduce repeated input operations C. Type checking D. Code optimization

Q2 — Easy

Which pointer normally identifies the start of the current lexeme?

A. `forward` B. `lexemeBegin` C. `stackTop` D. `follow`

Q3 — Medium

Suppose both `<` and `<=` are valid operators. Which ability allows the scanner to inspect `=` after reading `<`?

A. Parsing B. Lookahead C. Code generation D. Left factoring

Q4 — Medium

Why is the two-buffer arrangement useful?

A. A lexeme may cross a buffer boundary. B. It turns a CFG into a regular grammar. C. It avoids token generation. D. It makes semantic analysis unnecessary.

Q5 — Tricky

Which statement is **false**?

A. A sentinel can help detect buffer boundaries. B. A lexeme may extend from one buffer half into the other. C. The scanner must necessarily return a token immediately whenever it first reaches an accepting DFA state. D. `forward` can move beyond the end of a lexeme during recognition.

SOLUTIONS

Q1

' B — Reduce repeated input operations

Reading blocks into memory is more efficient than performing character-by-character file I/O.

Q2

' B — `lexemeBegin`

TEXT

```
lexemeBegin !' start  
forward !' scanning
```

Q3

' B — Lookahead

The lexer looks at upcoming input before deciding whether to return `<` or `<=`.

Q4

' A

A token can span two blocks.

Example:

TEXT

```
identi | fier
```

must still be recognized as one lexeme.

Q5

' C is false.

The first accepting state may correspond to a shorter token.

Example:

TEXT

```
>  
>=
```

After recognizing `>`, the scanner may continue to determine whether `>=` is possible.

30-SECOND REVISION

Why buffering?

`\boxed{\text{Efficient character input}}`

Instead of:

TEXT

one file read per character

USE:

TEXT

b l o c k ! ' b u f f e r ! ' s c a n

Two pointers

TEXT

l e x e m e B e g i n ! ' s t a r t o f l e x e m e
f o r w a r d ! ' m o v e s a h e a d

Memory:

Begin stays; Forward explores.

Two-buffer scheme

TEXT

B u f f e r 1 ! ' B u f f e r 2 ! ' r e f i l l B u f f e r 1 ! ' . . .

A lexeme can cross the boundary.

Sentinel

$\backslash \boxed{\text{Special end-of-buffer marker}}$

Purpose:

simplify boundary detection.

Lookahead

Inspect future characters before deciding token.

Retract

Back up after reading too far.

Maximal munch connection

TEXT

Read forward

!"

Find longest valid lexeme

!"

Possibly retract to last acceptance

CUMULATIVE REVISION — PARTS 1–4

We have now completed the entire **Lexical Analysis foundation**.

Compiler pipeline

LSSIOC

TEXT

Lexical

Syntax

Semantic

Intermediate

Optimization

Code generation

Lexer

$\backslash$ Characters $\rightarrow$ Tokens $\backslash$

Scanner

$\backslash$ $\boxed{\text{Scanner=Lexical Analyzer}}$ $\backslash$

TLP

TEXT

Token = category

Lexeme = actual text

Pattern = rule

Regular language connection

$\backslash$ $RE \equiv NFA \equiv DFA$ $\backslash$

Lexical vs parsing

TEXT

R e g u l a r ! ' L e x e r

C F G ! ' P a r s e r

Maximal munch

$\backslash$ $\boxed{\text{Longest valid prefix}}$ $\backslash$

Buffering

TEXT

lexemeBegin + forward
two buffers
sentinels
lookahead
retract

Next Major Module: PARSING

From the next part onward, we move from:

$\boxed{\text{Characters! Tokens}}$

to:

$\boxed{\text{Tokens! Grammatical Structure}}$

This is where the high-value GATE parsing portion begins.

Completed: Parts 1–4/31 | Next: Part 5 — Context-Free Grammar & Parsing Basics

Reply **NEXT** for **Part 5**, where we will build parsing from zero: **parser, CFG, terminals/non-terminals, productions, sentential forms, derivations, leftmost/rightmost derivations, parse trees, and the GATE relationship between derivations and parse trees.**

USER

NEXT

ASSISTANT

Compiler Design — Part 5: CFG & Parsing Fundamentals

Completed: Parts 1–4/31 | Current: Part 5 | Remaining: Parts 6–31

We now move from **lexical analysis** to the second major compiler phase:

$\boxed{\text{PARSING / SYNTAX ANALYSIS}}$

The lexer answers:

What are the tokens?

The parser answers:

Do these tokens form a grammatically valid program structure?

1. Where Does Parsing Fit?

Recall:

TEXT

```
Source Program
      !"
Lexical Analyzer
      !"
Token Stream
      !"
Syntax Analyzer / Parser
      !"
Parse / Syntax Structure
```

Therefore:

$\boxed{\text{Tokens} \rightarrow \text{Parser} \rightarrow \text{Syntactic Structure}}$

2. What Is a Parser?

CONCEPT

A **parser** receives tokens from the lexical analyzer and checks whether their sequence can be generated by the grammar of the programming language.

Example token stream:

TEXT

```
ID + ID * NUM
```

The parser determines whether this is a valid expression according to grammar rules.

Formal idea

Parsing is the process of determining the syntactic structure of a token sequence according to a grammar.

The syntax analyzer is also called:

$\boxed{\text{Parser}}$

WRITE THIS IN NOTES

TEXT

```
Lexical Analyzer:
  Characters !'  Tokens
```

```
Parser:
  Tokens !'  Grammatical structure
```

3. Why Do We Need a Grammar?

Consider valid expressions:

TEXT

a+b
a*b
a+b*c
(a+b)*c

We need rules describing how expressions can be formed.

Example:

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

These rules form a **Context-Free Grammar (CFG)**.

The parser uses these rules to recognize valid token sequences.

4. Context-Free Grammar

A CFG is formally represented as:

$G = (V, T, P, S)$

You may also see:

$G = (N, \Sigma, P, S)$

where:

- V or N = set of non-terminals
- T or Σ = set of terminals
- P = set of productions
- S = start symbol

Memory Trick

N-T-P-S

TEXT

N !' N o n - t e r m i n a l s
T !' T e r m i n a l s
P !' P r o d u c t i o n s
S !' S t a r t s y m b o l

Think:

Grammar Needs The Production Start.

5. Terminals

CONCEPT

Terminals are the actual symbols/tokens that ultimately appear in strings generated by the grammar.

Example:

$E \rightarrow E + E \mid id$

Terminals:

```
TEXT
+
id
```

They cannot normally be replaced further by grammar productions.

6. Non-Terminals

Non-terminals represent syntactic categories and can be expanded using productions.

For:

$E \rightarrow E + E \mid id$

E is a non-terminal.

Other common compiler grammar non-terminals:

```
TEXT
E !' Expression
T !' Term
F !' Factor
S !' Statement
```

Terminal vs Non-Terminal

Terminal	Non-terminal
Appears in final generated string	Used to describe structure
Cannot be further expanded	Can be replaced using productions
Example: id, +, *	Example: E, T, F

Memory Trick

Terminal = terminates.

A terminal has reached the end of grammatical expansion.

7. Production

A **production** is a grammar rule.

Example:

$\text{E} \rightarrow \text{E} + \text{T}$

It says:

An **E** may be replaced by **E+T**.

A context-free production generally has:

$A \rightarrow \alpha$

where:

- A = one non-terminal
- α = string of terminals/non-terminals, possibly ϵ

8. Why Is It Called Context-Free?

For a CFG production:

$A \rightarrow \alpha$

whenever **A** occurs, it can be replaced by α regardless of the surrounding symbols.

Example:

$A \rightarrow ab$

If we have:

TEXT

xAy

we can replace **A**:

TEXT

x a b y

The replacement does not depend on the **context** around **A**.

Hence:

Context-Free

9. Start Symbol

One distinguished non-terminal is the **start symbol**.

It represents the complete syntactic construct described by the grammar.

Example:

$E \rightarrow E + T \mid T$

If **E** is the start symbol:

$S = E$

Derivations begin from E .

10. Example Grammar

Consider:

$$S \rightarrow aSb \mid \epsilon$$

Here:

Non-terminal

$$\{S\}$$

Terminals

$$\{a, b\}$$

Productions

$$S \rightarrow aSb$$
$$S \rightarrow \epsilon$$

Start symbol

$$S$$

This grammar generates:

TEXT

μ
ab
aabb
aaabbb
...

11. Derivation

CONCEPT

A **derivation** is a sequence of applications of grammar productions starting from the start symbol.

Suppose:

$$S \rightarrow aSb \mid \epsilon$$

To generate:

TEXT

aabb

we can derive:

$$S \rightarrow aSb$$
$$\rightarrow aaSbb$$

$\rightarrow aabb$

because finally:

$\xrightarrow{\epsilon}$

WRITE THIS IN NOTES

Derivation: Sequence of production applications used to generate a string from the start symbol.

12. Meaning of $\rightarrow$

Notation:

$\alpha \rightarrow \beta$

means:

β can be obtained from α using one production step.

13. Meaning of $\rightarrow^*$

$\alpha \rightarrow^* \beta$

means:

β can be derived from α in zero or more steps.

Important:

Because zero steps are allowed:

$\boxed{\alpha \rightarrow^* \alpha}$

14. Meaning of $\rightarrow^+$

Sometimes:

$\alpha \rightarrow^+ \beta$

means:

one or more derivation steps.

This resembles the regular-expression distinction:

TEXT

*	!	z	e	r	o		o	r		m	o	r	e
+	!	o	n	e		o	r		m	o	r	e	

15. Sentential Form

Any string of terminals and/or non-terminals derivable from the start symbol is called a **sentential form**.

Formally, if:

$\text{S} \xrightarrow{*} \alpha$

then α is a sentential form.

Example

Using:

$\text{S} \rightarrow aSb \mid \epsilon$

Consider:

$\text{S} \rightarrow aSb \rightarrow aaSbb \rightarrow aabb$

All of these are sentential forms:

TEXT

S
aSb
aaSbb
aabb

16. Sentence

A **sentence** is a sentential form containing **only terminals**.

For the previous grammar:

TEXT

aabb

is a sentence.

But:

TEXT

aaSbb

is not a sentence because it still contains non-terminal S.

Sentence vs Sentential Form

Important relationship

$\boxed{\text{Every sentence is a sentential form}}$

but

$\boxed{\text{Not every sentential form is a sentence}}$

Why?

A sentential form can contain non-terminals.

A sentence cannot.

Memory Trick

Sentence = finished sentential form.

17. Language Generated by a Grammar

The language generated by grammar (G) is:

$L(G) = \{w \in T^+ \mid S \xrightarrow{*} w\}$

Don't panic about the notation.

It simply means:

All terminal-only strings that can be generated from the start symbol.

Example

Grammar:

$S \rightarrow a \mid b$

Possible derivations:

$S \rightarrow b$

$S \rightarrow a \rightarrow ab$

$S \rightarrow a \rightarrow aa \rightarrow aab$

Therefore:

$L(G) = \{b, ab, aab, aaab, \dots\}$

or:

$\boxed{a^*b}$

18. Leftmost Derivation

Now we reach an important GATE concept.

If a sentential form contains multiple non-terminals, which one do we expand first?

In a **leftmost derivation**, always expand the **leftmost non-terminal**.

Notation is sometimes:

$\text{\textbackslash}\text{\textbackslash}\text{Rightarrow}_{\{l m\}}\text{\textbackslash}$

or:

$\text{\textbackslash}\text{\textbackslash}\text{Rightarrow}_L\text{\textbackslash}$

Example Grammar

$\text{\textbackslash}\text{\textbackslash}\text{S}\text{\textbackslash}\text{Rightarrow} AB\text{\textbackslash}$

$\text{\textbackslash}\text{\textbackslash}\text{A}\text{\textbackslash}\text{Rightarrow} aA\text{\textbackslash}\text{mid} a\text{\textbackslash}$

$\text{\textbackslash}\text{\textbackslash}\text{B}\text{\textbackslash}\text{Rightarrow} bB\text{\textbackslash}\text{mid} b\text{\textbackslash}$

Generate:

TEXT

aabb

Start:

$\text{\textbackslash}\text{\textbackslash}\text{S}\text{\textbackslash}\text{Rightarrow} AB\text{\textbackslash}$

There are now two non-terminals:

TEXT

A B

For leftmost derivation, expand **A** first.

$\text{\textbackslash}\text{\textbackslash}\text{AB}\text{\textbackslash}\text{Rightarrow} aAB\text{\textbackslash}$

Again leftmost non-terminal is **A**:

$\text{\textbackslash}\text{\textbackslash}\text{aAB}\text{\textbackslash}\text{Rightarrow} aaB\text{\textbackslash}$

Now only **B** remains:

$\text{\textbackslash}\text{\textbackslash}\text{aaB}\text{\textbackslash}\text{Rightarrow} aabB\text{\textbackslash}$

$\text{\textbackslash}\text{\textbackslash}\text{aabB}\text{\textbackslash}\text{Rightarrow} aabb\text{\textbackslash}$

Thus:

$\text{\textbackslash}\text{\textbackslash}\text{boxed}\{\text{\textbackslash}\text{\textbackslash}\text{S}\text{\textbackslash}\text{Rightarrow} AB\text{\textbackslash}\text{Rightarrow} aAB\text{\textbackslash}\text{Rightarrow} aaB\text{\textbackslash}\text{Rightarrow} aabB\text{\textbackslash}\text{Rightarrow} aabb\}\text{\textbackslash}$

19. Rightmost Derivation

In a **rightmost derivation**, always expand the **rightmost non-terminal**.

Using the same grammar:

$\text{\textbackslash}\text{\textbackslash}\text{S}\text{\textbackslash}\text{Rightarrow} AB\text{\textbackslash}$

Generate:

TEXT

aabb

Start:

$\boxed{S \rightarrow AB}$

Rightmost non-terminal:

TEXT

B

So:

$\boxed{AB \rightarrow AbB}$

Again expand rightmost B:

$\boxed{AbB \rightarrow Abb}$

Now expand A:

$\boxed{Abb \rightarrow aAbb}$

$\boxed{aAbb \rightarrow aabb}$

Thus:

$\boxed{S \rightarrow AB \rightarrow AbB \rightarrow Abb \rightarrow aAbb \rightarrow aabb}$

20. Leftmost vs Rightmost Derivation

Leftmost	Rightmost
Expand leftmost non-terminal	Expand rightmost non-terminal
Commonly associated with top-down parsing	Reverse rightmost derivation is central to bottom-up parsing
Does not imply reading input backwards	Does not imply reading input backwards

That last point is a GATE trap.

GATE TRAP

Rightmost derivation does **not** mean:

Start from the right side of the input string.

It means:

Among the non-terminals currently present, expand the rightmost one.

21. Parse Tree

A **parse tree** is a tree representation of a derivation.

Suppose:

$\backslash E \rightarrow E + T \mid T \backslash$

$\backslash T \rightarrow id \backslash$

For:

TEXT

id + id

a parse tree could be:

TEXT

```
      E
     /\
    E + T
    |   |
    T   id
    |
    id
```

The root is the start symbol.

Internal nodes are non-terminals.

Leaves are terminals or $\backslash(\epsilon)\backslash$.

WRITE THIS IN NOTES

A parse tree has:

1. Root = start symbol
2. Internal nodes = non-terminals
3. Children = RHS of production used
4. Leaves = terminals or $\backslash(\epsilon)\backslash$
5. Reading leaves left-to-right gives the generated string

22. Parse Tree Example

Grammar:

$\backslash S \rightarrow aSb \mid \epsilon \backslash$

Generate:

TEXT

aabb

Parse tree:

TEXT

```
      S
     /\
    a S b
     /\
    a S b
```

| μ

Reading leaves left to right:

TEXT

a a μ b b

Ignoring ϵ :

TEXT

aabb

23. Derivation vs Parse Tree

A derivation records:

the order in which productions are applied.

A parse tree records:

the hierarchical syntactic structure.

This distinction is crucial.

Example

Grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Leftmost derivation

$S \Rightarrow AB \Rightarrow aB \Rightarrow ab$

Rightmost derivation

$S \Rightarrow AB \Rightarrow Ab \Rightarrow ab$

Different derivation sequences.

But the parse tree is the same:

TEXT

```
      S
     /\
    A  B
    |  |
    a  b
```

VERY IMPORTANT GATE FACT

Different **leftmost vs rightmost derivation orders** can correspond to the **same parse tree**.

So do not say:

Every distinct derivation means a distinct parse tree.

That is false.

24. Parse Tree and Derivation Relationship

For a fixed parse tree:

- there is a unique corresponding **leftmost derivation**
- there is a unique corresponding **rightmost derivation**

But there may be many arbitrary derivation orders that yield the same tree.

This fact becomes very important when studying **ambiguity** in Part 6.

25. What Is the Yield of a Parse Tree?

The **yield** or **frontier** of a parse tree is the sequence of leaf symbols read from left to right.

Example:

TEXT



Yield:

$\boxed{ab}$

26. Parse Tree vs Syntax Tree / AST

You will study syntax trees more deeply during intermediate code generation, but distinguish them now.

Parse Tree

Contains full grammar structure.

Example:

TEXT



```

      |   |
      T   F
      |   |
      F   id
      |
      id

```

Abstract Syntax Tree (AST)

Removes unnecessary grammar-detail nodes.

For:

```

TEXT
a+b

```

AST might simply be:

```

TEXT
      +
     / \
    a   b

```

Quick distinction

Parse tree = grammar details

AST = essential program structure

We will revisit this later.

27. Parser's Two Broad Families

Before we go deeper, know the overall classification:

```

TEXT
      Parsing
     /      \
  Top-Down  Bottom-Up

```

28. Top-Down Parsing

Starts from the **start symbol** and attempts to derive the input.

Conceptually:

```

TEXT
Start Symbol
      !"
Apply productions

```

```
!"  
Input String
```

It constructs the parse tree:

```
\[ \boxed{\text{Root !' Leaves}} \]
```

Examples we'll study:

- recursive descent
- predictive parsing
- LL(1)

29. Bottom-Up Parsing

Starts conceptually from the input string and reduces it back to the start symbol.

```
TEXT  
Input String  
!"  
Reductions  
!"  
Start Symbol
```

It constructs the parse tree:

```
\[ \boxed{\text{Leaves !' Root}} \]
```

Examples:

- shift-reduce parsing
- LR(0)
- SLR
- LALR
- CLR

Memory Trick

```
TEXT  
TOP-DOWN  
T o p / r o o t ! ' b o t t o m / l e a v e s  
  
BOTTOM-UP  
B o t t o m / l e a v e s ! ' t o p / r o o t
```

30. Important Derivation Relationship

Top-down parsing

tries to construct a:

```
\[ \boxed{\text{Leftmost derivation}} \]
```

for predictive/LL parsing.

Bottom-up parsing

constructs the:

$\boxed{\text{Reverse of a rightmost derivation}}$

This is a **very important GATE fact**.

WRITE THIS IN NOTES

TEXT

LL / Top-down

!' Leftmost derivation

LR / Bottom-up

!' Reverse of rightmost derivation

Memory Trick — LL and LR

We'll study this properly later, but start remembering now.

LL

First L:

scan input Left-to-right

Second L:

construct Leftmost derivation

LR

L:

scan input Left-to-right

R:

construct Rightmost derivation in reverse

Therefore:

$\boxed{LL = \text{Left-to-right}, \text{Leftmost}}$

$\boxed{LR = \text{Left-to-right}, \text{Reverse} \text{ Rightmost}}$

This mnemonic will be useful throughout parsing.

31. Example: Is a String Generated by the Grammar?

Grammar:

$S \rightarrow aSb \mid ab$

Question:

Is:

TEXT

aaabbb

generated?

Step 1

S

Step 2

Use:

$S \rightarrow aSb$

$S \rightarrow aSb$

Step 3

Again:

$aSb \rightarrow aaSbb$

Step 4

Now use:

$S \rightarrow ab$

$aaSbb \rightarrow aa(ab)bb$

Therefore:

$\boxed{aaabbb}$

So yes.

32. Number of Production Applications

A possible GATE pattern:

Grammar:

$S \rightarrow aSb \mid ab$

How many times is $(S \rightarrow aSb)$ used to generate:

$a^n b^n$

for $(n \geq 1)$?

Let's inspect.

For:

$\backslash[ab\backslash]$

use recursive production:

$\backslash[0\backslash]$

times.

For:

$\backslash[aabb\backslash]$

use:

$\backslash[1\backslash]$

time.

For:

$\backslash[aaabbb\backslash]$

use:

$\backslash[2\backslash]$

times.

Therefore for:

$\backslash[a^nb^n\backslash]$

recursive production is used:

$\backslash[\boxed{n-1}\backslash]$

times.

This is how grammar-based numerical reasoning may appear.

33. μ — Empty String

The symbol:

$\backslash[\epsilon\backslash]$

represents the **empty string**.

Its length is:

$\backslash[\boxed{|\epsilon|=0}\backslash]$

It is different from the empty set:

$\backslash[\emptyset\backslash]$

Important distinction

TEXT

μ = a string containing zero symbols

" = a set containing no elements

Therefore:

$\boxed{\epsilon \neq \emptyset}$

GATE TRAP — μ and $\{\mu\}$

Also:

ϵ

is a string.

But:

$\{\epsilon\}$

is a language/set containing one string.

Hence:

$|\{\epsilon\}| = 1$

whereas:

$|\emptyset| = 0$

as a set.

34. Nullable Non-Terminal — Preview

A non-terminal is **nullable** if it can derive ϵ .

Formally:

$A \xrightarrow{*} \epsilon$

Example:

$A \rightarrow aA \mid \epsilon$

Therefore:

A

is nullable.

Nullable non-terminals become very important for **FIRST** and **FOLLOW**.

We'll cover them deeply in Part 7.

35. GATE TRAPS

Trap 1 — Parser Input

Parser receives characters directly.

'L False.

Normally:

$\boxed{\text{Parser receives tokens from lexer}}$

Trap 2 — Terminal vs Non-terminal

A terminal can always be expanded with grammar rules.

'LFalse.

Non-terminals are expanded.

Trap 3 — Sentence vs Sentential Form

Every sentential form is a sentence.

'LFalse.

Example:

```
TEXT
aSb
```

is a sentential form but not a sentence.

Correct relation:

$\boxed{\text{Sentence}} \subseteq \text{Sentential forms}$

conceptually.

Trap 4 — Rightmost Derivation

Rightmost derivation means reading input right-to-left.

'LFalse.

It means expanding the rightmost **non-terminal**.

Trap 5 — Different Derivation Orders

Two derivations using productions in different orders must produce different parse trees.

'LFalse.

Leftmost and rightmost derivations can describe the same parse tree.

Trap 6 — Parse Tree Leaves

Only non-terminals can appear as leaves.

'LFalse.

Leaves are typically terminals or ϵ .

Trap 7 — Bottom-Up Parsing

Bottom-up parsing constructs a leftmost derivation directly.

'L False.

It corresponds to:

$\boxed{\text{Rightmost derivation in reverse}}$

Trap 8 — LL/LR Input Direction

LR means input is read right-to-left.

'L False.

Both LL and LR normally scan input:

$\boxed{\text{Left to right}}$

The **R** in LR refers to **rightmost derivation in reverse**.

36. GATE PATTERN— Identify Sentential Form

Grammar:

$S \rightarrow aSb \mid \epsilon$

Which is a sentential form?

A. aSb B. $aaSbb$ C. $aabb$ D. All of the above

FAST SOLUTION

All are derivable from S .

' **D**

But only $aabb$ is a **sentence**.

37. GATE PATTERN— Derivation Type

Suppose:

TEXT

S → A B → a B → a b

At AB , A was expanded first.

A is the leftmost non-terminal.

Therefore this is consistent with a:

$\boxed{\text{Leftmost derivation}}$

38. GATE PATTERN— Bottom-Up Parsing

Question:

An LR parser constructs which derivation?

Best precise wording:

$\boxed{\text{Rightmost derivation in reverse}}$

Do not answer merely "rightmost derivation" if the question asks what bottom-up parser **constructs through reductions**.

39. GATE-STYLE EXAMPLE

Grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Which two derivations are valid?

Leftmost

$S \rightarrow AB \rightarrow aB \rightarrow ab$

Rightmost

$S \rightarrow AB \rightarrow Ab \rightarrow ab$

Both correspond to:

TEXT



Key lesson

$\boxed{\text{Different derivation order does not automatically imply ambiguity}}$

This becomes crucial next.

MINI PRACTICE — PART 5

Try these before checking the answers.

Q1 — Easy

For:

$G = (N, T, P, S)$

what does (P) represent?

A. Parser B. Productions C. Parse tree D. Preprocessor

Q2 — Easy

Which is a sentence?

Grammar:

$[S \rightarrow aSb \mid \epsilon]$

A. aSb B. $aaSbb$ C. $aabb$ D. S

Q3 — Medium

A top-down predictive parser constructs:

A. Reverse of a leftmost derivation B. Leftmost derivation C. Reverse of rightmost derivation D. Only rightmost derivation

Q4 — Medium

A bottom-up LR parser corresponds to:

A. Leftmost derivation B. Reverse leftmost derivation C. Rightmost derivation in reverse D. Neither derivation

Q5 — Tricky

Suppose one string has a leftmost derivation and a rightmost derivation that apply productions in different orders.

Can you immediately conclude that the grammar is ambiguous?

A. Yes B. No

SOLUTIONS

Q1

' B — Productions

$[G = (N, T, P, S)]$

Q2

' C — $aabb$

A sentence contains terminals only.

aSb and $aaSbb$ contain S .

Q3

' B — Leftmost derivation

Remember:

$\boxed{LL \rightarrow \text{Leftmost}}$

Q4

' C — Rightmost derivation in reverse

Remember:

$\boxed{LR \rightarrow \text{Reverse} \text{ Rightmost}}$

Q5

' B — No

One parse tree naturally has:

- one corresponding leftmost derivation
- one corresponding rightmost derivation

Their production application order differs.

Ambiguity requires **more than one parse tree** for the same string, equivalently more than one leftmost derivation or more than one rightmost derivation.

We study that next.

WRITE THIS IN NOTES — PART 5 MASTER SHEET

CFG

$\boxed{G=(N,T,P,S)}$

TEXT

N	!	Non - terminals
T	!	Terminals
P	!	Productions
S	!	Start symbol

Production

$A \rightarrow \alpha$

One-step derivation

$\alpha \rightarrow \beta$

Zero or more steps

$\alpha \rightarrow^* \beta$

Sentential form

$S \rightarrow^* \alpha$

Sentence

A sentential form containing only terminals.

Relationship

$\boxed{\text{Every sentence is a sentential form}}$

but not vice versa.

Parsing

TEXT

T o p - d o w n ! ' R o o t t o l e a v e s ! ' L e f t m o s t d e r i v a t i o n

B o t t o m - u p ! ' L e a v e s t o r o o t ! ' R i g h t m o s t d e r i v a t i o n i n r e

LL

$\boxed{\text{Left-to-right input + Leftmost derivation}}$

LR

$\boxed{\text{Left-to-right input + Rightmost derivation in reverse}}$

30-SECOND REVISION

TEXT

Lexer:

C h a r a c t e r s ! ' T o k e n s

Parser:

T o k e n s ! ' S y n t a x

CFG

$G=(N,T,P,S)$

Terminal

Finished grammar symbol/token.

Non-terminal

Can be expanded.

Sentential form

May contain terminals + non-terminals.

Sentence

Terminals only.

Leftmost derivation

Always expand **leftmost non-terminal**.

Rightmost derivation

Always expand **rightmost non-terminal**.

Parse tree

TEXT

Root = start symbol

Internal = non-terminals

Crucial relationship

$\boxed{LL \rightarrow \text{Leftmost}}$

$\boxed{LR \rightarrow \text{Rightmost in reverse}}$

Very important trap

Different leftmost and rightmost derivations of the same tree do not prove ambiguity.

Completed: Parts 1–5/31 | Next: Part 6 — Ambiguity, Operator Precedence, Associativity, Left Recursion & Left Factoring

Reply **NEXT** for **Part 6**, where we'll cover one of the most important grammar-preparation areas: **how to identify ambiguous grammars, multiple parse trees, dangling-else intuition, precedence/associativity, immediate and indirect left recursion, elimination formulas, and left factoring with GATE shortcuts.**

USER

NEXT

ASSISTANT

Compiler Design — Part 6: Ambiguity, Precedence, Associativity, Left Recursion & Left Factoring

Completed: Parts 1–5/31 | Current: Part 6 | Remaining: Parts 7–31

This part prepares the grammar before predictive parsing. Three ideas must be clearly separated:

$\boxed{\text{Ambiguity} \neq \text{Left Recursion} \neq \text{Left Factoring}}$

They solve different problems.

1. What Is an Ambiguous Grammar?

CONCEPT

A context-free grammar is **ambiguous** if **at least one string** generated by the grammar has **more than one distinct parse tree**.

Formal definition

A CFG $\langle G \rangle$ is ambiguous if there exists some:

$w \in L(G)$

such that $\langle w \rangle$ has:

- two or more distinct parse trees

Equivalently, the same string has:

- more than one **leftmost derivation**, or
- more than one **rightmost derivation**.

WRITE THIS IN NOTES

$\boxed{\text{Ambiguous grammar} \iff \exists w \in L(G) \text{ having } > 1 \text{ parse tree}}$

Equivalent test:

$\boxed{\text{Same string has two distinct leftmost derivations}}$

or two distinct rightmost derivations.

2. Classic Ambiguous Expression Grammar

Consider:

$E \rightarrow E + E \mid E * E \mid id$

Take the string:

$id + id * id$

There are two possible interpretations.

Interpretation 1

$id + (id * id)$

Interpretation 2

$(id + id) * id$

These produce different parse trees.

3. Parse Tree 1 — Multiplication First

For:

$id + (id * id)$

TEXT



The root operator is $+$.

So multiplication occurs inside the right subtree.

To show ambiguity:

1. Find a suitable string.
2. Show two parse trees, or
3. Show two different leftmost derivations.

Done.

7. Two Arbitrary Derivations Are Not Enough

Recall from Part 5.

Suppose:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

We can derive **ab** as:

Derivation 1

$S \rightarrow AB \rightarrow aB \rightarrow ab$

Derivation 2

$S \rightarrow AB \rightarrow Ab \rightarrow ab$

These are different production orders, but represent the **same parse tree**.

So they do **not** prove ambiguity.

REMEMBER

To prove ambiguity using derivations, show:

- two **leftmost** derivations, or
- two **rightmost** derivations

for the same string.

8. Operator Precedence

CONCEPT

Precedence decides which operator binds more strongly.

Normally:

$a * b + c$ (where $*$ has higher precedence than $+$)

Thus:

$a + b * c$

means:

$a + (b * c)$

not:

$\backslash (a+b)*c \backslash$

Memory Trick

Precedence decides WHICH operator goes first.

9. Associativity

Associativity matters when operators have the **same precedence**.

Example:

$\backslash a-b-c \backslash$

If `-` is left associative:

$\backslash (a-b)-c \backslash$

If right associative:

$\backslash a-(b-c) \backslash$

Memory Trick

Precedence = different operators. Associativity = repeated same-level operators.

This is slightly simplified but excellent for GATE recall.

10. Encoding Precedence in Grammar

Instead of:

$\backslash E \rightarrow E+E \mid E * E \mid id \backslash$

use:

$\backslash E \rightarrow E+T \mid T \backslash$

$\backslash T \rightarrow T * F \mid F \backslash$

$\backslash F \rightarrow (E) \mid id \backslash$

Here the levels are:

TEXT

```
E !'  a d d i t i o n   l e v e l
T !'  m u l t i p l i c a t i o n   l e v e l
F !'  a t o m i c / f a c t o r   l e v e l
```

Because multiplication occurs deeper in the grammar, it gets higher precedence.

Memory Trick

Deeper grammar level = tighter binding.

So:

$\backslash [F > T > E \backslash]$

in binding strength.

11. Why This Grammar Gives * Higher Precedence

For:

$\backslash [\text{id} + \text{id} * \text{id} \backslash]$

the parse structure becomes:

TEXT



The multiplication is grouped inside **T**.

Hence:

$\backslash [\boxed{\text{id} + (\text{id} * \text{id})} \backslash]$

12. Encoding Left Associativity

Consider:

$\backslash [E \rightarrow E + T \mid T \backslash]$

The recursive occurrence of **E** is on the **left**.

This naturally produces left-associative grouping.

For:

$\backslash [\text{id} + \text{id} + \text{id} \backslash]$

we get:

$\backslash [(\text{id} + \text{id}) + \text{id} \backslash]$

General intuition

TEXT

A !' A o p B

tends to encode **left associativity**.

13. Encoding Right Associativity

A grammar such as:

$E \rightarrow T + E \mid T$

recurses to the right.

It tends to produce:

$\text{id} + (\text{id} + \text{id})$

Thus:

TEXT

Left recursive structure !' left associativity
Right recursive structure !' right associativity

Be careful: this is about grammar structure, not every possible recursive grammar.

14. Ambiguous Grammar vs Ambiguous Language

This distinction is important.

Ambiguous Grammar

A particular grammar gives multiple parse trees for some string.

Inherently Ambiguous Language

A context-free language for which **every possible CFG** describing it is ambiguous.

Therefore:

$\boxed{\text{Ambiguous grammar does NOT imply inherently ambiguous language}}$

A language may have:

- one ambiguous grammar
- another unambiguous grammar

Expression grammars are a classic example.

GATE TRAP

If a grammar is ambiguous, can we always conclude that its language is inherently ambiguous?

'L No.

15. Dangling Else — Classic Ambiguity

Consider:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$\backslash S \rightarrow \text{other} \backslash$

Now consider:

TEXT

```
if E1 then if E2 then S1 else S2
```

Question:

Which if does the else belong to?

Possibility 1:

TEXT

```
if E1 then
  (if E2 then S1 else S2)
```

Possibility 2:

TEXT

```
(if E1 then
  if E2 then S1)
else S2
```

This is the famous:

$\backslash \boxed{\text{Dangling-else ambiguity}} \backslash$

Many programming languages resolve it by associating an `else` with the **nearest unmatched** `if`.

16. Left Recursion

Now forget ambiguity for a moment.

CONCEPT

A grammar is **left recursive** if some non-terminal can derive a sentential form beginning with itself.

Formally:

$\backslash \boxed{A \rightarrow^+ A\alpha} \backslash$

for some $\backslash(\alpha)\backslash$.

The simplest case is:

$\backslash A \rightarrow A\alpha \backslash$

Example

$\backslash E \rightarrow E+T \mid T \backslash$

`E` immediately appears as the leftmost symbol on its own RHS:

$\backslash E \rightarrow E+T \backslash$

Therefore this grammar is left recursive.

17. Why Is Left Recursion a Problem?

It is especially problematic for **top-down recursive-descent parsing**.

Suppose we implement:

$E \rightarrow E + T \mid T$

as a function `E()`.

Conceptually:

```
TEXT
E()
  calls E()
    calls E()
      calls E()
        ...
```

The parser can recurse forever without consuming any input.

Thus:

$\boxed{\text{Left recursion must be removed for ordinary predictive top-down parsing}}$

18. Left Recursion Is NOT Necessarily Ambiguity

Very important.

Grammar:

$E \rightarrow E + T \mid T$

can be left recursive yet unambiguous when completed appropriately.

Therefore:

$\boxed{\text{Left recursion} \not\rightarrow \text{ambiguity}}$

Similarly:

$\boxed{\text{Ambiguity} \not\rightarrow \text{left recursion}}$

They are different properties.

19. Immediate Left Recursion

A grammar has **immediate/direct left recursion** if:

$A \rightarrow A\alpha$

appears directly.

Example:

$A \rightarrow Aa \mid b$

Here:

$\rightarrow Aa$

causes immediate left recursion.

20. Formula for Eliminating Immediate Left Recursion

This is a major note formula.

Suppose:

$\rightarrow A\alpha\mid\beta$

where β does **not** begin with A .

Transform it into:

$\rightarrow \boxed{A\rightarrow\beta A'}$

$\rightarrow \boxed{A\rightarrow\alpha A'\mid\epsilon}$

Memory Trick

Original:

$\rightarrow A\alpha\mid\beta$

Move the recursive part into new A' :

TEXT

$A \rightarrow \alpha A' \mid \beta$
 $A' \rightarrow \alpha A' \mid \epsilon$

Think:

Start with α , repeat ϵ .

21. Why Does the Formula Work?

Original grammar:

$\rightarrow A\mid b$

generates:

TEXT

b
 ba
 baa
 $baaa$
 $\dots$

because you eventually must choose b , after using A zero or more times.

Transformed grammar:

$\rightarrow bA'$

$A \rightarrow aA \mid \epsilon$

also generates:

TEXT

b
ba
baa
baaa
...

Therefore the language is preserved.

22. Example — Remove Left Recursion

Given:

$E \rightarrow E+T \mid T$

Identify:

$A=E$

$\alpha=+T$

$\beta=T$

Apply formula:

$E \rightarrow TE'$

$E \rightarrow +TE' \mid \epsilon$

Final answer

$\boxed{E \rightarrow TE'}$

$\boxed{E \rightarrow +TE' \mid \epsilon}$

23. Another Example

Given:

$A \rightarrow Aa \mid b$

Here:

$\alpha=a$

$\beta=b$

Therefore:

$A \rightarrow bA'$

$A \rightarrow aA' \mid \epsilon$

24. Multiple Left-Recursive Alternatives

Suppose:

$\boxed{A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2}$

Then:

$\boxed{A \rightarrow \beta_1 A \mid \beta_2 A}$

and:

$\boxed{A \rightarrow \alpha_1 A \mid \alpha_2 A \mid \epsilon}$

25. Worked Example with Multiple Alternatives

Given:

$A \rightarrow Aa \mid Ab \mid c \mid d$

Recursive portions:

TEXT

a
b

Non-recursive portions:

TEXT

c
d

Transform:

$A \rightarrow cA \mid dA$

$A \rightarrow aA \mid bA \mid \epsilon$

26. GATE Shortcut for Immediate Left Recursion

Whenever you see:

$A \rightarrow A(\text{something}) \mid (\text{nonrecursive})$

do:

TEXT

A ! ' n o n r e c u r s i v e A '
A ' ! ' s o m e t h i n g A ' | μ

If multiple alternatives exist, copy each one appropriately.

27. Indirect Left Recursion

Left recursion may not appear immediately.

Example:

$A \rightarrow B\alpha$

$\text{B} \rightarrow \text{A} \mid \gamma$

At first glance:

TEXT

A does not directly start with A.

But substitute B :

$\text{A} \rightarrow \text{B} \alpha \rightarrow \text{A} \beta \alpha$

So:

$\text{A} \rightarrow^+ \text{A}(\beta \alpha)$

Therefore the grammar is **indirectly left recursive**.

28. Simple Example of Indirect Left Recursion

$\text{A} \rightarrow \text{B} \mid c$

$\text{B} \rightarrow \text{A} \mid d$

From:

$\text{A} \rightarrow \text{B} \alpha$

and:

$\text{B} \rightarrow \text{A} \beta$

we get:

$\text{A} \rightarrow \text{A} \beta \alpha$

Therefore A derives something starting with itself.

So indirect left recursion exists.

29. Eliminating Indirect Left Recursion — Idea

General algorithm:

1. Order non-terminals:

$A_1, A_2, \dots, A_n$

2. For each (A_i) , replace productions beginning with earlier (A_j) by substituting (A_j) 's productions.
3. After substitution exposes immediate left recursion, eliminate it using the standard formula.

For GATE, the most important skill is usually:

- detecting indirect left recursion
- performing small substitutions correctly

rather than memorizing pseudocode mechanically.

30. Example — Eliminate Indirect Left Recursion

Given:

$A \rightarrow B \mid c$

$B \rightarrow Ab \mid d$

Let's substitute the productions of **A** into:

$B \rightarrow Ab$

Since:

$A \rightarrow B \mid c$

we obtain:

$B \rightarrow Bab \mid cb \mid d$

Now:

$B \rightarrow Bab$

is immediate left recursion.

So:

$B \rightarrow cbB' \mid dB'$

$B' \rightarrow abB' \mid \epsilon$

and retain:

$A \rightarrow B \mid c$

The exact order chosen matters during the systematic transformation, but the core method is:

$\boxed{\text{Substitute first ! ' expose direct recursion ! ' eliminate it}}$

31. Left Factoring

Now consider a completely different issue.

Grammar:

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

Both alternatives begin with the same prefix:

α

A predictive parser reading input cannot choose between the alternatives until it has seen more input.

We factor out the common prefix.

32. Left-Factoring Formula

Given:

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

transform to:

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2$

Memory Trick

Think ordinary algebra:

$$\alpha\beta_1 + \alpha\beta_2 = \alpha(\beta_1 + \beta_2)$$

Same idea.

Left factoring = take common left prefix outside.

33. Example

Given:

$$A \rightarrow ab \mid ac$$

Common prefix:

$$a$$

So:

$$A \rightarrow aA'$$

$$A' \rightarrow b \mid c$$

34. Another Example

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S$$

Common prefix:

TEXT

if E then S

Left-factor:

$$S \rightarrow \text{if } E \text{ then } S' \mid$$

$$S' \rightarrow \text{else } S \mid \epsilon$$

This delays the decision until the parser knows whether `else` follows.

Important Warning

Left factoring the dangling-else grammar does **not by itself necessarily remove its ambiguity**.

This is a major conceptual point.

Left factoring improves predictive decision-making by removing common prefixes.

It does not automatically guarantee:

- unambiguity
- LL(1)
- absence of left recursion

These are separate properties.

35. Why Left Factoring Helps Predictive Parsing

Consider:

$A \rightarrow ab \mid ac$

Input starts with:

TEXT

a

After seeing only **a**, which production should the parser choose?

Both begin with **a**.

So it cannot decide.

After factoring:

$A \rightarrow aA'$

$A' \rightarrow b \mid c$

the parser first matches **a**.

Then:

TEXT

next symbol b !' choose b
next symbol c !' choose c

Decision becomes easier.

36. Left Recursion vs Left Factoring

This comparison must go into your notes.

Left Recursion	Left Factoring
Same non-terminal can appear at left edge of its derivation	Alternatives share common left prefix
Can cause infinite recursion in top-down parser	Causes difficulty choosing an alternative
Example: $A \rightarrow Aa \mid b$	Example: $A \rightarrow ab \mid ac$
Remove using $A \rightarrow A'$ recursion transformation	Extract common prefix
Changes recursion form	Delays parser decision

Memory Trick

Left Recursion

I call myself first.

$\lceil A \rightarrow A \dots \rceil$

Left Factoring

My choices start the same.

$\lceil A \rightarrow \alpha \dots \mid \alpha \dots \rceil$

37. Neither Transformation Automatically Makes Grammar LL(1)

Very important.

A grammar may have:

' no left recursion ' be left factored

and still **not be LL(1)**.

Why?

There can still be FIRST/FOLLOW conflicts.

We'll learn the exact LL(1) conditions after FIRST/FOLLOW.

Thus:

$\lceil \boxed{\text{No left recursion} + \text{left factored}} \not\rightarrow \text{LL}(1) \rceil$

They are useful preparatory conditions, not sufficient conditions.

38. Does Removing Left Recursion Preserve Associativity?

This is a subtle compiler-design point.

Start with:

$\lceil E \rightarrow E + T \mid T \rceil$

This parse grammar encodes left associativity.

After eliminating left recursion:

$\lceil E \rightarrow TE' \rceil$

$\lceil E' \rightarrow +TE' \mid \epsilon \rceil$

the new grammar's parse-tree shape becomes more right-recursive.

However, a compiler can preserve the desired left-associative semantics through syntax-directed translation/AST construction.

GATE takeaway

The transformation preserves the **language**, but parse-tree structure can change.

Do not assume transformed grammar has identical parse trees.

39. Language Equivalence vs Grammar Equivalence

After transformations like:

- removing left recursion
- left factoring

we want the generated **language** to remain the same.

That is:

$L(G_1) = L(G_2)$

But the grammars themselves and their parse trees may be different.

REMEMBER

Same language does not mean same grammar or same parse trees.

40. Can Every Ambiguous CFG Be Made Unambiguous?

No.

Some context-free languages are **inherently ambiguous**.

For those languages, no equivalent unambiguous CFG exists.

Therefore:

$\boxed{\text{Not every ambiguity can be eliminated by rewriting the grammar}}$

For ordinary programming-language expression grammars, ambiguity is often deliberately removed using precedence/associativity rules or grammar restructuring.

GATE TRAPS

Trap 1 — Definition of Ambiguity

A grammar is ambiguous if it generates more than one string.

'LFalse.

Almost every useful grammar generates many strings.

Ambiguity means:

$\boxed{\text{same string ! ' multiple parse trees}}$

Trap 2 — Two Different Derivations

Two arbitrary derivations for the same string prove ambiguity.

'LFalse.

Use:

- two leftmost derivations, or

- two rightmost derivations, or
 - two parse trees.
-

Trap 3 — Left Recursion

Every left-recursive grammar is ambiguous.

'L False.

Left recursion and ambiguity are independent properties.

Trap 4 — Removing Left Recursion

Removing left recursion necessarily changes the language.

'L False when the transformation is performed correctly.

It preserves the generated language.

Trap 5 — Left Factoring

Left factoring removes left recursion.

'L Not necessarily.

They solve different issues.

Trap 6 — Left Factoring Removes Ambiguity

'L Not necessarily.

It removes common-prefix nondeterminism; ambiguity may remain.

Trap 7 — No Left Recursion Means LL(1)

'L False.

FIRST/FOLLOW conflicts can still prevent LL(1).

Trap 8 — Left Factored Means LL(1)

'L False.

Again, exact LL(1) conditions depend on FIRST/FOLLOW sets.

Trap 9 — Precedence vs Associativity

They mean the same thing.

'L False.

TEXT

P r e c e d e n c e ! ' w h i c h o p e r a t o r b i n d s s t r o n g e r
A s s o c i a t i v i t y ! ' g r o u p i n g a m o n g s a m e - l e v e l o p e r a t o r s

Trap 10 — Ambiguous Grammar ! inherently Ambiguous Language

'L False.

Another unambiguous grammar may generate the same language.

41. GATE-STYLE EXAMPLE — Ambiguity

Grammar:

$E \rightarrow E + E \mid E * E \mid id$

Is it ambiguous?

FAST SOLUTION

Take:

$id + id * id$

It can mean:

$(id + id) * id$

or:

$id + (id * id)$

Therefore two parse trees exist.

$\boxed{\text{Ambiguous}}$

42. GATE-STYLE EXAMPLE — Associativity

Grammar:

$E \rightarrow E - T \mid T$

For:

$a - b - c$

the recursive structure gives:

$\boxed{(a - b) - c}$

Thus subtraction is represented as left associative.

43. GATE-STYLE EXAMPLE — Remove Left Recursion

Given:

$A \rightarrow Aa \mid b$

Step 1

Identify:

$\alpha = a, \quad \beta = b$

Step 2

Apply:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

Result

$\boxed{A \rightarrow \beta A'}$

$\boxed{A' \rightarrow \alpha A' \mid \epsilon}$

44. GATE-STYLE Example — Multiple ² Alternatives

Given:

$A \rightarrow Aa \mid b \mid c$

Recursive:

Aa

Nonrecursive:

b, c

Therefore:

$A \rightarrow \beta A' \mid cA'$

$A' \rightarrow \alpha A' \mid \epsilon$

45. GATE-STYLE EXAMPLE — Left Factoring

Given:

$S \rightarrow aAB \mid aAC \mid d$

Common prefix among first two alternatives:

aA

Therefore:

$S \rightarrow aAS' \mid d$

$S' \rightarrow B \mid C$

46. Deeper Left Factoring Example

Given:

$A \rightarrow abc \mid abd \mid ae \mid f$

First identify common prefix among:

TEXT

abc
abd

Common:

$\backslash[ab \backslash]$

Factor:

$\backslash[A \rightarrow abA_1 \mid ae \mid f \backslash]$

$\backslash[A_1 \rightarrow c \mid d \backslash]$

But notice:

TEXT

abA1
ae

still share **a**.

So factor again:

$\backslash[A \rightarrow aA_2 \mid f \backslash]$

$\backslash[A_2 \rightarrow bA_1 \mid e \backslash]$

$\backslash[A_1 \rightarrow c \mid d \backslash]$

GATE Lesson

Left factoring may require **multiple rounds**.

47. How to Spot the Three Problems Fast

Given a grammar, ask these questions in order.

Question 1

Can one string have multiple parse trees?

!' **Ambiguity**

Question 2

Can:

$\backslash[A \rightarrow^+ A \alpha \backslash]$

occur?

!' **Left recursion**

Question 3

Do alternatives for the same non-terminal start with a common prefix?

!' **Left factoring needed**

48. Master Comparison

Property	Main Question
Ambiguity	Can one string have multiple parse trees?
Left recursion	Can a non-terminal derive itself at left edge?
Left factoring	Do multiple choices have same initial prefix?
Precedence	Which operator binds tighter?
Associativity	How do equal-precedence operators group?

WRITE THIS IN NOTES — TRANSFORMATION FORMULAS

Immediate Left Recursion

Before:

$A \rightarrow A\alpha \mid \beta$

After:

$A \rightarrow \beta A'$

$A' \rightarrow A\alpha \mid \epsilon$

General Immediate Left Recursion

Before:

$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$

After:

$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$

$A' \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \epsilon$

Left Factoring

Before:

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

After:

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2$

MINI PRACTICE — PART 6

Try these first.

Q1 — Easy

Which grammar is immediately left recursive?

A. $A \rightarrow a \mid b$ B. $A \rightarrow Aa \mid b$ C. $A \rightarrow a \mid b$ D. $A \rightarrow aB$

Q2 — Easy

Left-factor:

$A \rightarrow ab \mid ac$

Q3 — Medium

Remove immediate left recursion:

$E \rightarrow E+T \mid T$

Q4 — Medium

Is this grammar left recursive?

$A \rightarrow Ba \mid c$

$B \rightarrow Ab \mid d$

Explain why.

Q5 — Tricky

Consider the grammar:

$S \rightarrow a \mid aB$

$A \rightarrow c$

$B \rightarrow d$

Which statement is certainly true?

A. Grammar is ambiguous. B. Grammar is left recursive. C. Grammar needs left factoring for straightforward predictive choice. D. Grammar generates only one string.

SOLUTIONS

Q1

' B

$A \rightarrow Aa$

starts directly with A.

Q2

Original:

$\text{\textbackslash}[A\rightarrow ab\mid ac \text{\textbackslash}]$

Common prefix:

$\text{\textbackslash}[a \text{\textbackslash}]$

Therefore:

$\text{\textbackslash}[\boxed{A\rightarrow aA'} \text{\textbackslash}]$

$\text{\textbackslash}[\boxed{A'\rightarrow b\mid c} \text{\textbackslash}]$

Q3

Original:

$\text{\textbackslash}[E\rightarrow E+T\mid T \text{\textbackslash}]$

Here:

$\text{\textbackslash}[\alpha=+T,\quad\beta=T \text{\textbackslash}]$

Therefore:

$\text{\textbackslash}[\boxed{E\rightarrow TE'} \text{\textbackslash}]$

$\text{\textbackslash}[\boxed{E'\rightarrow +TE'\mid\epsilon} \text{\textbackslash}]$

Q4

Yes.

$\text{\textbackslash}[A\rightarrow Ba \text{\textbackslash}]$

and:

$\text{\textbackslash}[B\rightarrow Ab \text{\textbackslash}]$

therefore:

$\text{\textbackslash}[A\rightarrow Aba \text{\textbackslash}]$

So **A** derives a sentential form beginning with itself.

$\text{\textbackslash}[\boxed{\text{Indirect left recursion}} \text{\textbackslash}]$

Q5

' **C**

Both alternatives begin with:

$\text{\textbackslash}[a \text{\textbackslash}]$

so a parser seeing only **a** cannot immediately choose between them.

Left factoring gives:

$\{ S \rightarrow aS \}$

$\{ S \rightarrow A \mid B \}$

Nothing given proves ambiguity.

CUMULATIVE CHECK — PARSING SO FAR

We now know:

TEXT

Lexer

C h a r a c t e r s ! ' T o k e n s

and:

TEXT

Parser

T o k e n s ! ' P a r s e s t r u c t u r e

For CFG:

$G = (N, T, P, S)$

Parsing concepts:

TEXT

Derivation

Leftmost derivation

Rightmost derivation

Parse tree

Sentence

Sentential form

Grammar preparation:

TEXT

Ambiguity

Precedence

Associativity

Left recursion

Left factoring

Next we reach one of the **highest-value GATE calculation topics**:

FIRST + FOLLOW

This is essential for:

- predictive parsing
- LL(1) testing
- LL(1) parsing tables
- SLR parsing later

30-SECOND REVISION

Ambiguity

$\boxed{\text{Same string !' two parse trees}}$

Proof shortcut:

Find one string with two leftmost derivations.

Precedence

Which operator binds stronger?

Associativity

How same-level operators group?

Immediate left recursion

$A \rightarrow A\alpha \mid \beta$

remove as:

$\boxed{A \rightarrow \beta A'}$

$\boxed{A' \rightarrow \alpha A' \mid \epsilon}$

Memory:

Start ϵ , repeat α .

Indirect left recursion

$A \rightarrow A^i \alpha$

through other non-terminals.

Shortcut:

Substitute !' expose direct recursion !' eliminate.

Left factoring

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

becomes:

$\boxed{A \rightarrow \alpha A'}$

$\boxed{A' \rightarrow \beta_1 \mid \beta_2}$

Memory:

Take common left prefix outside.

Critical distinction

$\boxed{\text{Ambiguity} \neq \text{Left Recursion} \neq \text{Left Factoring}}$

And:

$\boxed{\text{No left recursion} + \text{left factored} \not\rightarrow \text{LL}(1)}$

We need **FIRST and FOLLOW** to determine LL(1).

Completed: Parts 1–6/31 | Next: Part 7 — NULLABLE, FIRST & FOLLOW

Reply **NEXT** for **Part 7**, where we'll learn the complete **FIRST/FOLLOW algorithm from zero**, including ϵ , sequences such as `ABC`, all rules, shortcuts, and several step-by-step GATE problems.

USER

NEXT

ASSISTANT

Compiler Design — Part 7: NULLABLE, FIRST & FOLLOW

Completed: Parts 1–6/31 | Current: Part 7 | Remaining: Parts 8–31

This is one of the **highest-value calculation topics in GATE Compiler Design**.

FIRST and FOLLOW are required for:

- predictive parsing
- LL(1) grammar checking
- LL(1) parsing-table construction
- SLR parsing later

The biggest mistake students make is memorizing rules without understanding **why** they work.

So first remember the intuition:

$\boxed{\text{FIRST} = \text{What can appear first?}}$

$\boxed{\text{FOLLOW} = \text{What can appear immediately after?}}$

1. Before FIRST: What Is NULLABLE?

CONCEPT

A non-terminal (A) is **nullable** if it can derive the empty string.

Formally:

$\boxed{A \rightarrow^* \epsilon}$

Example:

$\text{A} \rightarrow a \mid \epsilon$

Since:

$\text{A} \rightarrow \epsilon$

we say:

$\boxed{\text{A} \text{ text{ is nullable}}}$

2. Nullable Can Be Indirect

Consider:

$\text{A} \rightarrow \text{BC}$

$\text{B} \rightarrow \epsilon$

$\text{C} \rightarrow \epsilon$

Then:

$\text{A} \rightarrow \text{BC} \rightarrow \text{C} \rightarrow \epsilon$

Therefore:

$\boxed{\text{A, B, C text{ are nullable}}}$

So nullable does **not** require a direct production:

$\text{A} \rightarrow \epsilon$

3. Nullable Example

Grammar:

$\text{S} \rightarrow \text{AB}$

$\text{A} \rightarrow a \mid \epsilon$

$\text{B} \rightarrow b$

Which non-terminals are nullable?

A

$\text{A} \rightarrow \epsilon$

' Nullable.

B

$\text{B} \rightarrow b$

'L Not nullable.

S

For:

$\text{S} \rightarrow \text{AB}$

both A and B would have to disappear.

But B cannot derive ϵ .

Therefore:

'L S is not nullable.

Memory Trick

For:

$A \rightarrow XYZ$

A can become ϵ through this production only if:

$\boxed{X, Y, Z \text{ are ALL nullable}}$

Think:

For a sequence to vanish, everybody must vanish.

4. What Is FIRST?

CONCEPT

$\text{FIRST}(\alpha)$ is the set of terminals that can appear as the **first symbol** of some string derived from α .

If α itself can derive ϵ , then ϵ is also included.

Formally:

$\text{FIRST}(\alpha) = \{a \mid \alpha \rightarrow^* a\beta\}$

and:

$\epsilon \in \text{FIRST}(\alpha)$

if:

$\alpha \rightarrow^* \epsilon$

Simple Intuition

If:

$A \rightarrow^* aXYZ$

then:

$a \in \text{FIRST}(A)$

because a string derived from A can begin with a.

WRITE THIS IN NOTES

FIRST(X): terminals that may appear at the beginning of strings derived from X. Include ϵ if X can derive the empty string.

Memory:

FIRST = Who can come FIRST?

5. FIRST of a Terminal

If a is a terminal:

$$\boxed{\text{FIRST}(a) = \{a\}}$$

Example:

$$\text{FIRST}(+) = \{+\}$$

$$\text{FIRST}(\text{id}) = \{\text{id}\}$$

Very simple.

6. FIRST of ϵ

$$\boxed{\text{FIRST}(\epsilon) = \{\epsilon\}}$$

7. FIRST of a Non-Terminal

Suppose:

$$A \rightarrow aB \mid cD$$

Then strings from A can begin with:

TEXT

a
or
c

Therefore:

$$\boxed{\text{FIRST}(A) = \{a, c\}}$$

8. FIRST with μ Production

Suppose:

$$A \rightarrow aA \mid \epsilon$$

Then:

$\boxed{\text{FIRST}(A) = \{a, \epsilon\}}$

Why?

A can derive a string beginning with a, or disappear completely.

9. The Most Important FIRST Rule: Sequence of Symbols

Suppose:

$A \rightarrow XYZ$

How do we calculate:

$\text{FIRST}(XYZ)?$

This is where most GATE questions become interesting.

10. FIRST of (XYZ)

Start with X.

Case 1 — X is NOT nullable

Then every string generated from XYZ must begin with something generated by X.

Therefore:

$\text{FIRST}(XYZ) = \text{FIRST}(X)$

excluding ϵ , if relevant.

Case 2 — X is nullable

Then X may disappear:

$XYZ \rightarrow YZ$

So a string can begin with:

- something in $\text{FIRST}(X) - \{\epsilon\}$, or
 - something from Y.
-

Case 3 — X and Y are both nullable

Then both can disappear:

$XYZ \rightarrow Z$

So we must also consider $\text{FIRST}(Z)$.

Case 4 — X, Y and Z are ALL nullable

Then:

$XYZ \rightarrow \epsilon$

Therefore:

$\epsilon \in \text{FIRST}(XYZ)$

MASTER FIRST SEQUENCE RULE

For:

$X_1 X_2 \dots X_n$

1. Add:

$\text{FIRST}(X_1) - \epsilon$

2. If X_1 is nullable, also add:

$\text{FIRST}(X_2) - \epsilon$

3. Continue while previous symbols are nullable.

4. Add ϵ **only if every** X_i is nullable.

Memory Trick

FIRST walks left to right only while symbols can disappear.

This single sentence solves most FIRST problems.

11. Worked Example — FIRST of a Sequence

Suppose:

$\text{FIRST}(A) = \{a, \epsilon\}$

$\text{FIRST}(B) = \{b, \epsilon\}$

$\text{FIRST}(C) = \{c\}$

Find:

$\text{FIRST}(ABC)$

Step 1 — A

Add:

a

Since A is nullable, continue.

Step 2 — B

Add:

b

Since B is nullable, continue.

Step 3 — C

Add:

c

C is not nullable, so stop.

Therefore:

$$\boxed{\text{FIRST}(ABC) = \{a, b, c\}}$$

Do we add ϵ ?

No.

Because C is not nullable.

12. Same Example if C Is Also Nullable

Suppose:

$$\boxed{\text{FIRST}(C) = \{c, \epsilon\}}$$

Now every symbol can vanish.

Therefore:

$$\boxed{\text{FIRST}(ABC) = \{a, b, c, \epsilon\}}$$

13. FIRST Example 1

Grammar:

$$S \rightarrow AB$$

$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow b$$

Calculate FIRST.

FIRST(A)

$$\boxed{\text{FIRST}(A) = \{a, \epsilon\}}$$

FIRST(B)

$$\boxed{\text{FIRST}(B) = \{b\}}$$

FIRST(S)

Since:

$$S \rightarrow AB$$

Start with A.

Add:

$$a$$

Since A can become ϵ , inspect B.

Add:

$$b$$

B cannot become ϵ .

Therefore:

$$\boxed{\text{FIRST}(S) = \{a, b\}}$$

14. FIRST Example 2

Grammar:

$$S \rightarrow ABC$$

$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow b \mid \epsilon$$

$$C \rightarrow c \mid \epsilon$$

Then:

$$\text{FIRST}(A) = \{a, \epsilon\}$$

$$\text{FIRST}(B) = \{b, \epsilon\}$$

$$\text{FIRST}(C) = \{c, \epsilon\}$$

All three are nullable.

Therefore:

$$\boxed{\text{FIRST}(S) = \{a, b, c, \epsilon\}}$$

15. FIRST Example 3 — Multiple Productions

Grammar:

$$S \rightarrow aA \mid bB \mid c$$

Then:

$$\text{FIRST}(aA) = \{a\}$$

$$\text{FIRST}(bB) = \{b\}$$

$$\text{FIRST}(c) = \{c\}$$

Hence:

$$\boxed{\text{FIRST}(S) = \{a, b, c\}}$$

16. FIRST Example 4 — Recursive Grammar

Consider the familiar expression grammar after removing left recursion:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

Let's calculate FIRST sets.

FIRST(F)

From:

$F \rightarrow (E) \mid id$

Therefore:

$\boxed{\text{FIRST}(F) = \{ (, id \}}$

FIRST(T')

$T' \rightarrow *FT' \mid \epsilon$

Therefore:

$\boxed{\text{FIRST}(T') = \{ *, \epsilon \}}$

FIRST(T)

$T \rightarrow FT'$

F is not nullable.

Therefore:

$\text{FIRST}(T) = \text{FIRST}(F)$

Hence:

$\boxed{\text{FIRST}(T) = \{ (, id \}}$

FIRST(E')

$E' \rightarrow +TE' \mid \epsilon$

Therefore:

$\boxed{\text{FIRST}(E') = \{ +, \epsilon \}}$

FIRST(E)

$E \rightarrow TE'$

T is not nullable.

Therefore:

$\boxed{\text{FIRST}(E) = \{ (, id \}}$

17. FIRST GATE Shortcut

For each RHS:

Start at the left edge.

If the first symbol cannot become ϵ :

STOP.

If it can become ϵ :

MOVE RIGHT.

Continue until a non-nullable symbol is encountered.

Only include ϵ if **all symbols disappear**.

18. What Is FOLLOW?

Now the second major concept.

CONCEPT

For a non-terminal A , $FOLLOW(A)$ is the set of terminals that may appear **immediately to the right of A** in some sentential form.

Formally:

If:

$S \xRightarrow{\alpha} A \beta$

then:

$\alpha \in FOLLOW(A)$

Simple Intuition

Suppose somewhere during a derivation we can get:

TEXT

... A + ...

Then:

$+ \in FOLLOW(A)$

because **+** can immediately follow **A**.

WRITE THIS IN NOTES

$FOLLOW(A)$: set of terminals that can appear immediately after non-terminal A in a sentential form.

Memory:

$FOLLOW$ = Who can come after me?

19. FOLLOW Is Defined for Non-Terminals

Usually we calculate:

$\text{FOLLOW}(A)$

for non-terminals.

We don't normally calculate $\text{FOLLOW}(+)$, $\text{FOLLOW}(\text{id})$, etc. in parsing-table algorithms.

20. The End Marker $\$$

A special symbol:

$\$$

represents the **end of input**.

For the grammar's start symbol S :

$\$ \in \text{FOLLOW}(S)$

Always.

This is the first FOLLOW rule.

Memory Trick

Start symbol can be followed by the end of input.

Therefore:

$\$ \in \text{FOLLOW}(\text{Start})$

21. FOLLOW Rule 1

If S is the start symbol:

$\$ \in \text{FOLLOW}(S)$

22. FOLLOW Rule 2 — Terminal Comes After Non-Terminal

Suppose:

$A \rightarrow \alpha B \beta$

where a is a terminal.

Then:

$a \in \text{FOLLOW}(B)$

Example:

$S \rightarrow A + b$

Then immediately after A comes $+$.

Therefore:

$$\boxed{+ \in \text{FOLLOW}(A)}$$

23. FOLLOW Rule 3 — Non-Terminal Comes After Non-Terminal

Suppose:

$$A \rightarrow \alpha B C \beta$$

Then what can follow B ?

Anything that can **begin** strings generated by C .²

Therefore:

$$\boxed{\text{FIRST}(C\beta) - \{\epsilon\} \subseteq \text{FOLLOW}(B)}$$

This is the central FIRST/FOLLOW connection.

24. Example

Grammar:

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Since after A comes B :

$$\text{FIRST}(B) = \{b\}$$

Therefore:

$$\boxed{b \in \text{FOLLOW}(A)}$$

25. FOLLOW Rule 4 — Suffix Can Vanish

Suppose:

$$A \rightarrow \alpha B \beta$$

and:

$$\beta \xrightarrow{\epsilon}$$

Then B may effectively appear at the end of the RHS.

Whatever can follow A can then follow B .

Therefore:

$$\boxed{\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)}$$

26. Special Case: B at End of Production

If:

$$A \rightarrow \alpha B$$

then there is nothing after B .

Therefore:

$$\boxed{\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)}$$

This is one of the most-used rules.

Memory Trick for FOLLOW

Suppose:

$$A \xrightarrow{\alpha} B\beta$$

Ask:

What is after B ?

Case 1: Something remains (β)

Put:

$$\text{FIRST}(\beta - \epsilon)$$

into $\text{FOLLOW}(B)$.

Case 2: ϵ can disappear

Also copy:

$$\text{FOLLOW}(A)$$

into $\text{FOLLOW}(B)$.

Shortcut phrase

Look RIGHT. FIRST of right side. If right side vanishes, inherit FOLLOW of left side.

This is the master FOLLOW rule.

27. MASTER FOLLOW FORMULA

For production:

$$A \xrightarrow{\alpha} B\beta$$

add:

$$\boxed{\text{FIRST}(\beta - \epsilon) \subseteq \text{FOLLOW}(B)}$$

and if:

$$\epsilon \in \text{FIRST}(\beta)$$

then also add:

$$\boxed{\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)}$$

If B is already at the RHS end, treat $\beta = \epsilon$.

28. Worked FOLLOW Example 1

Grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Start symbol:

S

FOLLOW(S)

Always:

$\boxed{\text{FOLLOW}(S) = \{\$ \}}$

FOLLOW(A)

From:

$S \rightarrow AB$

after **A** comes **B**.

$\text{FIRST}(B) = \{b\}$

Thus:

$\boxed{\text{FOLLOW}(A) = \{b\}}$

FOLLOW(B)

B is at the end of:

$S \rightarrow AB$

Therefore:

$\text{FOLLOW}(S) \subseteq \text{FOLLOW}(B)$

Hence:

$\boxed{\text{FOLLOW}(B) = \{\$, b\}}$

29. Worked FOLLOW Example 2 — Nullable Symbol

Grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b \mid \epsilon$

Calculate FOLLOW(A).

After **A** is **B**.

First add:

$\text{FIRST}(B) = \{\epsilon\}$

Since:

$$\text{FIRST}(B) = \{b, \epsilon\}$$

add:

$$b$$

But B is nullable.

Therefore whatever follows S can also follow A .

Since:

$$\text{FOLLOW}(S) = \{\$, \epsilon\}$$

we also add $\$$.

Thus:

$$\boxed{\text{FOLLOW}(A) = \{b, \$\}}$$

30. Why $\$$ Appears in FOLLOW(A)

Because:

$$B \rightarrow \epsilon$$

allows:

$$S \rightarrow AB \rightarrow A$$

So A can be the last symbol of a complete sentence.

Therefore end-of-input can occur immediately after it.

Hence:

$$\$ \in \text{FOLLOW}(A)$$

31. Worked Example 3 — Longer Suffix

Grammar:

$$S \rightarrow ABC$$

Suppose:

$$\text{FIRST}(B) = \{b, \epsilon\}$$

$$\text{FIRST}(C) = \{c\}$$

What can follow A ?

Do **not** merely take $\text{FIRST}(B)$.

We need:

$$\text{FIRST}(BC)$$

Because B may disappear.

Since:

$$\text{FIRST}(B) = \{b, \epsilon\}$$

and C starts with c :

$$\text{FIRST}(BC) = \{b, c\}$$

Therefore:

$$\boxed{\{b, c\} \subseteq \text{FOLLOW}(A)}$$

This is a common GATE trap.

32. FOLLOW Must Use FIRST of the ENTIRE Suffix

For:

$$X \rightarrow ABCD$$

to calculate $\text{FOLLOW}(A)$, inspect:

$$BCD$$

not only B .

Use:

$$\boxed{\text{FIRST}(BCD) - \{\epsilon\}}$$

Then if all of B, C, D are nullable, also add:

$$\text{FOLLOW}(X)$$

GATE Shortcut

For a non-terminal inside a production:

Scan everything to its right until a non-nullable symbol stops you.

Exactly like FIRST computation.

33. Full Example — FIRST and FOLLOW Together

Consider:

$$S \rightarrow AB$$

$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow b \mid c$$

Let's solve from scratch.

Step 1 — FIRST(A)

$$A \rightarrow a \mid \epsilon$$

Therefore:

$$\boxed{\text{FIRST}(A) = \{a, \epsilon\}}$$

Step 2 — FIRST(B)

$[B \rightarrow bB \mid c]$

Therefore:

$\boxed{\text{FIRST}(B) = \{b, c\}}$

B is not nullable.

Step 3 — FIRST(S)

$[S \rightarrow AB]$

Start with A.

Add:

$[a]$

A is nullable, so inspect B.

Add:

$[b, c]$

Since B is not nullable, stop.

Thus:

$\boxed{\text{FIRST}(S) = \{a, b, c\}}$

Step 4 — FOLLOW(S)

S is start symbol:

$\boxed{\text{FOLLOW}(S) = \{ \$ \}}$

Step 5 — FOLLOW(A)

From:

$[S \rightarrow AB]$

after A comes B.

Since:

$\text{FIRST}(B) = \{b, c\}$

we get:

$\boxed{\text{FOLLOW}(A) = \{b, c\}}$

Step 6 — FOLLOW(B)

B occurs at the end of:

$\rightarrow AB$

Therefore copy:

$\text{FOLLOW}(S)$

into $\text{FOLLOW}(B)$.

Thus:

$\boxed{\text{FOLLOW}(B) = \{ \$ \}}$

34. Expression Grammar — Full FIRST/FOLLOW Example

This is worth mastering because it is the standard LL(1) grammar we'll use later:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

Start symbol:

E

35. FIRST Sets

We already know:

$\boxed{\text{FIRST}(E) = \{ (, id \}}$

$\boxed{\text{FIRST}(E') = \{ +, \epsilon \}}$

$\boxed{\text{FIRST}(T) = \{ (, id \}}$

$\boxed{\text{FIRST}(T') = \{ *, \epsilon \}}$

$\boxed{\text{FIRST}(F) = \{ (, id \}}$

Now FOLLOW.

36. FOLLOW(E)

Since E is start symbol:

$\$ \in \text{FOLLOW}(E)$

Also from:

$F \rightarrow (E)$

immediately after E is:

$)$

So:

$\epsilon \in \text{FOLLOW}(E)$

Therefore:

$\boxed{\text{FOLLOW}(E) = \{\epsilon, \$\}}$

37. FOLLOW(E')

From:

$E \rightarrow TE'$

E' occurs at the end.

Therefore:

$\text{FOLLOW}(E) \subseteq \text{FOLLOW}(E')$

Hence:

$\boxed{\text{FOLLOW}(E') = \{\epsilon, \$\}}$

38. FOLLOW(T)

Look at:

$E \rightarrow TE'$

After T comes E' .

$\text{FIRST}(E') = \{+, \epsilon\}$

First add:

$\{+\}$

Since E' is nullable, also add:

$\text{FOLLOW}(E) = \{\epsilon, \$\}$

Therefore:

$\boxed{\text{FOLLOW}(T) = \{+, \epsilon, \$\}}$

39. FOLLOW(T')

From:

$T \rightarrow FT'$

T' is at the end.

Therefore:

$\text{FOLLOW}(T) \subseteq \text{FOLLOW}(T')$

Hence:

$\boxed{\text{FOLLOW}(T') = \{+, \epsilon, \$\}}$

40. FOLLOW(F)

Look at:

$T \rightarrow FT'$

After **F** comes **T'**.

$FIRST(T') = \{\epsilon\}$

First add:

$*$

Since **T'** is nullable, add:

$FOLLOW(T) = \{+,), \$\}$

Therefore:

$\boxed{FOLLOW(F) = \{*, +,), \$\}}$

41. Final Expression Grammar Table

WRITE THIS IN NOTES

Non-terminal	FIRST	FOLLOW
(E)	$\{ (, id \}$	$\{), \$ \}$
(E')	$\{ +, \mu \}$	$\{), \$ \}$
(T)	$\{ (, id \}$	$\{ +,), \$ \}$
(T')	$\{ *, \mu \}$	$\{ +,), \$ \}$
(F)	$\{ (, id \}$	$\{ *, +,), \$ \}$

We will reuse this exact table to construct the LL(1) parsing table.

42. Why FOLLOW Needs Iteration

Sometimes FOLLOW sets depend on each other.

Example:

$S \rightarrow A$

$A \rightarrow B$

$B \rightarrow b$

Start with:

$FOLLOW(S) = \{ \$ \}$

Since **A** is at the end of:

$S \rightarrow A$

we get:

$\text{FOLLOW}(A) \supseteq \text{FOLLOW}(S)$

So:

$\text{FOLLOW}(A) = \{\$ \}$

Then from:

$A \rightarrow B$

we get:

$\text{FOLLOW}(B) \supseteq \text{FOLLOW}(A)$

So:

$\text{FOLLOW}(B) = \{\$ \}$

Information propagates through the grammar.

This is why FOLLOW calculation may require **repeated passes until no set changes**.

43. Fixed-Point Intuition

Algorithmically:

```
TEXT
Initialize FOLLOW sets
    !
Apply all FOLLOW rules
    !
Did any set change?
    ! Yes      ! No
Repeat         Stop
```

When nothing changes, we have reached a:

$\boxed{\text{fixed point}}$

You'll see this same idea again in **data-flow analysis** later.

44. Complete FIRST Algorithm

For each grammar symbol:

Rule 1

If X is terminal:

$\text{FIRST}(X) = \{X\}$

Rule 2

If:

$X \rightarrow \epsilon$

then:

$\epsilon \in \text{FIRST}(X)$

Rule 3

For:

$X \rightarrow Y_1 Y_2 \dots Y_k$

add:

$\text{FIRST}(Y_1) - \{\epsilon\}$

Then if (Y_1) is nullable, inspect (Y_2) , etc.

If every (Y_i) is nullable:

$\epsilon \in \text{FIRST}(X)$

Rule 4

Union contributions from every alternative.

45. Complete FOLLOW Algorithm

Rule 1 — Start Symbol

$\boxed{\$ \in \text{FOLLOW}(S)}$

Rule 2 — Production

For:

$A \rightarrow \alpha B \beta$

add:

$\boxed{\text{FIRST}(\beta) - \{\epsilon\}}$

to:

$\text{FOLLOW}(B)$

Rule 3 — Nullable Suffix

If:

$\beta \rightarrow^* \epsilon$

or if B is already at the end, add:

$\boxed{\text{FOLLOW}(A)}$

to:

$\text{FOLLOW}(B)$

Rule 4

Repeat until no FOLLOW set changes.

46. FIRST vs FOLLOW — Master Comparison

FIRST	FOLLOW
What can start strings from a symbol/sequence?	What can immediately come after a non-terminal?
Can include ϵ	Never includes ϵ
Defined for terminals, non-terminals, sequences	Mainly computed for non-terminals
Looks into derivations beginning at symbol	Looks at grammatical context to the right
Used heavily in predictive choices	Used especially when RHS can derive ϵ

CRITICAL FACT

$\epsilon \notin \text{FOLLOW}(A)$

for any non-terminal A .

Why?

FOLLOW contains symbols that can occur **after** A .

ϵ is not an input terminal that appears after anything.

This is a classic trap.

47. Can $\$$ Appear in FIRST?

Usually $\$$ is an end-of-input marker used by the parser, not a grammar terminal generated by ordinary productions.

So in standard FIRST/FOLLOW calculations:

TEXT

$\$$ appears in FOLLOW(start)

not FIRST sets, unless a question explicitly models $\$$ differently.

48. Common Error — Copying μ from FIRST into FOLLOW

Suppose:

$\text{FIRST}(B) = \{b, \epsilon\}$

and:

$A \rightarrow XB$

You should not put:

ϵ

into FOLLOW(X).

Instead:

1. Add b
2. Because B is nullable, add FOLLOW(A)

Thus:

$\text{FOLLOW}(X) \supseteq \text{FOLLOW}(A)$

Never add ϵ to FOLLOW.

49. Common Error — Looking Only One Symbol Right

Consider:

$A \rightarrow ABCD$

Suppose:

TEXT

B nullable

C nullable

D begins with d

To calculate $\text{FOLLOW}(A)$:

- $\text{FIRST}(B)$ contributes non-terminals
- since B nullable, inspect C
- since C nullable, inspect D
- stop if D non-nullable

So don't stop at the immediately adjacent symbol.

50. Common Error — FOLLOW of RHS Non-Terminal

Suppose:

$A \rightarrow BC$

C is at end.

Which FOLLOW is copied into $\text{FOLLOW}(C)$?

Correct:

$\text{FOLLOW}(A)$

Not $\text{FOLLOW}(B)$.

Remember:

RHS-end symbol inherits FOLLOW of LHS.

51. GATE Shortcut — FIRST

For $\text{FIRST}(XYZ)$:

TEXT

Take FIRST(X) without μ .
If X nullable !' take FIRST(Y) without μ .
If Y nullable !' take FIRST(Z) without μ .
...
If ALL nullable !' add μ .

52. GATE Shortcut — FOLLOW

For every occurrence:

$\backslash A \rightarrow \alpha B \beta \backslash$

say aloud:

"FIRST of what is right."

Then:

- add $\backslash \text{FIRST}(\beta) - \backslash \{\epsilon\} \backslash$
- if $\backslash(\beta)$ can vanish:

"FOLLOW of what is left."

Meaning:

$\backslash \text{FOLLOW}(A) \rightarrow \text{FOLLOW}(B) \backslash$

Mnemonic

FOLLOW: Right-FIRST; if right vanishes, Left-FOLLOW.

This is one of the best shortcuts to memorize.

53. GATE-STYLE EXAMPLE 1

Given:

$\backslash S \rightarrow ABC \backslash$

$\backslash \text{FIRST}(A) = \backslash \{a, \epsilon\} \backslash$

$\backslash \text{FIRST}(B) = \backslash \{b, \epsilon\} \backslash$

$\backslash \text{FIRST}(C) = \backslash \{c, d\} \backslash$

Find FIRST(S).

FAST SOLUTION

A contributes:

$\backslash a \backslash$

A nullable !' B contributes:

$\{b\}$

B nullable !' C contributes:

$\{c,d\}$

C non-nullable !' stop.

Therefore:

$\boxed{\text{FIRST}(S) = \{a,b,c,d\}}$

54. GATE-STYLE EXAMPLE 2

Grammar:

$S \rightarrow ABC$

Suppose:

$\text{FIRST}(B) = \{b, \epsilon\}$

$\text{FIRST}(C) = \{c, \epsilon\}$

Find what must be added to $\text{FOLLOW}(A)$.

After A comes:

BC

So:

$\text{FIRST}(BC) = \{b,c, \epsilon\}$

Add non symbols:

$\{b,c\}$

Since the entire suffix BC is nullable, also add:

$\text{FOLLOW}(S)$

Therefore:

$\boxed{\text{FOLLOW}(A) \supseteq \{b,c\} \cup \text{FOLLOW}(S)}$

55. GATE-STYLE EXAMPLE 3

Grammar:

$S \rightarrow aA$

$A \rightarrow b$

What is $\text{FOLLOW}(A)$?

A is at RHS end.

Thus:

$\text{FOLLOW}(S) \subseteq \text{FOLLOW}(A)$

S is start symbol:

$\text{FOLLOW}(S) = \{\$ \}$

Therefore:

$$\boxed{\text{FOLLOW}(A) = \{\$ \}}$$

56. GATE-STYLE Example 4 — Recursive FOLLOW

Grammar:

$$S \rightarrow Aa$$

$$A \rightarrow bA \mid c$$

Find FOLLOW(A).

From:

$$S \rightarrow Aa$$

terminal `a` immediately follows `A`.

Therefore:

$$a \in \text{FOLLOW}(A)$$

From:

$$A \rightarrow bA$$

the RHS `A` is at the end, so:

$$\text{FOLLOW}(A) \subseteq \text{FOLLOW}(A)$$

which adds nothing new.

Thus:

$$\boxed{\text{FOLLOW}(A) = \{a\}}$$

Notice `$` is **not** automatically added to every non-terminal.

Only the start symbol gets `$` directly.

57. GATE TRAPS

Trap 1

If:

$$\epsilon \in \text{FIRST}(A)$$

then:

$$\epsilon \in \text{FOLLOW}(A)$$

False.

$$\boxed{\epsilon \text{ never belongs to FOLLOW}}$$

Trap 2

`$` is put in FOLLOW of every non-terminal.

'LFalse.

Initially:

$\boxed{\$ \text{in FOLLOW(Start)}}$

It may later propagate to others.

Trap 3

For:

$X \rightarrow ABC$

FOLLOW(A) only depends on FIRST(B).

'LFalse.

If B is nullable, C also matters.

Use:

$\text{FIRST}(BC)$

Trap 4

For:

$A \rightarrow \alpha B$

add FOLLOW(B) to FOLLOW(A).

'LBackwards.

Correct:

$\boxed{\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)}$

Trap 5

If first symbol of RHS is nullable, immediately add μ to FIRST(lhs).

'LFalse.

You must continue to the next symbol.

Only if **all RHS symbols are nullable** can μ be added.

Trap 6

FIRST contains only terminals, never μ .

'LFalse.

FIRST may include ϵ .

Trap 7

FOLLOW can be computed in exactly one left-to-right pass.

'L Not always.

Dependencies may require repeated iterations.

Trap 8

Nullable means the non-terminal has an explicit production $(A \rightarrow \epsilon)$.

'L False.

It can derive μ indirectly.

58. MINI PRACTICE — PART 7

Try these first.

Q1 — Easy

Given:

$[A \rightarrow a \mid \epsilon]$

what is:

$[FIRST(A)?]$

A. $\{a\}$ B. $\{\mu C\}$ $\{a\}$, D. $\{\$, a\}$

Q2 — Easy

If S is the start symbol, what must belong to:

$[FOLLOW(S)?]$

A. μB $\$$ C. Start symbol itself D. Every terminal

Q3 — Medium

Given:

$[S \rightarrow ABC]$

with:

$[FIRST(A) = \{a, \epsilon\}]$

$[FIRST(B) = \{b, \epsilon\}]$

$[FIRST(C) = \{c, \epsilon\}]$

Find:

$[FIRST(S)]$

Q4 — Medium

Given:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b \mid \epsilon$

Find:

$\text{FOLLOW}(A)$

Q5 — Tricky

Given:

$S \rightarrow ABCD$

where:

$\text{FIRST}(B) = \{b, \epsilon\}$

$\text{FIRST}(C) = \{c, \epsilon\}$

$\text{FIRST}(D) = \{d\}$

What terminals must be in $\text{FOLLOW}(A)$ because of this production?

SOLUTIONS

Q1

' C

$\boxed{\text{FIRST}(A) = \{a, \epsilon\}}$

Q2

' B

$\boxed{\$ \in \text{FOLLOW}(S)}$

for start symbol S .

Q3

All A , B , and C are nullable.

Therefore collect:

TEXT

a
b
c

and because all can disappear:

ϵ

Hence:

$\text{FIRST}(S) = \{a, b, c, \epsilon\}$

Q4

From:

$S \rightarrow AB$

after A comes B.

$\text{FIRST}(B) = \{b, \epsilon\}$

Add:

b

Since B is nullable, also add:

$\text{FOLLOW}(S) = \{\$ \}$

Therefore:

$\text{FOLLOW}(A) = \{b, \$ \}$

Q5

To find $\text{FOLLOW}(A)$, inspect:

BCD

B contributes:

b

B nullable ! C contributes:

c

C nullable ! D contributes:

d

D is not nullable, so stop.

Therefore:

$\{b, c, d\} \subseteq \text{FOLLOW}(A)$

Do not add $\text{FOLLOW}(S)$, because the entire suffix BCD cannot disappear.

59. WRITE THIS IN NOTES — FIRST/FOLLOW MASTER SHEET

NULLABLE

$A \text{ is nullable} \iff A \xrightarrow{\epsilon}$

FIRST

$\boxed{\text{FIRST}(\alpha) = \text{possible first terminals of strings from } \alpha}$

For terminal a :

$\text{FIRST}(a) = \{a\}$

For sequence:

$X_1 X_2 \dots X_n$

scan left to right.

Continue only while previous symbols are nullable.

Add μ only if **all** are nullable.

FOLLOW

$\boxed{\text{FOLLOW}(A) = \text{terminals that can immediately follow } A}$

Start symbol:

$\boxed{\$ \in \text{FOLLOW}(S)}$

For:

$A \xrightarrow{\alpha} B \beta$

add:

$\boxed{\text{FIRST}(\beta) - \{\epsilon\}}$

to $\text{FOLLOW}(B)$.

If:

$\beta \xrightarrow{\epsilon}$

also add:

$\boxed{\text{FOLLOW}(A)}$

to $\text{FOLLOW}(B)$.

60. The Two Best Memory Shortcuts

FIRST

Walk RIGHT while previous symbols can vanish.

FOLLOW

Right-FIRST; if right vanishes, Left-FOLLOW.

For:

$\boxed{A \rightarrow \alpha B \mid \beta}$

TEXT

What follows B?

!"

FIRST(α) - μ

Can α vanish?

!" Yes

FOLLOW(A) also follows B

30-SECOND REVISION

Nullable

$\boxed{A \xrightarrow{\epsilon}}$

FIRST

What can come first?

For:

$\boxed{ABC}$

TEXT

FIRST(A) - μ

Nullable? !' FIRST(B) - μ

Nullable? !' FIRST(C) - μ

ALL nullable? !' add μ

FOLLOW

What can come immediately after a non-terminal?

Start symbol:

$\boxed{\$}$

For:

$\boxed{A \rightarrow \alpha B \mid \beta}$

$\boxed{\text{FIRST}(\beta) \xrightarrow{\epsilon} \text{FOLLOW}(B)}$

If α nullable:

$\boxed{\text{FOLLOW}(A) \rightarrow \text{FOLLOW}(B)}$

Never forget

$\boxed{\epsilon \text{ can be in FIRST}}$

but:

ϵ can NEVER be in FOLLOW

Expression grammar values worth recognizing

$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{ (, id \}$

$\text{FIRST}(E') = \{ +, \epsilon \}$

$\text{FIRST}(T') = \{ *, \epsilon \}$

and:

$\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{ \}, \{ \$ \}$

$\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +, \epsilon, \$ \}$

$\text{FOLLOW}(F) = \{ *, +, \epsilon, \$ \}$

We now have the exact machinery required to understand **predictive parsing and LL(1)**.

Completed: Parts 1–7/31 | Next: Part 8 — Recursive-Descent & Predictive Parsing

Reply **NEXT** for **Part 8**, where we'll cover **top-down parsing, recursive descent, backtracking, predictive parsing, why left recursion breaks top-down parsers, why left factoring helps, and how one-symbol lookahead chooses productions**.

USER

NEXT

ASSISTANT

Compiler Design — Part 8: Recursive-Descent & Predictive Parsing

Completed: Parts 1–7/31 | Current: Part 8 | Remaining: Parts 9–31

Now we know:

- CFG
- derivations
- ambiguity
- left recursion
- left factoring
- FIRST
- FOLLOW

We can finally understand how a **top-down parser actually works**.

The central idea is:

$\text{Start Symbol} \rightarrow \text{Input String}$

1. What Is Top-Down Parsing?

CONCEPT

A **top-down parser** starts with the grammar's **start symbol** and tries to construct a parse tree until its leaves match the input string.

Conceptually:

TEXT

```
Start Symbol      ! "  
Apply productions ! "  
Non-terminals    ! "  
Tokens
```

The parse tree is built:

$\boxed{\text{Root} \rightarrow \text{Leaves}}$

Hence the name **top-down**.

WRITE THIS IN NOTES

Top-down parsing: Construct the parse tree from the root (start symbol) toward the leaves by choosing productions that generate the input.

2. Top-Down Parsing and Leftmost Derivation

Top-down parsers generally construct a:

$\boxed{\text{Leftmost derivation}}$

That means:

At every step, expand the leftmost non-terminal.

Recall our memory trick:

LL

First **L**:

$\text{Input scanned Left-to-right}$

Second **L**:

$\text{Leftmost derivation}$

So:

$\boxed{LL = \text{Left-to-right input} + \text{Leftmost derivation}}$

3. Basic Top-Down Example

Grammar:

$S \rightarrow aA$

$A \rightarrow b$

Input:

TEXT

ab

Start with:

S

Apply:

$S \rightarrow aA$

So:

$S \Rightarrow aA$

Input begins with **a**, so match it.

Then:

$A \rightarrow b$

Thus:

$aA \Rightarrow ab$

Input completely matched.

Therefore:

$\boxed{\text{Accepted}}$

4. Parser's Main Problem: Which Production Should It Choose?

Suppose:

$S \rightarrow aA \mid bB$

and input begins with:

TEXT

a

It is easy.

Choose:

$S \rightarrow aA$

because the other alternative starts with **b**.

But suppose:

$S \rightarrow aA \mid aB$

Input begins with:

```
TEXT
a
```

Now both alternatives appear possible.

The parser cannot immediately choose.

This leads to two approaches:

1. Backtracking
2. Predictive parsing

5. Recursive-Descent Parsing

CONCEPT

A **recursive-descent parser** is a top-down parser implemented as a collection of procedures/functions, usually one procedure for each non-terminal.

Suppose grammar:

$[E \rightarrow T + E \mid T]$

$[T \rightarrow id]$

We can conceptually create:

```
TEXT
procedure E()
procedure T()
```

Each non-terminal corresponds to a parsing routine.

WRITE THIS IN NOTES

Recursive-descent parser: Top-down parser consisting of mutually recursive procedures, typically one for each non-terminal.

6. Simple Recursive-Descent Example

Grammar:

$[S \rightarrow aSb \mid \epsilon]$

Conceptually:

```
TEXT
S():
    if lookahead == 'a':
        match('a')
```

```

    S()
    match('b')
else:
    use S !' μ

```

Don't memorize programming syntax.

Understand the mapping:

$S \rightarrow aSb$

becomes conceptually:

```

TEXT
match(a)
call S()
match(b)

```

Grammar recursion becomes function recursion.

7. What Does `match()` Mean?

A parser maintains a current input token called the:

`lookahead token`

A conceptual operation:

```

TEXT
match(t)

```

means:

1. Check whether current input token equals `t`.
2. If yes, consume it.
3. Advance to the next token.
4. Otherwise report syntax error.

Example input:

```

TEXT
id + id $
! '
lookahead

```

After:

```

TEXT
match(id)

```

we get:

```

TEXT
id + id $
! '

```


8. Lookahead Token

The parser generally does not need to inspect the entire remaining input.

A predictive parser may use a limited number of upcoming tokens.

For **LL(1)**, the **1** means:

$\boxed{\text{1 lookahead token}}$

So the parser looks at the next input token to decide which production to use.

9. Recursive Descent With Backtracking

Consider:

$S \rightarrow aA \mid aB$

$A \rightarrow c$

$B \rightarrow d$

Input:

TEXT

ad

At **S**, both alternatives begin with **a**.

Suppose parser first tries:

$S \rightarrow aA$

Then:

TEXT

a

matches.

Now it expects:

TEXT

c

because:

$A \rightarrow c$

But input has:

TEXT

d

So this alternative fails.

The parser may:

1. return to the earlier decision point,
2. restore the input position,
3. try:

$\text{[S} \rightarrow \text{aB]}$

Now:

$\text{[B} \rightarrow \text{d]}$

works.

This process is:

$\text{[\boxed{\text{Backtracking}}]}$

10. What Is Backtracking?

CONCEPT

Backtracking means:

Try one production. If it eventually fails, return to a previous choice point and try another production.

Conceptually:

```
TEXT
Choose production 1
      !"
Try parsing
      !"
Failure?
      !"  Y e s
Go back
      !"
Choose production 2
```

11. Why Is Backtracking Undesirable?

Backtracking may cause:

- repeated work
- inefficient parsing
- complicated implementation
- potentially large search

We would prefer the parser to choose the **correct production immediately**.

That is the goal of:

Predictive Parsing

12. Predictive Parsing

CONCEPT

A **predictive parser** is a top-down parser that chooses the appropriate production using lookahead information **without backtracking**.

So:

$\boxed{\text{Predictive Parsing} = \text{Top-down parsing without backtracking}}$

This is one of the most important definitions.

WRITE THIS IN NOTES

Predictive parser: A top-down parser that uses lookahead to select the correct production without backtracking.

13. Example of Easy Predictive Choice

Grammar:

$S \rightarrow aA \mid bB$

If lookahead:

TEXT

a

choose:

$S \rightarrow aA$

If lookahead:

TEXT

b

choose:

$S \rightarrow bB$

No guessing.

No backtracking.

14. FIRST Sets Explain Predictive Choice

For:

$S \rightarrow aA \mid bB$

we have:

$\text{FIRST}(aA) = \{a\}$

$\text{FIRST}(bB) = \{b\}$

The sets are disjoint:

$\{a\} \cap \{b\} = \emptyset$

Therefore one-token lookahead is sufficient to distinguish the alternatives.

This is the basic intuition behind LL(1).

15. What if FIRST Sets Overlap?

Consider:

$S \rightarrow aA \mid aB$

Then:

$\text{FIRST}(aA) = \{a\}$

and:

$\text{FIRST}(aB) = \{a\}$

So:

$\text{FIRST}(aA) \cap \text{FIRST}(aB) = \{a\}$

The lookahead `a` does not uniquely identify a production.

Therefore this grammar is not directly suitable for one-symbol predictive choice.

What can we do?

Left factoring.

16. Why Left Factoring Helps

Original:

$S \rightarrow aA \mid aB$

Factor common prefix:

$S \rightarrow aS'$

$S' \rightarrow A \mid B$

Now the parser does not need to decide between `A` and `B` before consuming `a`.

It first matches:

TEXT

a

Then examines the next input token.

Thus:

Left factoring delays the decision until enough input has been seen.

Memory Trick

Left factoring removes "same beginning" choices.

17. But Left Factoring Does NOT Guarantee LL(1)

Suppose after factoring:

$$S \rightarrow A \mid B$$

and:

$$\text{FIRST}(A) = \text{FIRST}(B)$$

Then the conflict still exists.

Therefore:

$$\boxed{\text{Left factored}} \not\rightarrow \text{LL}(1)$$

Exact LL(1) conditions come in **Part 9**.

18. Why Left Recursion Breaks Recursive Descent

Consider:

$$E \rightarrow E + T \mid T$$

A direct recursive-descent procedure for `E` might conceptually attempt:

```
TEXT
E():
    E()
    match('+')
    T()
```

The first thing `E()` does is call:

```
TEXT
E()
```

which again calls:

```
TEXT
E()
```

and so on:

TEXT

E
!"
E
!"
E
!"
E
!"
...

No input has been consumed.

Therefore:

$\boxed{\text{Infinite recursion}}$

19. Left Recursion and Predictive Parsing

A grammar containing left recursion cannot be directly handled by the usual LL predictive parser.

Therefore before constructing an LL parser:

$\boxed{\text{Remove left recursion}}$

Example:

$E \rightarrow E+T \mid T$

becomes:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

Now recursive parsing can proceed while consuming input.

20. Important Distinction

Left recursion causes:

Recursive procedure may call itself before consuming input.

Left factoring causes:

Multiple alternatives look identical initially.

Hence:

TEXT

L e f t r e c u r s i o n ! ' r e c u r s i o n p r o b l e m

L e f t f a c t o r i n g ! ' p r o d u c t i o n - c h o i c e p r o b l e m

These are different.

21. Recursive Descent vs Predictive Parser

These terms are closely related but not identical.

Recursive-Descent Parser

Uses recursive procedures.

It **may** use backtracking.

Predictive Recursive-Descent Parser

Uses recursive procedures **without backtracking**, because lookahead uniquely predicts the production.

Therefore:

$\boxed{\text{Predictive recursive descent is a restricted form of recursive descent}}$

Master Comparison

Recursive Descent	Predictive Parsing
Top-down	Top-down
Procedures for non-terminals	Can be recursive or table-driven
May involve backtracking	No backtracking
May guess productions	Uses lookahead to select production
Not automatically LL(1)	LL(1) is the standard one-lookahead predictive case

GATE TRAP

Every recursive-descent parser is predictive.

'L False.

A recursive-descent parser can use backtracking.

22. Two Forms of Predictive Parser

Predictive parsing can be implemented mainly in two conceptual ways:

1. Recursive Predictive Parser

One procedure per non-terminal.

2. Non-Recursive Table-Driven Predictive Parser

Uses:

- stack
- input
- parsing table

We will study the table-driven version deeply in Part 9.

23. Predictive Parsing With μ - P r o d u c t i o n s

Now things become interesting.

Consider:

$A \rightarrow aA \mid \epsilon$

Suppose lookahead is:

TEXT

a

Clearly choose:

$A \rightarrow aA$

But when should we choose:

$A \rightarrow \epsilon$?

We need to know:

Which tokens are legally allowed to appear after A?

That is exactly:

$FOLLOW(A)$

This explains **why FOLLOW exists** in predictive parsing.

24. FIRST Chooses Normal Productions; FOLLOW Helps Choose μ

This is a crucial intuition.

For production:

$A \rightarrow \alpha$

if current lookahead is in:

$FIRST(\alpha)$

choose that production.

If:

$\epsilon \in FIRST(\alpha)$

then the production may disappear.

It is appropriate when the lookahead belongs to:

$FOLLOW(A)$

This becomes the exact LL(1) table rule in Part 9.

Memory Trick

Normal RHS:

Choose using FIRST.

μ RHS:

Choose using FOLLOW.

This is not the whole formal rule, but it is an excellent first recall.

25. Worked Predictive Example

Grammar:

$[S \rightarrow aA]$

$[A \rightarrow bA \mid \epsilon]$

Input:

TEXT
abbb

with end marker:

TEXT
abbb\$

Step 1

Start:

$[S]$

Lookahead:

TEXT
a

Choose:

$[S \rightarrow aA]$

Match **a**.

Remaining:

TEXT

bbb\$

Step 2

Current non-terminal:

$\backslash A \backslash$

Lookahead:

TEXT

b

Since:

$\backslash b \text{ in } \text{FIRST}(bA) \backslash$

choose:

$\backslash A \rightarrow bA \backslash$

Match **b**.

Step 3

Lookahead is again **b**.

Again choose:

$\backslash A \rightarrow bA \backslash$

Continue.

After all three **b**s:

TEXT

\$

is lookahead.

Now:

$\backslash \text{FOLLOW}(A) = \backslash \{ \$ \} \backslash$

Therefore choose:

$\backslash A \rightarrow \epsilon \backslash$

Nothing is consumed.

Input and grammar both finish.

$\backslash \boxed{\text{Accepted}} \backslash$

26. Why FOLLOW Matters Here

If we always chose:

$\backslash A \rightarrow bA \backslash$

we could not stop.

If we always chose:

$\backslash A \rightarrow \epsilon \backslash$

we would stop too early.

The lookahead tells us:

TEXT

```
b ! '   c o n t i n u e
$ ! '   s t o p
```

Thus:

$\backslash \text{FIRST}(bA) = \{b\} \backslash$

and:

$\backslash \text{FOLLOW}(A) = \{\$, \epsilon\} \backslash$

make the decision deterministic.

27. Another Example

Grammar:

$\backslash S \rightarrow aAB \backslash$

$\backslash A \rightarrow b \mid \epsilon \backslash$

$\backslash B \rightarrow c \backslash$

Input:

TEXT

ac

After matching **a**, current non-terminal is **A**.

Lookahead is:

TEXT

c

Now:

$\backslash \text{FIRST}(b) = \{b\} \backslash$

so production:

$\backslash A \rightarrow b \backslash$

does not match.

Since:

$\{A \rightarrow \epsilon\}$

is possible, check FOLLOW(A).

From:

$\{S \rightarrow aAB\}$

after A comes B, and:

$\{FIRST(B) = \{c\}\}$

Thus:

$\{c \in FOLLOW(A)\}$

So choose:

$\{A \rightarrow \epsilon\}$

Then parse B! c

Input accepted.

28. Basic Predictive Parsing Decision Rule

Suppose:

$\{A \rightarrow \alpha_1 \mid \alpha_2\}$

We would like the lookahead token to uniquely indicate which alternative applies.

Rough intuition:

TEXT

```
l o o k a h e a d "   F I R S T ( ± 1 ) ! '   c h o o s e ± 1
l o o k a h e a d "   F I R S T ( ± 2 ) ! '   c h o o s e ± 2
```

If an alternative can derive $\mu \in FOLLOW(A)$ must also be considered.

The exact conflict rules are the heart of **LL(1)**, which is Part 9.

29. Predictive Parser Error Detection

Suppose:

$\{S \rightarrow aA\}$

and current lookahead is:

TEXT

b

But:

$\{FIRST(aA) = \{a\}\}$

No valid production can begin with b.

Therefore:

$\boxed{\text{Syntax error}}$

Predictive parsers can detect errors when no valid table entry/production exists for:

TEXT

(current non-terminal, lookahead)

30. Recursive-Descent Example With Expression Grammar

Use the left-recursion-free grammar:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

A recursive parser conceptually has:

TEXT

$E()$
 $E'()$
 $T()$
 $T'()$
 $F()$

31. Procedure for $F()$

Because:

$F \rightarrow (E) \mid \text{id}$

we can choose using lookahead.

If lookahead:

TEXT

(

choose:

$F \rightarrow (E)$

If lookahead:

TEXT

id

choose:

$F \rightarrow \text{id}$

Thus $F()$ conceptually behaves as:

TEXT

```
if lookahead == '(':
    match('(')
    E()
    match(')')
else if lookahead == id:
    match(id)
else:
    syntax error
```

No backtracking is needed because:

$\text{FIRST}((E)) = \{ (\}$

and:

$\text{FIRST}(id) = \{ id \}$

are disjoint.

32. Procedure for $\text{FIRST}(E')$

Recall:

$E' \xrightarrow{+TE'} \epsilon$

We know:

$\text{FIRST}(+TE') = \{ + \}$

and:

$\text{FOLLOW}(E') = \{ \}, \{ \$ \}$

So:

Lookahead +

Choose:

$E' \xrightarrow{+TE'}$

Lookahead) or \$

Choose:

$E' \xrightarrow{\epsilon}$

This is predictive parsing in action.

33. Procedure for $\text{FIRST}(T')$

$T' \xrightarrow{*FT'} \epsilon$

We know:

$\text{FIRST}(FT') = \{ \wedge \}$

and:

$\text{FOLLOW}(T') = \{ +,), \$ \}$

So:

TEXT

l o o k a h e a d = * ! ' u s e * F T '

l o o k a h e a d = + ,) , o r \$! ' u s e μ

Again no backtracking.

34. Complete Predictive Decisions for Expression Grammar

$\backslash(E\backslash)$

$\backslash[E\rightarrow TE' \backslash]$

Use when lookahead:

TEXT

id
(

$\backslash(E'\backslash)$

$\backslash[E'\rightarrow +TE' \backslash]$

when lookahead:

TEXT

+

Use:

$\backslash[E'\rightarrow \epsilon \backslash]$

when lookahead:

TEXT

)
\$

$\backslash(T\backslash)$

$\backslash[T\rightarrow FT' \backslash]$

when lookahead:

TEXT

id
(

$\backslash(T'\backslash)$

$\backslash[T'\rightarrow *FT' \backslash]$

when lookahead:

TEXT
*

Use:

$\backslash [T \rightarrow \epsilon]$

when:

TEXT
+
)
\$

$\backslash (F)$

$\backslash [F \rightarrow (E)]$

when:

TEXT
(

and:

$\backslash [F \rightarrow id]$

when:

TEXT
id

This will become the LL(1) parsing table in the next part.

35. What Makes Predictive Parsing Possible?

The grammar must provide enough information so that the parser can make a unique choice based on lookahead.

Common problems include:

1. Left recursion

Example:

$\backslash [A \rightarrow A\alpha]$

causes recursion before input consumption.

2. Common prefixes

Example:

$\backslash [A \rightarrow ab \mid ac]$

lookahead **a** does not distinguish choices.

3. FIRST/FIRST conflicts

Two alternatives predict the same terminal.

4. FIRST/FOLLOW conflicts

An μ -production conflicts with another alternative.

We'll formalize the last two in Part 9.

36. Top-Down Parser Limitations

Not every CFG can be parsed by an LL(1) predictive parser.

LL(1) grammars are a restricted class of context-free grammars.

A grammar may need:

- left recursion removal
- left factoring
- rewriting

and even after transformations, it may still not be LL(1).

Later, bottom-up LR parsers will handle a **larger class** of grammars.

37. Top-Down vs Bottom-Up — Preview

Top-Down	Bottom-Up
Start symbol !' input	Input !' start symbol
Root !' leaves	Leaves !' root
Leftmost derivation	Reverse rightmost derivation
LL family	LR family
Cannot directly handle left recursion	Can handle left-recursive grammars
Generally simpler	More powerful

We will study bottom-up parsing after LL(1).

38. Important GATE Fact — Left Recursion Can Be Useful for LR

Do not conclude:

Left recursion is bad for all parsers.

It is especially problematic for **top-down** parsing.

Bottom-up LR parsers can handle many left-recursive grammars naturally.

Therefore:

$\boxed{\text{Left recursion problematic for LL, not inherently for LR}}$

39. Predictive Parsing and Backtracking

The defining operational goal is:

$\boxed{\text{Choose once and move forward}}$

No:

TEXT

```
t r y ! '   f a i l ! '   r e t u r n ! '   t r y   a n o t h e r
```

Instead:

TEXT

```
lookahead
! "
unique production
! "
continue
```

40. GATE TRAPS

Trap 1

Top-down parsing constructs the parse tree from leaves to root.

'LFalse.

$\boxed{\text{Root !' leaves}}$

Trap 2

Recursive descent always means predictive parsing.

'LFalse.

Recursive descent may use backtracking.

Trap 3

Predictive parsing uses backtracking to select productions.

'LFalse.

Its purpose is to avoid backtracking.

Trap 4

Left recursion causes an ordinary recursive-descent parser to possibly recurse indefinitely without consuming input.

' True.

Trap 5

Left factoring always makes a grammar LL(1).

'L False.

It only removes common-prefix problems.

Trap 6

Removing left recursion guarantees LL(1).

'L False.

FIRST/FIRST or FIRST/FOLLOW conflicts may remain.

Trap 7

LL(1)'s **1** refers to one production on the RHS.

'L False.

It means:

$\boxed{1 \text{ lookahead token}}$

Trap 8

LL parser scans input right-to-left.

'L False.

First **L** means:

$\boxed{\text{Left-to-right input}}$

Trap 9

Top-down parsing constructs a rightmost derivation.

'L Generally false for LL parsing.

$\boxed{\text{LL constructs leftmost derivation}}$

Trap 10

FOLLOW is useless when a grammar has μ - p r o d u c t i o n s .

'L Exactly opposite.

FOLLOW becomes especially important for deciding **when to use μ**

41. GATE-STYLE EXAMPLE 1 — Backtracking

Grammar:

$\text{\textbackslash}[S\text{\textbackslash}rightarrow ab\text{\textbackslash}mid ac \text{\textbackslash}]$

Input:

TEXT

ac

A naive recursive-descent parser chooses:

$\text{\textbackslash}[S\text{\textbackslash}rightarrow ab \text{\textbackslash}]$

first.

It matches **a**, then fails at **b**.

It returns and tries:

$\text{\textbackslash}[S\text{\textbackslash}rightarrow ac \text{\textbackslash}]$

This behavior is:

$\text{\textbackslash}[\text{\textbackslash}boxed{\text{\textbackslash}text{Backtracking}} \text{\textbackslash}]$

42. GATE-STYLE EXAMPLE 2 — Why Left Factoring?

Given:

$\text{\textbackslash}[S\text{\textbackslash}rightarrow ab\text{\textbackslash}mid ac \text{\textbackslash}]$

Both alternatives have:

$\text{\textbackslash}[FIRST=\{a\} \text{\textbackslash}]$

So one lookahead token **a** cannot distinguish them.

Left-factor:

$\text{\textbackslash}[\text{\textbackslash}boxed{S\text{\textbackslash}rightarrow aS'} \text{\textbackslash}]$

$\text{\textbackslash}[\text{\textbackslash}boxed{S'\text{\textbackslash}rightarrow b\text{\textbackslash}mid c} \text{\textbackslash}]$

Now after consuming **a**, next lookahead determines the choice.

43. GATE-STYLE EXAMPLE 3 — Left Recursion

Grammar:

$\text{\textbackslash}[A\text{\textbackslash}rightarrow Aa\text{\textbackslash}mid b \text{\textbackslash}]$

Can it be used directly by a standard predictive recursive-descent parser?

'LNo.

Because:

TEXT

A () ! ' c a l l s A () ! ' c a l l s A () ! ' . . .

before consuming input.

Remove left recursion:

$A \rightarrow bA'$

$A' \rightarrow \epsilon$

44. GATE-STYLE EXAMPLE 4 — μ Choice

Grammar:

$A \rightarrow aA \mid \epsilon$

Suppose:

$\text{FOLLOW}(A) = \{b, \$\}$

What should a predictive parser do?

Lookahead a

Use:

$A \rightarrow aA$

Lookahead b

Use:

$A \rightarrow \epsilon$

Lookahead \$

Use:

$A \rightarrow \epsilon$

This is the FIRST/FOLLOW mechanism.

45. GATE-STYLE Example 5 — Detect Syntax Error

Same grammar:

$A \rightarrow aA \mid \epsilon$

with:

$\text{FOLLOW}(A) = \{b, \$\}$

Suppose lookahead is:

TEXT

c

Then:

TEXT

c " F I R S T (a A)

and:

TEXT

c " F O L L O W (A)

No production is valid.

Therefore:

$\boxed{\text{Syntax error}}$

46. MINI PRACTICE — PART 8

Try first.

Q1 — Easy

A predictive parser is primarily designed to avoid:

A. Tokens B. Backtracking C. Grammars D. Parse trees

Q2 — Easy

In LL(1), the number 1 denotes:

A. One non-terminal B. One grammar production C. One lookahead token D. One parse tree

Q3 — Medium

Why is:

$A \rightarrow a \mid b$

unsuitable directly for a standard recursive-descent parser?

A. It is necessarily ambiguous B. It contains immediate left recursion C. It has no terminals D. It is regular

Q4 — Medium

Given:

$S \rightarrow ab \mid ac$

what transformation helps a predictive parser?

Q5 — Tricky

Given:

$A \rightarrow a \mid \epsilon$

with:

$\text{FOLLOW}(A) = \{b, \$\}$

Which production should be used when lookahead is **b** ?

A. $\rightarrow a$ B. $\rightarrow \epsilon$ C. Either one arbitrarily D. Syntax error

SOLUTIONS

Q1

' **B — Backtracking**

$\boxed{\text{Predictive = no backtracking}}$

Q2

' **C — One lookahead token**

$LL(1)$

means:

TEXT

```
L !'   L e f t - t o - r i g h t   i n p u t
L !'   L e f t m o s t   d e r i v a t i o n
1 !'   o n e   l o o k a h e a d   t o k e n
```

Q3

' **B**

The production:

$\rightarrow Aa$

causes immediate left recursion.

Q4

Factor common prefix **a** :

$\rightarrow aS$

$\rightarrow S \mid c$

Q5

' **B**

Since lookahead **b** is in:

$FOLLOW(A)$

and the μ -p r o d u c t i o n is possible:

$\rightarrow \epsilon$

WRITE THIS IN NOTES — PART 8 MASTER SHEET

Top-Down Parsing

$\boxed{\text{Start symbol !' input}}$

TEXT

R o o t ! ' L e a v e s

Uses leftmost derivation.

Recursive Descent

One procedure per non-terminal.

May use backtracking.

Backtracking

TEXT

Choose

!"

Fail

!"

Return

!"

Try another

Predictive Parsing

$\boxed{\text{Top-down + lookahead + no backtracking}}$

LL(1)

TEXT

L = Left-to-right input

L = Leftmost derivation

1 = One lookahead token

Left Recursion

Bad for ordinary top-down recursive descent:

TEXT

A () !' A () !' A () !' . . .

Remove it first.

Left Factoring

Useful when alternatives share a prefix:

$\text{\textbackslash}[A\rightarrow\alpha\beta_1\mid\alpha\beta_2\text{\textbackslash}]$

becomes:

$\text{\textbackslash}[A\rightarrow\alpha A'\text{\textbackslash}]$

$\text{\textbackslash}[A'\rightarrow\beta_1\mid\beta_2\text{\textbackslash}]$

Predictive Selection

For:

$\text{\textbackslash}[A\rightarrow\alpha\text{\textbackslash}]$

normally choose using:

$\text{\textbackslash}[\text{FIRST}(\alpha)\text{\textbackslash}]$

If α can derive μ then:

$\text{\textbackslash}[\text{FOLLOW}(A)\text{\textbackslash}]$

helps determine when the μ alternative is valid.

30-SECOND REVISION

Top-down

$\text{\textbackslash}[\boxed{\text{Root !' Leaves}}\text{\textbackslash}]$

LL

$\text{\textbackslash}[\boxed{\text{Left-to-right + Leftmost + 1 lookahead}}\text{\textbackslash}]$

Recursive descent

Functions/procedures for non-terminals.

Predictive parser

$\text{\textbackslash}[\boxed{\text{No backtracking}}\text{\textbackslash}]$

Left recursion problem

TEXT

A !' A ±

can cause:

TEXT

A () !' A () !' A () !' "

Left factoring problem

TEXT

A !' a b | a c

lookahead **a** cannot choose.

Factor:

TEXT

A !' a A '
A ' !' b | c

Main decision memory

FIRST tells when to enter a production.

FOLLOW tells when a nullable production may disappear.

The next part is one of the **most important parsing sections for GATE**: we will use FIRST and FOLLOW to construct actual **LL(1) parsing tables**, determine whether grammars are LL(1), identify FIRST/FIRST and FIRST/FOLLOW conflicts, and manually parse input strings using a stack.

Completed: Parts 1–8/31 | Next: Part 9 — LL(1) Grammar & Predictive Parsing Table

Reply **NEXT** for **Part 9**.

USER

NEXT

ASSISTANT

Compiler Design — Part 9: LL(1) Grammar & Predictive Parsing Table

Completed: Parts 1–8/31 | Current: Part 9 | Remaining: Parts 10–31

This is one of the **most important GATE parsing topics**.

By the end of this part, you should be able to:

- understand exactly what LL(1) means

- test whether a grammar is LL(1)
- detect **FIRST/FIRST** conflicts
- detect **FIRST/FOLLOW** conflicts
- construct an LL(1) parsing table
- parse a string using the table and stack
- identify errors and conflicts quickly

1. What Does LL(1) Mean?

The notation is:

$\boxed{LL(1)}$

Break it into three parts.

First L

Scan the input:

$\boxed{\text{Left to Right}}$

Second L

Construct a:

$\boxed{\text{Leftmost Derivation}}$

1

Use:

$\boxed{\text{One Lookahead Token}}$

Therefore:

WRITE THIS IN NOTES

$\boxed{LL(1) = \text{Left-to-right scan} + \text{Leftmost derivation} + 1\text{ lookahead}}$

Memory Trick

LL(1) = Left, Left, Look-one

2. What Is an LL(1) Grammar?

CONCEPT

An LL(1) grammar is one in which the parser can uniquely decide which production to use by examining only **one lookahead token**.

Suppose:

$A \rightarrow \alpha \mid \beta$

If the next input token is **a**, the parser must be able to determine uniquely:

TEXT

U s e A ! ' ±

or:

TEXT

U s e A ! ' ²

There must be no uncertainty.

3. Simple LL(1) Example

Grammar:

$S \rightarrow aA \mid bB$

We have:

$FIRST(aA) = \{a\}$

$FIRST(bB) = \{b\}$

These sets are disjoint:

$\{a\} \cap \{b\} = \emptyset$

So:

TEXT

l o o k a h e a d a ! ' c h o o s e S ! ' a A
l o o k a h e a d b ! ' c h o o s e S ! ' b B

One token is enough.

Therefore this choice is LL(1)-friendly.

4. Non-LL(1) Example

Consider:

$S \rightarrow aA \mid aB$

Then:

$FIRST(aA) = \{a\}$

and:

$FIRST(aB) = \{a\}$

So:

$FIRST(aA) \cap FIRST(aB) = \{a\}$

When lookahead is **a**, parser cannot choose uniquely.

Therefore:

$\boxed{\text{Not LL(1)}}$

in its current form.

This is a:

$\boxed{\text{FIRST/FIRST conflict}}$

5. FIRST/FIRST Conflict

For:

$A \rightarrow \alpha \mid \beta$

if:

$\boxed{\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) \neq \emptyset}$

then the same lookahead token can predict both productions.

Hence there is a conflict.

Example

$A \rightarrow aB \mid aC$

Both have **a** in FIRST.

So table cell:

$M[A, a]$

would need to contain two productions.

That means:

$\boxed{\text{Grammar is not LL(1)}}$

6. LL(1) Condition When Neither Alternative Is Nullable

Suppose:

$A \rightarrow \alpha \mid \beta$

and neither α nor β can derive ϵ .

Then we need:

$\boxed{\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset}$

That's the basic LL(1) condition.

7. What Happens With μ - Productions ?

Consider:

$A \rightarrow a \mid \epsilon$

Now:

$\text{FIRST}(a) = \{a\}$

and:

$\text{FIRST}(\epsilon) = \epsilon$

These FIRST sets appear disjoint.

But is that enough?

No.

Suppose:

$a \in \text{FOLLOW}(A)$

Then on lookahead a :

- $A \rightarrow a$ is possible because a is in FIRST of A
- $A \rightarrow \epsilon$ is also potentially possible because after disappearing, a is legal after A

Then there is a conflict.

This is called:

$\boxed{\text{FIRST/FOLLOW conflict}}$

8. FIRST/FOLLOW Conflict

Suppose:

$A \rightarrow \alpha \mid \epsilon$

Then for $LL(1)$, we need:

$\boxed{\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset}$

Why?

If the same token belongs to both:

TEXT

FIRST(ϵ)
and
FOLLOW(A)

the parser cannot decide whether to:

TEXT

expand $A \rightarrow \epsilon$

or:

TEXT

make A disappear using $A \rightarrow \mu$

9. Example of FIRST/FOLLOW Conflict

Grammar:

$S \rightarrow Aa$

$A \rightarrow a \mid \epsilon$

Let's calculate.

FIRST of non alternative

$\text{FIRST}(a) = \{a\}$

FOLLOW(A)

From:

$S \rightarrow Aa$

the terminal **a** immediately follows **A**.

Therefore:

$\text{FOLLOW}(A) = \{a\}$

Now:

$\text{FIRST}(a) \cap \text{FOLLOW}(A) = \{a\}$

So:

$\boxed{\text{FIRST/FOLLOW conflict}}$

When lookahead is **a**, should parser choose:

$A \rightarrow a$

or:

$A \rightarrow \epsilon$?

It cannot tell with one lookahead.

Thus grammar is **not LL(1)**.

10. General LL(1) Conditions

Suppose:

$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$

For LL(1), the prediction sets for the alternatives must be pairwise disjoint.

A useful practical test is:

Condition 1 — FIRST/FIRST

For any $i \neq j$:

$\boxed{\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset}$

ignoring ϵ appropriately.

Condition 2 — Nullable Alternative

If:

$\epsilon \in \text{FIRST}(\alpha_i)$

then for every other alternative α_j :

$\boxed{(\text{FIRST}(\alpha_j) - \epsilon) \cap \text{FOLLOW}(A) = \emptyset}$

Important

At most one alternative for a non-terminal can derive ϵ in an LL(1) grammar's predictive choices.

11. SELECT / PREDICT Set

A very clean way to test LL(1) is to compute the **SELECT** set for each production.

For:

$A \rightarrow \alpha$

define:

If α is not nullable:

$\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$

If α is nullable:

$\text{SELECT}(A \rightarrow \alpha) = (\text{FIRST}(\alpha) - \epsilon) \cup \text{FOLLOW}(A)$

Some books call this:

- SELECT set
- PREDICT set

The idea is the same:

These are the lookahead tokens for which this production should be chosen.

12. Best LL(1) Test

For all productions of the **same non-terminal**:

$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$

calculate their SELECT sets.

The grammar is LL(1) if these sets are pairwise disjoint:

$\text{SELECT}_i \cap \text{SELECT}_j = \emptyset \quad \text{for all } i \neq j$

This is one of the safest exam methods.

Memory Trick

LL(1) = One lookahead !' One production.

Therefore:

SELECT sets must not overlap.

13. Parsing Table

An LL(1) parser can use a two-dimensional table:

$$M[A,a]$$

where:

- A = non-terminal at top of parsing stack
- a = current lookahead input token

Each valid table cell contains the production the parser should apply.

Example:

TEXT

		INPUT															
		a	b	\$													
S		%	%	%	%	%	%	%	%	%,	%	%	%	%	%	%,	%
		%	S!	a	A		%	S!	b	B		%					%
		%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%

The row comes from the non-terminal.

The column comes from lookahead terminal.

14. LL(1) Table Construction Algorithm

For every production:

$$A \rightarrow \alpha$$

perform:

Rule 1

For every terminal:

$$a \in \text{FIRST}(\alpha) - \{\epsilon\}$$

put:

$$M[A,a] = A \rightarrow \alpha$$

into:

$$M[A,a]$$

Rule 2

If:

$$\epsilon \in \text{FIRST}(\alpha)$$

then for every:

$$b \in \text{FOLLOW}(A)$$

put:

$$M[A,b] = A \rightarrow \alpha$$

into:

$\boxed{M[A,b]}$

This includes $\$$ if:

$\$ \in \text{FOLLOW}(A)$

Rule 3

All remaining cells are error entries.

Memory Trick

For a production:

$A \rightarrow \alpha$

Normal production

Put it under **FIRST**.

Nullable production

Also put it under **FOLLOW(A)**.

15. Why the Table Detects Non-LL(1)

Suppose two different productions must be placed into the same cell:

$M[A,a]$

Then on lookahead a , there are two choices.

That violates the core property:

$\boxed{\text{One lookahead ! one production}}$

Therefore:

$\boxed{\text{Multiple entries in one table cell ! Grammar not LL(1)}}$

16. Our Standard Expression Grammar

We will construct the complete table for:

$E \rightarrow TE'$

$E \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

This is one of the most important standard examples.

17. FIRST and FOLLOW Recap

From Part 7:

Non-terminal	FIRST	FOLLOW
$\backslash(E\backslash)$	$\{ (, id \}$	$\{), \$ \}$
$\backslash(E'\backslash)$	$\{ +, \mu \}$	$\{), \$ \}$
$\backslash(T\backslash)$	$\{ (, id \}$	$\{ +,), \$ \}$
$\backslash(T'\backslash)$	$\{ *, \mu \}$	$\{ +,), \$ \}$
$\backslash(F\backslash)$	$\{ (, id \}$	$\{ *, +,), \$ \}$

Now let's build the table.

18. Production 1

$\backslash E \rightarrow TE' \backslash$

We need:

$\backslash FIRST(TE') \backslash$

Since:

$\backslash FIRST(T) = \{ (, id \} \backslash$

and T is non-nullable:

$\backslash FIRST(TE') = \{ (, id \} \backslash$

Therefore put:

$\backslash E \rightarrow TE' \backslash$

in:

$\backslash M[E, (] \backslash$

and:

$\backslash M[E, id] \backslash$

19. Production 2

$\backslash E' \rightarrow +TE' \backslash$

Its FIRST is:

$\backslash \{ + \} \backslash$

So put:

$\backslash E' \rightarrow +TE' \backslash$

in:

$\backslash M[E', +] \backslash$

20. Production 3 — μ Production

$\backslash E' \rightarrow \epsilon \backslash$

Because RHS is nullable, use:

$\backslash FOLLOW(E') = \{ \}, \{ \$ \} \backslash$

Therefore put:

$\backslash E' \rightarrow \epsilon \backslash$

in:

$\backslash M[E',)] \backslash$

and:

$\backslash M[E', \$] \backslash$

21. Production 4

$\backslash T \rightarrow FT' \backslash$

Since:

$\backslash FIRST(F) = \{ (, id \} \backslash$

put:

$\backslash T \rightarrow FT' \backslash$

in:

$\backslash M[T, (] \backslash$

and:

$\backslash M[T, id] \backslash$

22. Production 5

$\backslash T' \rightarrow *FT' \backslash$

Its FIRST is:

$\backslash \{ * \} \backslash$

Put it in:

$\backslash M[T', *] \backslash$

23. Production 6 — μ

$\backslash T' \rightarrow \epsilon \backslash$

Use:

$\text{FOLLOW}(T') = \{+,), \$\}$

Therefore put:

$T \rightarrow \epsilon$

in:

$M[T', +]$

$M[T',)]$

$M[T', \$]$

24. Production 7

$F \rightarrow (E)$

FIRST:

$\{($

Put:

$F \rightarrow (E)$

in:

$M[F, (]$

25. Production 8

$F \rightarrow id$

FIRST:

$\{id\}$

Put:

$F \rightarrow id$

in:

$M[F, id]$

26. Complete LL(1) Table

WRITE THIS IN NOTES

Non-termin al	id	+	*	(	)	\$
E	$\{E \rightarrow TE\}$	—	—	$\{E \rightarrow TE\}$	—	—
E'	—	$\{E' \rightarrow +TE\}$	—	—	$\{E' \rightarrow \epsilon\}$	$\{E' \rightarrow \epsilon\}$
T	$\{T \rightarrow FT\}$	—	—	$\{T \rightarrow FT\}$	—	—

Non-terminal	id	+	*	(	)	\$
T'	—	$\backslash(T'\text{to}\epsilon)$	$\backslash(T'\text{to } *FT')$	—	$\backslash(T'\text{to}\epsilon)$	$\backslash(T'\text{to}\epsilon)$
F	$\backslash(F\text{to id})$	—	—	$\backslash(F\text{to}(E))$	—	—

— means:

$\backslash[\boxed{\text{error}}]$

Notice:

No cell has more than one production.

Therefore this grammar is LL(1).

27. Table-Driven Predictive Parser

The parser uses three main things:

1. Input buffer
2. Stack
3. LL(1) parsing table

Conceptually:

TEXT

INPUT

id + id * id \$

! "

% % % % % % % %

% P a r s e r %

% % % % % % % %

! '

Stack

! • P a r s i n g T a b l e

28. Initial Stack

If start symbol is E , initialize:

TEXT

\$

E

We will write the stack horizontally as:

TEXT

\$ E

and assume the **rightmost symbol is the top**.

Input:

TEXT

```
id + id * id $
```

Thus initially:

TEXT

```
Stack: $ E
```

```
Input: id + id * id $
```

29. General Parsing Algorithm

Let:

- $\backslash(X\backslash)$ = top of stack
- $\backslash(a)$ = current input/lookahead token

There are three main cases.

30. Case 1 — Top Is a Terminal

If:

$\backslash[X=a \backslash]$

then:

1. pop X
2. consume input token a

This is called:

$\backslash[\boxed{\text{match}}] \backslash]$

Example:

TEXT

```
Stack top = id
```

```
Lookahead = id
```

Match them and advance.

31. Case 2 — Top Is Terminal but Does Not Match

If top is terminal $+$, but lookahead is $*$:

TEXT

```
Stack top = +
```

```
Input      = *
```

then:

$\backslash[\boxed{\text{Syntax error}}] \backslash]$

32. Case 3 — Top Is Non-Terminal

Suppose top is:

$\backslash [A \]$

and lookahead is:

$\backslash [a \]$

consult:

$\backslash [M[A,a] \]$

If:

$\backslash [M[A,a]=A \rightarrow XYZ \]$

then:

1. pop A
2. push RHS in reverse order

Why reverse?

Because the leftmost RHS symbol must appear on top of the stack.

33. Why Push RHS in Reverse?

Suppose:

$\backslash [A \rightarrow XYZ \]$

We need parser to process:

TEXT

X first
then Y
then Z

Since top is at right, push:

TEXT

Z
Y
X

so resulting stack ends in:

TEXT

... Z Y X

and X is on top.

Memory Trick

RHS goes onto stack backwards.

34. μ Production

If:

$\backslash \text{A} \rightarrow \epsilon \backslash$

then:

- pop A
- push nothing

Because μ represents nothing.

35. Acceptance Condition

Successful parsing ends when:

TEXT

Stack: \$
Input: \$

Then:

$\backslash \boxed{\text{ACCEPT}} \backslash$

36. Full Parsing Example

Parse:

$\backslash \boxed{\text{id} + \text{id} * \text{id}} \backslash$

using our expression grammar.

Input with end marker:

TEXT

id + id * id \$

Initial stack:

TEXT

\$ E

Remember:

rightmost = stack top.

37. Step 1

TEXT

Stack: \$ E

Input: id + id * id \$

Top:

$\backslash [E]$

Lookahead:

TEXT

id

From table:

$\backslash [M[E, id] = E \rightarrow TE']$

Replace E:

TEXT

Stack: \$ E' T

38. Step 2

TEXT

Stack: \$ E' T

Input: id + id * id \$

Top:

$\backslash [T]$

Lookahead:

TEXT

id

Table:

$\backslash [T \rightarrow FT']$

Push reverse:

TEXT

Stack: \$ E' T' F

39. Step 3

Top:

$\backslash [F]$

Lookahead:

TEXT

id

Table:

$\backslash [\text{F} \rightarrow \text{id}]$

So:

TEXT

Stack: \$ E' T' id

40. Step 4 — Match

Top:

TEXT

id

Lookahead:

TEXT

id

Match.

New configuration:

TEXT

Stack: \$ E' T'

Input: + id * id \$

41. Step 5

Top:

$\backslash [T']$

Lookahead:

TEXT

+

Table:

$\backslash [T' \rightarrow \epsilon]$

Pop T' ; push nothing.

TEXT

Stack: \$ E'

Input: + id * id \$

42. Step 6

Top:

$\backslash E' \backslash$

Lookahead:

```
TEXT
+

```

Table:

$\backslash E' \rightarrow +TE' \backslash$

RHS:

```
TEXT
+ T E'

```

Push in reverse:

```
TEXT
E'
T
+

```

Result:

```
TEXT
Stack: $ E' T +

```

43. Step 7 — Match +

```
TEXT
Stack: $ E' T +
Input: + id * id $

```

Match +.

Now:

```
TEXT
Stack: $ E' T
Input: id * id $

```

44. Step 8

Top:

$\backslash T \backslash$

Lookahead:

```
TEXT
id
```

Use:

\[T\rightarrow FT' \]

Stack:

```
TEXT
$ E' T' F
```

45. Step 9

Top:

\[F \]

Lookahead:

```
TEXT
id
```

Use:

\[F\rightarrow id \]

Stack:

```
TEXT
$ E' T' id
```

Then match `id`.

New:

```
TEXT
Stack: $ E' T'
Input: * id $
```

46. Step 10

Top:

\[T' \]

Lookahead:

```
TEXT
*
```

Use:

$\backslash [T \backslash \text{rightarrow} *FT' \backslash]$

Push RHS reverse:

TEXT

T'
F
*

Stack:

TEXT

\$ E' T' F *

47. Step 11

Top `*`, lookahead `*`.

Match.

TEXT

Stack: \$ E' T' F
Input: id \$

48. Step 12

Top:

$\backslash [F \backslash]$

Lookahead:

TEXT

id

Use:

$\backslash [F \backslash \text{rightarrow} id \backslash]$

Then match `id`.

Now:

TEXT

Stack: \$ E' T'
Input: \$

49. Step 13

Top:

$\backslash [T' \backslash]$

Lookahead:

$\backslash \ \$ \ \backslash$

Use:

$\backslash \ T \rightarrow \epsilon \ \backslash$

Now:

TEXT

Stack: \$ E'

Input: \$

50. Step 14

Top:

$\backslash \ E' \ \backslash$

Lookahead:

$\backslash \ \$ \ \backslash$

Use:

$\backslash \ E' \rightarrow \epsilon \ \backslash$

Now:

TEXT

Stack: \$

Input: \$

Therefore:

$\backslash \ \boxed{\text{ACCEPT}} \ \backslash$

51. Complete Parsing Trace Table

WRITE THIS IN NOTES ONLY IF USEFUL

Stack	Input	Action
\$ E	id + id * id \$	$\backslash (E \rightarrow TE' \backslash)$
\$ E' T	id + id * id \$	$\backslash (T \rightarrow FT' \backslash)$
\$ E' T' F	id + id * id \$	$\backslash (F \rightarrow id \backslash)$
\$ E' T' id	id + id * id \$	match id
\$ E' T'	+ id * id \$	$\backslash (T' \rightarrow \epsilon \backslash)$
\$ E'	+ id * id \$	$\backslash (E' \rightarrow +TE' \backslash)$
\$ E' T +	+ id * id \$	match +

Stack	Input	Action
\$ E' T	id * id \$	\(T\to FT\)
\$ E' T' F	id * id \$	\(F\to id\)
\$ E' T' id	id * id \$	match id
\$ E' T'	* id \$	\(T\to *FT\)
\$ E' T' F *	* id \$	match *
\$ E' T' F	id \$	\(F\to id\)
\$ E' T' id	id \$	match id
\$ E' T'	\$	\(T\to \epsilon\)
\$ E'	\$	\(E\to \epsilon\)
\$	\$	ACCEPT

52. GATE Shortcut for Parsing Trace

Do not try to mentally build the whole parse tree.

At each step ask only:

If stack top is terminal:

Does it equal lookahead?

If yes !' match.

If stack top is non-terminal:

Look up table entry.

If μ :

pop only.

If empty/error table entry:

syntax error.

53. Checking Whether a Grammar Is LL(1) — Method 1

The safest full method:

Step 1

Compute FIRST sets.

Step 2

Compute FOLLOW sets.

Step 3

Construct SELECT sets for alternatives.

Step 4

Check whether alternatives of the same LHS have overlapping SELECT sets.

If overlap:

$\boxed{\text{Not LL(1)}}$

54. Checking LL(1) — Method 2

Construct the predictive parsing table.

If every table cell contains at most one production:

$\boxed{\text{LL(1)}}$

If any cell requires two or more productions:

$\boxed{\text{Not LL(1)}}$

This is mathematically equivalent to the conflict test.

55. Worked LL(1) Test — No μ

Grammar:

$S \rightarrow aA \mid bB$

FIRST:

$\text{FIRST}(aA) = \{a\}$

$\text{FIRST}(bB) = \{b\}$

Intersection:

$\emptyset$

Therefore this choice has no FIRST/FIRST conflict.

Assuming the rest of grammar also has no conflicts:

$\boxed{\text{LL(1)}}$

56. Worked Non-LL(1) Test — FIRST/FIRST

Grammar:

$S \rightarrow aA \mid aB$

FIRST:

$\{a\}$

and:

$\{a\}$

Intersection:

$\{a\} \neq \emptyset$

Therefore:

$\boxed{\text{Not LL}(1)}$

Table cell:

$M[S,a]$

would contain both productions.

57. Worked LL(1) Test With μ

Grammar:

$S \rightarrow Ac$

$A \rightarrow a \mid \epsilon$

For A :

$\text{FIRST}(a) = \{a\}$

Now:

$\text{FOLLOW}(A) = \{c\}$

because:

$S \rightarrow Ac$

So:

$\text{FIRST}(a) \cap \text{FOLLOW}(A) = \{a\} \cap \{c\} = \emptyset$

Therefore no FIRST/FOLLOW conflict.

Predictions:

TEXT

l	o	o	k	a	h	e	a	d	a	!	'	A	!	'	a
l	o	o	k	a	h	e	a	d	c	!	'	A	!	'	μ

Hence this portion is LL(1).

58. Worked Non-LL(1) Test With μ

Grammar:

$S \rightarrow Aa$

$A \rightarrow a \mid \epsilon$

Now:

$\text{FIRST}(a) = \{a\}$

and:

$\text{FOLLOW}(A) = \{a\}$

Intersection:

$\{a\}$

Thus:

$\boxed{\text{Not LL}(1)}$

59. SELECT Set Example

For:

$A \rightarrow aB \mid \epsilon$

suppose:

$\text{FOLLOW}(A) = \{c, d\}$

Then:

Production 1

$A \rightarrow aB$

$\text{SELECT} = \{a\}$

Production 2

$A \rightarrow \epsilon$

$\text{SELECT} = \{c, d\}$

They are disjoint.

So there is no conflict for A .

60. More General SELECT Example

Suppose:

$A \rightarrow BC$

and:

$\text{FIRST}(BC) = \{a, b, \epsilon\}$

while:

$\text{FOLLOW}(A) = \{c, d\}$

Then:

$\text{SELECT}(A \rightarrow BC) = \{a, b\} \cup \{c, d\}$

because RHS can derive μ .

Thus:

$\boxed{\text{SELECT}(A \rightarrow BC) = \{a, b, c, d\}}$

61. LL(1) and Left Recursion

An LL(1) grammar cannot be left recursive.

Why?

Suppose:

$A \rightarrow A^\alpha$

A top-down LL parser could attempt to expand A recursively without consuming input.

This is incompatible with deterministic predictive expansion.

Therefore:

$\boxed{\text{LL(1) grammar is not left recursive}}$

62. Converse Is False

If a grammar is not left recursive, is it necessarily LL(1)?

'L No.

Example:

$S \rightarrow aA \mid aB$

No left recursion.

But FIRST/FIRST conflict exists.

Therefore:

$\boxed{\text{No left recursion} \not\rightarrow \text{LL(1)}}$

63. LL(1) and Ambiguity

An LL(1) grammar is:

$\boxed{\text{Unambiguous}}$

Why?

At each step, one lookahead uniquely chooses one production.

There cannot be two different valid leftmost derivations for the same input.

Therefore:

$\boxed{\text{LL(1)} \rightarrow \text{Unambiguous}}$

64. Converse Is False

Does every unambiguous grammar have to be LL(1)?

'L No.

There are many unambiguous grammars that cannot be parsed with one-symbol LL lookahead.

Therefore:

$\boxed{\text{Unambiguous} \not\rightarrow \text{LL}(1)}$

65. Ambiguous Grammar Cannot Be LL(1)

Since LL(1) grammars are unambiguous:

$\boxed{\text{Ambiguous grammar} \rightarrow \text{Not LL}(1)}$

This is a useful GATE shortcut.

66. Left Factoring and LL(1)

If grammar is LL(1), alternatives must already be distinguishable with one lookahead.

A common prefix often produces a FIRST/FIRST conflict.

Therefore left factoring frequently helps.

But:

$\boxed{\text{Left factored} \not\rightarrow \text{LL}(1)}$

You must still test FIRST/FOLLOW.

67. Important Grammar Relationship Summary

Guaranteed:

$\boxed{\text{LL}(1) \rightarrow \text{Unambiguous}}$

$\boxed{\text{LL}(1) \rightarrow \text{No left recursion}}$

Not guaranteed:

$\boxed{\text{Unambiguous} \not\rightarrow \text{LL}(1)}$

$\boxed{\text{No left recursion} \not\rightarrow \text{LL}(1)}$

$\boxed{\text{Left factored} \not\rightarrow \text{LL}(1)}$

68. What About μ in Multiple Alternatives?

Suppose:

$A \rightarrow \epsilon \mid \epsilon$

Clearly the same lookahead from FOLLOW(A) would select both productions.

So not LL(1).

More generally, if two alternatives can both derive μ their SELECT sets both contain FOLLOW(A), producing conflict whenever FOLLOW(A) is nonempty.

Thus an LL(1) choice cannot have two distinct nullable alternatives.

69. Parsing Table Size

Suppose a grammar has:

- m non-terminals
- n terminals

The predictive parsing table has approximately:

$m \times (n+1)$ cells if \$ is included as an input-column marker.

Many cells may be error entries.

GATE may ask simple table-size/count questions.

70. Error Entries

An empty cell:

$M[A, a]$

means:

No production of A is valid when the current lookahead is a .

Therefore:

Syntax error

Later compiler courses sometimes add **synch** entries for panic-mode error recovery, but that is not central to your pasted syllabus unless needed in a question.

71. Important Table Construction Trap

For:

$A \rightarrow \epsilon$

students sometimes place the production under the column:

TEXT
 μ

'LWrong.

There is normally **no input column**.

Instead put the μ - p r o d u c t i o n

$\text{FOLLOW}(A)$

columns.

Example:

$\text{FOLLOW}(A) = \{\epsilon, \$\}$

Then:

TEXT

$$\begin{aligned} M[A, \mid] &= A \mid \mu \\ M[A, \$] &= A \mid \mu \end{aligned}$$

72. Another Trap — Put Production Under FOLLOW Even When RHS Isn't Nullable?

No.

For:

$$\mid \rightarrow aB \mid$$

if:

$$\epsilon \notin \text{FIRST}(aB) \mid$$

place it only under:

$$\text{FIRST}(aB) \mid$$

Do not use FOLLOW(A).

FOLLOW is added to SELECT only when RHS is nullable.

73. GATE-STYLE EXAMPLE — Determine LL(1)

Grammar:

$$S \rightarrow aA \mid b$$
$$A \rightarrow c \mid \epsilon$$

Suppose:

$$\text{FOLLOW}(A) = \{ \$ \} \mid$$

For S alternatives:

$$\text{FIRST}(aA) = \{ a \} \mid$$
$$\text{FIRST}(b) = \{ b \} \mid$$

disjoint.

For A:

$$\text{FIRST}(c) = \{ c \} \mid$$

and ϵ alternative uses:

$$\text{FOLLOW}(A) = \{ \$ \} \mid$$

Again disjoint.

Therefore:

$$\boxed{\text{LL}(1)}$$

74. GATE-STYLE EXAMPLE — Hidden FIRST/FIRST Conflict

Grammar:

$$A \rightarrow BC \mid bD$$

Suppose:

$$\text{FIRST}(B) = \{\epsilon, b\}$$

$$\text{FIRST}(C) = \{c\}$$

Then:

$$\text{FIRST}(BC) = \{b, c\}$$

because B may start with **b** or disappear and let C start with **c**.

Other alternative:

$$\text{FIRST}(bD) = \{b\}$$

Intersection:

$$\{b, c\} \cap \{b\} = \{b\}$$

Therefore:

$$\boxed{\text{FIRST/FIRST conflict}}$$

Not LL(1).

GATE Lesson

Always compute FIRST of the **whole RHS**, not just its first grammar symbol.

75. GATE-STYLE Example — Hidden FIRST/FOLLOW Conflict

Grammar:

$$S \rightarrow ABc$$

$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow b \mid \epsilon$$

For **A**, suppose we want to check:

$$A \rightarrow a \mid \epsilon$$

What is FOLLOW(A)?

After A comes:

$$Bc$$

Since:

$$\text{FIRST}(B) = \{b, \epsilon\}$$

we get:

$$\text{FIRST}(Bc) = \{b, c\}$$

Thus:

$$\text{FOLLOW}(A) \supseteq \{b, c\}$$

Since:

$\text{FIRST}(a) = \{a\}$

there is no conflict:

$\{a\} \cap \{b, c\} = \emptyset$

So A's alternatives are okay.

76. GATE Shortcut — Fast LL(1) Inspection

When given:

$A \rightarrow \alpha \mid \beta$

ask:

Step 1

Do the FIRST sets overlap?

If yes:

$\boxed{\text{Not LL(1)}}$

Step 2

Does one side derive μ ?

If yes, check:

$\text{FIRST}(\text{other side}) \cap \text{FOLLOW}(A)$

If nonempty:

$\boxed{\text{Not LL(1)}}$

This solves many questions without constructing the entire table.

77. GATE TRAPS

Trap 1

LL(1)'s **1** means one production per non-terminal.

'L False.

It means:

$\boxed{1 \text{ lookahead token}}$

Trap 2

An LL(1) parser constructs a rightmost derivation.

'L False.

It constructs a:

$\boxed{\text{Leftmost derivation}}$

Trap 3

If FIRST sets of alternatives are disjoint, grammar must be LL(1).

'L Not always.

If μ is involved, FIRST/FOLLOW conflict must also be checked.

Trap 4

μ is a column in the LL(1) parsing table.

'L Normally false.

μ is not an input token.

Use FOLLOW columns for μ - p r o d u c t i o n s .

Trap 5

Two productions in one table cell are acceptable if one contains μ .

'L False.

Any multiple-entry cell means the grammar is not LL(1).

Trap 6

Every unambiguous grammar is LL(1).

'L False.

LL(1) is only a subset of unambiguous CFGs.

Trap 7

Every non-left-recursive grammar is LL(1).

'L False.

FIRST conflicts may remain.

Trap 8

Every left-factored grammar is LL(1).

'L False.

FIRST/FOLLOW conflicts may remain.

Trap 9

An LL(1) grammar can be ambiguous.

'LFalse.

$\boxed{LL(1) \rightarrow \text{Unambiguous}}$

Trap 10

When using:

$A \rightarrow XYZ$

push **X**, then **Y**, then **Z** onto a stack whose right end is the top.

'LThat would leave **Z** on top.

Push RHS in **reverse** so **X** is processed first.

78. MINI PRACTICE — PART 9

Try these before the solutions.

Q1 — Easy

What does the second **L** in LL(1) mean?

A. Left recursion B. Leftmost derivation C. Left factoring D. Longest match

Q2 — Easy

If one LL(1) table cell contains two productions, then:

A. Grammar is definitely LL(1) B. Grammar is not LL(1) C. Choose either production D. Use both simultaneously

Q3 — Medium

Consider:

$A \rightarrow aB \mid bC$

Is there a FIRST/FIRST conflict?

Q4 — Medium

Consider:

$S \rightarrow Aa$

$A \rightarrow a \mid \epsilon$

Is this grammar LL(1)?

Q5 — Tricky

Suppose:

$\mid \rightarrow BC \mid d$

and:

$\text{FIRST}(B) = \{\epsilon, d\}$

$\text{FIRST}(C) = \{c\}$

Is there a FIRST/FIRST conflict between the two alternatives of A?

SOLUTIONS

Q1

' B — Leftmost derivation

$\text{LL}(1)$

means:

TEXT

Left-to-right
Leftmost derivation
1 lookahead

Q2

' B — Not $\text{LL}(1)$

One table cell must never require two production choices.

Q3

$\text{FIRST}(aB) = \{a\}$

$\text{FIRST}(bC) = \{b\}$

Intersection:

$\emptyset$

So:

$\boxed{\text{No FIRST/FIRST conflict}}$

Q4

For:

$\mid \rightarrow a \mid \epsilon$

we have:

$\text{FIRST}(a) = \{a\}$

From:

$\rightarrow Aa$

we have:

$\text{FOLLOW}(A) = \{a\}$

Thus:

$\text{FIRST}(a) \cap \text{FOLLOW}(A) = \{a\}$

Therefore:

$\boxed{\text{Not LL}(1)}$

It has a FIRST/FOLLOW conflict.

Q5

Find:

$\text{FIRST}(BC)$

Since:

$\text{FIRST}(B) = \{\epsilon, d\}$

B can contribute:

d

and because B is nullable, C contributes:

c

Thus:

$\text{FIRST}(BC) = \{d, c\}$

Other alternative:

$\text{FIRST}(d) = \{d\}$

Intersection:

$\{d, c\} \cap \{d\} = \{d\}$

Therefore:

$\boxed{\text{Yes, FIRST/FIRST conflict}}$

So this grammar is not LL(1).

79. WRITE THIS IN NOTES — LL(1) MASTER SHEET

Meaning

$\boxed{\text{LL}(1) = \text{Left-to-right} + \text{Leftmost} + 1 \text{ lookahead}}$

FIRST/FIRST Condition

For:

$\forall A \rightarrow \alpha \beta$

need:

$\boxed{\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset}$

with proper treatment.

FIRST/FOLLOW Condition

If:

$\forall \epsilon \in \text{FIRST}(\alpha)$

then:

$\boxed{(\text{FIRST}(\beta) - \{\epsilon\}) \cap \text{FOLLOW}(A) = \emptyset}$

SELECT

For:

$\forall A \rightarrow \alpha$

if RHS non-nullable:

$\boxed{\text{SELECT} = \text{FIRST}(\alpha)}$

If RHS nullable:

$\boxed{\text{SELECT} = (\text{FIRST}(\alpha) - \{\epsilon\}) \cup \text{FOLLOW}(A)}$

LL(1) requires SELECT sets for alternatives of the same LHS to be pairwise disjoint.

Table Construction

For:

$\forall A \rightarrow \alpha$

put production in:

$M[A, a]$

for all:

$a \in \text{FIRST}(\alpha) - \{\epsilon\}$

If RHS is nullable, also put it under:

$\text{FOLLOW}(A)$

LL(1) Table Test

$\boxed{\text{At most one production per table cell}}$

80. RELATIONSHIP SHEET

Always true:

$\boxed{LL(1) \rightarrow \text{Unambiguous}}$

$\boxed{LL(1) \rightarrow \text{No left recursion}}$

Not always true:

$\boxed{\text{Unambiguous} \not\rightarrow LL(1)}$

$\boxed{\text{No left recursion} \not\rightarrow LL(1)}$

$\boxed{\text{Left factored} \not\rightarrow LL(1)}$

30-SECOND REVISION

LL(1)

Left, Left, Look-one

Conflict 1

$\boxed{\text{FIRST/FIRST overlap}}$

means two normal alternatives predict the same token.

Conflict 2

$\boxed{\text{FIRST/FOLLOW overlap}}$

happens when μ is possible and the parser cannot decide between disappearing and taking another production.

Best shortcut

For each production alternative compute:

$\boxed{\text{SELECT}}$

Then ask:

Do SELECT sets overlap?

Yes!' not LL(1).

No !' LL(1), assuming this holds for every non-terminal.

Parsing table

TEXT

Row = non-terminal

Column = lookahead terminal

Cell = production

μproduction

Put under FOLLOW columns, not an μcolumn.

Stack

TEXT

```
Terminal top + same lookahead !' match
Non-terminal top !' consult table
μ production !' pop, push nothing
$ and $ !' ACCEPT
```

We have now completed the core **top-down LL parsing machinery**. Next we move to the other major parsing philosophy: **bottom-up parsing**, which is the foundation for LR(0), SLR, CLR and LALR.

Completed: Parts 1–9/31 | Next: Part 10 — Shift-Reduce Parsing, Handles, Handle Pruning, Viable Prefixes & Parsing Conflicts

Reply **NEXT** for **Part 10**.

USER

NEXT

ASSISTANT

Compiler Design — Part 10: Shift-Reduce Parsing, Handles & Viable Prefixes

Completed: Parts 1–9/31 | Current: Part 10 | Remaining: Parts 11–31

We now move from **top-down parsing** to **bottom-up parsing**.

This is the foundation for:

- LR(0)
- SLR(1)
- CLR(1)
- LALR(1)

The central idea changes from:

$\text{[} \text{\textit{Start symbol}} \text{]} \rightarrow \text{\textit{input}} \text{]}$

to:

$\text{\textit{Input}} \rightarrow \text{\textit{Start symbol}} \text{]}$

1. What Is Bottom-Up Parsing?

CONCEPT

A bottom-up parser starts with the input string and repeatedly reduces substrings until only the start symbol remains.

Conceptually:

```
TEXT
Input string
! "
Find suitable substring
! "
Reduce it to a non-terminal
! "
Repeat
! "
Start symbol
```

It builds a parse tree:

$\boxed{\text{Leaves}} \rightarrow \text{Root}$

Hence the name **bottom-up**.

WRITE THIS IN NOTES

Bottom-up parsing: Construct the parse tree from leaves to root by repeatedly reducing suitable substrings of the input to grammar non-terminals.

2. Most Important Relationship

A bottom-up parser constructs:

$\boxed{\text{Rightmost derivation in reverse}}$

This fact is fundamental for LR parsing.

Recall:

```
TEXT
Top-down LL
! ' L e f t m o s t d e r i v a t i o n

Bottom-up LR
! ' R i g h t m o s t d e r i v a t i o n i n r e v e r s e
```

Memory Trick

LL

Leftmost forward

LR

Rightmost reversed

Or:

$\boxed{LR = \text{Left-to-right input} + \text{Rightmost derivation in reverse}}$

3. Simple Bottom-Up Example

Grammar:

$S \rightarrow aAb$

$A \rightarrow c$

Input:

acb

Start with:

TEXT

a c b

Recognize:

c

as RHS of:

$A \rightarrow c$

Reduce:

TEXT

a c b
! "
a A b

Now:

aAb

matches RHS of:

$S \rightarrow aAb$

Reduce:

$aAb \rightarrow S$

So:

TEXT

acb
! "
aAb
! "
S

Accepted.

4. What Does "Reduce" Mean?

Suppose grammar contains:

$A \rightarrow \beta$

If the parser identifies a suitable occurrence of β , it replaces that occurrence with A .

That is a:

$\boxed{\text{Reduction}}$

Example:

$A \rightarrow id$

Then:

```
TEXT
id
!" reduce
A
```

Reduction is essentially the **reverse use of a production**.

Production:

$A \rightarrow \beta$

Bottom-up reduction:

$\boxed{\beta \rightarrow A}$

5. Shift-Reduce Parsing

A common bottom-up method is:

$\boxed{\text{Shift-Reduce Parsing}}$

It uses a stack and performs four main types of actions:

1. Shift
2. Reduce
3. Accept
4. Error

Memory Trick

SRAE

```
TEXT
S !' Shift
R !' Reduce
A !' Accept
E !' Error
```

6. Shift Operation

CONCEPT

Shift means:

Move the next input symbol onto the stack.

Example:

TEXT

Stack: \$
Input: id + id \$

Shift **id**:

TEXT

Stack: \$ id
Input: + id \$

Nothing is replaced yet.

We simply move input to the stack.

WRITE THIS IN NOTES

$\boxed{\text{SHIFT = move next input symbol onto stack}}$

7. Reduce Operation

Suppose stack contains:

TEXT

. . . ²

and grammar has:

$A \rightarrow \beta$

If that occurrence of β is the correct one to reduce, replace it by **A**.

Example:

$E \rightarrow id$

Stack:

TEXT

\$ id

Reduce:

TEXT

\$ E

WRITE THIS IN NOTES

$\boxed{\text{REDUCE}} \rightarrow \text{using } A \rightarrow \text{replaced by } A$

8. Accept

When the complete input has been consumed and the stack represents the start symbol appropriately:

$\boxed{\text{ACCEPT}}$

Conceptually:

TEXT

Stack: \$ S

Input: \$

! Accept.

Exact LR implementations also use parser states, which we'll study later.

9. Error

If neither a valid shift nor a valid reduction nor acceptance is possible:

$\boxed{\text{Syntax error}}$

10. Example of Shift-Reduce Parsing

Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow \text{id}$

Input:

$\text{id} + \text{id}$

Let's understand conceptually.

Initial:

TEXT

Stack: \$

Input: id + id \$

Step 1 — Shift

Shift `id`.

TEXT

Stack: \$ `id`
Input: + `id` \$

Step 2 — Reduce

Using:

$\backslash [T \rightarrow id \]$

reduce:

TEXT

\$ `id`
! "
\$ `T`

Step 3 — Reduce

Using:

$\backslash [E \rightarrow T \]$

reduce:

TEXT

\$ `T`
! "
\$ `E`

Step 4 — Shift +

TEXT

Stack: \$ `E` +
Input: `id` \$

Step 5 — Shift `id`

TEXT

Stack: \$ `E` + `id`
Input: \$

Step 6 — Reduce

$\backslash [T \rightarrow id \]$

gives:

TEXT

\$ E + T

Step 7 — Reduce

Using:

$E \rightarrow E+T$

gives:

TEXT

\$ E

Input:

TEXT

\$

Accepted.

11. Notice the Reverse Derivation

Our reductions were:

$id+id$

$T+id$

$E+id$

$E+T$

E

Read this backwards:

$E \rightarrow E+T \rightarrow E+id \rightarrow T+id \rightarrow id+id$

This is a **rightmost derivation**.

Therefore bottom-up reductions give:

$\boxed{\text{Rightmost derivation in reverse}}$

12. The Big Question: What Exactly Should Be Reduced?

Suppose stack contains symbols matching the RHS of a grammar production.

Can we always reduce them?

'L No.

Only a special substring called the:

$\boxed{\text{HANDLE}}$

should be reduced.

This is one of the most important concepts in bottom-up parsing.

13. What Is a Handle?

INTUITION

A **handle** is the substring that corresponds to the production used in the **most recent step of the rightmost derivation**.

When reversing that derivation, this is the substring that should be reduced next.

Formal Definition

Suppose a right-sentential form is:

$\gamma = \alpha \beta w$

and there is a production:

$A \rightarrow \beta$

such that in the rightmost derivation:

$S \xrightarrow{*} \alpha A w \xrightarrow{} \alpha \beta w$

Then the occurrence of β in:

$\alpha \beta w$

is a **handle**.

For GATE, focus more on the intuition:

Handle = correct RHS occurrence to reduce in order to reverse a rightmost derivation.

WRITE THIS IN NOTES

$\boxed{\text{Handle} = \text{RHS occurrence that should be reduced next in reverse rightmost derivation}}$

14. Handle Is More Than "Any RHS"

This is a classic trap.

Suppose grammar contains:

$A \rightarrow ab$

and the current stack/string contains **ab**.

Does that automatically make it a handle?

'L Not necessarily.

It is a handle only if reducing **that occurrence** corresponds to reversing the appropriate rightmost derivation.

Therefore:

$\boxed{\text{Every handle matches some production RHS}}$

but:

$\boxed{\text{Every RHS occurrence is not necessarily a handle}}$

15. Handle Example

Grammar:

$E \rightarrow E+T \mid T$

$T \rightarrow \text{id}$

String:

$\text{id}+\text{id}$

A rightmost derivation is:

$E \rightarrow E+T \rightarrow E+\text{id} \rightarrow T+\text{id} \rightarrow \text{id}+\text{id}$

Reverse it:

$\text{id}+\text{id}$

First handle:

$\boxed{\text{left id}}$

because the last forward derivation step was:

$T+\text{id} \rightarrow \text{id}+\text{id}$

using:

$T \rightarrow \text{id}$

Reduce:

$\text{id}+\text{id} \rightarrow T+\text{id}$

Next handle:

T

using:

$E \rightarrow T$

Then:

$T+\text{id} \rightarrow E+\text{id}$

Next handle:

id

etc.

16. Handle Includes Position

Suppose the same substring appears more than once.

Example:

```
TEXT
id + id
```

There are two occurrences of `id`.

Saying:

"The handle is id"

may be incomplete.

Which `id` ?

The **position** matters.

A handle is technically:

- a production RHS
- plus its particular occurrence/position in the right-sentential form.

17. Handle Pruning

Repeatedly finding the handle and reducing it is called:

$\boxed{\text{Handle Pruning}}$

Conceptually:

```
TEXT
Input string
! "
Find handle
! "
Reduce
! "
Find next handle
! "
Reduce
! "
...
! "
Start symbol
```

Thus:

Bottom-up parsing is often described as handle pruning.

WRITE THIS IN NOTES

Handle pruning: Repeatedly identify the handle of a right-sentential form and reduce it until the start symbol is reached.

18. Why "Pruning"?

Imagine the parse tree being built from bottom to top.

Reducing a handle collapses children into their parent.

Example:

TEXT



At the bottom-up stage:

TEXT

a b

is replaced with:

TEXT

A

It's like pruning/collapsing part of the tree.

19. Another Handle Example

Grammar:

$\backslash \ S \rightarrow aABe \ \backslash$

$\backslash \ A \rightarrow Abc \mid b \ \backslash$

$\backslash \ B \rightarrow d \ \backslash$

Suppose we have some right-sentential form. GATE may ask:

Which substring is the handle?

The safest method is:

1. reconstruct or inspect the rightmost derivation,
2. identify the production used most recently,
3. its RHS occurrence is the next handle when going backward.

Do not simply look for the longest RHS.

20. Handle Is Not Necessarily the Longest Matching RHS

Very important.

Suppose current stack/string has multiple substrings matching productions.

The handle is selected based on **derivation structure**, not:

- longest substring
- leftmost substring
- rightmost substring alone

LR parsing machinery determines this systematically.

21. Shift-Reduce Parser Stack

A conceptual shift-reduce parser has:

TEXT

Input

```
      ! "
% % % % % % % % % % % % %
%   P a r s e r   %
% % % % % % % % % % % % %
      ! '
Stack
```

It repeatedly decides:

TEXT

Shift?
Reduce?
Accept?
Error?

In a real LR parser, **states** on the stack and ACTION/GOTO tables make these decisions deterministic.

We begin that in Part 11.

22. Shift-Reduce Conflict

Now one of the most famous compiler terms.

Suppose at some point the parser can validly:

1. **shift** the next input symbol, or
2. **reduce** the current stack handle

but cannot uniquely decide which.

This is:

$\boxed{\text{Shift/Reduce Conflict}}$

WRITE THIS IN NOTES

Shift/reduce conflict: Parser cannot decide whether to shift the next input token or reduce the current stack content.

23. Intuition for Shift/Reduce Conflict

Imagine:

TEXT

S t a c k : . . . ±
Input: a ...

The parser has evidence for:

TEXT

R e d u c e ± n o w

but also evidence for:

TEXT

Shift a and continue building a longer construct

If both are possible in the same parsing-table situation:

$\boxed{\text{S/R conflict}}$

24. Classic Dangling-Else Conflict

Recall:

TEXT

if E1 then if E2 then S1 else S2

When parser reaches:

TEXT

if E1 then if E2 then S1

and lookahead is:

TEXT

else

it may face a conceptual choice:

Reduce

Treat:

TEXT

if E2 then S1

as complete.

or

Shift

Shift `else` so it attaches to the inner `if`.

This is the classic **shift/reduce conflict** in the ambiguous dangling-else grammar.

Programming-language convention generally shifts `else` to attach it to the nearest unmatched `if`.

25. Reduce/Reduce Conflict

Suppose the parser has two different possible reductions and cannot choose between them.

For example:

$\backslash A \rightarrow \beta \backslash$

and:

$\backslash B \rightarrow \beta \backslash$

and the parser cannot determine whether the current $\backslash(\beta)\backslash$ should reduce to `A` or `B`.

This is:

$\backslash \boxed{\text{Reduce/Reduce Conflict}} \backslash$

WRITE THIS IN NOTES

Reduce/reduce conflict: Parser cannot decide which of two or more productions should be used for reduction.

26. S/R vs R/R

Conflict	Choice
Shift/Reduce	Shift next token or reduce
Reduce/Reduce	Reduce using production 1 or production 2

Memory Trick

TEXT

S / R ! ' Move forward OR collapse
R / R ! ' Which collapse ?

27. Which Conflict Is Usually Considered More Serious?

In parser construction, both mean the grammar/parser specification is not deterministic for that parser class.

Reduce/reduce conflicts are often especially problematic because there are two competing completed constructs.

But for GATE:

Do not memorize "one is always worse" as a formal property.

Focus on correctly identifying the conflict type.

28. Example of Reduce/Reduce Conflict

Grammar:

$$[S \rightarrow A \mid B]$$
$$[A \rightarrow a]$$
$$[B \rightarrow a]$$

For input:

TEXT

a

after recognizing **a**, it could potentially reduce:

$$[a \rightarrow A]$$

or:

$$[a \rightarrow B]$$

This is the intuitive form of a:

$$[\boxed{\text{Reduce/Reduce conflict}}]$$

The grammar is also ambiguous because:

$$[S \rightarrow A \rightarrow a]$$

and:

$$[S \rightarrow B \rightarrow a]$$

give distinct parse trees.

29. Ambiguous Grammar and LR Parsing

A deterministic LR parsing table cannot parse an ambiguous grammar **without conflict/resolution conventions**.

Thus an ambiguous grammar is not LR(k) in the strict deterministic grammar-class sense.

A common GATE implication is:

$$[\boxed{\text{LR grammar}} \rightarrow \text{Unambiguous}]$$

We will formalize parser-class relationships later.

30. What Is a Viable Prefix?

This is another major LR concept.

INTUITION

During shift-reduce parsing, the stack contains a prefix of a right-sentential form that has not gone **past the handle**.

Such a prefix is called a:

$\boxed{\text{Viable Prefix}}$

Formal Definition

A **viable prefix** is a prefix of a right-sentential form that does not extend beyond the right end of the handle.

In simpler language:

It is a prefix that can legitimately appear on the stack of a shift-reduce parser.

WRITE THIS IN NOTES

$\boxed{\text{Viable prefix} = \text{prefix of right-sentential form not extending beyond handle}}$

Equivalent exam intuition:

Exactly the kinds of symbol sequences that can appear on the stack of an LR parser.

31. Why Do We Need Viable Prefixes?

The parser must know:

Is the current stack still a valid partial path toward some successful reduction?

LR states recognize viable prefixes.

This is why LR item automata are useful.

Conceptually:

TEXT

Stack content

!"

Is it a viable prefix?

!"

LR state / DFA tracks this

This becomes crystal clear once we study LR(0) items.

32. Example Intuition

Suppose production:

$\backslash [E \rightarrow E+T] \backslash$

During parsing, stack might contain:

TEXT

E

or:

TEXT

E +

or:

TEXT

E + T

These can be meaningful prefixes of a right-sentential form up to the handle boundary.

But once the parser shifts symbols that go beyond where a valid handle could occur, it would no longer represent an appropriate viable prefix.

33. Handle vs Viable Prefix

Do not confuse them.

Handle

The exact substring that should be reduced next.

Viable prefix

A legal stack prefix that does not extend beyond the handle.

Thus there can be many viable prefixes on the way toward recognizing a handle.

Memory Trick

Handle = what to reduce.

Viable prefix = how far the stack can safely grow.

34. Important Property

Every prefix of a handle is a viable prefix.

Also, viable prefixes may include symbols to the left of the handle.

For example, if right-sentential form is conceptually:

$$[\alpha\beta w]$$

where β is handle, then prefixes ending anywhere within:

$$[\alpha\beta]$$

can be viable.

But a prefix extending beyond the handle into w is not viable for that configuration.

35. Why "Right-Sentential Form"?

A **right-sentential form** is a sentential form that occurs in a rightmost derivation.

Bottom-up parsing reverses rightmost derivations.

So handles and viable prefixes are naturally defined with respect to right-sentential forms.

36. Shift-Reduce Parsing and Viable Prefixes

At any valid point during an LR shift-reduce parse:

$$[\boxed{\text{Stack contents form a viable prefix}}]$$

This is a very useful GATE fact.

The LR automaton is effectively a DFA for the set of viable prefixes.

We will revisit this immediately in LR(0).

37. What Is the "Handle" at Acceptance?

Suppose the entire RHS of the start production has been recognized.

Eventually reduction gives:

$$[S]$$

Then no further ordinary reduction is required; parser accepts when the augmented start condition is reached.

To make this precise, LR parsing uses an **augmented grammar**:

$$[S \rightarrow S]$$

We'll introduce this in Part 11.

38. Bottom-Up Can Handle Left Recursion

Recall left-recursive grammar:

$$[E \rightarrow E + T \mid T]$$

This is inconvenient for top-down recursive descent.

But bottom-up parsing works naturally with it.

For example:

TEXT

id + id

can reduce:

TEXT

i d ! ' T ! ' E

then shift **+**, recognize another **T**, and reduce:

$[E + T \rightarrow E]$

Therefore:

$\boxed{\text{Left recursion is not inherently a problem for LR parsing}}$

39. Bottom-Up Is Generally More Powerful Than LL

LR parsing can handle a larger class of deterministic context-free grammars than LL(1) parsing.

Intuitively:

- LL must predict which production to use **before** seeing its entire RHS.
- LR can wait until it has seen enough input to identify a handle and then reduce.

This extra information gives LR more power.

We'll formalize the hierarchy later.

40. Why Does LR "Wait Longer"?

Suppose grammar alternatives share prefixes.

A top-down parser may need to choose early.

A bottom-up parser shifts input and accumulates information before deciding which reduction applies.

So:

LL predicts a production. LR recognizes a completed RHS.

This is a very useful mental model.

41. Top-Down vs Bottom-Up Master Table

Top-Down / LL	Bottom-Up / LR
Start symbol !' input	Input !' start symbol
Root !' leaves	Leaves !' root
Expands non-terminals	Reduces RHS to LHS
Leftmost derivation	Rightmost derivation in reverse

Top-Down / LL	Bottom-Up / LR
Predicts production early	Recognizes handle after seeing input
Left recursion problematic	Left recursion generally acceptable
Simpler parser class	More powerful parser class

42. Shift vs Reduce Example

Grammar:

$E \rightarrow E + E \mid id \mid \epsilon$

Suppose:

TEXT

Stack: E + E
Input: + id \$

Should parser reduce:

$E + E \rightarrow E$

or shift the next + ?

If grammar does not encode associativity, this can lead to a shift/reduce conflict.

If + should be left-associative, reducing first corresponds to:

$(E + E) + E$

If shifting can lead to:

$E + (E + E)$

This shows the connection between ambiguity and shift/reduce conflicts.

43. Precedence and Shift/Reduce Conflicts

Consider ambiguous grammar:

$E \rightarrow E + E \mid E * E \mid id \mid \epsilon$

For:

$id + id * id$

at some parser configuration it may need to decide whether to:

- reduce $E + E$
- shift $*$

If $*$ has higher precedence, desired decision is:

$\boxed{\text{Shift } *}$

so multiplication can form first.

Parser generators often allow precedence/associativity declarations to resolve such conflicts, but for GATE the central

idea is:

Ambiguous expression grammars naturally lead to shift/reduce conflicts.

44. Associativity and Shift/Reduce

For:

$\{ E \rightarrow E + E \mid id \}$

input:

$\{ id + id + id \}$

At the second $+$, parser may choose:

Reduce

$\{ (id + id) + id \}$

! left associativity.

Shift

Potentially produce:

$\{ id + (id + id) \}$

! right associativity.

So precedence and associativity can affect conflict resolution.

45. Important Caveat

Do not conclude:

Every shift/reduce conflict means the language itself is ambiguous.

A conflict may occur because the chosen parsing method is too weak for the grammar, even when the grammar/language situation is more nuanced.

For specific LR tables, conflict means:

That particular grammar is not deterministic for that parser construction.

We'll distinguish LR(0), SLR, LALR, CLR conflicts later.

46. GATE-STYLE EXAMPLE 1 — Identify Action

Stack:

TEXT

\$ E +

Input:

```
TEXT
id $
```

If parser moves `id` from input to stack, the action is:

$\boxed{\text{SHIFT}}$

47. GATE-STYLE EXAMPLE 2 — Identify Reduction

Grammar:

$T \rightarrow id$

Stack:

```
TEXT
$ E + id
```

If parser replaces `id` by `T`:

$\boxed{\text{REDUCE using } T \rightarrow id}$

48. GATE-STYLE EXAMPLE 3 — Handle

If a rightmost derivation has:

$S \xrightarrow{*} \alpha A w \xrightarrow{\quad} \alpha \beta w$

using:

$A \rightarrow \beta$

then in:

$\alpha \beta w$

the occurrence of:

$\boxed{\beta}$

is the handle.

49. GATE-STYLE EXAMPLE 4 — Conflict

Parser can either:

- shift current lookahead `*`
- reduce `E+E` to `E`

What conflict?

$\boxed{\text{Shift/Reduce conflict}}$

50. GATE-STYLE EXAMPLE 5 — Conflict

Parser can reduce the same stack portion using either:

$\text{A} \rightarrow \beta$

or:

$\text{B} \rightarrow \beta$

What conflict?

$\boxed{\text{Reduce/Reduce conflict}}$

51. GATE-STYLE EXAMPLE 6 — Viable Prefix

Which statement best describes a viable prefix?

A. Any prefix of the input B. Any RHS of a production C. A prefix of a right-sentential form that does not extend beyond the handle D. A prefix beginning only with terminals

' C

52. GATE TRAPS

Trap 1

Bottom-up parser constructs a leftmost derivation.

'L False.

$\boxed{\text{Reverse rightmost derivation}}$

Trap 2

Reduce means replacing a non-terminal by its RHS.

'L That's forward derivation.

Bottom-up reduction does:

$\boxed{\text{RHS} \rightarrow \text{LHS}}$

Trap 3

Any RHS occurrence is a handle.

'L False.

It must be the **correct occurrence to reduce next** in reverse rightmost derivation.

Trap 4

A handle is necessarily the longest RHS currently present.

'L False.

Trap 5

Shift/reduce means choosing between two reductions.

'L False.

That's reduce/reduce.

Trap 6

Viable prefix must be a complete handle.

'L False.

It can end before the handle is complete.

Trap 7

A viable prefix can extend beyond the right end of the handle.

'L False by definition.

Trap 8

Left recursion prevents LR parsing.

'L False.

LR parsers handle left recursion naturally.

Trap 9

LR scans the input right-to-left.

'L False.

The L means:

$\boxed{\text{Left-to-right scan}}$

Trap 10

R in LR means right-to-left input.

'L False.

It refers to:

$\boxed{\text{Rightmost derivation in reverse}}$

53. MINI PRACTICE — PART 10

Try before checking solutions.

Q1 — Easy

A bottom-up parser builds a parse tree:

A. Root to leaves B. Leaves to root C. Left to right only D. Right to left only

Q2 — Easy

A reduction using:

$A \rightarrow xyz$

replaces:

A. A by xyz B. xyz by A C. x by A D. A by μ

Q3 — Medium

Bottom-up parsing corresponds to:

A. Leftmost derivation B. Rightmost derivation C. Reverse leftmost derivation D. Rightmost derivation in reverse

Q4 — Medium

What is a handle?

A. Any terminal on the stack B. Any production RHS appearing in the input C. The RHS occurrence that should be reduced next when reversing a rightmost derivation D. The start symbol

Q5 — Tricky

A parser configuration allows:

- reducing stack symbols using $A \rightarrow \beta$
- reducing the same configuration using $B \rightarrow \gamma$

with no unique choice.

What conflict is this?

SOLUTIONS

Q1

' B — Leaves to root

$\boxed{\text{Bottom-up = input !' start symbol}}$

Q2

' B

Production:

$\text{\[A\rightarrow xyz \]}$

Reduction:

$\text{\[\boxed{xyz\rightarrow A} \]}$

Q3

' D

$\text{\[\boxed{\text{Rightmost derivation in reverse}} \]}$

Q4

' C

A handle is not just any matching RHS. It is the correct RHS occurrence for the next reduction.

Q5

There are two competing reductions.

Therefore:

$\text{\[\boxed{\text{Reduce/Reduce conflict}} \]}$

54. WRITE THIS IN NOTES — PART 10 MASTER SHEET

Bottom-Up

$\text{\[\boxed{\text{Input}\rightarrow\text{Start Symbol}} \]}$

$\text{\[\boxed{\text{Leaves}\rightarrow\text{Root}} \]}$

$\text{\[\boxed{\text{Reverse Rightmost Derivation}} \]}$

Shift-Reduce Actions

Shift

$\text{\[\boxed{\text{Input token}\rightarrow\text{Stack}} \]}$

Reduce

For:

$\text{\[A\rightarrow\beta \]}$

perform:

$\text{\[\boxed{\beta\rightarrow A} \]}$

Accept

Successful complete parse.

Error

No valid action.

Handle

$\boxed{\text{Correct RHS occurrence to reduce next}}$

Handle Pruning

$\boxed{\text{Repeated handle detection + reduction}}$

until start symbol.

Viable Prefix

$\boxed{\text{Prefix of a right-sentential form not extending beyond handle}}$

GATE intuition:

Possible stack contents of an LR parser.

Conflicts

Shift/Reduce

TEXT

Shift next input?

OR

Reduce now?

Reduce/Reduce

TEXT

Reduce by production 1?

OR

Reduce by production 2?

30-SECOND REVISION

Bottom-up

Start with input, collapse toward start symbol.

LR

$\boxed{\text{Left-to-right scan + Rightmost derivation in reverse}}$

Four actions

SRAE

TEXT

Shift
Reduce
Accept
Error

Handle

What should I reduce now?

Viable prefix

What can legally be on the LR stack?

Conflicts

TEXT

S/R = shift OR reduce
R/R = reduce this OR reduce that

Critical difference from LL

TEXT

LL !' predicts production

LR !' recognizes handle

Left recursion

TEXT

Bad for ordinary LL recursive descent
Fine for LR-style bottom-up parsing

The next part begins the actual **LR machinery**. We will take productions such as:

$A \rightarrow XYZ$

insert a dot:

$A \rightarrow X \cdot YZ$

and use these **LR(0) items** to represent exactly how much of a production has been recognized.

Completed: Parts 1–10/31 | Next: Part 11 — LR(0) Items, Augmented Grammar, Closure, GOTO & Canonical Collection

Reply **NEXT** for **Part 11**.

USER

NEXT

ASSISTANT

Compiler Design — Part 11: LR(0) Items, Augmented Grammar, CLOSURE, GOTO & Canonical Collection

Completed: Parts 1–10/31 | Current: Part 11 | Remaining: Parts 12–31

This is the **foundation of the entire LR family**.

If you clearly understand these four things:

$\boxed{\text{LR(0) Item}}$

$\boxed{\text{Augmented Grammar}}$

$\boxed{\text{CLOSURE}}$

$\boxed{\text{GOTO}}$

then LR(0), SLR, CLR and LALR become much easier.

1. First: What Does LR Mean?

Recall:

$\boxed{\text{LR}}$

L

Input is scanned:

$\boxed{\text{Left to Right}}$

R

Parser constructs:

$\boxed{\text{Rightmost derivation in reverse}}$

So:

$\boxed{\text{LR} = \text{Left-to-right scan} + \text{Reverse Rightmost derivation}}$

2. What Does the 0 in LR(0) Mean?

In:

$\boxed{\text{LR(0)}}$

the 0 means:

$\boxed{\text{zero explicit lookahead symbols in an LR item}}$

An LR(0) item contains only a production plus a dot.

Example:

$\rightarrow \alpha \cdot \beta$

There is no lookahead attached to the item.

Later, LR(1) items will look like:

$\rightarrow \alpha \cdot \beta, a$

where a is one lookahead symbol.

3. Why Do We Need LR Items?

Suppose the grammar has:

$\rightarrow XYZ$

During bottom-up parsing, the parser may have recognized:

- nothing
- X
- XY
- all of XYZ

We need a way to represent this progress.

That is exactly what the **dot** does.

4. LR(0) Item

CONCEPT

An **LR(0) item** is a grammar production with a dot placed somewhere in its RHS.

For:

$\rightarrow XYZ$

the LR(0) items are:

$\rightarrow \cdot XYZ$

$\rightarrow X \cdot YZ$

$\rightarrow XY \cdot Z$

$\rightarrow XYZ \cdot$

WRITE THIS IN NOTES

LR(0) item: A production with a dot indicating how much of its RHS has already been recognized.

5. Meaning of the Dot

For:

$\backslash [A \rightarrow \alpha \cdot \beta]$

think:

```text id="kps9mk" ±= already recognized ²= still expected

```
Memory Trick

> **Left of dot = Done**
> **Right of dot = Remaining**

6. Example

Production:

\[
E \rightarrow E+T
\]

Item 1

\[
E \rightarrow \cdot E+T
\]

Meaning:

> Nothing from `E+T` has been recognized yet.

Item 2

\[
E \rightarrow E \cdot +T
\]

Meaning:

> `E` has been recognized; expecting `+`.

Item 3

\[
E \rightarrow E+ \cdot T
\]

Meaning:

> `E+` recognized; expecting `T`.

Item 4

\[
E \rightarrow E+T \cdot
\]

Meaning:

> Entire RHS recognized.

This is a **completed item**.
```

---

#### # 7. Number of LR(0) Items From One Production

Suppose:

$$A \rightarrow X_1 X_2 \cdots X_n$$

RHS has  $n$  grammar symbols.

The dot can be placed:

- before symbol 1
- between symbols
- after symbol  $n$

So number of LR(0) items is:

$$n+1$$

---

#### # Example

$$A \rightarrow abc$$

RHS length:

$$3$$

Items:

$$A \rightarrow \cdot abc$$

$$A \rightarrow a \cdot bc$$

$$A \rightarrow ab \cdot c$$

$$A \rightarrow abc \cdot$$

Total:

$$4$$

---

#### # GATE Shortcut



For an ordinary production with RHS containing  $\backslash(n\backslash)$  grammar symbols:

```
\[
\boxed{\text{Number of LR(0) items}=n+1}
\]
```

B u t b e c a r e f u l w i t h  $\mu$  - p r o d u c t i o n s .

---

# 8 . W h a t A b o u t a n  $\mu$  - P r o d u c t i o n ?

Suppose:

```
\[
A\rightarrow\epsilon
\]
```

S i n c e  $\mu$  c o n t a i n s n o g r a m m a r s y m b o l , c o n c e p t u a l l y t h e c o

```
\[
\boxed{A\rightarrow\cdot}
\]
```

Some notations write:

```
\[
A\rightarrow\epsilon\cdot
\]
```

but standard LR construction usually treats the empty RHS as having the dot already at the end.

So it contributes:

```
\[
\boxed{1\text{ item}}
\]
```

---

# 9. Total Number of LR(0) Items – GATE Pattern

Suppose grammar:

```
\[
S\rightarrow ABC
\]
```

```
\[
A\rightarrow a
\]
```

```
\[
B\rightarrow bc
\]
```

Count possible LR(0) items from productions.

###  $\backslash(S\rightarrow ABC\backslash)$

RHS length = 3

```
\[
3+1=4
\]
```

```
\(\text{A} \rightarrow a\)
```

RHS length = 1

```
\[
1+1=2
\]
```

```
\(\text{B} \rightarrow bc\)
```

RHS length = 2

```
\[
2+1=3
\]
```

Total:

```
\[
4+2+3=\boxed{9}
\]
```

If augmented grammar is added, the augmented production contributes more items too.

---

# 10. What Is an Augmented Grammar?

Before constructing an LR parser, we add a new start symbol.

Suppose original start symbol is:

```
\[
S
\]
```

Create a new symbol:

```
\[
S'
\]
```

and add:

```
\[
\boxed{S' \rightarrow S}
\]
```

The resulting grammar is called the:

```
\[
\boxed{\text{Augmented Grammar}}
\]
```

---

# WRITE THIS IN NOTES

Original:

```
\[
S \rightarrow \cdots
\]
```

Augment with:

$$\boxed{S \rightarrow S}$$

where  $(S')$  is a new start symbol.

---

#### # 11. Why Do We Augment the Grammar?

The parser needs a unique way to recognize:

> The whole start symbol has been parsed, so accept.

The special item:

$$S' \rightarrow S \cdot$$

means:

$$\boxed{\text{Entire input corresponding to original start symbol has been recognized}}$$

Thus it leads to the **ACCEPT** action.

---

#### # Memory Trick

> **Augmented start gives us an explicit acceptance production.**

---

#### # 12. Example of Augmentation

Original grammar:

$$S \rightarrow CC$$

$$C \rightarrow c \mid d$$

Augmented grammar:

$$\boxed{S' \rightarrow S}$$

$$S \rightarrow CC$$

$$C \rightarrow c \mid d$$

Initial LR item:

$$S' \rightarrow \cdot S$$

```
\]
```

Final accepting item:

```
\[
S'\rightarrow S\cdot
\]
```

---

### # 13. What Is an Item Set?

An LR parser does not usually work with one item at a time.

It groups compatible items into a **state**.

Example:

```
\[
I_0=
\{
S'\rightarrow\cdot S,\;
S'\rightarrow\cdot CC,\;
C'\rightarrow\cdot cC,\;
C'\rightarrow\cdot d
\}
\]
```

Such a collection is called an:

- item set
- LR state
- set of LR(0) items

These become states of a DFA.

---

### # 14. Why Does One State Contain Multiple Items?

Suppose:

```
\[
S'\rightarrow\cdot S
\]
```

The dot is immediately before non-terminal `S`.

To recognize an `S`, the parser must be ready to begin any production of `S`.

If:

```
\[
S'\rightarrow CC
\]
```

then we also need:

```
\[
S'\rightarrow\cdot CC
\]
```

But now the dot is before `C`.

So we must also be ready to begin any production of `C`.

If:

$$C \rightarrow c \mid d$$

we add:

$$C \rightarrow cC$$

and:

$$C \rightarrow dC$$

This expansion is called:

$$\boxed{\text{CLOSURE}}$$

---

#### # 15. CLOSURE – Intuition

Suppose an item contains:

$$A \rightarrow \alpha \cdot B \beta$$

Dot is before a **non-terminal**  $B$ .

That means:

> We are now expecting to recognize a  $B$ .

Therefore the parser must be ready for every possible way  $B$  can begin.

If:

$$B \rightarrow \gamma_1$$
$$B \rightarrow \gamma_2$$

add:

$$B \rightarrow \gamma_1 \cdot$$
$$B \rightarrow \gamma_2 \cdot$$

Then repeat this process if those new items also have dots before non-terminals.

---

# WRITE THIS IN NOTES

For:

$\backslash[$   
 $A \rightarrow \alpha \cdot B \beta$   
 $\backslash]$

if  $\backslash(B)$  is a non-terminal, add:

$\backslash[$   
 $\boxed{B \rightarrow \cdot \gamma}$   
 $\backslash]$

for every production:

$\backslash[$   
 $B \rightarrow \gamma$   
 $\backslash]$

Repeat until no new item can be added.

---

# 16. CLOSURE Algorithm

Given an item set  $\backslash(I)$ :

### Step 1

Start with all items already in  $\backslash(I)$ .

### Step 2

If an item has:

$\backslash[$   
 $A \rightarrow \alpha \cdot B \beta$   
 $\backslash]$

where  $\backslash(B)$  is a non-terminal:

For every production:

$\backslash[$   
 $B \rightarrow \gamma$   
 $\backslash]$

add:

$\backslash[$   
 $B \rightarrow \cdot \gamma$   
 $\backslash]$

if not already present.

### Step 3

Repeat for newly added items.

### Step 4

Stop when no new item can be added.

Result:

```
\[
\boxed{CLOSURE(I)}
\]
```

---

# Memory Trick

> \* \* D o t b e f o r e n o n - t e r m i n a l ! ' o p e n a l l i t s p r o d u c t i o n s

That's the easiest way to remember CLOSURE.

---

# 17. What if Dot Is Before a Terminal?

Suppose:

```
\[
A\rightarrow\alpha\cdot a\beta
\]
```

where `a` is a terminal.

Do we add productions for `a`?

'L N o .

Terminals do not have grammar productions.

Therefore CLOSURE does nothing for that item.

---

# 18. What if Dot Is at the End?

Suppose:

```
\[
A\rightarrow\alpha\cdot
\]
```

There is nothing after the dot.

So no CLOSURE expansion occurs.

This is a completed item.

---

# 19. CLOSURE Example 1

Grammar:

```
\[
S'\rightarrow S
\]
```

```
\[
S\rightarrow AB
\]
```

```
\[
A\rightarrow a
\]
```

$\backslash[$   
 $B \rightarrow b$   
 $\backslash]$

Find:

$\backslash[$   
 $CLOSURE(\{S' \rightarrow \cdot S\})$   
 $\backslash]$

### Start

$\backslash[$   
 $S' \rightarrow \cdot S$   
 $\backslash]$

Dot before `S`, a non-terminal.

Add:

$\backslash[$   
 $S \rightarrow \cdot AB$   
 $\backslash]$

Now dot before `A`.

Add:

$\backslash[$   
 $A \rightarrow \cdot a$   
 $\backslash]$

Dot before `a`, a terminal.

Stop expanding there.

So:

$\backslash[$   
 $\boxed{$   
 $I_0 =$   
 $\backslash\{$   
 $S' \rightarrow \cdot S, \backslash;$   
 $S \rightarrow \cdot AB, \backslash;$   
 $A \rightarrow \cdot a$   
 $\backslash\}$   
 $\}$   
 $\backslash]$

Notice:

$\backslash[$   
 $B \rightarrow \cdot b$   
 $\backslash]$

is **not** yet added.

Why?

Because the dot in:

$\backslash[$   
 $S \rightarrow \cdot AB$   
 $\backslash]$

is currently before `A`, not `B`.



```

GATE TRAP

Do not add productions of every non-terminal appearing anywhere after the dot.

Only expand the non-terminal **immediately after the dot**.

20. CLOSURE Example 2

Grammar:

\[
S' \rightarrow S
\]

\[
S \rightarrow A
\]

\[
A \rightarrow B
\]

\[
B \rightarrow b
\]

Start:

\[
S' \rightarrow \cdot S
\]

Dot before `S` !' add :

\[
S \rightarrow \cdot A
\]

Dot before `A` !' add :

\[
A \rightarrow \cdot B
\]

Dot before `B` !' add :

\[
B \rightarrow \cdot b
\]

Dot before terminal `b` !' stop .

Thus:

\[
\boxed{
\begin{aligned}
&\text{CLOSURE=} \\
&\{ \\
&S' \rightarrow \cdot S, \\
&S \rightarrow \cdot A, \\
&A \rightarrow \cdot B,
\end{aligned}
}

```

```
B\to\cdot b
\}
}
\]
```

This shows closure can propagate through several non-terminals.

---

## # 21. Kernel vs Non-Kernel Items – Useful GATE Concept

LR states sometimes distinguish:

### ### Kernel items

Items that determine the state's core.

They are:

1. the initial augmented item:  
     $\left[ S' \rightarrow \cdot S \right]$
2. items whose dot is **not** at the extreme left

Example:

```
\[
A\rightarrow a\cdot B
\]
```

is kernel.

### ### Non-kernel items

Items with dot at the extreme left:

```
\[
A\rightarrow \cdot \alpha
\]
```

except:

```
\[
S'\rightarrow \cdot S
\]
```

which is treated as kernel.

---

## # Memory Trick

- > **Dot moved? Kernel.**
- > **Dot never moved? Usually non-kernel.**
- > Exception: augmented starting item.

This becomes particularly useful when discussing LALR cores.

---

## # 22. What Is GOTO?

CLOSURE tells us:

- > What possibilities exist in this state?

GOTO tells us:

> Where do we go after recognizing one grammar symbol?

Suppose state contains:

```
\[
A\rightarrow\alpha\cdot X\beta
\]
```

If we recognize symbol `X`, move the dot past `X`:

```
\[
A\rightarrow\alpha X\cdot\beta
\]
```

Do this for **every item in the state with `X` immediately after the dot**, and then take CLOSURE.

This is:

```
\[
\boxed{\text{GOTO}(I,X)}
\]
```

---

# WRITE THIS IN NOTES

```
\[
GOTO(I,X)
=
CLOSURE(
\{A\rightarrow\alpha X\cdot\beta
\mid
A\rightarrow\alpha\cdot X\beta\in I
\}
)
\]
```

Simple language:

> **Move dot over X in every applicable item, then take closure.**

---

# Memory Trick

### CLOSURE

> **Open productions.**

### GOTO

> **Move dot, then open productions.**

---

# 23. GOTO Example

Suppose:

```
\[
I=
\{
S\rightarrow\cdot AB,\;
```

```
A\rightarrow\cdot a
\}
\]
```

Find:

```
\[
GOTO(I,A)
\]
```

Which items have dot immediately before `A`?

```
\[
S\rightarrow\cdot AB
\]
```

Move dot over `A`:

```
\[
S\rightarrow A\cdot B
\]
```

Now take CLOSURE.

Dot before non-terminal `B`.

Suppose:

```
\[
B\rightarrow b
\]
```

Add:

```
\[
B\rightarrow\cdot b
\]
```

Therefore:

```
\[
\boxed{
GOTO(I,A)=
\{
S\rightarrow A\cdot B,\;
B\rightarrow\cdot b
\}
}
\]
```

---

# 24. GOTO on a Terminal

Using same state:

```
\[
I=
\{
S\rightarrow\cdot AB,\;
A\rightarrow\cdot a
\}
\]
```

Find:

```
\[
GOTO(I,a)
\]
```

Item with dot before `a`:

```
\[
A\rightarrow\cdot a
\]
```

Move:

```
\[
A\rightarrow a\cdot
\]
```

Nothing after dot.

Therefore:

```
\[
\boxed{
GOTO(I,a)=
\{A\rightarrow a\cdot\}
}
\]
```

---

# 25. Empty GOTO

If no item in state  $\{I\}$  has `X` immediately after the dot:

```
\[
\boxed{GOTO(I,X)=\emptyset}
\]
```

No transition is created.

---

# 26. Canonical Collection of LR(0) Items

Now we combine everything.

The **canonical collection** is the complete set of LR(0) item sets/states reachable from the initial state.

Algorithm:

### Step 1 – Augment grammar

Add:

```
\[
S'\rightarrow S
\]
```

### Step 2 – Initial state

```
\[
\boxed{
I_0=CLOSURE(\{S'\rightarrow\cdot S\})
}
\]
```

### Step 3

For every state  $(I_i)$  and every grammar symbol  $(X)$ :

Compute:

```
\[
GOTO(I_i,X)
\]
```

### Step 4

If the result is:

- nonempty
- not already present

add it as a new state.

### Step 5

Repeat until no new states appear.

---

# WRITE THIS IN NOTES

```text id="c5wikz"

Augment grammar

!"

$I_0 = \text{CLOSURE}(S' \cdot S)$

!"

Compute GOTO on possible symbols

!"

Create new item sets

!"

Repeat

!"

No new sets

!"

Canonical $LR(0)$ collection

27. Full Worked Example

Let's construct the canonical $LR(0)$ collection for:

$S \rightarrow CC$

$C \rightarrow c \mid d$

This is a classic example.

28. Step 1 — Augment Grammar

Add:

$S' \rightarrow S$

Final grammar:

$0 \rightarrow S' \rightarrow S$

$\backslash[1.\text{quad } S\rightarrow CC \backslash]$

$\backslash[2.\text{quad } C\rightarrow cC \backslash]$

$\backslash[3.\text{quad } C\rightarrow d \backslash]$

29. Step 2 — Construct $\backslash(I_0)$

Start:

$\backslash[S'\rightarrow\cdot S \backslash]$

Dot before S .

Add:

$\backslash[S\rightarrow\cdot CC \backslash]$

Dot before first C .

Add all C productions:

$\backslash[C\rightarrow\cdot cC \backslash]$

$\backslash[C\rightarrow\cdot d \backslash]$

Therefore:

$\backslash[\boxed{I_0 = \left\{ \begin{array}{l} S'\rightarrow\cdot S \\ S\rightarrow\cdot CC \\ C\rightarrow\cdot cC \\ C\rightarrow\cdot d \end{array} \right\}} \backslash]$

30. What Symbols Can We GOTO From $\backslash(I_0)$?

Look immediately after each dot:

``text id="17b5s3" $S' \cdot S \quad !' S S \cdot !' \cdot C C \quad !' C C \cdot !' \cdot c C \quad !' c C \cdot !' \cdot d \quad !' d$

Possible transitions:

$\backslash[$
 $S, \backslash C, \backslash c, \backslash d$
 $\backslash]$

31. Compute $\backslash(\text{GOTO}(I_0, S)) \backslash$

From:

$\backslash[$
 $S'\rightarrow\cdot S$
 $\backslash]$

move dot:

$\backslash[$
 $S'\rightarrow S\cdot$
 $\backslash]$

Nothing after dot.

Thus:

```
\[
\boxed{
I_1=
\{S'\rightarrow S\cdot\}
}
\]
```

This will be the accepting state.

32. Compute $\text{GOTO}(I_0, C)$

From:

```
\[
S\rightarrow\cdot CC
\]
```

move over first `C`:

```
\[
S\rightarrow C\cdot C
\]
```

Dot now before non-terminal `C`.

Take closure:

```
\[
C\rightarrow\cdot cC
\]
```

```
\[
C\rightarrow\cdot d
\]
```

Therefore:

```
\[
\boxed{
I_2=
\left\{
\begin{array}{l}
S\rightarrow C\cdot C\\
C\rightarrow\cdot cC\\
C\rightarrow\cdot d
\end{array}
\right\}
}
\]
```

33. Compute $\text{GOTO}(I_0, c)$

From:

```
\[
C\rightarrow\cdot cC
\]
```

move dot:

$$\begin{array}{l} \backslash[\\ C \rightarrow c \cdot C \\ \backslash] \end{array}$$

Dot before `C`, so closure adds:

$$\begin{array}{l} \backslash[\\ C \rightarrow \cdot C \\ \backslash] \end{array}$$

$$\begin{array}{l} \backslash[\\ C \rightarrow \cdot d \\ \backslash] \end{array}$$

Thus:

$$\begin{array}{l} \backslash[\\ \boxed{ \\ I_3 = \\ \left\{ \begin{array}{l} C \rightarrow c \cdot C \\ C \rightarrow \cdot C \\ C \rightarrow \cdot d \end{array} \right. \\ \right. \\ \backslash] \end{array}$$

34. Compute $\text{GOTO}(I_0, d)$

From:

$$\begin{array}{l} \backslash[\\ C \rightarrow \cdot d \\ \backslash] \end{array}$$

move dot:

$$\begin{array}{l} \backslash[\\ C \rightarrow d \cdot \\ \backslash] \end{array}$$

So:

$$\begin{array}{l} \backslash[\\ \boxed{ \\ I_4 = \\ \{ C \rightarrow d \cdot \} \\ \} \\ \backslash] \end{array}$$

35. Now Process $\text{GOTO}(I_2)$

Recall:

$$\begin{array}{l} \backslash[\\ I_2 = \\ \{ \\ S \rightarrow C \cdot, \\ C \rightarrow \cdot C, \\ C \rightarrow \cdot d \end{array}$$

```
\}  
\]
```

Symbols after dots:

```
```text id="v5r51a"  
C
c
d
```

### 36. $\backslash(GOTO(I_2, C))$

From:

$\backslash \ S \rightarrow C \cdot C \ \backslash$

move dot:

$\backslash \ S \rightarrow CC \cdot \ \backslash$

Therefore:

$\backslash \ \boxed{I_5 = \backslash \ S \rightarrow CC \cdot \} \ \backslash$

Completed production.

### 37. $\backslash(GOTO(I_2, c))$

From:

$\backslash \ C \rightarrow \cdot cC \ \backslash$

moving over **c** produces:

$\backslash \ C \rightarrow c \cdot C \ \backslash$

and closure adds the same C-starting items as before.

This is exactly:

$\backslash \ I_3 \ \backslash$

Therefore:

$\backslash \ \boxed{GOTO(I_2, c) = I_3} \ \backslash$

Important:

**Do not create a duplicate state if the item set is identical to an existing state.**

### 38. $\backslash(GOTO(I_2, d))$

Produces:

$\backslash \ C \rightarrow d \cdot \ \backslash$

which is:

$I_4$

Thus:

$\boxed{\text{GOTO}(I_2, d) = I_4}$

---

### 39. Process $I_3$

$I_3 = \{ C \rightarrow c \cdot C, C \rightarrow cC \cdot, C \rightarrow cd \cdot \}$

Possible symbols:

$C, c, d$

---

### 40. $\text{GOTO}(I_3, C)$

From:

$C \rightarrow c \cdot C$

move dot:

$C \rightarrow cC \cdot$

So new state:

$\boxed{I_6 = \{ C \rightarrow cC \cdot \}}$

---

### 41. $\text{GOTO}(I_3, c)$

Again produces:

$I_3$

So there is a self-loop:

$\boxed{I_3 \xrightarrow{c} I_3}$

---

### 42. $\text{GOTO}(I_3, d)$

Produces:

$I_4$

Therefore:

$\boxed{I_3 \xrightarrow{d} I_4}$

---

### 43. Completed Canonical Collection

We have:

$I_0$

$\{ S \rightarrow \cdot S, S \rightarrow CC \cdot, C \rightarrow \cdot cC, C \rightarrow cd \cdot \}$

---

**$\backslash(l_1)$**

$\backslash[ \{S \rightarrow S \cdot\} ]$

**$\backslash(l_2)$**

$\backslash[ \{S \rightarrow C \cdot, C \rightarrow c \cdot, C \rightarrow d \cdot\} ]$

**$\backslash(l_3)$**

$\backslash[ \{C \rightarrow c \cdot, C \rightarrow c \cdot, C \rightarrow d \cdot\} ]$

**$\backslash(l_4)$**

$\backslash[ \{C \rightarrow d \cdot\} ]$

**$\backslash(l_5)$**

$\backslash[ \{S \rightarrow C \cdot\} ]$

**$\backslash(l_6)$**

$\backslash[ \{C \rightarrow c \cdot\} ]$

Total states:

$\backslash[ \boxed{7} ]$

---

## 44. DFA / LR(0) Automaton

Transitions:

```text id="799u93" I0 --S--> I1 I0 --C--> I2 I0 --c--> I3 I0 --d--> I4`

`I2 --C--> I5 I2 --c--> I3 I2 --d--> I4`

`I3 --C--> I6 I3 --c--> I3 I3 --d--> I4`

This DFA recognizes viable prefixes.

WRITE THIS IN NOTES

$\backslash[\boxed{\text{Canonical LR(0) item DFA recognizes viable prefixes}} \backslash]$

That connects Part 10 with Part 11.

45. Terminal vs Non-Terminal Transitions

In the LR automaton:

Transition on a terminal

Example:

$\backslash[I_0 \xrightarrow{c} I_3 \backslash]$

will later correspond to a:

```
\[
\boxed{\text{SHIFT action}}
\]
```

Transition on a non-terminal

Example:

```
\[
I_0 \xrightarrow{C} I_2
\]
```

will later go into the:

```
\[
\boxed{\text{GOTO table}}
\]
```

This distinction is essential for constructing the LR parsing table in Part 12.

Memory Trick

```
> * * T e r m i n a l   e d g e   ! '   S H I F T * *
> * * N o n - t e r m i n a l   e d g e   ! '   G O T O * *
```

46. Completed Items

An item of the form:

```
\[
A \rightarrow \alpha \cdot
\]
```

is called a:

```
\[
\boxed{\text{Completed Item}}
\]
```

It means:

> Entire RHS of production $(A \rightarrow \alpha)$ has been recognized.

This usually suggests a **reduce action**.

But exactly **where** to place that reduction depends on the LR parser type:

- LR(0): all terminal columns
- SLR: FOLLOW(A)
- CLR/LR(1): item's lookahead(s)

This distinction is hugely important.

47. Special Completed Item

```
\[
```

$S' \rightarrow S \cdot$
]

is different from ordinary completed items.

It means:

[
boxed{\text{ACCEPT}}
]

not reduce by:

[
 $S' \rightarrow S$
]

in the ordinary sense.

48. Item Types at a Glance

Beginning item

[
 $A \rightarrow \cdot \alpha$
]

Nothing recognized.

Intermediate item

[
 $A \rightarrow \alpha \cdot \beta$
]

Some recognized, some remaining.

Completed item

[
 $A \rightarrow \alpha \cdot$
]

Entire RHS recognized.

Accepting item

[
 $S' \rightarrow S \cdot$
]

Accept input.

49. CLOSURE vs GOTO – Master Comparison

CLOSURE	GOTO
Expands possibilities in same state	Moves to another state
Dot does not move in original item	Dot moves across one symbol
Triggered by non-terminal after dot	Can be computed on terminal or non-terminal
Adds $(B \rightarrow \cdot \gamma)$	First move dot, then apply closure
Does not consume/recognize symbol	Represents recognition of symbol X

```

---

# Memory Trick

> **CLOSURE = what could start here?**

> **GOTO = what happens after I recognize X?**

---

# 50. GATE Shortcut for Constructing States

For each item set:

### Step 1

Circle/identify every distinct symbol immediately after a dot.

### Step 2

For each such symbol  $\backslash(X\backslash)$ :

- move all dots across  $\backslash(X\backslash)$ 
- put those moved items together
- take closure

### Step 3

Compare with existing states.

If identical:

> reuse old state.

If new:

> create new state.

This prevents most LR state-construction mistakes.

---

# 51. Common Mistake: Moving Only One Item

Suppose a state contains:

 $\backslash[$ 
 $A \rightarrow \alpha \cdot X \beta$ 
 $\backslash]$ 

and:

 $\backslash[$ 
 $B \rightarrow \gamma \cdot X \delta$ 
 $\backslash]$ 

When computing:

 $\backslash[$ 
 $GOTO(I, X)$ 
 $\backslash]$ 

you must move the dot in both:

 $\backslash[$ 

```

$A \rightarrow \alpha X \cdot \beta$
]

[
 $B \rightarrow \gamma X \cdot \delta$
]

Then take closure.

Do not create separate states for each item.

52. Common Mistake: Forgetting Closure After GOTO

Suppose:

[
GOTO
]

produces:

[
 $A \rightarrow \alpha X \cdot B \beta$
]

Since dot is now before non-terminal `B`, you must add all:

[
 $B \rightarrow \cdot \gamma$
]

items.

So:

[
 $\boxed{\text{GOTO} = \text{Move} \cdot + \text{CLOSURE}}$
]

53. Common Mistake: Closure After a Terminal

If the dot is before terminal `a`:

[
 $A \rightarrow \alpha \cdot a \beta$
]

do not add anything from closure.

You wait for a GOTO transition on `a`.

54. Common Mistake: Duplicate States

If newly computed GOTO produces an item set identical to an existing state:

'L d o n o t m a k e a n o t h e r s t a t e .

Just draw a transition to the existing state.

This is important when counting number of LR states.

55. Number of Items vs Number of States

Do not confuse:

Number of possible individual LR items

and:

Number of canonical LR item sets/states

They are different quantities.

A state contains **multiple items**.

So if grammar has 15 possible LR items, it does **not** imply 15 LR states.

56. GATE-STYLE EXAMPLE – Number of Items

Production:

```
\[
A\rightarrow XYZP
\]
```

How many LR(0) items?

RHS has:

```
\[
4
\]
```

symbols.

Therefore:

```
\[
\boxed{5}
\]
```

57. GATE-STYLE Example – Closure

Grammar:

```
\[
S'\rightarrow S
\]
```

```
\[
S\rightarrow AB
\]
```

```
\[
A\rightarrow a\mid b
\]
```

Find initial closure.

Start:

$\backslash[$
 $S' \rightarrow \cdot S$
 $\backslash]$

Add:

$\backslash[$
 $S \rightarrow \cdot AB$
 $\backslash]$

Dot before `A`, so add:

$\backslash[$
 $A \rightarrow \cdot aA$
 $\backslash]$

$\backslash[$
 $A \rightarrow \cdot b$
 $\backslash]$

Therefore:

$\backslash[$
 $\boxed{\begin{array}{l} I_0 = \\ \{ \\ S' \rightarrow \cdot S, \\ S \rightarrow \cdot AB, \\ A \rightarrow \cdot aA, \\ A \rightarrow \cdot b \end{array}}$
 $\backslash]$

Do not add B productions yet because dot is not before B.

58. GATE-STYLE Example – GOTO

Using previous state, compute:

$\backslash[$
 $GOTO(I_0, a)$
 $\backslash]$

Only:

$\backslash[$
 $A \rightarrow \cdot aA$
 $\backslash]$

has dot before `a`.

Move:

$\backslash[$
 $A \rightarrow a \cdot A$
 $\backslash]$

Now closure because `A` follows dot:

$\backslash[$
 $A \rightarrow \cdot aA$
 $\backslash]$

```
\[
A\rightarrow\cdot b
\]
```

So:

```
\[
\boxed{
GOTO(I_0,a)=
\{
A\to a\cdot A,
A\to\cdot aA,
A\to\cdot b
\}
}
\]
```

59. GATE-STYLE Example – Completed Item

State contains:

```
\[
A\rightarrow xyz\cdot
\]
```

What does this mean?

' Complete RHS `xyz` has been recognized .

This state potentially calls for:

```
\[
\boxed{\text{Reduction using }A\rightarrow xyz}
\]
```

The exact lookahead columns depend on parser type.

60. GATE TRAPS

Trap 1

In:

```
\[
A\rightarrow\alpha\cdot\beta
\]
```

² has already been recognized .

'L False .

```text id="2i9vik"

<sup>±</sup> = recognized

<sup>2</sup> = remaining

## Trap 2

A production with  $\backslash(n\backslash)$  RHS symbols produces  $\backslash(n\backslash)$  LR(0) items.

'L False.

$\boxed{n+1}$

---

### Trap 3

CLOSURE moves the dot.

'L False.

CLOSURE adds new initial items.

**GOTO** moves the dot.

---

### Trap 4

If dot is before a terminal, CLOSURE adds that terminal's productions.

'L Terminals do not have productions.

---

### Trap 5

CLOSURE adds productions of every non-terminal somewhere to the right of the dot.

'L Only the non-terminal **immediately after the dot**, recursively.

---

### Trap 6

GOTO moves only one item's dot.

'L Move the dot in **every item** having the chosen symbol immediately after the dot.

---

### Trap 7

GOTO does not require closure.

'L After moving dots, take closure.

---

### Trap 8

$S \rightarrow S \cdot$

means reduce using  $(S \rightarrow S)$ .

'L In LR parsing, this signals:

$\boxed{\text{ACCEPT}}$

---

### Trap 9

Every completed item means accept.

---

'L Only augmented:

$\{ S \rightarrow S \cdot \}$

means accept.

Others imply potential reductions.

---

## Trap 10

---

Number of LR items equals number of LR states.

'L False.

States are **sets of items**.

---

## 61. MINI PRACTICE — PART 11

---

Try these before checking the solutions.

### Q1 — Easy

How many LR(0) items are generated by:

$\{ A \rightarrow abcd \}$

A. 4 B. 5 C. 6 D. 1

---

### Q2 — Easy

What does:

$\{ A \rightarrow ab \cdot cd \}$

mean?

A. `abcd` completely recognized B. `ab` recognized, `cd` remains C. `cd` recognized D. Reduction must happen immediately

---

### Q3 — Medium

Grammar:

$\{ S \rightarrow S \}$

$\{ S \rightarrow A \}$

$\{ A \rightarrow a \}$

Find:

$\{ \text{CLOSURE}(\{ S \rightarrow \cdot S \}) \}$

---

### Q4 — Medium

Suppose:

$\{ I = \{ A \rightarrow a \cdot B, B \rightarrow \cdot b \} \}$

and:

---

$\rightarrow b$

What is:

$\text{GOTO}(I, B)?$

### Q5 — Tricky

Suppose state  $I$  contains:

$A \rightarrow \alpha \cdot X \beta$

and:

$B \rightarrow \gamma \cdot X \delta$

When computing:

$\text{GOTO}(I, X)$

which dots must move?

A. Only first B. Only second C. Both D. Neither

## SOLUTIONS

### Q1

RHS length:

4

Therefore:

$4 + 1 = \boxed{5}$

' B

### Q2

' B

For:

$A \rightarrow ab \cdot cd$

ab = recognized cd = still expected

---

## Q3

Start:

$S' \rightarrow \cdot S$

Dot before S !' add :

```
\[
S\rightarrow\cdot A
\]
```

Dot before A !' add :

```
\[
A\rightarrow\cdot a
\]
```

Thus:

```
\[
\boxed{
\{
S'\rightarrow\cdot S,\;
S\rightarrow\cdot A,\;
A\rightarrow\cdot a
\}
}
\]
```

---

## Q4

Item with dot before `B`:

```
\[
A\rightarrow a\cdot B
\]
```

Move dot:

```
\[
A\rightarrow aB\cdot
\]
```

No closure expansion.

Therefore:

```
\[
\boxed{
GOTO(I,B)=
\{A\rightarrow aB\cdot\}
}
\]
```

The item:

```
\[
B\rightarrow\cdot b
\]
```

has dot before `b`, not `B`, so it does not move for  $\backslash(GOTO(I,B)\backslash)$ .

---

## Q5

' \* \* C — B o t h \* \*

GOTO must move every dot that has `X` immediately to its right.

```

62. WRITE THIS IN NOTES – LR(0) MASTER SHEET

LR(0) Item

\[
\boxed{A\rightarrow\alpha\cdot\beta}
\]

```text id="vi5czc"
left of dot = recognized
right of dot = remaining

```

For (n) RHS symbols:

$\boxed{n+1\text{ items}}$

Augmented Grammar

$\boxed{S\rightarrow S}$

Initial item:

$\boxed{S\rightarrow\cdot S}$

Accepting item:

$\boxed{S\rightarrow S\cdot}$

CLOSURE

If:

$\boxed{A\rightarrow\alpha\cdot B\beta}$

add:

$\boxed{B\rightarrow\cdot\gamma}$

for every:

$\boxed{B\rightarrow\gamma}$

and repeat.

Shortcut

Dot before non-terminal !' open its productions.

GOTO

For:

$\boxed{A\rightarrow\alpha\cdot X\beta}$

move:

$\{A \rightarrow \alpha X \cdot \beta\}$

for all applicable items, then take closure.

Shortcut

Move dot over X !' CLOSURE.

Canonical Collection

$I_0 = \text{CLOSURE}(\{S \rightarrow \cdot S\})$

Then repeatedly compute all nonempty GOTOs until no new item set appears.

30-SECOND REVISION

LR

$\{\text{Left-to-right} + \text{Reverse Rightmost}\}$

LR(0)

0 = no explicit lookahead in items.

Dot

``text id="8cfp00" A !' ± · ²

± = done ² = remaining

```
### Completed item

\[
A \rightarrow \alpha \cdot
\]

!' potential ** reduce **.

### Special completed item

\[
S \rightarrow S \cdot
\]

!' ** ACCEPT **.

### CLOSURE

> Dot before a non-terminal? Add all its productions with dot at beginning.

### GOTO

> Move every applicable dot across X, then take closure.

### Transition meaning

``text id="ok4qfy"
Terminal transition      !' later SHIFT
```

Fundamental connection

$\boxed{\text{LR(0) item DFA recognizes viable prefixes}}$

Next we use these states to build the actual **LR(0) ACTION/GOTO parsing table**, learn exactly where **shift, reduce and accept entries** go, identify **LR(0) conflicts**, and parse strings using the LR stack.

Completed: Parts 1–11/31 | Next: Part 12 — LR(0) Parsing Table, ACTION/GOTO & Conflict Detection

Reply **NEXT** for **Part 12**.

USER

NEXT

ASSISTANT

Compiler Design — Part 12: LR(0) Parsing Table, ACTION/GOTO & Conflict Detection

Completed: Parts 1–11/31 | Current: Part 12 | Remaining: Parts 13–31

In Part 11, we constructed the **canonical collection of LR(0) item sets**.

Now we turn those states into an actual parser.

The two most important structures are:

$\boxed{\text{ACTION}}$

and

$\boxed{\text{GOTO}}$

Once you understand exactly where **shift, reduce, accept and error** entries come from, LR(0) questions become much more mechanical.

1. Overall LR Parser Structure

An LR parser uses:

- input
- stack
- ACTION table
- GOTO table

Conceptually:

TEXT

Input tokens
!"

```

% % % % % % % % % % % % % % %
%      L R   P a r s e r      %   ! •   A C T I O N   /   G O T O   T a b l e
% % % % % % % % % % % % % % %
      ! '
Stack

```

Unlike the simple shift-reduce stack from Part 10, an LR parser keeps track of **states**.

2. ACTION Table vs GOTO Table

This distinction is essential.

ACTION

Columns correspond to:

$\boxed{\text{Terminals} + \$}$

ACTION tells the parser whether to:

- shift
- reduce
- accept
- report error

GOTO

Columns correspond to:

$\boxed{\text{Non-terminals}}$

GOTO tells us which state to enter after a reduction produces a non-terminal.

WRITE THIS IN NOTES

Table	Columns	Purpose
ACTION	Terminals + \$	Shift / Reduce / Accept / Error
GOTO	Non-terminals	State transition after recognizing/reducing a non-terminal

Memory Trick

Terminals cause ACTION. Non-terminals cause GOTO.

3. Four Possible ACTION Entries

An ACTION cell can contain:

Shift

TEXT
s5

means:

Shift current input token and move to state 5.

Reduce

TEXT
r3

means:

Reduce using production number 3.

Accept

TEXT
acc

means:

Input has been successfully parsed.

Blank/Error

means:

$\boxed{\text{Syntax Error}}$

4. How Do We Create SHIFT Entries?

Suppose LR(0) state (I_i) contains:

$A \rightarrow \alpha \cdot a \beta$

where a is a **terminal**.

And:

$GOTO(I_i, a) = I_j$

Then:

$\boxed{ACTION[i, a] = \text{shift } j}$

or simply:

$\boxed{s_j}$

Intuition

The dot says:

Next expected grammar symbol is terminal a .

If input lookahead is also a , consume it.

That's a **shift**.

WRITE THIS IN NOTES

If:

$$I_i \xrightarrow{a} I_j$$

where a is a terminal:

$$\boxed{\text{ACTION}[i,a]=s_j}$$

5. How Do We Create GOTO Entries?

Suppose:

$$GOTO(I_i, A) = I_j$$

where A is a **non-terminal**.

Then:

$$\boxed{GOTO[i,A]=j}$$

This transition goes into the GOTO part of the parsing table, not ACTION.

Memory Trick

TEXT

Terminal transition	!	SHIFT in ACTION
Non-terminal transition	!	State number in GOTO

6. How Do We Create REDUCE Entries in LR(0)?

Suppose state I_i contains a completed item:

$$A \rightarrow \alpha \cdot$$

where:

$$A \neq S'$$

This means the entire RHS α has been recognized.

In **LR(0)**, the parser has **zero lookahead information attached to the item**.

Therefore it places the reduction:

$\rightarrow \alpha$

under:

$\{\text{EVERY terminal column, including } \$\}$

That is:

$\text{ACTION}[i,a] = \text{reduce } \rightarrow \alpha$

for every terminal/input symbol a .

This is the key limitation of LR(0).

7. Why Does LR(0) Reduce Everywhere?

Because an LR(0) completed item:

$\rightarrow \alpha \cdot$

only says:

I have completely recognized α .

It carries **no information about what token is allowed to come next**.

So LR(0) blindly places the reduction under all terminal lookaheads.

This can create many conflicts.

Later:

SLR improves this

Reduce only under:

$\text{FOLLOW}(A)$

CLR/LR(1) improves further

Reduce only under the item's actual lookahead.

This distinction is extremely important.

WRITE THIS IN NOTES — REDUCTION COMPARISON

For completed item:

$\rightarrow \alpha \cdot$

LR(0)

Reduce under:

$\{\text{all terminals}\}$

SLR(1)

Reduce under:

$$\boxed{\text{FOLLOW}(A)}$$

CLR(1)

Reduce under:

$$\boxed{\text{text{item lookahead}}}$$

We'll build this comparison throughout the next parts.

8. ACCEPT Entry

Suppose state I_i contains:

$$\boxed{S \rightarrow S \cdot}$$

Then:

$$\boxed{\text{ACTION}[i, \$] = \text{accept}}$$

Only on $\$$.

This is the special completed augmented item.

9. LR(0) Table Construction Rules — Complete

For every state I_i :

Rule 1 — Shift

If:

$$A \rightarrow \alpha \cdot a \beta \text{ in } I_i$$

where a is terminal and:

$$\text{GOTO}(I_i, a) = I_j$$

then:

$$\boxed{\text{ACTION}[i, a] = s_j}$$

Rule 2 — Reduce

If:

$$A \rightarrow \alpha \cdot \text{ in } I_i$$

and $A \neq S$, then for **every terminal a , including $\$$** :

$$\boxed{\text{ACTION}[i, a] = r(A \rightarrow \alpha)}$$

Rule 3 — Accept

If:

$$S \rightarrow S \cdot \text{ in } I_i$$

then:

$\boxed{\text{ACTION}[i, \$] = \text{acc}}$

Rule 4 — GOTO

If:

$\text{GOTO}(I_i, A) = I_j$

for non-terminal (A) :

$\boxed{\text{GOTO}[i, A] = j}$

Rule 5

All unfilled entries are:

$\boxed{\text{error}}$

10. When Is a Grammar LR(0)?

After constructing the table:

If every ACTION cell contains at most one action, then the grammar is LR(0).

If any cell contains:

- shift + reduce

or

- reduce + reduce

then the grammar is **not** LR(0).

WRITE THIS IN NOTES

$\boxed{\text{LR(0) grammar} \iff \text{LR(0) ACTION table has no conflicts}}$

11. Conflict Detection Directly From Item Sets

You can often identify conflicts **without constructing the full table**.

This is a valuable GATE shortcut.

12. Shift/Reduce Conflict in LR(0)

Suppose a state contains:

$A \rightarrow \alpha \cdot a \beta$

and:

$B \rightarrow \gamma \cdot$

The first item can cause a shift on terminal a .

The completed second item causes reduction on **every terminal**, including **a**.

So:

$\text{ACTION}[i,a]$

gets:

- shift
- reduce

Thus:

$\boxed{\text{Shift/Reduce Conflict}}$

GATE Shortcut

In LR(0), if one state contains:

1. a completed ordinary item, and
2. another item with a terminal immediately after the dot,

then expect a:

$\boxed{\text{Shift/Reduce conflict}}$

because the completed item reduces on every terminal.

13. Reduce/Reduce Conflict in LR(0)

Suppose one state contains two different completed items:

$A \rightarrow \alpha \cdot$

and:

$B \rightarrow \beta \cdot$

Both reductions are entered on every terminal column.

Thus every terminal column would receive both reductions.

Therefore:

$\boxed{\text{Reduce/Reduce Conflict}}$

GATE Shortcut

Two distinct completed ordinary items in the same LR(0) state:

$\boxed{\text{R/R conflict}}$

14. Important Exception — Accepting Item

What if state contains:

$S \rightarrow S \cdot$

This creates **accept only under** $\$$.

It is not treated as an ordinary reduce item.

So don't apply the "completed item reduces everywhere" rule to the augmented item.

15. Let's Reuse the Grammar From Part 11

Grammar:

$S \rightarrow CC$

$C \rightarrow c \mid d$

Augment:

$0. S \rightarrow S$

$1. S \rightarrow CC$

$2. C \rightarrow cC$

$3. C \rightarrow d$

We constructed 7 states.

Let's use them to build the LR(0) table.

16. State I_0

$I_0 = \left\{ \begin{array}{l} S \rightarrow \cdot S \\ S \rightarrow \cdot CC \\ C \rightarrow \cdot cC \\ C \rightarrow \cdot d \end{array} \right\}$

Transitions:

$I_0 \xrightarrow{S} I_1$

$I_0 \xrightarrow{C} I_2$

$I_0 \xrightarrow{c} I_3$

$I_0 \xrightarrow{d} I_4$

Therefore:

Terminal transitions

$ACTION[0, c] = s3$

$ACTION[0, d] = s4$

Non-terminal transitions

$GOTO[0, S] = 1$

$GOTO[0, C] = 2$

17. State I_1

$I_1 = \{ S \rightarrow S \cdot \}$

This is the accepting item.

Therefore:

$\boxed{\text{ACTION}[1,\$]=\text{acc}}$

No ordinary reduction.

18. State I_2

$I_2 = \left\{ \begin{array}{l} S \rightarrow C \cdot C \\ C \rightarrow \cdot C \\ C \rightarrow C \cdot d \end{array} \right\}$

Transitions:

$I_2 \xrightarrow{C} I_5$

$I_2 \xrightarrow{c} I_3$

$I_2 \xrightarrow{d} I_4$

Thus:

$\text{ACTION}[2,c]=s_3$

$\text{ACTION}[2,d]=s_4$

and:

$\text{GOTO}[2,C]=5$

19. State I_3

$I_3 = \left\{ \begin{array}{l} C \rightarrow \cdot c \\ C \rightarrow c \cdot C \\ C \rightarrow C \cdot d \end{array} \right\}$

Transitions:

$I_3 \xrightarrow{C} I_6$

$I_3 \xrightarrow{c} I_3$

$I_3 \xrightarrow{d} I_4$

Therefore:

$\text{ACTION}[3,c]=s_3$

$\text{ACTION}[3,d]=s_4$

and:

$\text{GOTO}[3,C]=6$

20. State I_4

$I_4 = \{ C \rightarrow d \cdot \}$

This is a completed ordinary item.

Production:

$3 \rightarrow d$

LR(0) says reduce on **every terminal and `\\$`**.

Terminals are:

TEXT

c
d
\$

Therefore:

$\backslash[\text{ACTION}[4,c]=r3 \backslash]$

$\backslash[\text{ACTION}[4,d]=r3 \backslash]$

$\backslash[\text{ACTION}[4,\$]=r3 \backslash]$

21. State $\backslash(I_5\backslash)$

$\backslash[I_5= \backslash\{S\rightarrow CC\cdot\} \backslash]$

Completed item.

Production:

$\backslash[1.\backslash S\rightarrow CC \backslash]$

Therefore reduce everywhere:

$\backslash[\text{ACTION}[5,c]=r1 \backslash]$

$\backslash[\text{ACTION}[5,d]=r1 \backslash]$

$\backslash[\text{ACTION}[5,\$]=r1 \backslash]$

22. State $\backslash(I_6\backslash)$

$\backslash[I_6= \backslash\{C\rightarrow cC\cdot\} \backslash]$

Completed item.

Production:

$\backslash[2.\backslash C\rightarrow cC \backslash]$

Thus:

$\backslash[\text{ACTION}[6,c]=r2 \backslash]$

$\backslash[\text{ACTION}[6,d]=r2 \backslash]$

$\backslash[\text{ACTION}[6,\$]=r2 \backslash]$

23. Complete LR(0) Table

Productions

TEXT

```
1 . S ! ' C C
2 . C ! ' c C
3 . C ! ' d
```

Parsing table

State	c	d	\$	GOTO S	GOTO C
0	s3	s4	—	1	2
1	—	—	acc	—	—
2	s3	s4	—	—	5
3	s3	s4	—	—	6
4	r3	r3	r3	—	—
5	r1	r1	r1	—	—
6	r2	r2	r2	—	—

There are no conflicting ACTION cells.

Therefore:

$\boxed{\text{The grammar is LR(0)}}$

24. Notice Why This Grammar Works

Every state is essentially one of:

Shift-oriented

Example:

$[I_0, I_2, I_3]$

They contain no completed ordinary item.

Reduce-oriented

Example:

$[I_4, I_5, I_6]$

They contain completed items but no shift possibility.

Accept state

$[I_1]$

Thus no state mixes incompatible actions.

25. LR Stack Representation

A full LR parser stack alternates conceptually between:

TEXT

state symbol state symbol state ...

For example:

TEXT

0 C 2 c 3 d 4

But a compact implementation can maintain state information appropriately.

For exam traces, commonly use:

TEXT

State stack / symbol stack

or a combined form.

The critical fact is:

The top state and current input lookahead determine ACTION.

26. LR Parsing Algorithm

Initially:

TEXT

Stack: 0
Input: w\$

where state 0 is the initial LR state.

Let:

- $\text{top}(s)$ = top state
- $\text{lookahead}(a)$ = current lookahead

Consult:

$\text{ACTION}[s, a]$

27. If ACTION = Shift $\text{lookahead}(j)$

Suppose:

$\text{ACTION}[s, a] = \text{shift } j$

Then:

1. push input symbol $\text{lookahead}(a)$
2. push state $\text{lookahead}(j)$
3. advance input

Conceptually:

TEXT

... s

becomes:

TEXT

... s a j

28. If ACTION = Reduce $(A \rightarrow \beta)$

Suppose:

$[ACTION[s,a]=r(A \rightarrow \beta)]$

Then:

Step 1

Pop symbols/states corresponding to (β) .

If $(|\beta|=k)$, in a combined stack representation you remove $(2k)$ entries:

TEXT

symbol-state pairs

Step 2

Let new top state be (t) .

Step 3

Push LHS non-terminal (A) .

Step 4

Consult:

$[GOTO[t,A]=u]$

and push state (u) .

Important

Reduction does not consume the input lookahead.

This is a classic trap.

WRITE THIS IN NOTES

SHIFT

consumes input.

REDUCE

does not consume input.

29. Why Reduction Uses GOTO

Suppose we reduce:

$\beta \rightarrow A$

After replacing RHS by A , the parser needs to know:

From the previous state, where do I go after recognizing non-terminal A ?

That is exactly:

$\text{GOTO}[\text{state}, A]$

This is why ACTION and GOTO are both required.

30. Acceptance

If:

$\text{ACTION}[s, \$] = \text{acc}$

then:

Successful Parse

31. Error

If:

$\text{ACTION}[s, a]$

is empty/error:

Syntax Error

32. Let's Parse a String

Using grammar:

$S \rightarrow CC$

$C \rightarrow cC \mid d$

Let's parse:

cdd

Input:

TEXT

c d d \$

Does the grammar generate it?

Yes:

$\{ \rightarrow CC \rightarrow cCC \rightarrow cdC \rightarrow cdd \}$

Let's see LR parsing.

33. Initial Configuration

TEXT

Stack: 0

Input: c d d \$

Top state = 0.

Lookahead = c .

From table:

$\{ ACTION[0,c]=s3 \}$

Shift.

34. After Shift c

TEXT

Stack: 0 c 3

Input: d d \$

Top state = 3.

Lookahead = d .

$\{ ACTION[3,d]=s4 \}$

Shift.

35. After Shift d

TEXT

Stack: 0 c 3 d 4

Input: d \$

State 4 has:

$\{ ACTION[4,d]=r3 \}$

Production 3:

$\{ C \rightarrow d \}$

Reduce.

36. Reduce $(C \rightarrow d)$

Pop:

```
TEXT
d 4
```

Stack becomes:

```
TEXT
0 c 3
```

Top state:

$[3]$

Push **C**.

Find:

$[GOTO[3,C]=6]$

So:

```
TEXT
Stack: 0 c 3 C 6
Input: d $
```

Notice input **d** was **not consumed** during reduction.

37. State 6

Lookahead remains:

```
TEXT
d
```

Table:

$[ACTION[6,d]=r2]$

Production 2:

$(C \rightarrow cC)$

Reduce.

RHS length = 2 grammar symbols:

```
TEXT
c C
```

Pop:

TEXT

c 3 C 6

Stack becomes:

TEXT

0

Push C.

\[GOTO[0,C]=2 \]

So:

TEXT

Stack: 0 C 2

Input: d \$

38. State 2, Lookahead d

\[ACTION[2,d]=s4 \]

Shift:

TEXT

Stack: 0 C 2 d 4

Input: \$

39. State 4, Lookahead \$

\[ACTION[4,\$]=r3 \]

Reduce:

\[C\rightarrow d \]

Pop d 4.

Stack:

TEXT

0 C 2

Push C using:

\[GOTO[2,C]=5 \]

Result:

TEXT

Stack: 0 C 2 C 5

Input: \$

40. State 5, Lookahead \$

$\backslash$ ACTION[5,\$]=r1 $\backslash$

Production 1:

$\backslash$ S $\rightarrow$ CC $\backslash$

Pop:

TEXT
C 2 C 5

Stack becomes:

TEXT
0

Push S.

$\backslash$ GOTO[0,S]=1 $\backslash$

So:

TEXT
Stack: 0 S 1
Input: \$

41. State 1

$\backslash$ ACTION[1,\$]=acc $\backslash$

Therefore:

$\backslash$ $\boxed{\text{cdd}}$ is ACCEPTED $\backslash$

42. Parsing Trace

Stack	Input	Action
0	c d d \$	s3
0 c 3	d d \$	s4
0 c 3 d 4	d \$	r3: $\backslash$ (C to d)
0 c 3 C 6	d \$	r2: $\backslash$ (C to cC)
0 C 2	d \$	s4
0 C 2 d 4	\$	r3: $\backslash$ (C to d)
0 C 2 C 5	\$	r1: $\backslash$ (S to CC)

Stack	Input	Action
0 S 1	\$	accept

43. Critical Stack Rule

If production:

$A \rightarrow XYZ$

is reduced:

RHS contains 3 grammar symbols.

In a combined stack like:

TEXT

state symbol state symbol state ...

we remove:

$\boxed{2 \times 3 = 6 \text{ entries}}$

Then use GOTO for the LHS.

But don't blindly memorize 2k if the question represents stack differently.

The conceptual rule is:

Pop the RHS symbols and their associated states.

44. μ - Production Reduction

Suppose:

$A \rightarrow \epsilon$

RHS length:

$|0|$

During reduction:

Pop how many RHS symbols?

$\boxed{0}$

Nothing is removed from the stack.

Then:

1. look at current top state t
2. push A
3. push:

$\text{GOTO}[t, A]$

This can be a GATE trap.

45. Example of an LR(0) Conflict

Consider grammar:

$S \rightarrow Aa$

$A \rightarrow a$

Let's inspect a possible LR state after recognizing **A**:

$S \rightarrow A \cdot a$

This wants to **shift 'a'**.

Suppose the same state also had:

$B \rightarrow \beta \cdot$

Then that completed item wants to **reduce on every terminal**, including **a**.

So `ACTION[state, a]` would contain:

```
TEXT
shift
reduce
```

Therefore:

$\boxed{\text{Shift/Reduce Conflict}}$

46. A Concrete Non-LR(0) Grammar

Consider:

$S \rightarrow AA$

$A \rightarrow a \mid b$

This grammar may have states where completed items and shift possibilities coexist. Rather than guess from the grammar alone, the correct test is to construct canonical LR(0) states/table and inspect conflicts.

Important lesson

Do not say:

"This grammar looks simple, so it must be LR(0)."

LR(0) is determined by **conflict-free LR(0) item states**.

47. Completed Item + Terminal-after-Dot = S/R

Suppose state:

$I = \{ A \rightarrow \alpha \cdot, B \rightarrow \beta \cdot a \gamma \}$

Then:

- first item: reduce on **a**

- second item: shift **a**

Therefore:

$\boxed{S/R \text{ conflict on } a}$

This is one of the fastest LR(0) conflict checks.

48. Two Completed Items = R/R

State:

$I = \{ A \rightarrow \alpha \cdot, B \rightarrow \beta \cdot \}$

with distinct reductions.

Both reductions go on all terminals.

Therefore:

$\boxed{R/R \text{ conflict}}$

49. Is a State With a Completed Item Always a Conflict?

No.

Example:

$I_4 = \{ C \rightarrow d \cdot \}$

There is only one completed item and no shift item.

So all ACTION columns simply contain the same reduction.

No conflict.

50. Is a State With Multiple Non-Completed Items a Conflict?

Not necessarily.

Example:

$I_0 = \{ C \rightarrow \cdot c, C \rightarrow \cdot d \}$

One shifts on **c**, another shifts on **d**.

These go into **different columns**:

TEXT

ACTION[0, c]
ACTION[0, d]

No conflict.

Conflict requires competing actions in the same cell.

WRITE THIS IN NOTES

Multiple items in a state do not imply conflict. Multiple incompatible actions for the same terminal lookahead imply conflict.

51. Why LR(0) Is Restrictive

Suppose state has:

$[A \rightarrow \alpha \cdot]$

Maybe in the actual grammar, reduction by $(A \rightarrow \alpha)$ is valid only when next token is:

TEXT

)
or
\$

But LR(0) places reduction under **every** terminal:

TEXT

+
*
id
(
)
\$
...

This can create false conflicts.

Therefore LR(0) recognizes a relatively small class of grammars.

SLR fixes some of this by using:

$[\boxed{\text{FOLLOW}(A)}]$

52. Preview: Why SLR Can Resolve an LR(0) Conflict

Suppose state has:

$[A \rightarrow \alpha \cdot]$

and:

$[B \rightarrow \beta \cdot + \gamma]$

LR(0):

- reduction by $(A \rightarrow \alpha)$ under +
- shift +

! S/R conflict.

But suppose:

$[+ \notin \text{FOLLOW}(A)]$

Then SLR does **not** place the reduction under +.

Only shift remains.

Conflict disappears.

Therefore:

$\boxed{\text{Every LR(0) grammar is SLR(1), but not vice versa}}$

We'll establish this properly in Part 13.

53. ACTION vs GOTO During Shift and Reduce

Another common confusion.

On SHIFT

Parser uses:

$\text{ACTION}[\text{state}, \text{terminal}]$

and shifts directly to the state given there.

On REDUCE

Parser:

1. reduces RHS to non-terminal
2. then uses:

$\text{GOTO}[\text{state}, \text{nonterminal}]$

So GOTO is particularly important **after reductions**.

54. GATE-STYLE EXAMPLE 1

Suppose state 6 has:

$A \rightarrow xy \cdot$

in an LR(0) parser.

What entries are made?

Assuming terminals:

$\{a, b, \$\}$

Then reduction $(A \rightarrow xy)$ is entered in:

$\text{ACTION}[6, a]$

$\text{ACTION}[6, b]$

$\text{ACTION}[6, \$]$

All three.

55. GATE-STYLE EXAMPLE 2

Suppose:

$I_2 \xrightarrow{+} I_7$

Then:

$\boxed{\text{ACTION}[2,+] = s7}$

because $+$ is terminal.

56. GATE-STYLE EXAMPLE 3

Suppose:

$I_2 \xrightarrow{E} I_5$

Then:

$\boxed{\text{GOTO}[2,E] = 5}$

because E is non-terminal.

Do not write $s5$.

57. GATE-STYLE EXAMPLE 4

State contains:

$S \rightarrow S \cdot$

What entry?

$\boxed{\text{ACTION}[\text{state}, \$] = \text{accept}}$

Only $\$$.

58. GATE-STYLE EXAMPLE 5

State contains:

$A \rightarrow a \cdot$

and:

$B \rightarrow b \cdot c$

What happens in LR(0)?

- $(A \rightarrow a \cdot)$!' reduce under all terminals, including c
- $(B \rightarrow b \cdot c)$!' shift on c

Therefore:

$\boxed{\text{Shift/Reduce conflict on } c}$

59. GATE-STYLE EXAMPLE 6

State contains:

$A \rightarrow a \cdot$

and:

$B \rightarrow b \cdot$

LR(0):

Both produce reductions on every terminal.

Therefore:

$\boxed{\text{Reduce/Reduce conflict}}$

60. GATE TRAPS

Trap 1

A terminal transition goes into the GOTO table.

'L False.

Terminal transition:

$\boxed{\text{ACTION} = \text{shift}}$

Trap 2

A non-terminal transition produces a shift entry.

'L False.

It produces:

$\boxed{\text{GOTO}}$

Trap 3

LR(0) reduction is entered only under FOLLOW(LHS).

'L That's **SLR**, not LR(0).

LR(0):

$\boxed{\text{reduce on all terminal columns}}$

Trap 4

Completed augmented item reduces on every terminal.

'L False.

$S \rightarrow S \cdot$

gives:

$\boxed{\text{accept on } \$}$

Trap 5

Reduction consumes the current input lookahead.

'LFalse.

$\boxed{\text{Reduce does not consume input}}$

Trap 6

Shift leaves input unchanged.

'LFalse.

Shift consumes the lookahead token.

Trap 7

Two shift possibilities in the same state always mean a conflict.

'LNot if they occur on different terminals.

Example:

TEXT

shift on a
shift on b

Different table cells !' no conflict.

Trap 8

A completed item always creates a conflict.

'LNo.

It conflicts only if another incompatible action is required in the same cell.

Trap 9

Two completed ordinary items in one LR(0) state are fine if RHSs differ.

'LNo.

They generate competing reductions:

$\boxed{R/R}$

Trap 10

After reduction, the parser uses ACTION on the LHS non-terminal.

'LFalse.

Use:

$\boxed{\text{GOTO}}$

for the LHS.

61. MINI PRACTICE — PART 12

Try these before reading solutions.

Q1 — Easy

If:

$\text{GOTO}(I_3, a) = I_7$

where a is terminal, what table entry is created?

A. $\text{GOTO}[3, a] = 7$ B. $\text{ACTION}[3, a] = s7$ C. $\text{ACTION}[7, a] = s3$ D. Reduce 7

Q2 — Easy

In LR(0), a completed ordinary item:

$A \rightarrow \alpha \cdot$

causes reduction on:

A. FOLLOW(A) only B. FIRST(A) only C. Every terminal including $\$$ D. No terminal

Q3 — Medium

State:

$I = \{ A \rightarrow a \cdot, B \rightarrow b \cdot c \}$

What conflict occurs in LR(0)?

Q4 — Medium

State:

$I = \{ A \rightarrow a \cdot, B \rightarrow b \cdot \}$

What conflict occurs?

Q5 — Tricky

An LR parser reduces:

$A \rightarrow xyz$

Current lookahead is $+$.

Does the parser consume $+$ during reduction?

A. Yes B. No

SOLUTIONS

Q1

Since a is terminal:

$\boxed{\text{ACTION}[3,a]=s7}$

' B

Q2

LR(0) has no lookahead restriction.

Therefore:

$\boxed{\text{Every terminal including } \$}$

' C

Q3

Completed:

$A \rightarrow a \cdot$

causes reduce under c .

Other item:

$B \rightarrow b \cdot c$

causes shift on c .

Therefore:

$\boxed{\text{Shift/Reduce conflict}}$

Q4

Two completed items cause two reductions in the same terminal columns.

Therefore:

$\boxed{\text{Reduce/Reduce conflict}}$

Q5

' B — No

Reduction changes the stack but does not consume the lookahead.

62. WRITE THIS IN NOTES — LR(0) TABLE MASTER SHEET

SHIFT

If:

$\text{A} \rightarrow \alpha \cdot \beta$

and:

$\text{GOTO}(I_i, a) = I_j$

where a is terminal:

$\text{ACTION}[i, a] = s_j$

REDUCE

If:

$\text{A} \rightarrow \alpha \cdot$

with $(A \neq S)$:

$\text{Reduce } A \text{ to } \alpha \text{ under ALL terminal columns}$

ACCEPT

If:

$S \rightarrow \cdot$

then:

$\text{ACTION}[i, \$] = \text{acc}$

GOTO

If:

$\text{GOTO}(I_i, A) = I_j$

where A is non-terminal:

$\text{GOTO}[i, A] = j$

63. Conflict Shortcuts

LR(0) S/R

State has:

TEXT

completed item

+

terminal after dot in another item

!' likely:

S/R

LR(0) R/R

State has:

TEXT

two different completed ordinary items

!'

$\boxed{R/R}$

64. Parser Operation Memory

SHIFT

TEXT

Consume input

Push symbol/state

REDUCE

TEXT

Do NOT consume input

Pop RHS

Push LHS

Use GOTO

ACCEPT

TEXT

S ' !' S .

and

lookahead \$

30-SECOND REVISION

ACTION

$\boxed{\text{Terminals} + \$}$

contains:

TEXT

Shift

Reduce

Accept

Error

GOTO

$\boxed{\text{Non-terminals}}$

Terminal DFA edge

$\boxed{\text{SHIFT}}$

Non-terminal DFA edge

$\boxed{\text{GOTO}}$

Completed LR(0) item

$A \rightarrow \alpha \cdot$

means:

$\boxed{\text{reduce on EVERY terminal}}$

Accept

$S \rightarrow \cdot$

means:

$\boxed{\text{accept on } \$}$

Conflict tests

TEXT

Completed item + shift possibility
!' S / R conflict

Two completed items
!' R / R conflict

Critical difference coming next

TEXT

LR(0):
reduce everywhere

SLR:
reduce only on FOLLOW(LHS)

That single improvement allows SLR to parse more grammars than LR(0).

Completed: Parts 1–12/31 | Next: Part 13 — SLR(1): FOLLOW-Based Reductions, SLR Table Construction, Conflicts & LR(0) vs SLR

Reply **NEXT** for **Part 13**.

USER

NEXT

ASSISTANT

Compiler Design — Part 13: SLR(1) Parsing

Completed: Parts 1–12/31 | Current: Part 13 | Remaining: Parts 14–31

In Part 12, we found the biggest weakness of LR(0):

$A \rightarrow \alpha \cdot$

causes a reduction under **every terminal column**.

That is often unnecessarily aggressive.

SLR improves this using one idea you already know:

$\boxed{\text{FOLLOW}}$

The core shortcut is:

$\boxed{\text{LR(0): Reduce everywhere}}$

$\boxed{\text{SLR(1): Reduce only on FOLLOW(LHS)}}$

1. What Is SLR?

SLR stands for:

$\boxed{\text{Simple LR}}$

It uses:

- the **same LR(0) item sets**
- the **same LR(0) DFA**
- FIRST/FOLLOW information
- a more selective rule for reductions

This is extremely important.

WRITE THIS IN NOTES

SLR(1) uses LR(0) items and LR(0) states, but places a completed item's reduction only in the FOLLOW set of its LHS non-terminal.

2. Why Do We Need SLR?

Suppose an LR(0) state contains:

$A \rightarrow \alpha \cdot$

and:

$B \rightarrow \beta \cdot + \gamma$

The second item says:

On +, shift.

But LR(0) completed item says:

Reduce $(A \rightarrow \alpha)$ on every terminal.

Therefore on + :

TEXT
SHIFT
and
REDUCE

!' conflict.

But suppose:

$\notin \text{FOLLOW}(A)$

Then it makes no sense to reduce $\rightarrow A$ when lookahead is + , because + cannot legally follow A .

SLR uses this fact and does **not** put the reduction in the + column.

The conflict disappears.

3. Main Difference: LR(0) vs SLR

For a completed item:

$A \rightarrow \alpha \cdot$

LR(0)

Put:

$A \rightarrow \alpha$

as a reduction under:

$\boxed{\text{ALL terminals } + \$}$

SLR

Put the reduction only under:

$\boxed{\text{FOLLOW}(A)}$

That's the entire fundamental difference.

Master Memory Trick

SLR = Same LR(0) States + Limited Reductions using FOLLOW.

Or:

S = Simple

F = FOLLOW controls reduction

4. Does SLR Use LR(1) Items?

'L No.

SLR states still contain items such as:

$[A \rightarrow \alpha \cdot \beta]$

There is no explicit lookahead attached.

That is still an:

$[LR(0) \text{ item}]$

CLR/LR(1), which comes next, will use:

$[A \rightarrow \alpha \cdot \beta, a]$

with an explicit lookahead.

5. What Does the 1 in SLR(1) Mean?

SLR parsing makes parsing decisions based on the current **one input lookahead token**.

However, unlike canonical LR(1), that lookahead is not stored individually in each item.

Instead SLR uses FOLLOW sets globally for completed items.

For GATE, remember the operational distinction rather than overthinking the notation:

TEXT

```
S L R ! ' L R ( 0 ) i t e m s + F O L L O W
C L R ! ' L R ( 1 ) i t e m s w i t h e x p l i c i t l o o k a h e a d
```

6. SLR Table Construction

We begin exactly like LR(0).

Step 1

Augment grammar:

$[S \rightarrow S]$

Step 2

Build canonical collection of **LR(0) items**.

Step 3

Build terminal shift entries.

Step 4

Build non-terminal GOTO entries.

Step 5

For completed items, use FOLLOW to restrict reductions.

7. SLR SHIFT Rule

If state (I_i) contains:

$\text{GOTO}(\alpha, a) = \beta$

where a is terminal, and:

$\text{GOTO}(I_i, a) = I_j$

then:

$\text{ACTION}[i, a] = s_j$

Exactly the same as LR(0).

8. SLR GOTO Rule

If:

$\text{GOTO}(I_i, A) = I_j$

where A is non-terminal:

$\text{GOTO}[i, A] = j$

Again, same as LR(0).

9. SLR REDUCE Rule

Suppose:

$\alpha \rightarrow \beta$

occurs in state I_i , with $A \neq S'$.

Then for every:

$a \in \text{FOLLOW}(A)$

put:

$\text{ACTION}[i, a] = \text{reduce } \alpha \rightarrow \beta$

Do **not** put the reduction into terminal columns outside $\text{FOLLOW}(A)$.

WRITE THIS IN NOTES

For completed item:

$\alpha \rightarrow \beta$

SLR reduction:

$\text{ACTION}[i, a] = r(\alpha \rightarrow \beta) \quad \forall a \in \text{FOLLOW}(A)$

10. ACCEPT Rule

If state contains:

$S' \rightarrow S$

then:

$\boxed{\text{ACTION}[i, \$] = \text{acc}}$

Same as LR(0).

11. SLR Construction Summary

Situation	Table entry
$(A\alpha \cdot a\beta)$, terminal a	SHIFT
$(A\alpha \cdot B\beta)$, non-terminal B	GOTO
$(A\alpha \cdot)$	REDUCE only under FOLLOW(A)
$(S \cdot)$	ACCEPT under $\$$

12. Why FOLLOW Is Safe for Reduction

Suppose we reduce:

$\alpha \rightarrow A$

The next input symbol must be one that can legally occur after A .

By definition:

$\text{FOLLOW}(A)$

contains exactly such possible terminals.

So SLR asks:

"The RHS is complete—but is the next input symbol even allowed after A ?"

If not, don't reduce.

13. A Grammar That Shows SLR's Advantage

Consider:

$S \rightarrow Aa$

$A \rightarrow b$

Augment:

$S' \rightarrow S$

Let's build enough states to see the key idea.

14. Initial State

$I_0 = \{ S' \rightarrow \cdot S, S \rightarrow \cdot Aa, A \rightarrow \cdot b \}$

Transitions:

- on (S)
- on (A)
- on b

Suppose:

$I_0 \xrightarrow{b} I_3$

with:

$I_3 = \{A \rightarrow b \cdot\}$

15. FOLLOW(A)

From:

$S \rightarrow Aa$

terminal a follows A .

Therefore:

$\boxed{\text{FOLLOW}(A) = \{a\}}$

16. LR(0) Reduction in I_3

State:

$I_3 = \{A \rightarrow b \cdot\}$

LR(0) would reduce by:

$A \rightarrow b$

under **every terminal**.

If terminals are:

TEXT

a
b
\$

then LR(0) gives:

TEXT

a	!	r	e	d	u	c	e
b	!	r	e	d	u	c	e
\$	!	r	e	d	u	c	e

Even though only a can actually follow A here.

17. SLR Reduction

SLR uses:

$\text{FOLLOW}(A) = \{a\}$

Therefore:

TEXT

`ACTION[3, a] = reduce A! ' b`

but:

TEXT

`ACTION[3, b] = error`
`ACTION[3, $] = error`

This is more precise.

18. Important Point

This particular grammar may already happen to be LR(0), so restricting reductions might not change whether a conflict exists.

But it illustrates the principle:

$\boxed{\text{SLR produces fewer unnecessary reduce entries}}$

That extra precision lets it parse grammars that LR(0) cannot.

19. Classic SLR Example

Use grammar:

$S \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow id$

This grammar is left recursive, which is fine for LR parsing.

Augment:

$S' \rightarrow S$

We will focus on the relevant states rather than rederive every transition.

20. Some Important Item States

A state may contain:

$I = \{ E \rightarrow T \cdot \}$

This completed item means:

$E \rightarrow T$

is ready for reduction.

21. FOLLOW(E)

From:

$S \rightarrow E$

and E is at RHS end:

$FOLLOW(S) \subseteq FOLLOW(E)$

Since S ultimately finishes input:

$\$ \in FOLLOW(E)$

Also from:

$E \rightarrow E + T$

+ follows E.

Thus:

$FOLLOW(E) = \{+, \$\}$

22. LR(0) Behavior

For:

$E \rightarrow T \cdot$

LR(0) places reduction under all terminal columns, possibly including:

TEXT

id
+
\$

even though an `id` cannot legally follow E in this grammar.

SLR restricts the reduction to:

$\{+, \$\}$

This is more accurate.

23. Full Example Showing Conflict Removal

Consider the grammar:

$S \rightarrow Aa$

$S \rightarrow bAc$

$S \rightarrow dc$

$S \rightarrow bda$

$A \rightarrow d$

This is a classic kind of grammar useful for distinguishing LR-family power.

Suppose an LR(0) state contains:

$A \rightarrow d \cdot$

and also an item that can shift c .

LR(0) puts the reduction $(A \rightarrow d)$ under **all terminals**, including c , potentially creating:

$\boxed{\text{Shift/Reduce}}$

But calculate:

$\text{FOLLOW}(A)$

24. FOLLOW(A)

Occurrences of A:

From

$S \rightarrow Aa$

we get:

$a \in \text{FOLLOW}(A)$

From

$S \rightarrow bAc$

we get:

$c \in \text{FOLLOW}(A)$

So:

$\boxed{\text{FOLLOW}(A) = \{a, c\}}$

SLR reduces $d!$ only under a and c .

Whether this particular state's conflict disappears depends on which terminal transition coexists in that same state.

The key is to perform the exact state-local comparison.

25. Conflict Detection in SLR

Unlike LR(0), a completed item does **not** automatically conflict with every terminal shift.

Suppose state contains:

$A \rightarrow \alpha \cdot$

and:

$B \rightarrow \beta \cdot x \gamma$

where x is terminal.

Shift exists on:

x

Reduction exists on x only if:

$x \in \text{FOLLOW}(A)$

Therefore:

$\boxed{x \in \text{FOLLOW}(A) \rightarrow S/R \text{ conflict}} \}$

but:

$\boxed{x \notin \text{FOLLOW}(A) \rightarrow \text{no } S/R \text{ conflict on } x} \}$

26. SLR Shift/Reduce Shortcut

For a state containing:

$A \rightarrow \alpha \cdot$

and an item with terminal x after dot:

Ask:

$\boxed{x \in \text{FOLLOW}(A)?} \}$

If yes:

$\boxed{S/R \text{ conflict}} \}$

If no:

$\boxed{\text{No conflict on } x} \}$

This is a major GATE shortcut.

27. SLR Reduce/Reduce Conflict

Suppose same state contains:

$A \rightarrow \alpha \cdot$

and:

$B \rightarrow \beta \cdot$

SLR reduces:

- $A \rightarrow \alpha$ on $\text{FOLLOW}(A)$
- $B \rightarrow \beta$ on $\text{FOLLOW}(B)$

A reduce/reduce conflict occurs only where:

$\boxed{\text{FOLLOW}(A) \cap \text{FOLLOW}(B) \neq \emptyset} \}$

Huge Difference From LR(0)

LR(0):

Two completed ordinary items in one state !'

$\boxed{\text{Always R/R conflict}} \}$

SLR:

Two completed items cause R/R conflict **only if their FOLLOW sets overlap.**

28. Example

State contains:

$A \rightarrow a \cdot$

$B \rightarrow b \cdot$

Suppose:

$FOLLOW(A) = \{x\}$

$FOLLOW(B) = \{y\}$

with:

$x \neq y$

Then SLR table entries are:

TEXT

```
o n x !' r e d u c e A ! ' a
o n y !' r e d u c e B ! ' b
```

No same cell receives two reductions.

Therefore:

$\boxed{\text{No R/R conflict}}$

even though LR(0) would definitely have one.

29. SLR R/R Shortcut

For completed items:

$A \rightarrow \alpha \cdot$

$B \rightarrow \beta \cdot$

ask:

$FOLLOW(A) \cap FOLLOW(B)$

Empty:

$\boxed{\text{No R/R conflict}}$

Nonempty:

$\boxed{\text{R/R conflict on those common terminals}}$

30. LR(0) vs SLR Conflict Test

Situation	LR(0)	SLR
Completed A item + shift on x	Always S/R	Only if $x \in FOLLOW(A)$

Situation	LR(0)	SLR
Two completed items A, B	Always R/R	Only if FOLLOW(A) and FOLLOW(B) overlap

This table is extremely important.

31. SLR Uses Global FOLLOW Information

This is both its strength and its weakness.

Suppose:

$\text{FOLLOW}(A) = \{a, b, c\}$

SLR places an $\text{FOLLOW}(A)$ -reduction under all three:

TEXT

a
b
c

even if, in **this particular LR state**, only **a** is actually a valid lookahead.

SLR does not know state-specific lookahead information.

It uses the same global $\text{FOLLOW}(A)$ wherever an A-production completes.

This can still create unnecessary conflicts.

32. This Is Why CLR Is More Powerful

Canonical LR(1) attaches lookahead directly to items:

$[A \rightarrow \alpha \cdot, a]$

Then reduction occurs only on:

$[a]$

not all of $\text{FOLLOW}(A)$.

So the progression is:

TEXT

LR(0)
No reduction lookahead
!"
SLR
Global FOLLOW information
!"
CLR/LR(1)
State-specific item lookahead

Memory Trick

LR(0)

Blind reduction

SLR

FOLLOW-aware reduction

CLR

Item-specific lookahead reduction

33. Relative Power: LR(0) and SLR

Every LR(0) grammar is also SLR(1).

Why?

If the LR(0) table has no conflicts even when reductions are placed everywhere, restricting those reductions to FOLLOW sets cannot introduce a new conflict.

Therefore:

$$\boxed{\text{LR}(0) \subseteq \text{SLR}(1)}$$

More precisely, the inclusion is proper:

$$\boxed{\text{LR}(0) \subset \text{SLR}(1)}$$

because some grammars are SLR(1) but not LR(0).

34. Why the Reverse Does Not Hold

SLR may eliminate conflicts that LR(0) creates.

Therefore:

$$\boxed{\text{SLR}(1) \not\rightarrow \text{LR}(0)}$$

Example idea:

A state has:

TEXT

```
completed A item
+
shift on x
```

but:

$$x \notin \text{FOLLOW}(A)$$

Then:

- LR(0) !' conflict
- SLR !' no conflict

So grammar can be SLR but not LR(0).

35. Are SLR Grammars Unambiguous?

Yes.

If a grammar has a conflict-free deterministic SLR parsing table, it has a unique deterministic parse under that grammar.

Therefore:

$\boxed{\text{SLR}(1) \rightarrow \text{Unambiguous}}$

As with LL(1), the converse is false:

$\boxed{\text{Unambiguous} \not\rightarrow \text{SLR}(1)}$

36. Is Every LL(1) Grammar SLR(1)?

This is an important parser-family relationship that can be subtle if stated carelessly.

For deterministic context-free parsing classes, LL(1) grammars are within the LR(1) family. However, relationships involving the *same grammar* and SLR can be nuanced in textbook questions.

For your GATE preparation, do **not** infer SLR solely from LL(1) unless the question expects a standard grammar-class theorem. We'll build the exact hierarchy after CLR and LALR in Part 16.

For now, lock only:

$\boxed{\text{LR}(0) \subset \text{SLR}(1)}$

which is safe and central.

37. Important SLR Workflow

When a question asks:

Is this grammar SLR(1)?

Use this procedure.

Step 1

Augment grammar.

Step 2

Construct canonical LR(0) item states.

Step 3

Calculate FOLLOW sets.

Step 4

For terminal transitions, insert shifts.

Step 5

For completed item:

$A \rightarrow \alpha \cdot$

insert reduction only on:

$\text{FOLLOW}(A)$

Step 6

Check table cells for conflicts.

No conflict:

$\boxed{\text{SLR}(1)}$

Conflict:

$\boxed{\text{Not SLR}(1)}$

38. Can We Skip Full Table Construction?

Often yes.

If the question only asks whether an item state causes a conflict:

For S/R

Check:

$\text{shift terminal} \in \text{FOLLOW}(\text{completed LHS})?$

For R/R

Check:

$\text{FOLLOW}(A) \cap \text{FOLLOW}(B)$

This can save a lot of time.

39. Example — Fast S/R Check

State:

$I = \{ A \rightarrow x \cdot, B \rightarrow y \cdot z \}$

Suppose:

$\text{FOLLOW}(A) = \{+, \$\}$

The second item wants to shift:

z

Is:

$z \in \text{FOLLOW}(A)?$

No.

Therefore:

$\boxed{\text{No S/R conflict in SLR}}$

But LR(0) would have had a conflict on z .

40. Example — S/R Conflict Remains

Same state:

$I = \{ A \rightarrow x \cdot, B \rightarrow y \cdot z \}$

but now:

$\text{FOLLOW}(A) = \{z, \$\}$

Then:

- shift z
- reduce A on z

Therefore:

$\boxed{\text{SLR S/R conflict on } z}$

41. Example — Fast R/R Check

State:

$I = \{ A \rightarrow a \cdot, B \rightarrow b \cdot \}$

Suppose:

$\text{FOLLOW}(A) = \{x, y\}$

$\text{FOLLOW}(B) = \{z, \$\}$

Intersection:

$\emptyset$

Therefore SLR has:

$\boxed{\text{No R/R conflict}}$

42. If FOLLOW Sets Overlap

Suppose:

$\text{FOLLOW}(A) = \{x, y\}$

$\text{FOLLOW}(B) = \{y, z\}$

Intersection:

$\{y\}$

Then in column y :

TEXT

r	e	d	u	c	e		A	!	'	a
r	e	d	u	c	e		B	!	'	b

Therefore:

$\boxed{\text{R/R conflict on } y}$

43. Why FIRST Is Not Used for SLR Reduction

A common confusion:

Why FOLLOW(A), not FIRST(A)?

Suppose we have completed:

$A \rightarrow \alpha \cdot$

We have already recognized the RHS and are deciding whether to reduce it to A .

The relevant question is:

What can appear after A?

That is:

$\boxed{\text{FOLLOW}(A)}$

FIRST(A) tells what can start strings derived from A, which is not what we need at this point.

44. Compare With LL(1)

Interesting connection:

LL(1)

FOLLOW helps decide when a nullable non-terminal can disappear.

SLR

FOLLOW helps decide when a completed RHS can reduce to its LHS.

In both cases FOLLOW answers:

What may legally come after this non-terminal?

45. SLR Parsing Operation

Once the table is constructed, actual parsing is exactly like LR(0):

SHIFT

Consumes input.

REDUCE

Does not consume input.

After reduction

Use GOTO on the LHS.

Accept

At:

$[S \rightarrow S \cdot]$

with lookahead \$.

The difference lies only in **how the parsing table was constructed**.

46. LR(0) and SLR Use the Same DFA

This is a very frequent exam fact.

Given the same grammar:

$\boxed{\text{LR(0) canonical item collection} = \text{SLR canonical item collection}}$

The states do not change.

The transitions do not change.

Only reduction placement in ACTION changes.

WRITE THIS IN NOTES

LR(0) and SLR use exactly the same LR(0) item DFA.

Difference:

$\boxed{\text{ACTION reduce entries}}$

47. Therefore State Count Is the Same

For a given grammar:

$\boxed{\# \text{LR(0) states} = \# \text{SLR states}}$

because SLR doesn't build a new item automaton.

This is an excellent GATE fact.

CLR, however, generally has more states because LR(1) lookaheads distinguish states that LR(0) would treat as the same core.

48. GATE PATTERN— Identify Parser Type From Reduction Rule

Question says:

A reduction $\alpha \rightarrow \beta$ is placed only in columns corresponding to FOLLOW(A).

Answer:

$\boxed{\text{SLR}}$

If reduction is placed in every terminal:

$\boxed{\text{LR}(0)}$

If reduction uses explicit LR(1) item lookahead:

$\boxed{\text{CLR/LR}(1)}$

49. GATE-STYLE EXAMPLE 1

State contains:

$A \rightarrow ab \cdot$

$B \rightarrow c \cdot d$

and:

$\text{FOLLOW}(A) = \{e, \$\}$

Does SLR have a shift/reduce conflict?

Shift possibility is on:

d

Reduction $A \rightarrow a \cdot$ exists only under:

$\{e, \$\}$

Since:

$d \notin \text{FOLLOW}(A)$

there is:

$\boxed{\text{No conflict}}$

50. What Would LR(0) Do?

LR(0) reduces $A \rightarrow a \cdot$ on all terminals, including d .

So:

TEXT

on d :
shift
reduce

Therefore LR(0) has:

$\boxed{\text{S/R conflict}}$

This is the perfect demonstration of SLR's improvement.

51. GATE-STYLE EXAMPLE 2

State:

$\{ A \rightarrow a \cdot, B \rightarrow b \cdot \}$

with:

$FOLLOW(A) = \{c, d\}$

$FOLLOW(B) = \{d, e\}$

Find SLR conflict.

Intersection:

$FOLLOW(A) \cap FOLLOW(B) = \{d\}$

Therefore:

$\boxed{\text{R/R conflict on } d}$

52. GATE-STYLE EXAMPLE 3

State:

$\{ A \rightarrow a \cdot, B \rightarrow b \cdot c \}$

$FOLLOW(A) = \{c, \$ \}$

There is:

- shift on **c**
- reduce $A \rightarrow a$ on **c**

Therefore:

$\boxed{\text{S/R conflict on } c}$

Grammar is not SLR if this conflict cannot otherwise be removed by the formal grammar/table construction.

53. GATE-STYLE Example 4 — Same State, Different FOLLOW

Change:

$FOLLOW(A) = \{d, \$ \}$

Now reduction only on:

TEXT

d
\$

Shift is on:

TEXT

c

No overlap.

Therefore:

$\boxed{\text{No conflict}}$

54. Can FOLLOW Contain μ ?

Recall Part 7:

$\boxed{\epsilon \notin \text{FOLLOW}(A)}$

Therefore there is never an SLR table column for μ .

SLR reductions are placed under actual terminal lookaheads and possibly $\$$.

55. SLR vs Predictive LL(1) Table

Don't mix the two tables.

LL(1) table

Rows:

TEXT

Non-terminals

Columns:

TEXT

Terminals

Cells:

TEXT

Productions

SLR table

Rows:

TEXT

LR states

Columns split into:

TEXT

ACTION: terminals

GOTO: non-terminals

Cells contain:

TEXT

shift / reduce / accept / state number

56. GATE TRAP — SLR Uses FIRST to Restrict Reductions

'L False.

SLR reduction restriction is based on:

$\boxed{\text{FOLLOW}(\text{LHS})}$

57. GATE TRAP — SLR Constructs LR(1) Items

'L False.

It uses:

$\boxed{\text{LR}(0) \text{ items}}$

58. GATE TRAP — SLR and LR(0) Have Different State Counts

'L False for the same grammar.

$\boxed{\text{\#states are the same}}$

59. GATE TRAP — Every LR(0) Grammar Is SLR

' True.

$\boxed{\text{LR}(0) \subset \text{SLR}(1)}$

60. GATE TRAP — Every SLR Grammar Is LR(0)

'L False.

SLR can resolve some LR(0) conflicts using FOLLOW.

61. GATE TRAP — A Completed Item Reduces on All Terminals in SLR

'L False.

That's LR(0).

SLR:

$\boxed{\text{Reduce only on FOLLOW}(\text{LHS})}$

62. GATE TRAP — Two Completed Items Always Mean SLR R/R

'L False.

Only if their FOLLOW sets overlap.

63. GATE TRAP — Completed Item + Shift Item Always Means SLR S/R

'L False.

Only if the shift terminal belongs to FOLLOW of the completed item's LHS.

64. GATE TRAP — FOLLOW Is State-Specific

'L False.

FOLLOW(A) is a **global grammar property**.

The same FOLLOW(A) is used wherever an A-production completes.

This lack of state-specific precision is why SLR is weaker than CLR.

65. LR Family Precision Ladder

Start memorizing this:

```
TEXT
LR(0)
reduce everywhere
! " m o r e p r e c i s e
SLR
reduce on FOLLOW
! " m o r e p r e c i s e
CLR / LR(1)
reduce on item lookahead
```

More precision generally means:

$\boxed{\text{fewer unnecessary conflicts}}$

66. MINI PRACTICE — PART 13

Try first.

Q1 — Easy

SLR uses which item sets?

A. LL(1) items B. LR(0) items C. LR(1) items only D. No items

Q2 — Easy

For completed item:

$A \rightarrow xyz \cdot$

SLR places reduction under:

A. FIRST(A) B. Every terminal C. FOLLOW(A) D. FIRST(xyz)

Q3 — Medium

State:

$\{ A \rightarrow a \cdot, B \rightarrow b \cdot c \}$

and:

$\text{FOLLOW}(A) = \{d, \$\}$

Is there an SLR shift/reduce conflict?

Q4 — Medium

State:

$\{ A \rightarrow a \cdot, B \rightarrow b \cdot \}$

with:

$\text{FOLLOW}(A) = \{x, y\}$

$\text{FOLLOW}(B) = \{z\}$

Is there an SLR reduce/reduce conflict?

Q5 — Tricky

A grammar is LR(0). Which must be true?

A. It cannot be SLR(1). B. It is necessarily SLR(1). C. It is necessarily ambiguous. D. SLR uses fewer states than LR(0).

SOLUTIONS

Q1

' B — LR(0) items

SLR uses exactly the same canonical LR(0) item collection as LR(0).

Q2

' C — FOLLOW(A)

$\boxed{\text{SLR reduction !' FOLLOW(LHS)}}$

Q3

The shift terminal is:

c

But:

$c \notin \text{FOLLOW}(A)$

Therefore A' does not reduce under c .

Only shift remains.

$\boxed{\text{No S/R conflict}}$

Q4

Check:

$\text{FOLLOW}(A) \cap \text{FOLLOW}(B)$

$\{x, y\} \cap \{z\} = \emptyset$

Therefore:

$\boxed{\text{No R/R conflict}}$

Q5

' B

$\boxed{\text{LR}(0) \subset \text{SLR}(1)}$

So every LR(0) grammar is also SLR(1).

WRITE THIS IN NOTES — SLR MASTER SHEET

State Construction

$\boxed{\text{Same canonical LR}(0) \text{ item sets as LR}(0)}$

Shift

$A \rightarrow \alpha \cdot a \beta$

terminal transition:

$\boxed{\text{ACTION}[i, a] = \text{shift}}$

Reduce

$A \rightarrow \alpha \cdot$

SLR:

$\boxed{\text{reduce only on } \text{FOLLOW}(A)}$

Accept

$S \rightarrow \cdot$

$\boxed{\text{ACTION}[i, \$] = \text{accept}}$

S/R Conflict Test

State has:

$A \rightarrow \alpha \cdot$

and shift on x .

Conflict iff:

$\boxed{x \in \text{FOLLOW}(A)}$

R/R Conflict Test

State has:

$A \rightarrow \alpha \cdot$

$B \rightarrow \beta \cdot$

Conflict iff:

$\boxed{\text{FOLLOW}(A) \cap \text{FOLLOW}(B) \neq \emptyset}$

30-SECOND REVISION

LR(0)

$\boxed{\text{Reduce everywhere}}$

SLR

$\boxed{\text{Reduce on FOLLOW(LHS)}}$

Same states?

$\boxed{\text{YES}}$

LR(0) and SLR use the same LR(0) DFA.

Power

$\boxed{LR(0) \subset SLR(1)}$

S/R shortcut

TEXT

completed A item
+
shift on x

Check:

$x \in \text{FOLLOW}(A)?$

Yes! conflict.

No! safe.

R/R shortcut

TEXT

completed A item
+
completed B item

Check:

$\{ \text{FOLLOW}(A) \cap \text{FOLLOW}(B) \}$

Nonempty $\Rightarrow$ conflict.

Limitation

FOLLOW is global, so SLR can still make reductions in states where a particular FOLLOW symbol is not actually valid.

Canonical LR(1) fixes this using **item-specific lookahead**.

Completed: Parts 1–13/31 | Next: Part 14 — CLR(1) / Canonical LR(1): LR(1) Items, Lookahead, CLOSURE, GOTO & State Construction

Reply **NEXT** for **Part 14**.

USER

NEXT

ASSISTANT

Compiler Design — Part 14: CLR(1) / Canonical LR(1)

Completed: Parts 1–13/31 | Current: Part 14 | Remaining: Parts 15–31

We now move to a more powerful LR parser.

Recall the progression:

$\{ \text{LR}(0): \text{reduce everywhere} \}$

$\{ \text{SLR: reduce on FOLLOW(LHS)} \}$

Now:

$\{ \text{CLR}(1): \text{reduce only on the lookahead attached to the item} \}$

This extra precision removes many conflicts that SLR cannot remove.

1. What Does CLR Mean?

CLR means:

$\{ \text{Canonical LR} \}$

Usually, when we say CLR in GATE Compiler Design, we mean:

$\{ \text{Canonical LR}(1) \}$

It constructs states using **LR(1) items**, rather than LR(0) items.

2. LR(0) Item vs LR(1) Item

An LR(0) item looks like:

$[A \rightarrow \alpha \cdot \beta]$

An LR(1) item looks like:

$[\boxed{A \rightarrow \alpha \cdot \beta, a}]$

where a is a terminal or $\$$.

That final symbol a is called the:

$\boxed{\text{Lookahead}}$

WRITE THIS IN NOTES

LR(0)

$[A \rightarrow \alpha \cdot \beta]$

LR(1)

$[\boxed{A \rightarrow \alpha \cdot \beta, a}]$

a = one-symbol lookahead associated with that item.

3. Meaning of an LR(1) Item

Consider:

$[A \rightarrow \alpha \cdot \beta, a]$

It means roughly:

We have recognized α , still need to recognize β , and when the entire A is recognized, terminal a is a valid symbol that may follow this occurrence of A .

The dot still has the same meaning:

``text id="ir14a" left of dot '!' already recognized right of dot '!' still expected

The lookahead adds **context**.

4. Why Is Lookahead Useful?

Suppose we have completed:

$[A \rightarrow \alpha \cdot \beta, +]$

This tells us:

> Reduce $\backslash(\backslash\alpha\backslash)$ to $\backslash A \backslash$ only when the current input lookahead is $\backslash + \backslash$.

We do **not** reduce on every terminal.

We don't even necessarily reduce on every member of $\text{FOLLOW}(A)$.

Only:

```
\[
\boxed{+}
\]
```

for this specific item.

This gives CLR much more precision.

5. Compare Reduction Rules

This table is extremely important.

Parser	Completed information	Reduce on
LR(0)	$\backslash(A \rightarrow \alpha \cdot \backslash)$	Every terminal
SLR	$\backslash(A \rightarrow \alpha \cdot \backslash)$	$\text{FOLLOW}(A)$
CLR(1)	$\backslash([A \rightarrow \alpha \cdot \backslash, a] \backslash)$	Only $\backslash a \backslash$

Memory Trick

```
```text id="d1cgvp"
L R (0) ! ' A L L
S L R ! ' F O L L O W
C L R ! ' L O O K A H E A D
```

## 6. Why SLR Is Sometimes Too Crude

Suppose globally:

$\backslash \text{FOLLOW}(A) = \{a, b, c\} \backslash$

But in one particular parser state, an occurrence of  $A$  can actually be followed only by:

$\backslash a \backslash$

SLR still reduces under:

```
```text id="rwq71j" a b c
```

That may create unnecessary conflicts.

CLR can represent the completed item as:

```
\[
[A \rightarrow \alpha \cdot \backslash, a]
\]
```

and reduce only on $\backslash a \backslash$.

7. The ϵ in LR(1)

An LR(1) item contains:

```
\[
\boxed{1\text{ lookahead symbol}}
\]
```

Hence:

```
\[
LR(1)
\]
```

If there were k lookahead symbols, conceptually we'd speak of LR(k).

For GATE, LR(1) is the important form.

8. Augmented Grammar in CLR

Same idea as before.

If original start symbol is:

```
\[
S
\]
```

add:

```
\[
S' \rightarrow S
\]
```

The initial LR(1) item is:

```
\[
\boxed{[S' \rightarrow \cdot S, \epsilon]}
\]
```

Why ϵ ?

Because after the complete start symbol, the input must end.

WRITE THIS IN NOTES

Initial CLR item:

```
\[
\boxed{[S' \rightarrow \cdot S, \epsilon]}
\]
```

Accepting item:

```
\[
\boxed{[S' \rightarrow S \cdot, \epsilon]}
\]
```

9. CLR Acceptance

When a state contains:

```
\[
[S'\rightarrow S\cdot, \ \$]
\]
```

and current input is $\$$:

```
\[
\boxed{\text{ACCEPT}}
\]
```

Same high-level idea as LR(0)/SLR.

10. The Hardest Part: LR(1) CLOSURE

This is where CLR differs significantly.

Suppose an LR(1) state contains:

```
\[
[A\rightarrow\alpha\cdot B\beta, \ a]
\]
```

The dot is before non-terminal B .

So we must add productions of B .

But **which** lookaheads should those new B -items receive?

Answer:

```
\[
\boxed{\text{FIRST}(\beta a)}
\]
```

11. LR(1) CLOSURE Rule

If:

```
\[
[A\rightarrow\alpha\cdot B\beta, \ a]
\]
```

is in the state, then for every production:

```
\[
B\rightarrow\gamma
\]
```

and every terminal:

```
\[
b\in \text{FIRST}(\beta a)
\]
```

add:

```
\[
\boxed{[B\rightarrow\cdot\gamma, \ b]}
\]
```


Repeat until no new items appear.

WRITE THIS IN NOTES

From:

$$[A \rightarrow \alpha \cdot B\beta, a]$$

add:

$$[B \rightarrow \gamma \cdot b]$$

for every $B \rightarrow \gamma$ and:

$$\boxed{b \in \text{FIRST}(\beta a)}$$

This formula is **very important**.

12. What Is β ?

In:

$$[A \rightarrow \alpha \cdot B\beta, a]$$

we have:

- B = non-terminal being expanded
- β = symbols after B in the production
- a = existing lookahead of the original item

To determine what can follow B in this context, calculate:

$$\boxed{\text{FIRST}(\beta a)}$$

13. Why $\text{FIRST}(\beta a)$?

Suppose:

$$A \rightarrow \alpha B\beta$$

Once B is completed, the next input could come from β .

So normally B 's lookahead comes from:

$$\text{FIRST}(\beta a)$$

But if:

$$\backslash[\backslash\beta\rightarrow^*\epsilon\backslash]$$

then $\backslash(\backslash\beta\backslash)$ can disappear, and B can be followed by whatever followed A in this particular item.

That is the old lookahead $\backslash a \backslash$.

Therefore:

$$\backslash[\boxed{\text{FIRST}(\backslash\beta a)}\backslash]$$

automatically handles both cases.

Memory Trick

> ****CLR closure lookahead = FIRST of "what remains after B + old lookahead."****

Or:

> ****Look right of B; if it can vanish, inherit old lookahead.****

This is very similar to FOLLOW logic.

14 . C a s e 1 - 2 B e g i n s W i t h N o n - N u l l a b l e T e r m i n a l

Suppose:

$$\backslash[\backslash A \rightarrow \alpha \cdot BcD, \backslash \backslash \$]\backslash]$$

Here:

$$\backslash[\backslash\beta = cD\backslash]$$

The first symbol is terminal $\backslash c \backslash$.

Therefore:

$$\backslash[\text{FIRST}(cD\backslash \$) = \backslash \{c\}\backslash]$$

So every new B-item gets lookahead:

$$\backslash[\boxed{c}\backslash]$$

The old $\backslash \$ \backslash$ does not propagate because $\backslash cD \backslash$ cannot vanish.

15. Example

Grammar:

```
\[
S' \rightarrow S
\]
```

```
\[
S \rightarrow Ac
\]
```

```
\[
A \rightarrow a
\]
```

Start:

```
\[
[S' \rightarrow \cdot S, \$]
\]
```

Closure adds:

```
\[
[S \rightarrow \cdot Ac, \$]
\]
```

Now dot is before A.

Here:

```
\[
\beta = c
\]
```

old lookahead:

```
\[
\$
\]
```

Compute:

```
\[
FIRST(c\$) = \{c\}
\]
```

Thus add:

```
\[
\boxed{[A \rightarrow \cdot a, c]}
\]
```

Not:

```
\[
[A \rightarrow \cdot a, \$]
\]
```

This is state-specific precision.

16 . Case 2 - ² Is Empty

Suppose:

```
\[
[A\rightarrow\alpha\cdot B,\ a]
\]
```

There is nothing after B.

So:

```
\[
\beta=\epsilon
\]
```

Then:

```
\[
FIRST(\epsilon a)=FIRST(a)=\{a\}
\]
```

Therefore new B-items inherit the original lookahead:

```
\[
\boxed{a}
\]
```

17. Example

Grammar:

```
\[
S'\rightarrow S
\]
```

```
\[
S\rightarrow A
\]
```

```
\[
A\rightarrow a
\]
```

Initial:

```
\[
[S'\rightarrow\cdot S,\ \$]
\]
```

Add:

```
\[
[S\rightarrow\cdot A,\ \$]
\]
```

Here after A there is nothing.

Thus new A-production gets lookahead '\$':

```
\[
\boxed{[A\rightarrow\cdot a,\ \$]}
\]
```

18 . Case 3 - 2 Is Nullable

Suppose:

$$[A \rightarrow \alpha \cdot BC, a]$$

and:

$$FIRST(C) = \{c, \epsilon\}$$

Then:

$$FIRST(Ca)$$

contains:

- c
- and because C can vanish, old lookahead a

So:

$$\boxed{FIRST(Ca) = \{c, a\}}$$

Therefore B-productions are added with both lookaheads.

19. Worked Nullable Example

Suppose:

$$[S \rightarrow \cdot AB, \$]$$

and:

$$FIRST(B) = \{b, \epsilon\}$$

For A-items, compute:

$$FIRST(B\$)$$

B may start with:

$$b$$

or disappear, exposing:

$$\$$$

Therefore:

```
\[
FIRST(B\$)=\{b,\$\}
\]
```

So if:

```
\[
A\rightarrow a
\]
```

we add:

```
\[
[A\rightarrow\cdot a,b]
\]
```

and:

```
\[
[A\rightarrow\cdot a,\$]
\]
```

Often these are compactly written:

```
\[
\boxed{[A\rightarrow\cdot a,\{b,\$\}]}
\]
```

But formally, different lookaheads correspond to different LR(1) items.

20. Multiple Lookaheads – Notation

Instead of writing:

```
\[
[A\rightarrow\cdot a,b]
\]
```

```
\[
[A\rightarrow\cdot a,c]
\]
```

```
\[
[A\rightarrow\cdot a,\$]
\]
```

books often combine them for convenience:

```
\[
\boxed{[A\rightarrow\cdot a,\{b,c,\$\}]}
\]
```

This is compact notation.

Important

The underlying LR(1) items are technically separate items differing in lookahead.

21. LR(1) GOTO

GOTO is similar to LR(0), but the lookahead stays attached.

If state contains:

```
\[
[A\rightarrow\alpha\cdot X\beta,\ a]
\]
```

move the dot across X:

```
\[
[A\rightarrow\alpha X\cdot\beta,\ a]
\]
```

The lookahead:

```
\[
a
\]
```

does not change merely because the dot moved.

Then take LR(1) CLOSURE.

WRITE THIS IN NOTES

```
\[
GOTO(I,X)=
CLOSURE(
\{
[A\to\alpha X\cdot\beta,a]
\mid
[A\to\alpha\cdot X\beta,a]\in I
\}
)
\]
```

Memory Trick

> **GOTO moves the dot; lookahead travels with the item.**

22. LR(1) GOTO Example

Suppose:

```
\[
I=
\{
[A\rightarrow\cdot aB,\ \$]
\}
\]
```

On terminal `a`:

```
\[
GOTO(I,a)
\]
```

moves:

```
\[
```

```
[A $\rightarrow$  a $\cdot$  B,  $\$$ ]
\]
```

Then because dot is before B, closure uses old $\$$.

If:

```
\[
B $\rightarrow$  b
\]
```

we add:

```
\[
[B $\rightarrow$   $\cdot$  b,  $\$$ ]
\]
```

Thus:

```
\[
\boxed{
GOTO(I, a)
=
\{
[A $\rightarrow$  a $\cdot$  B,  $\$$ ],
[B $\rightarrow$   $\cdot$  b,  $\$$ ]
\}
}
\]
```

23. Canonical Collection of LR(1) Items

The overall algorithm resembles LR(0).

Step 1

Augment grammar.

Step 2

Construct:

```
\[
I_0=
CLOSURE(
\{[S' $\rightarrow$   $\cdot$  S,  $\$$ ]\}
)
\]
```

using **LR(1) closure**.

Step 3

For every state and every grammar symbol x:

```
\[
GOTO(I, x)
\]
```

Step 4

Add every new nonempty LR(1) item set.

Step 5

Repeat until no new states appear.

The result is the:

```
\[
\boxed{\text{Canonical Collection of LR(1) Items}}
\]
```

24. Why CLR Usually Has More States Than LR(0)/SLR

Suppose two states contain the same dotted productions:

State A:

```
\[
[A\rightarrow a\dot{B},c]
\]
```

State B:

```
\[
[A\rightarrow a\dot{B},d]
\]
```

If we ignore lookaheads, both have the same LR(0) core:

```
\[
A\rightarrow a\dot{B}
\]
```

But in canonical LR(1), they are **different states** because lookaheads differ.

Therefore CLR can produce many more states.

WRITE THIS IN NOTES

```
\[
\boxed{\text{\#CLR\ states\ \ge\ \#LR(0)/SLR\ states}}
\]
```

for the same grammar, generally.

Often strictly greater.

25. What Is the Core of an LR(1) Item?

If we remove the lookahead from:

```
\[
[A\rightarrow\alpha\dot{\beta},\ a]
\]
```

we get:

```
\[
A\rightarrow\alpha\dot{\beta}
\]
```

This LR(0) item is called its:

$$\boxed{\text{Core}}$$

Example:

$$[E \rightarrow E \cdot T, \]$$

has core:

$$\boxed{E \rightarrow E \cdot T}$$

This becomes crucial for **LALR** in Part 15.

26. State Core

Suppose CLR state is:

$$I = \{ [A \rightarrow a \cdot B, c], [B \rightarrow \cdot b, c] \}$$

Its LR(0) core is:

$$\boxed{\{ [A \rightarrow a \cdot B, B \rightarrow \cdot b] \}}$$

Different CLR states can have the same core but different lookahead sets.

LALR merges such states.

27. CLR Parsing Table Construction

Once CLR states are constructed:

SHIFT

If:

$$[A \rightarrow \alpha \cdot a \beta, b] \in I_i$$

and:

$$GOTO(I_i, a) = I_j$$

```

\]

where `a` is terminal:

\[
\boxed{\text{ACTION}[i,a]=s_j}
\]

---

### GOTO

For non-terminal A:

\[
\text{GOTO}(I_i,A)=I_j
\]

put:

\[
\boxed{\text{GOTO}[i,A]=j}
\]

---

### REDUCE

If:

\[
[A \rightarrow \alpha \cdot a] \in I_i
\]

then:

\[
\boxed{\text{ACTION}[i,a]=\text{reduce } A \rightarrow \alpha}
\]

Only under that item's lookahead `a`.

---

### ACCEPT

If:

\[
[S' \rightarrow S \cdot, \$]
\]

then:

\[
\boxed{\text{ACTION}[i,\$]=\text{acc}}
\]

---

# 28. CLR Reduction Is the Key

Suppose state contains:

\[
[A \rightarrow \alpha \cdot c]

```

\]

SLR might use:

\[

FOLLOW(A)={c,d,e\}

\]

and reduce under:

```text id="kjs35m"

c

d

e

CLR reduces only under:

```text id="ozqdw" c

That may eliminate conflicts on `d` and `e`.

29. CLR Shift/Reduce Conflict Test

Suppose state contains:

\[

[A $\rightarrow$ α $\cdot$ x]

\]

and another item can shift terminal:

\[

x

\]

Then:

- completed item reduces on x
- other item shifts on x

So:

\[

\boxed{\text{S/R conflict}}

\]

But if the completed item's lookahead is `y` and shift is on `x` with:

\[

x $\neq$ y

\]

there is no conflict.

30. CLR Reduce/Reduce Conflict Test

Suppose same state contains:

\[

$$[A \rightarrow \alpha \cdot x]$$

and:

$$[B \rightarrow \beta \cdot y]$$

A reduce/reduce conflict occurs only if the same terminal is in both items' lookahead sets.

If:

$$x=y$$

then both reductions compete on x.

If lookahead sets are disjoint:

$$\boxed{\text{No R/R conflict}}$$

31. Compare Conflict Precision

For completed A:

LR(0)

Potential reduction set:

$$\boxed{\text{All terminals}}$$

SLR

Potential reduction set:

$$\boxed{\text{FOLLOW}(A)}$$

CLR

Potential reduction set:

$$\boxed{\text{actual LR(1) lookahead set in that state}}$$

So:

$$\text{CLR is most precise of these three}$$

32. Full Small CLR Example

Consider grammar:

```
\[
S\rightarrow CC
\]
```

```
\[
C\rightarrow cC\mid d
\]
```

Augment:

```
\[
S'\rightarrow S
\]
```

Let's construct the initial CLR state carefully.

33. Start Item

```
\[
[S'\rightarrow\cdot S,\$]
\]
```

Dot before S.

Add:

```
\[
[S\rightarrow\cdot CC,\$]
\]
```

Now dot is before first C.

We must find its lookahead.

Item is:

```
\[
[S\rightarrow\cdot C C,\$]
\]
```

Here:

- B in closure rule = first `C`
- β = second `C`
- old lookahead = `\$`

Therefore compute:

```
\[
FIRST(C\$)
\]
```

34. FIRST(C)

Grammar:

```
\[
C\rightarrow cC\mid d
\]
```

So:

$$\begin{aligned} & \backslash[\\ & \text{FIRST}(C) = \{c, d\} \\ & \backslash] \end{aligned}$$

C is not nullable.

Therefore:

$$\begin{aligned} & \backslash[\\ & \text{FIRST}(C\$) = \{c, d\} \\ & \backslash] \end{aligned}$$

Thus add C-productions with lookaheads c and d:

$$\begin{aligned} & \backslash[\\ & [C \rightarrow \cdot cC, c] \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[\\ & [C \rightarrow \cdot d, c] \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[\\ & [C \rightarrow \cdot cC, d] \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[\\ & [C \rightarrow \cdot d, d] \\ & \backslash] \end{aligned}$$

Compactly:

$$\begin{aligned} & \backslash[\\ & [C \rightarrow \cdot cC, \{c, d\}] \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[\\ & [C \rightarrow \cdot d, \{c, d\}] \\ & \backslash] \end{aligned}$$

35. Initial CLR State

Thus:

$$\begin{aligned} & \backslash[\\ & \boxed{ \\ & I_0 = \\ & \left\{ \begin{array}{l} S \rightarrow \cdot S, \$ \\ S \rightarrow \cdot CC, \$ \\ C \rightarrow \cdot cC, c/d \\ C \rightarrow \cdot d, c/d \end{array} \right. \\ & \right. \\ & \backslash] \end{aligned}$$

where c/d is shorthand for lookaheads $\{c, d\}$.

Notice how this differs from $LR(0)$:

$LR(0)$ had no lookaheads.

36. GOTO on First C

From:

```
\[
[S\rightarrow\cdot CC,\$]
\]
```

move dot:

```
\[
[S\rightarrow C\cdot C,\$]
\]
```

Now dot before second C.

There is nothing after this C before old lookahead `\\$`.

So:

```
\[
FIRST(\$)=\{\$\}
\]
```

Thus closure adds:

```
\[
[C\rightarrow\cdot cC,\$]
\]
```

```
\[
[C\rightarrow\cdot d,\$]
\]
```

This produces a state such as:

```
\[
\boxed{
I=
\{
[S\to C\cdot C,\$],
[C\to\cdot cC,\$],
[C\to\cdot d,\$]
\}
}
\]
```

Lookahead context has changed.

37. Why This Matters

Notice the same C-productions appeared:

Initially with lookaheads

```
\[
\{c,d\}
\]
```

Later with lookahead

$$\backslash[\\{\backslash \$\backslash} \\]$$

The LR(0) cores may be similar, but CLR preserves the different contexts.

That's exactly why CLR is more powerful—and why it gets more states.

38. Important LR(1) Closure Shortcut

Given:

$$\backslash[A\rightarrow\alpha\cdot B\beta, \backslash a]$$

do this:

Step 1

Ignore everything left of the dot.

Step 2

Focus on:

```
```text id="i8m0ei"
B 2 , a
```

### Step 3

Compute:

$$\backslash[ \text{FIRST}(\backslash\beta a) \backslash]$$

### Step 4

Give every B-production those lookaheads.

### Step 5

Repeat recursively.

## 39. Common Case: Terminal Immediately After B

$$\backslash[ A\rightarrow\alpha\cdot Bx\gamma, a] \backslash]$$

where x is a terminal.

Then:

$$\backslash[ \text{FIRST}(x\gamma a) = \{x\} \backslash]$$

So B-items receive:

$$\backslash[ \boxed{x} \backslash]$$

You don't even need to inspect the old lookahead.

## 40. Common Case: B at End

$\rightarrow \alpha \cdot B, a$

Then:

$\boxed{\text{B-items inherit } a}$

This is a very fast exam shortcut.

## 41. Common Case: Nullable Sequence After B

$\rightarrow \alpha \cdot BCD, a$

Suppose:

- C nullable
- D starts with `d` and is non-nullable

Then:

$\text{FIRST}(CDa)$

contains:

- $\text{FIRST}(C) - \epsilon$
- then `d`

but not old `a`, because D blocks nullability.

If both C and D are nullable, then old `a` also propagates.

Exactly the same FIRST-sequence logic from Part 7 applies.

## 42. SLR FOLLOW vs CLR Lookahead

This deserves a clear example.

Grammar context:

`... A x ... A y ...`

Globally:

```
\[
FOLLOW(A) = {x, y}
\]
```

SLR completed A-production reduces on:

```
```text id="3e7gnq"
x and y
```

in every state where it completes.

CLR may have:

State 1

$\rightarrow \alpha \cdot x$

Reduce only on x.

State 2

$\backslash [A \rightarrow \alpha \cdot y] \backslash$

Reduce only on y.

This avoids mixing contexts.

43. Relative Power

Because CLR uses more precise information than SLR:

$\backslash \boxed{\text{SLR}(1) \subseteq \text{CLR}(1)} \backslash$

In standard parser hierarchy, this is a proper inclusion:

$\backslash \boxed{\text{SLR}(1) \subset \text{CLR}(1)} \backslash$

Meaning some grammars are CLR(1) but not SLR(1).

44. Why Every SLR Grammar Is CLR

If global FOLLOW-based reductions already cause no conflicts, using even more precise state-specific lookaheads cannot make the parser less capable.

Therefore:

$\backslash \boxed{\text{SLR}(1) \rightarrow \text{CLR}(1)} \backslash$

But the reverse is false.

45. CLR State Count

CLR's major disadvantage:

$\backslash \boxed{\text{Large number of states}} \backslash$

The same LR(0) core can occur repeatedly with different lookaheads.

So tables can become large.

This motivates:

LALR

LALR will merge CLR states with the **same LR(0) core**.

That's Part 15.

46. CLR vs SLR — Master Table

Feature	SLR(1)	CLR(1)
Items	LR(0)	LR(1)
Lookahead stored in item?	No	Yes
Reduction	FOLLOW(LHS)	Item lookahead
States	Same as LR(0)	Usually more
Precision	Lower	Higher
Power	Lower	Higher
Table size	Smaller	Larger

47. CLR vs LR(0)

Feature	LR(0)	CLR(1)
Item	$\{A \rightarrow \alpha \cdot \beta\}$	$\{A \rightarrow \alpha \cdot \beta, a\}$
Reduction	all terminals	explicit lookahead
Lookahead	none	1 symbol
States	fewer	generally more
Conflicts	more likely	fewer unnecessary conflicts

48. Important: Same Core $\Rightarrow$ Same CLR State

Suppose:

$$I_1 = \{A \rightarrow \alpha \cdot c\}$$

and:

$$I_2 = \{A \rightarrow \alpha \cdot d\}$$

Their cores are both:

$$\{A \rightarrow \alpha \cdot\}$$

But in CLR:

$$\boxed{I_1 \neq I_2}$$

because lookaheads differ.

In LALR, they may be merged.

49. Do Lookaheads Affect Shift Transitions?

The terminal immediately after the dot determines whether a shift is possible.

Example:

$[\rightarrow \alpha \cdot x \beta, a]$

shift occurs on:

$[x]$

not on the item's lookahead a .

This is a classic confusion.

WRITE THIS IN NOTES

In:

$[\rightarrow \alpha \cdot x \beta, a]$

- x after the dot !' next grammar symbol / shift candidate
- a after comma !' reduction lookahead once production completes

They perform completely different jobs.

50. Example

Item:

$[\rightarrow E \cdot + T, \$]$

What does it mean?

- Already recognized: E
- Next grammar symbol expected: $+$
- If this E -production eventually completes in this context, associated lookahead is $\$$

Current shift candidate:

$\boxed{+}$

not $\$$.

51. Lookahead Is Used Mainly at Completed Items

Consider:

$[\rightarrow \alpha \cdot \beta, a]$

while $(\beta \neq \epsilon)$, the parser still needs to recognize more RHS symbols.

The lookahead a matters for propagating context through closure, and eventually for reduction.

When completed:

$[\rightarrow \alpha \beta \cdot, a]$

then:

$\boxed{\text{reduce only when current input} = a}$

52. GATE-STYLE Example 1 — LR(1) Closure

Given item:

$[A \rightarrow \alpha \cdot BcD, \$]$

and productions:

$B \rightarrow x \mid y$

What lookahead gets attached to new B-items?

Compute:

$FIRST(cD\$) = \{c\}$

Therefore add:

$\boxed{[B \rightarrow \cdot x, c]}$

$\boxed{[B \rightarrow \cdot y, c]}$

53. GATE-STYLE Example 2 — Empty ²

Given:

$[A \rightarrow \alpha \cdot B, \$]$

and:

$B \rightarrow b$

Since nothing follows B:

$FIRST(\$) = \{\$\}$

Add:

$\boxed{[B \rightarrow \cdot b, \$]}$

54. GATE-STYLE Example 3 — Nullable ²

Given:

$[A \rightarrow \cdot BC, \$]$

and:

$FIRST(C) = \{c, \epsilon\}$

Then:

$FIRST(C\$) = \{c, \$\}$

So B-production items receive:

$\boxed{\{c, \$\}}$

as lookaheads.

55. GATE-STYLE Example 4 — Reduction

State contains:

$[A \rightarrow xyz \cdot]$

Where does CLR place the reduction?

Only:

$\boxed{\text{ACTION}[\text{state}] = \text{reduce} \rightarrow xyz}$

Not under every FOLLOW(A) token.

56. GATE-STYLE Example 5 — Conflict

State contains:

$[A \rightarrow x \cdot a]$

and:

$[B \rightarrow y \cdot aZ, b]$

First item reduces on:

a

Second item shifts:

a

Therefore:

$\boxed{\text{Shift/Reduce conflict on } a}$

57. GATE-STYLE Example 6 — No Conflict

State:

$[A \rightarrow x \cdot b]$

$[B \rightarrow y \cdot aZ, c]$

First item reduces only on:

b

Second shifts on:

a

Different ACTION columns.

Therefore:

$\boxed{\text{No S/R conflict}}$

58. GATE TRAPS

Trap 1

CLR uses LR(0) items plus FOLLOW sets.

'LFalse.

That's SLR.

CLR uses:

$\boxed{LR(1) \text{ items}}$

Trap 2

The LR(1) lookahead determines what symbol is immediately shifted next.

'LFalse.

The symbol immediately after the dot determines the next grammar-symbol transition.

Trap 3

For closure of:

$[A \rightarrow \alpha \cdot B \beta, a]$

new B-items get lookahead **a** directly in every case.

'LFalse.

Use:

$\boxed{\text{FIRST}(\beta a)}$

Only if (βa) can vanish fully does **a** propagate.

Trap 4

CLR reductions use FOLLOW(A).

'LFalse.

Use the item's own lookahead.

Trap 5

Two CLR items with same dotted production but different lookaheads are identical.

'LFalse.

They are distinct LR(1) items.

Trap 6

LR(0) and CLR always have the same number of states.

'LFalse.

CLR usually has more.

Trap 7

When computing GOTO, the lookahead changes automatically.

'L False.

The existing lookahead travels with the shifted-dot item. Closure may then generate additional items with derived lookaheads.

Trap 8

Completed CLR item reduces on every terminal.

'L False.

Only on its lookahead(s).

Trap 9

CLR is less powerful than SLR.

'L False.

$\boxed{\text{SLR}(1) \subset \text{CLR}(1)}$

Trap 10

$[S \rightarrow S \cdot, \$]$

means ordinary reduce.

'L It means:

$\boxed{\text{ACCEPT}}$

59. MINI PRACTICE — PART 14

Try first.

Q1 — Easy

Which is an LR(1) item?

A. $(A \rightarrow a \cdot B)$ B. $([A \rightarrow a \cdot B, c])$ C. $(\text{FIRST}(A))$ D. $(\text{FOLLOW}(A))$

Q2 — Easy

For:

$[A \rightarrow xyz \cdot, a]$

a CLR parser reduces on:

A. Every terminal B. FOLLOW(A) C. ☒ a only D. FIRST(A)

Q3 — Medium

Given:

$[A \rightarrow \alpha \cdot Bc, \$]$

and:

$[B \rightarrow b]$

what lookahead is attached to the new B-item?

Q4 — Medium

Given:

$[A \rightarrow \alpha \cdot BC, \$]$

and:

$\text{FIRST}(C) = \{c, \epsilon\}$

what lookahead set is propagated to B-items?

Q5 — Tricky

Two CLR states contain:

$I_1 = \{[A \rightarrow a \cdot b]\}$

and:

$I_2 = \{[A \rightarrow a \cdot c]\}$

Are they the same CLR state merely because their dotted productions are identical?

SOLUTIONS

Q1

' B

$\boxed{[A \rightarrow a \cdot B, c]}$

contains both:

- dotted production
 - one-symbol lookahead
-

Q2

' C — 'a' only

CLR reduction uses the item's explicit lookahead.

Q3

We have:

$\rightarrow \alpha \cdot Bc, \$$

After B comes terminal c .

Compute:

$\text{FIRST}(c\$) = \{c\}$

Therefore:

$\boxed{B \rightarrow \cdot b, c}$

Q4

Compute:

$\text{FIRST}(C\$)$

Since:

$\text{FIRST}(C) = \{c, \epsilon\}$

C can start with c or disappear.

So:

$\text{FIRST}(C\$) = \{c, \$\}$

Therefore lookaheads:

$\boxed{\{c, \$\}}$

Q5

'L No.

They have the same **LR(0) core**, but different LR(1) lookaheads.

Therefore:

$\boxed{I_1 \neq I_2 \text{ in CLR}}$

This is exactly the type of state pair that LALR may merge.

60. WRITE THIS IN NOTES — CLR MASTER SHEET

LR(1) Item

$\boxed{A \rightarrow \alpha \cdot \beta, a}$

``text id="sag83i" ±= recognized ²= remaining a = lookahead

Initial Item

$\boxed{S' \rightarrow \cdot S, \$}$

```

\]

## Accept

\[
\boxed{[S'\rightarrow S\cdot,\$]}
\]

---

## CLOSURE

From:

\[
[A\rightarrow\alpha\cdot B\beta,a]
\]

for every:

\[
B\rightarrow\gamma
\]

add:

\[
[B\rightarrow\cdot\gamma,b]
\]

for all:

\[
\boxed{b\in \text{FIRST}(\beta a)}
\]

---

## GOTO

Move dot across X:

\[
[A\rightarrow\alpha\cdot X\beta,a]
\]

becomes:

\[
[A\rightarrow\alpha X\cdot\beta,a]
\]

Then take LR(1) closure.

---

## Reduce

Completed:

\[
[A\rightarrow\alpha\cdot,a]
\]

means reduce:

```

```

\[
A\rightarrow\alpha
\]

only under:

\[
\boxed{a}
\]

---

# 30-SECOND REVISION

### Precision ladder

```text id="5mhspr"
L R (0) !' A L L t e r m i n a l s
S L R !' F O L L O W (L H S)
C L R !' I t e m l o o k a h e a d

```

## CLR item

$\boxed{[A\rightarrow\alpha\cdot\beta,a]}$

## Closure magic formula

$\boxed{\text{FIRST}(\beta a)}$

For:

$[A\rightarrow\alpha\cdot B\beta,a]$

new B-items receive lookaheads from:

$\boxed{\text{FIRST}(\beta a)}$

## Fast cases

```text id="r854bm" B followed by terminal x !' lookahead x

B at RHS end !' inherit old lookahead

B followed by nullable sequence !' $\text{FIRST}(\text{sequence})$ plus old lookahead if entire sequence vanishes

```

### GOTO

> Move dot; old lookahead moves with it; then closure.

### Core

Remove lookahead:

\[
[A\rightarrow\alpha\cdot\beta,a]
\rightarrow
A\rightarrow\alpha\cdot\beta
\]

### Power

\[
\boxed{\text{SLR}(1)\subset\text{CLR}(1)}
\]

```

Cost

\[
\boxed{\text{CLR is powerful but can have many states}}
\]

That motivates **LALR**, which keeps much of CLR's power while reducing table size by merging states with identical LR(0) cores.

Completed: Parts 1–14/31 | **Next:** Part 15 – LALR(1): Same-Core State Merging, Lookahead Union, New Conflicts & CLR vs LALR

Reply **NEXT** for **Part 15**.

USER

NEXT

ASSISTANT

Compiler Design — Part 15: LALR(1)

Completed: Parts 1–14/31 | **Current:** Part 15 | **Remaining:** Parts 16–31

We now reach the last major LR parser type in your syllabus.

We already know:

\[\boxed{\text{LR(0)}} \]

\[\boxed{\text{SLR(1)}} \]

\[\boxed{\text{CLR(1)}} \]

CLR is very powerful, but it can create a **large number of states** because the same LR(0) core may appear repeatedly with different lookaheads.

LALR solves this by:

\[\boxed{\text{Merging CLR states having the same LR(0) core}} \]

1. What Does LALR Mean?

LALR stands for:

\[\boxed{\text{Look-Ahead LR}} \]

It starts conceptually from the **Canonical LR(1) states** and combines certain states.

The goal is:

Keep LR(1)-style lookahead information while reducing the number of states.

2. Why Is LALR Needed?

Suppose CLR has two states:

$$I_5 = \{ [A \rightarrow a \cdot B, c] \}$$

and:

$$I_{11} = \{ [A \rightarrow a \cdot B, d] \}$$

These states differ only in lookahead.

If we ignore lookaheads, both have:

$$[A \rightarrow a \cdot B]$$

So they have the same:

$$\boxed{\text{LR}(0) \text{ core}}$$

CLR keeps them separate.

LALR merges them.

3. Recall: What Is a Core?

For LR(1) item:

$$[A \rightarrow \alpha \cdot \beta, a]$$

remove the lookahead:

$$[A \rightarrow \alpha \cdot \beta]$$

This dotted production is the item's **LR(0) core**.

Example

Item:

$$[E \rightarrow E + \cdot T,)]$$

Core:

$$\boxed{E \rightarrow E + \cdot T}$$

4. Core of a State

Suppose CLR state:

$$I = \{ [A \rightarrow a \cdot B, c], [B \rightarrow \cdot b, d] \}$$

Its core is:

$$\boxed{\{ [A \rightarrow a \cdot B,], [B \rightarrow \cdot b] \}}$$

Lookaheads are ignored while finding the core.

WRITE THIS IN NOTES

Two CLR states have the same core if, after removing all lookaheads, they contain exactly the same set of LR(0) items.

5. LALR Construction — Central Idea

Start with CLR(1) item states.

If two CLR states have the same LR(0) core:

$\boxed{\text{Merge them}}$

For corresponding items, take the:

$\boxed{\text{Union of their lookahead sets}}$

6. Simple Merge Example

CLR state 1:

$I_1 = \{[A \rightarrow a \cdot c], [A \rightarrow a \cdot c] \}$

CLR state 2:

$I_2 = \{[A \rightarrow a \cdot d], [A \rightarrow a \cdot d] \}$

Same core:

$A \rightarrow a \cdot$

LALR merges them into:

$\boxed{I = \{[A \rightarrow a \cdot c, d], [A \rightarrow a \cdot c, d] \}}$

Formally this means:

$[A \rightarrow a \cdot c]$

and:

$[A \rightarrow a \cdot d]$

are represented together.

Memory Trick

Same core !' merge.

Different lookaheads !' union.

7. Example With Multiple Items

Suppose:

$$I_4 = \left\{ [A \rightarrow a \cdot B, c], [B \rightarrow a \cdot b, c] \right\}$$

and:

$$I_9 = \left\{ [A \rightarrow a \cdot B, d], [B \rightarrow a \cdot b, d] \right\}$$

Their cores are identical:

$$\{ [A \rightarrow a \cdot B, \cdot], [B \rightarrow a \cdot b, \cdot] \}$$

Merge:

$$\boxed{\left\{ [A \rightarrow a \cdot B, \{c, d\}], [B \rightarrow a \cdot b, \{c, d\}] \right\}}$$

8. Can States With Different Cores Be Merged?

No.

Example:

$$I_1 = \{ [A \rightarrow a \cdot B, c] \}$$

and:

$$I_2 = \{ [A \rightarrow a b \cdot c] \}$$

Their dotted productions differ.

Therefore their cores differ.

They cannot be merged.

GATE TRAP

LALR does **not** merge states merely because:

- they contain the same non-terminal, or
- they have similar productions, or
- they have the same lookahead.

They must have:

$$\boxed{\text{Exactly the same LR(0) core}}$$

9. State Count — Major GATE Fact

Recall:

LR(0) / SLR

Use LR(0) item sets.

CLR

May split one LR(0) core into several states because lookaheads differ.

LALR

Merges CLR states back together when their cores are equal.

Therefore, for the same grammar:

$$\boxed{\#LALR \text{ states} = \#LR(0) \text{ states} = \#SLR \text{ states}}$$

under the standard canonical-core construction.

This is a very important exam fact.

10. Why LALR State Count Equals LR(0) State Count

Each LR(0) state corresponds to one distinct LR(0) core.

CLR may have:

core X + lookahead set 1 core X + lookahead set 2 core X + lookahead set 3

LALR combines all of them back into one state:

core X + union of lookaheads

So the number of merged states corresponds to the LR(0) cores.

WRITE THIS IN NOTES

$$\boxed{\#LR(0) = \#SLR = \#LALR}$$

for the same grammar's canonical constructions.

CLR generally has:

$$\boxed{\text{same or more states}}$$

11. Why Is LALR Useful?

It offers a practical compromise:

CLR

- powerful
- precise
- large parsing tables

SLR

- smaller
- less precise
- more conflicts

LALR

- LR(0)-like state count
- better lookahead precision than SLR

- often close to CLR in practical parsing capability

This is why LALR became historically popular in parser generators such as Yacc-style systems.

12. Does LALR Use FOLLOW Sets for Reduction?

LNNo.

That is SLR.

LALR retains LR(1) lookahead information.

For a completed merged item:

$[A \rightarrow \alpha \cdot, \{a, b\}]$

reduce only on:

```text id="ty9q7l" a b

not on all of FOLLOW(A), unless they happen to coincide.

---

### # 13. LALR Reduction Rule

If merged state contains:

$[A \rightarrow \alpha \cdot, \{a_1, a_2, \dots, a_k\}]$

then place reduction:

$A \rightarrow \alpha$

under:

$\boxed{a_1, a_2, \dots, a_k}$

only.

So reduction is based on LR(1) lookaheads, not global FOLLOW.

---

### # 14. CLR vs LALR Reduction

Suppose CLR has:

### State  $(I_1)$

$[A \rightarrow \alpha \cdot, a]$

### State  $(I_2)$

$[$

```
[A\rightarrow\alpha\cdot,b]
\]
```

After LALR merge:

```
\[
[A\rightarrow\alpha\cdot,\{a,b\}]
\]
```

Now the merged state reduces on:

```
\[
a,b
\]
```

This union can sometimes introduce a new conflict.

That is the key limitation of LALR.

---

# 15. Can Merging Introduce Conflicts?

' Yes .

This is very important.

Two CLR states can individually be conflict-free.

But after merging their lookahead sets, a **reduce/reduce conflict** may appear.

Therefore:

```
\[
\boxed{\text{CLR grammar need not be LALR grammar}}
\]
```

---

# 16. How Can a New Conflict Appear?

Consider two CLR states with the same core.

State 1:

```
\[
I_1=
\{
[A\rightarrow c\cdot,d],
[B\rightarrow c\cdot,e]
\}
\]
```

State 2:

```
\[
I_2=
\{
[A\rightarrow c\cdot,e],
[B\rightarrow c\cdot,d]
\}
\]
```

Both have same core:

```
\[
```

```

\{
A\rightarrow c\dot{,}\;
B\rightarrow c\dot{,}
\}
\}

17. Check CLR State 1

\[
[A\rightarrow c\dot{,}d]
\]

reduces A!' c on :

\[
d
\]

\[
[B\rightarrow c\dot{,}e]
\]

reduces B!' c on :

\[
e
\]

Different columns.

No R/R conflict.

18. Check CLR State 2

\[
[A\rightarrow c\dot{,}e]
\]

reduces on:

\[
e
\]

\[
[B\rightarrow c\dot{,}d]
\]

reduces on:

\[
d
\]

Again different columns.

No R/R conflict.

Thus each CLR state is conflict-free.

```

# 19. Merge for LALR

Union corresponding lookaheads:

For A:

$$\backslash[\backslash\{d,e\}\backslash]$$

For B:

$$\backslash[\backslash\{d,e\}\backslash]$$

Merged state:

$$\backslash[\backslash\boxed{\{[A\rightarrow c\cdot,\backslash\{d,e\}], [B\rightarrow c\cdot,\backslash\{d,e\}]\}}\backslash]$$

Now on `d`:

```
```text id="lzvphi"
  reduce  A!' c
  reduce  B!' c
```

and on e:

```
```text id="yp3hvj" reduce  A!' reduce  B!' c
```

Therefore:

$$\backslash[\backslash\boxed{\text{Reduce/Reduce conflict}}\backslash]$$

---

# 20. Major Result

A grammar can be:

$$\backslash[\backslash\boxed{\text{CLR(1) but not LALR(1)}}\backslash]$$

because merging same-core CLR states can create a conflict.

Therefore CLR is strictly more powerful than LALR:

$$\backslash[\backslash\boxed{\text{LALR(1)}\subset\text{CLR(1)}}\backslash]$$

---

## # 21. Can Merging Create a New Shift/Reduce Conflict?

This is a subtle and highly useful GATE fact.

Standard merging of LR(1) states with identical cores **cannot** create a new shift/reduce conflict if none existed before.

Why?

A shift possibility depends on the LR(0) core:

```
\[
A \rightarrow \alpha \cdot a \beta
\]
```

Since same-core states contain the same shift items, a shift on `a` exists in **both** states.

Suppose merging introduces a reduction on `a` because one state contributes lookahead `a`.

Then that original state already contained:

- the shift on `a` (same core)
- the reduction on `a`

So that state would already have had an S/R conflict.

Therefore merging conflict-free CLR states cannot newly introduce S/R.

---

## # 22. What New Conflict Can LALR Merging Introduce?

The important answer:

```
\[
\boxed{\text{Reduce/Reduce conflict}}
\]
```

So remember:

> **Merging CLR states with identical cores may introduce R/R conflicts, but not new S/R conflicts.**

This is a very good GATE fact.

---

## # 23. Why Can R/R Be New?

Because two reductions may have disjoint lookaheads in each individual CLR state but their unions may overlap after merging.

Example we just saw:

```
```text id="rf38ac"
```

State 1:

A reduce on d

B reduce on e

State 2:

A reduce on e

B reduce on d

After union:

```
```text id="5oi5w4" A reduce on d,e B reduce on d,e
```

!' R / R .

---

#### # 24. Relative Power So Far

We now know:

$\backslash$   
 $\boxed{\text{LR}(0) \subset \text{SLR}(1)}$   
 $\backslash$

and:

$\backslash$   
 $\boxed{\text{SLR}(1) \subset \text{LALR}(1)}$   
 $\backslash$

and:

$\backslash$   
 $\boxed{\text{LALR}(1) \subset \text{CLR}(1)}$   
 $\backslash$

Thus:

$\backslash$   
 $\boxed{\text{LR}(0) \subset \text{SLR}(1) \subset \text{LALR}(1) \subset \text{CLR}(1)}$   
 $\backslash$

This is one of the most important hierarchy lines to remember.

We'll consolidate all grammar-class relationships in Part 16.

---

#### # 25. Why Is LALR More Powerful Than SLR?

SLR uses:

$\backslash$   
 $\text{FOLLOW}(A)$   
 $\backslash$

globally.

LALR uses lookaheads derived from  $\text{LR}(1)$  contexts.

Suppose:

$\backslash$   
 $\text{FOLLOW}(A) = \{a, b, c\}$   
 $\backslash$

but in a particular merged LALR state, A's actual lookaheads are:

$\backslash$



$\{a, c\}$   
 $\setminus$

LALR does not reduce on  $b$ .

SLR would.

Therefore LALR may avoid an SLR conflict.

---

#### # 26. Why Is CLR More Powerful Than LALR?

CLR keeps same-core states separate.

LALR merges them.

Keeping them separate preserves contextual distinctions.

Merging can combine lookahead sets and create R/R conflicts.

Thus:

$\boxed{\text{CLR has maximum precision among these standard LR methods}}$

but at the cost of more states.

---

#### # 27. Table-Size Comparison

Generally:

### LR(0)

small state count.

### SLR

same state count as LR(0).

### LALR

same state count as LR(0)/SLR.

### CLR

can have significantly more states.

So in terms of number of states:

$\boxed{\text{LR}(0) = \text{SLR} = \text{LALR} \leq \text{CLR}}$

for a fixed grammar under standard constructions.

---

#### # 28. Power vs State Count

A useful visualization:

```
```text id="acjcjs"
Parsing power:
LR(0) < SLR < LALR < CLR

State count:
LR(0) = SLR = LALR "d CLR
```

This is why LALR is attractive:

More power than SLR without CLR's state explosion.

29. LALR Construction Method — Exam Steps

If asked to construct LALR from CLR:

Step 1

Construct canonical LR(1) states.

Step 2

Remove lookaheads mentally and identify each state's LR(0) core.

Step 3

Find states having identical cores.

Step 4

Merge each same-core group.

Step 5

For corresponding items, union their lookahead sets.

Step 6

Reconstruct transitions between merged states.

Step 7

Build ACTION/GOTO table.

Step 8

Check conflicts.

30. GATE Shortcut for State Comparison

Suppose:

$I_i = \{ [A \rightarrow a \cdot B, c], [B \rightarrow a \cdot b, d] \}$

and:

$I_j = \{ [A \rightarrow a \cdot B, x], [B \rightarrow a \cdot b, y] \}$

Ignore:

```text id="8uwpa5" c, d, x, y

Compare only:

```
\[
A\to a\cdot B
\]
```

and:

```
\[
B\to\cdot b
\]
```

If those dotted-production sets match exactly:

```
\[
\boxed{\text{same core}}
\]
```

!' merge in L A L R .

---

# 31. Kernel Items and LALR

Recall from Part 11:

Kernel items are essentially:

- $\backslash(S'\rightarrow\cdot S)$
- items whose dot has moved from the left edge

In practice, LALR state equivalence is often described using the same **LR(0) kernel/core**.

So you may see:

> Merge LR(1) states having identical kernels.

For standard canonical states, "same LR(0) core/kernel" is the key idea.

---

# 32. Does LALR Start From SLR States?

Conceptually, the textbook construction is:

```
\[
\boxed{\text{CLR states !' merge same-core states !' L A L R}}
\]
```

There are more efficient algorithms that construct LALR without explicitly building full CLR tables, but they are implementation details.

For GATE, remember the conceptual definition above.

---

# 33. Does LALR Simply Add FOLLOW Sets?

'L N o .

Do not confuse LALR with SLR.

### SLR

### ### LALR

Very different.

— — —

Suppose completed production:

```
LR(0)
```

```
```text id="s7vwjm"
every terminal
```

if:

reduce on:

```
#### CLR state 1
```

reduce only on:

reduce only on:

```
#### LALR merged
```

```
\[
[A\rightarrow x\cdot, \{a,c\}]
\]
```

reduce on:

```
```text id="57rlq0"
a,c
```

This one example captures all four parser types.

## 35. Very Important Comparison

$\boxed{\text{LALR lookahead set} \subseteq \text{FOLLOW}(A)}$

in the relevant sense for a completed A item, because LR(1)-derived lookaheads are valid possible followers of that contextual occurrence of A.

The LALR lookahead may be more selective than global FOLLOW.

That is why LALR can succeed where SLR fails.

## 36. GATE-STYLE Example 1 — Same Core?

Consider:

$I_1 = \{ [A \rightarrow a \cdot B, c], [B \rightarrow \cdot d, c] \}$

$I_2 = \{ [A \rightarrow a \cdot B, e], [B \rightarrow \cdot d, e] \}$

Same core?

Remove lookaheads.

Both become:

$\{ [A \rightarrow a \cdot B, B \rightarrow \cdot d] \}$

Therefore:

$\boxed{\text{Yes}}$

They are merged in LALR.

## 37. Resulting Merge

The merged state is:

$\boxed{\{ [A \rightarrow a \cdot B, \{c,e\}], [B \rightarrow \cdot d, \{c,e\}] \}}$

## 38. GATE-STYLE Example 2 — Different Core

$I_1 = \{ [A \rightarrow a \cdot B, c] \}$

$I_2 = \{ [A \rightarrow a \cdot b, c] \}$

Remove lookaheads:

```text id="kyd1dv" A !' a · B A !' ab ·

Different.

Therefore:

\[
\boxed{\text{Do not merge}}
\]

39. GATE-STYLE Example 3 – Number of States

Suppose:

- LR(0) automaton has 12 states.
- CLR automaton has 19 states.

How many states will standard LALR automaton have?

\[
\boxed{12}
\]

because merging CLR states by core returns one state per LR(0) core.

40. GATE-STYLE Example 4 – Conflict Created by Merge

CLR states:

\[
I_1=
\{
[A\to x\cdot,a],
[B\to x\cdot,b]
\}
\]

\[
I_2=
\{
[A\to x\cdot,b],
[B\to x\cdot,a]
\}
\]

Both states individually:

```text id="ovzdkl"  
A and B reductions on different terminals

No conflict.

Merged:

\[ A: \{a,b\} \]

\[ B: \{a,b\} \]

Thus both reduce on **a** and **b**.

Therefore:

$\boxed{\text{R/R conflict}}$

So grammar can be CLR but not LALR.

---

## 41. GATE-STYLE Example 5 — Power

Which is more powerful?

A. SLR B. LALR

Answer:

$\boxed{\text{LALR}}$

because:

$\text{SLR}(1) \subset \text{LALR}(1)$

---

## 42. GATE-STYLE Example 6 — State Count

Which usually has more states?

A. LALR B. CLR

$\boxed{\text{CLR}}$

because LALR merges CLR states with identical cores.

---

## 43. GATE TRAPS

### Trap 1

LALR merges CLR states having the same lookahead sets.

'L False.

It merges states having the same:

$\boxed{\text{LR}(0) \text{ core}}$

---

### Trap 2

Different cores can be merged if lookaheads are identical.

'L False.

Same core is mandatory.

---

### Trap 3

When merging lookaheads, take intersection.

'L False.

Take:

$\boxed{\text{UNION}}$

---

#### Trap 4

---

LALR uses FOLLOW sets for reduction.

'L False.

That's SLR.

---

#### Trap 5

---

LALR always has the same number of states as CLR.

'L False.

CLR can have more.

---

#### Trap 6

---

LALR has fewer states than LR(0).

'L Under standard construction, no.

$\boxed{\#LALR = \#LR(0)}$

---

#### Trap 7

---

If a grammar is CLR(1), it must be LALR(1).

'L False.

Merging can create an R/R conflict.

---

#### Trap 8

---

If grammar is LALR(1), it must be CLR(1).

' True.

Because:

$\boxed{LALR(1) \subset CLR(1)}$

---

#### Trap 9

---

If grammar is SLR(1), it may not be LALR(1).

'L In standard hierarchy:

$\boxed{SLR(1) \subset LALR(1)}$

So SLR implies LALR.

---



---

## Trap 10

---

Merging conflict-free CLR same-core states can introduce a new S/R conflict.

'LNo.

The important new conflict possible from merging is:

$\boxed{R/R}$

---

## 44. Parser Family Memory Ladder

---

Memorize this visual:

LR(0) | v SLR(1) | v LALR(1) | v CLR(1)

Power increases downward

Or mathematically:

```
\[
\boxed{
LR(0)
\subset
SLR(1)
\subset
LALR(1)
\subset
CLR(1)
}
\]
```

---

# 45. Reduction Precision Ladder

LR(0)

ALL terminals

!"

SLR

FOLLOW(LHS)

!"

LALR

Merged LR(1) lookaheads

!"

CLR

Exact state-specific LR(1) lookahead

Be careful: LALR vs CLR both use LR(1)-style lookaheads; CLR simply preserves more context by not merging same-core states.

---

## 46. State Count Ladder

LR(0) = SLR = LALR  $\neq$  CLR

This is one of the best quick GATE memories.

---

# 47. Table Size vs Power Trade-Off

| Parser | Relative state count | Relative power           |
|--------|----------------------|--------------------------|
| LR(0)  | Small                | Lowest of LR family here |
| SLR    | Small                | Higher                   |
| LALR   | Small                | Higher still             |
| CLR    | Largest              | Highest                  |

This is why LALR is often described as a compromise between:

Table size and parsing power

---

# 48. What If No Same-Core CLR States Exist?

Then no merging occurs.

Therefore:

LALR automaton = CLR automaton

for that grammar.

Number of states is the same in that special case.

---

# 49. Can LALR Ever Have More States Than CLR?

No.

It is created by merging CLR states.

Therefore:

$$\#LALR \leq \#CLR$$

---

# 50. Can LALR Have More States Than SLR?

Under standard construction:

No.

They have the same LR(0)-core state count.

\[  
\boxed{\backslash\#LALR=\backslash\#SLR}  
\]

Their **items/lookahead information** differ, but state count is the same.

---

#### # 51. Why Doesn't Merging Destroy Shift Transitions?

Same-core CLR states contain the same dotted productions.

Therefore they have the same grammar symbols immediately after dots.

So their shift/GOTO structure is compatible.

The key change after merging is in:

\[  
\boxed{\backslash\text{lookahead unions}}  
\]

and consequently reduction entries.

---

#### # 52. MINI PRACTICE – PART 15

Try first.

##### ### Q1 – Easy

LALR merges CLR states having:

- A. Same FOLLOW sets
- B. Same LR(0) core
- C. Same state number
- D. Same lookahead only

---

##### ### Q2 – Easy

When two same-core LR(1) states are merged, their corresponding lookaheads are:

- A. Intersected
- B. Discarded
- C. Unioned
- D. Replaced by FOLLOW

---

##### ### Q3 – Medium

If a grammar's LR(0) automaton has 10 states and CLR has 16 states, how many states does standard LALR have?

---

##### ### Q4 – Medium

Can a grammar be CLR(1) but not LALR(1)?

- A. Yes
- B. No

---

### ### Q5 – Tricky

Which new conflict can merging two individually conflict-free same-core CLR states introduce?

- A. Only shift/shift
- B. New reduce/reduce
- C. New shift/reduce only
- D. No conflict can ever be introduced

---

### # SOLUTIONS

#### ## Q1

' \* \* B \* '

\[  
\boxed{\text{Same LR(0) core}}  
\]

---

#### ## Q2

' \* \* C – U n i o n e d \* '

Example:

\[  
\{a\}+\{b\}  
\rightarrow  
\{a,b\}  
\]

---

#### ## Q3

\[  
\boxed{10}  
\]

because:

\[  
\#LALR=\#LR(0)  
\]

under standard canonical merging.

---

#### ## Q4

' \* \* A – Y e s \* '

CLR states can be individually conflict-free but their same-core merge can create an R/R conflict.

Thus:

\[  
\boxed{LALR\subset CLR}

\]

---

## Q5

' \* \* B — New reduce / reduce conflict \* '

Same-core merging can union previously separate reduction lookaheads and make two reductions overlap.

---

# 53. WRITE THIS IN NOTES — LALR MASTER SHEET

## Construction

```text id="h9xzu4"

Construct CLR(1) states

!"

Find states with same LR(0) core

!"

Merge them

!"

Union corresponding lookaheads

!"

Build ACTION/GOTO

Same Core

Ignore lookaheads.

If dotted-production sets are identical:

$\boxed{\text{same core}}$

Merge

$[A \rightarrow \alpha \cdot a]$

and:

$[A \rightarrow \alpha \cdot b]$

become:

$\boxed{[A \rightarrow \alpha \cdot, \{a, b\}]}$

State Count

$\boxed{\#LR(0) = \#SLR = \#LALR \leq \#CLR}$

Power

$\boxed{LR(0) \subset SLR(1) \subset LALR(1) \subset CLR(1)}$

Main Risk

Merging same-core CLR states can introduce:

$\boxed{\text{Reduce/Reduce conflict}}$

Therefore:

$\boxed{\text{CLR}(1) \not\rightarrow \text{LALR}(1)}$

but:

$\boxed{\text{LALR}(1) \rightarrow \text{CLR}(1)}$

30-SECOND REVISION

LALR

CLR with same-core states merged.

How?

$\boxed{\text{Same core ! Union lookaheads}}$

Core

Remove lookahead from LR(1) items.

Reduction information

LR(0) ! all terminals SLR ! FOLLOW LALR ! merged LR(1) lookaheads CLR ! exact LR(1) lookaheads

Number of states

$\boxed{\text{LR}(0) = \text{SLR} = \text{LALR} \leq \text{CLR}}$

Parsing power

$\boxed{\text{LR}(0) < \text{SLR} < \text{LALR} < \text{CLR}}$

Big trap

> CLR state merging can create **R/R conflict**, so a CLR grammar need not be LALR.

We have now completed the individual parsing families. The next part is extremely important for GATE because questions often give a grammar and ask **which parser classes can parse it**, or ask whether statements such as “every SLR grammar is LALR” are true.

Completed: Parts 1-15/31 | Next: Part 16 – Complete LL/LR Parser Hierarchy, Grammar-Class Relationships & GATE Classification Shortcuts

Reply ****NEXT**** for ****Part 16****.

USER

NEXT

ASSISTANT

We've reached the part where small hierarchy mistakes can cost an entire GATE MCQ. I'll separate **guaranteed inclusions** from relationships you should *not* infer, then give fast grammar-classification rules.

ASSISTANT

Compiler Design — Part 16: Complete LL/LR Parser Hierarchy & GATE Classification Shortcuts

Completed: Parts 1–15/31 | Current: Part 16 | Remaining: Parts 17–31

We have now individually studied:

- LL(1)
- LR(0)
- SLR(1)
- LALR(1)
- CLR(1)

Now we connect everything.

This part is extremely important because GATE often asks questions like:

If a grammar is SLR(1), what else must it be?

Can an ambiguous grammar be LR(1)?

Is every LL(1) grammar LALR(1)?

A grammar has an LR(0) conflict but no SLR conflict. What is its smallest LR class?

There is one major subtlety:

$\boxed{\text{Grammar-class relationships}} \neq \boxed{\text{Language-class relationships}}$

For GATE grammar questions, we must be precise.

1. First Lock the LR Hierarchy

The standard LR grammar hierarchy is:

$\boxed{LR(0) \subset SLR(1) \subset LALR(1) \subset CLR(1)}$

where:

$\boxed{CLR(1) = \text{Canonical LR}(1)}$

and usually we simply call it:

$\boxed{LR(1)}$

The containments are proper: each successive method can handle some grammars the preceding one cannot.

WRITE THIS IN NOTES

$\boxed{LR(0) < SLR(1) < LALR(1) < CLR(1)}$

Memory Trick

0 ! ' S ! ' LA ! ' C

TEXT

LR(0)

! "

SLR

! "

LALR

! "

CLR

Parsing power increases downward.

2. Why Does Power Increase?

Recall their reduction rules.

LR(0)

Completed item:

$A \rightarrow \alpha \cdot$

reduces on:

$\boxed{\text{ALL terminals}}$

Very crude.

SLR

Reduces on:

$\boxed{\text{FOLLOW}(A)}$

More selective.

LALR

Uses merged LR(1) lookahead information.

Even more selective.

CLR

Uses exact state-specific LR(1) lookahead.

Most precise.

Master Precision Ladder

```
TEXT
LR(0)
ALL terminals
    ! "
SLR
FOLLOW(A)
    ! "
LALR
Merged LR(1) lookaheads
    ! "
CLR
Exact LR(1) lookaheads
```

Greater precision generally means fewer unnecessary conflicts.

3. LR(0) $\Rightarrow$ SLR

Suppose a grammar is LR(0).

That means:

Even after putting every reduction under every terminal, there is no conflict.

SLR only **removes unnecessary reductions** by limiting them to FOLLOW sets.

Removing entries cannot create a new conflict.

Therefore:

$\boxed{LR(0)} \Rightarrow SLR(1)$

4. But SLR $\nRightarrow$ LR(0)

A grammar may have an LR(0) conflict:

```
TEXT
shift x
vs
reduce A' ± on x
```

but:

$x \notin \text{FOLLOW}(A)$

SLR removes the reduction on x .

Conflict disappears.

Hence:

$\boxed{\text{SLR}(1) \not\rightarrow \text{LR}(0)}$

5. SLR $\nRightarrow$ LALR

SLR uses global:

$\text{FOLLOW}(A)$

for reductions.

LALR uses more precise LR(1)-derived lookaheads.

Therefore anything already conflict-free under the less precise SLR method remains parsable under LALR.

$\boxed{\text{SLR}(1) \rightarrow \text{LALR}(1)}$

6. LALR $\nRightarrow$ SLR

Suppose:

$\text{FOLLOW}(A) = \{a, b, c\}$

SLR might reduce on all three and conflict on b .

But the relevant LALR state may have only:

$\{a, c\}$

as A's lookaheads.

No reduction on b .

Therefore:

$\boxed{\text{LALR}(1) \not\rightarrow \text{SLR}(1)}$

7. LALR $\Rightarrow$ CLR

LALR is produced by merging same-core CLR states.

If the merged LALR states remain conflict-free, then the original unmerged CLR states are also conflict-free.

Hence:

$\boxed{\text{LALR}(1) \rightarrow \text{CLR}(1)}$

8. CLR $\nRightarrow$ LALR

Part 15 showed why.

Two conflict-free CLR states may have the same LR(0) core.

After merging them, their lookahead unions can create:

$\boxed{\text{Reduce/Reduce conflict}}$

Hence:

$\boxed{\text{CLR}(1) \not\rightarrow \text{LALR}(1)}$

9. The Complete LR Implication Chain

If grammar is LR(0):

Then it is automatically:

$\boxed{\text{LR}(0), \text{SLR}(1), \text{LALR}(1), \text{CLR}(1)}$

If grammar is SLR:

Guaranteed:

$\boxed{\text{SLR}(1), \text{LALR}(1), \text{CLR}(1)}$

but not necessarily LR(0).

If grammar is LALR:

Guaranteed:

$\boxed{\text{LALR}(1), \text{CLR}(1)}$

but not necessarily SLR/LR(0).

If grammar is only CLR:

Nothing below CLR is guaranteed.

10. Reverse-Direction GATE Shortcut

Think of the hierarchy:

TEXT

LR(0) ! SLR ! LALR ! CLR

If a grammar belongs to some class:

Everything to its right is guaranteed.

Nothing to its left is guaranteed.

Example:

TEXT

SLR

! "

LALR

! "

SLR implies LALR and CLR.

11. Now Add LL(1)

This is where students often memorize a wrong single chain.

It is tempting to write:

TEXT

L L (1) ! ' L R (0) ! ' S L R ! ' L A L R ! ' C L R

'L Do not memorize this.

LL(1) is a different **top-down** grammar family.

The guaranteed relationship is:

$\boxed{LL(1) \subset CLR(1)}$

Every LL(1) grammar is LR(1), but not every LR(1) grammar is LL(1).

WRITE THIS IN NOTES

$\boxed{LL(1) \rightarrow CLR(1)}$

but:

$\boxed{CLR(1) \not\rightarrow LL(1)}$

12. Very Important: LL(1) vs SLR/LALR

At the **grammar level**, there is no simple containment relationship between LL(1) and SLR(1) or LALR(1).

There are LL(1) grammars that are not SLR(1), and even LL(1) grammars that are not LALR(1). Conversely, left-recursive LALR/SLR grammars give easy examples of LR-style grammars that are not LL(1).

So for GATE grammar questions:

$\boxed{LL(1) \not\rightarrow SLR(1)}$

in general.

Also:

$\boxed{LL(1) \not\rightarrow LALR(1)}$

in general.

And:

$\boxed{SLR/LALR \not\rightarrow LL(1)}$

13. LL(1) vs LR(0)

They are also not one simple containment chain.

A left-recursive LR grammar cannot be LL(1), while many LL(1) grammars are not LR(0).

Therefore do not infer:

$\nexists [LL(1) \rightarrow LR(0)]$

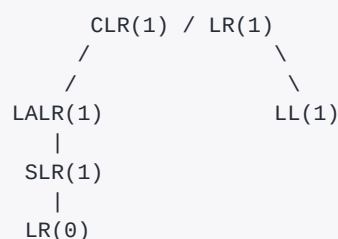
or:

$\nexists [LR(0) \rightarrow LL(1)]$

14. Best Hierarchy Picture for GATE

Draw this:

TEXT



But remember:

This picture only shows the guaranteed inclusion of LL(1) inside LR(1); it does not claim LL(1) is disjoint from LALR/SLR.

Some LL(1) grammars may also be SLR/LALR.

The classes overlap.

15. Grammar Classes vs Language Classes — Important Caveat

When we say:

"This grammar is LL(1)"

we are talking about the **particular grammar**.

A language may have:

- one grammar that is not LL(1)
- another equivalent grammar that is LL(1)

Similarly, transformations can change which parsing class the **grammar** belongs to without changing its language.

This distinction is why parser-class diagrams can become confusing.

For GATE

Unless explicitly asking about language classes, treat statements like:

as properties of the **given grammar**.

16. All Deterministic Parser Grammars Here Are Unambiguous

If a grammar has a conflict-free:

- LL(1) parser
- LR(0) parser
- SLR parser
- LALR parser
- CLR parser

then that grammar is:

$\boxed{\text{Unambiguous}}$

Therefore:

$\text{LL(1)} \rightarrow \text{unambiguous}$

$\text{LR(0)} \rightarrow \text{unambiguous}$

$\text{SLR} \rightarrow \text{unambiguous}$

$\text{LALR} \rightarrow \text{unambiguous}$

$\text{CLR} \rightarrow \text{unambiguous}$

17. Huge GATE Shortcut — Ambiguous Grammar

Suppose a grammar is proven ambiguous.

Then can it be LL(1)?

'LNo.

LR(0)?

'LNo.

SLR?

'LNo.

LALR?

'LNo.

CLR/LR(1)?

'LNo.

Therefore:

$\boxed{\text{Ambiguous} \rightarrow \text{none of these deterministic grammar classes}}$

18. Converse Is False

If a grammar is not LL(1), does that mean it is ambiguous?

'LNo.

It might simply need a more powerful parser.

Example:

$E \rightarrow E+T \mid T$

can be unambiguous yet left recursive.

It may not be directly LL(1), while LR parsing can handle it.

Thus:

$\boxed{\text{Not LL(1)} \not\rightarrow \text{Ambiguous}}$

19. Left Recursion and LL(1)

An LL(1) grammar cannot be left recursive.

Therefore:

$\boxed{\text{Left recursive grammar} \rightarrow \text{Not LL(1)}}$

This is an extremely fast rejection test.

20. But Left Recursion Is Fine for LR

Do not continue the inference.

From:

Left recursive

you cannot conclude:

TEXT

Not LR(0)

Not SLR

Not LALR

Not CLR

LR parsers can naturally handle many left-recursive grammars.

Therefore:

$\boxed{\text{Left recursion only immediately eliminates LL-style parsing}}$

21. Left Factoring and LL(1)

If alternatives share prefixes such as:

$A \rightarrow ab \mid ac$

the given grammar has a FIRST/FIRST conflict and is not LL(1).

Left factoring may transform it:

$A \rightarrow aA'$

$A' \rightarrow b \mid c$

But remember:

$\boxed{\text{Left factoring does not guarantee LL(1)}}$

FIRST/FOLLOW conflicts may remain.

22. LL(1) Recognition Checklist

When asked whether a grammar is LL(1):

Step 1

Left recursion?

If yes:

$\boxed{\text{Not LL(1)}}$

Step 2

Calculate FIRST for alternatives.

Overlap?

$\boxed{\text{FIRST/FIRST conflict ! Not LL(1)}}$

Step 3

If μ alternative exists:

Check FIRST/FOLLOW.

Overlap?

$\boxed{\text{Not LL(1)}}$

Step 4

If no conflict:

$\boxed{\text{LL(1)}}$

23. LR(0) Recognition Checklist

Build LR(0) canonical item sets.

Look for:

S/R

State contains:

$A \rightarrow \alpha \cdot$

plus another item with terminal after dot.

Because completed item reduces everywhere:

$\boxed{S/R}$

R/R

Two completed ordinary items:

$\boxed{R/R}$

If no conflict:

$\boxed{LR(0)}$

And immediately conclude:

$\{SLR, LALR, CLR\}$

too.

24. SLR Recognition Checklist

Use the same LR(0) states.

For completed:

$A \rightarrow \alpha \cdot$

reduce only on:

$\{FOLLOW(A)\}$

S/R check

If shift on x , ask:

$x \in FOLLOW(A)?$

Yes! ' conflict.

R/R check

Two completed A/B items:

$\{FOLLOW(A) \cap FOLLOW(B)\}$

Nonempty ! ' conflict.

No conflicts:

$\boxed{SLR(1)}$

25. LALR Recognition Checklist

Conceptually:

1. Construct CLR states.
2. Find states with same LR(0) core.
3. Merge.
4. Union lookaheads.
5. Check conflicts.

If merging creates R/R conflict:

$\boxed{\text{Not LALR}}$

even though the original CLR states may be conflict-free.

26. CLR Recognition Checklist

Construct LR(1) items:

$[A \rightarrow \alpha \cdot \beta, a]$

using:

$\boxed{\text{FIRST}(\beta a)}$

for closure lookaheads.

Completed item:

$[A \rightarrow \alpha \cdot \beta, a]$

reduces only on:

a

If no ACTION conflicts:

$\boxed{\text{CLR}(1)}$

27. The "Smallest LR Class" Question

This is a common GATE pattern.

Suppose grammar:

- not LR(0)
- SLR conflict-free

Then smallest LR class among the four is:

$\boxed{\text{SLR}(1)}$

Suppose:

- LR(0): conflict
- SLR: conflict
- LALR: no conflict

Then:

$\boxed{\text{LALR}(1)}$

Suppose:

- LR(0): conflict
- SLR: conflict
- LALR: conflict
- CLR: no conflict

Then:

$\boxed{\text{CLR}(1)}$

28. Fast Decision Ladder

Use:

```
TEXT
LR(0) conflict?
    !"    n o
LR(0) – STOP

    !"    y e s
SLR conflict?
    !"    n o
SLR – STOP

    !"    y e s
LALR conflict?
    !"    n o
LALR – STOP

    !"    y e s
CLR conflict?
    !"    n o
CLR – STOP

    !"    y e s
Not LR(1)
```

This is an excellent exam strategy.

29. Important Logic Direction

Suppose grammar is **not SLR**.

Can you say it is not LALR?

'LNo.

It may still be:

$\boxed{\text{LALR}}$

or:

$\boxed{\text{CLR}}$

because those are more powerful.

30. Suppose Grammar Is Not CLR

Can it still be LALR?

'LNo.

Because:

$\boxed{\text{LALR} \subset \text{CLR}}$

Therefore:

$\boxed{\text{Not } \text{CLR} \rightarrow \text{Not } \text{LALR}, \text{Not } \text{SLR}, \text{Not } \text{LR}(0)}$

Also, because:

$\text{LL}(1) \subset \text{CLR}(1)$

if the **same grammar** is not LR(1)/CLR(1), it cannot be LL(1) either.

31. Suppose Grammar Is Not LR(0)

What can we conclude?

Almost nothing about higher classes.

It may still be:

- SLR
- LALR
- CLR

Therefore:

$\boxed{\text{Not } \text{LR}(0) \rightarrow \text{Not } \text{SLR}}$

32. Suppose Grammar Is CLR

Can we conclude SLR?

'LNo.

CLR is more powerful.

It could be:

TEXT

CLR only

among the LR methods.

33. Suppose Grammar Is SLR

Then definitely:

$\boxed{\text{LALR}}$

and:

$\boxed{\text{CLR}}$

But not necessarily:

$\boxed{\text{LR}(0)}$

34. Suppose Grammar Is LALR

Then definitely:

$\boxed{\text{CLR}}$

but not necessarily:

- SLR
- LR(0)
- LL(1)

35. Suppose Grammar Is LL(1)

Guaranteed:

$\boxed{\text{CLR}(1)}$

and:

$\boxed{\text{Unambiguous}}$

and:

$\boxed{\text{Not left recursive}}$

But do **not** automatically mark:

- LR(0)
- SLR
- LALR

for the same grammar.

36. LL vs LR Fundamental Difference

LL parser

tries to answer:

Which production should I choose now?

It predicts the production before seeing the whole RHS.

LR parser

tries to answer:

Which production have I just completed?

It waits until enough input has been recognized to identify a handle.

Memory Trick

$\boxed{\text{LL predicts}}$

$\boxed{\text{LR recognizes}}$

This explains much of LR's greater power.

37. Derivation Difference

LL

$\boxed{\text{Leftmost derivation}}$

LR

$\boxed{\text{Rightmost derivation in reverse}}$

Both normally scan actual input:

$\boxed{\text{Left to right}}$

38. Stack / Parsing Style

| LL(1) | LR |
|----------------------------|------------------------------|
| Top-down | Bottom-up |
| R o o t ! ' l e a v e s | L e a v e s ! ' r o o t |
| Predict production | Recognize handle |
| Leftmost derivation | Reverse rightmost derivation |
| FIRST/FOLLOW parsing table | ACTION/GOTO |
| Rows = non-terminals | Rows = states |
| Left recursion problematic | Left recursion allowed |
| Simpler | More powerful |

39. LL(1) Table vs LR Table

LL(1)

Rows:

$\boxed{\text{Non-terminals}}$

Columns:

$\boxed{\text{Terminals}}$

Entries:

$\boxed{\text{Productions}}$

LR

Rows:

$\boxed{\text{States}}$

ACTION columns:

$\boxed{\text{Terminals} + \$}$

GOTO columns:

$\boxed{\text{Non-terminals}}$

Entries:

TEXT

shift
reduce
accept
goto state
error

40. Number-of-States Comparison

For the same grammar under standard canonical constructions:

$\boxed{\#LR(0) = \#SLR = \#LALR}$

and:

$\boxed{\#CLR \geq \#LALR}$

because:

- LR(0) and SLR use the same LR(0) states
- LALR merges CLR states by LR(0) core
- CLR keeps context-separated LR(1) states

41. SLR vs LALR — Don't Mix Them

SLR

States:

$\boxed{LR(0)}$

Reduction:

$\boxed{FOLLOW(LHS)}$

LALR

States after merging:

same core count as LR(0)

but reductions use:

$\boxed{\text{merged LR(1) lookaheads}}$

Therefore:

$\boxed{\text{SLR and LALR can have the same number of states but different parsing power}}$

42. LALR vs CLR

CLR

Keeps same-core states separate.

LALR

Merges them.

Therefore:

TEXT

```
C L R ! '   m o r e   p r e c i s e
L A L R ! '   s m a l l e r   t a b l e
```

Possible result of merging:

$\boxed{R/R \text{ conflict}}$

43. Which Parser Uses Which Items?

LL(1)

No LR items.

Uses:

- FIRST
- FOLLOW
- predictive table

LR(0)

$\boxed{\text{LR}(0) \text{ items}}$

SLR

$\boxed{\text{LR}(0) \text{ items}}$

LALR

Conceptually derived from:

$\boxed{\text{LR}(1) \text{ items}}$

then same-core states merged.

CLR

$\boxed{\text{LR}(1) \text{ items}}$

44. Which Parser Uses FOLLOW for Reductions?

Only among these LR variants:

$\boxed{\text{SLR}}$

This is a classic MCQ.

TEXT

```
L R ( 0 ) ! '   a l l   t e r m i n a l s
S L R ! '   F O L L O W
L A L R ! '   m e r g e d   l o o k a h e a d s
C L R ! '   e x a c t   l o o k a h e a d
```

45. GATE-STYLE QUESTION 1

If grammar (G) is LR(0), which must be true?

A. G is SLR but not LALR B. G is SLR, LALR and CLR C. G must be LL(1) D. G is ambiguous

Solution

Hierarchy:

$LR(0) \subset SLR \subset LALR \subset CLR$

Therefore:

$\boxed{B}$

46. GATE-STYLE QUESTION 2

If a grammar is LALR(1), which is guaranteed?

A. LR(0) B. SLR(1) C. CLR(1) D. LL(1)

Answer

$\boxed{C}$

Because:

$LALR(1) \subset CLR(1)$

47. GATE-STYLE QUESTION 3

If a grammar is ambiguous, which could it be?

A. LL(1) B. LR(0) C. SLR(1) D. None of these

Answer

$\boxed{D}$

Deterministic grammar classes here are unambiguous.

48. GATE-STYLE QUESTION 4

A grammar is left recursive.

Which conclusion is immediately valid?

A. It cannot be LR(1). B. It cannot be LL(1). C. It must be ambiguous. D. It must be LR(0).

Answer

$\boxed{B}$

Left recursion prevents LL(1), but LR methods may handle it.

49. GATE-STYLE QUESTION 5

A grammar is SLR(1) but not LR(0).

Which are definitely true?

1. It is LALR(1).
2. It is CLR(1).
3. It is LL(1).
4. It is unambiguous.

Correct:

$\boxed{1,2,4}$

LL(1) is not guaranteed.

50. GATE-STYLE QUESTION 6

A grammar is CLR(1) but LALR construction develops a conflict.

Possible?

$\boxed{\text{Yes}}$

Same-core CLR states may be individually conflict-free, but merging can create:

$\boxed{R/R}$

51. GATE-STYLE QUESTION 7

Grammar's LR(0) table has a shift/reduce conflict on **a**.

Completed item:

$A \rightarrow \alpha \cdot$

and:

$a \notin \text{FOLLOW}(A)$

Could SLR remove this conflict?

$\boxed{\text{Yes}}$

because SLR won't put the A-reduction in column **a**.

52. GATE-STYLE QUESTION 8

SLR table has a shift/reduce conflict.

Can CLR still be conflict-free?

$\boxed{\text{Yes}}$

CLR uses more precise state-specific lookaheads.

53. GATE-STYLE QUESTION 9

A grammar is not CLR(1).

Could the same grammar be LL(1)?

$\boxed{\text{No}}$

because:

$\text{LL}(1) \subset \text{CLR}(1)$

54. GATE-STYLE QUESTION 10

A grammar is not LL(1).

Could it still be CLR(1)?

$\boxed{\text{Absolutely yes}}$

Many useful LR grammars are not LL(1).

For example, left recursion alone rules out LL(1) but need not rule out LR parsing.

55. TRUE/FALSE RAPID FIRE

Statement 1

Every LR(0) grammar is SLR(1).

' TRUE

Statement 2

Every SLR(1) grammar is LR(0).

' FALSE

Statement 3

Every SLR(1) grammar is LALR(1).

' TRUE

Statement 4

Every LALR(1) grammar is CLR(1).

' TRUE

Statement 5

Every CLR(1) grammar is LALR(1).

'L FALSE

Statement 6

Every LL(1) grammar is CLR(1).

' TRUE

Statement 7

Every CLR(1) grammar is LL(1).

'L FALSE

Statement 8

Every LL(1) grammar is necessarily LALR(1).

'L FALSE at the grammar level.

Statement 9

Every LALR grammar is unambiguous.

' TRUE

Statement 10

An LR grammar may be left recursive.

' TRUE

56. Most Important GATE Classification Shortcut

If given a grammar, use this order:

First ask: Is it ambiguous?

If definitely yes:

$\boxed{\text{Reject all LL/LR deterministic classes}}$

Then LL(1), if asked

Check:

TEXT

Left recursion
FIRST/FIRST
FIRST/FOLLOW

For LR classes

Test from weakest to strongest:

TEXT

LR(0)
! "
SLR
! "
LALR
! "
CLR

Stop at first conflict-free class.

57. Conflict Information Tells You the Class

Suppose:

LR(0): conflict

Does not settle anything.

Check SLR.

SLR: no conflict

Immediately:

$\boxed{\text{SLR, LALR, CLR}}$

SLR: conflict

Could still be:

$\boxed{\text{LALR}}$

or:

$\boxed{\text{CLR}}$

LALR: conflict

Could still be:

$\boxed{\text{CLR}}$

CLR: conflict

Grammar is not any of the standard LR classes:

[LR(0),SLR,LALR,CLR]

and cannot be LL(1) either.

58. Master Grammar-Class Table

| If grammar is... | Must also be... |
|------------------|--------------------------------------|
| LR(0) | SLR, LALR, CLR |
| SLR(1) | LALR, CLR |
| LALR(1) | CLR |
| CLR(1) | Unambiguous |
| LL(1) | CLR, unambiguous, non-left-recursive |
| Ambiguous | None of LL(1)/LR(0)/SLR/LALR/CLR |

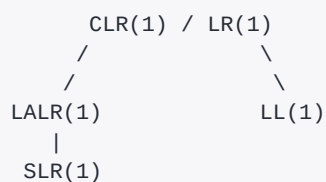
59. Parser Feature Master Table

| Feature | LL(1) | LR(0) | SLR | LALR | CLR |
|----------------------------------|----------|-------------------|-------------------|-------------------|-------------------|
| Style | Top-down | Bottom-up | Bottom-up | Bottom-up | Bottom-up |
| Input | L! R | L! R | L! R | L! R | L! R |
| Derivation | Leftmost | Reverse rightmost | Reverse rightmost | Reverse rightmost | Reverse rightmost |
| LR items | — | LR(0) | LR(0) | LR(1), merged | LR(1) |
| Reduction info | — | All terminals | FOLLOW | Merged lookahead | Exact lookahead |
| Left recursion directly allowed? | No | Yes | Yes | Yes | Yes |
| Relative LR power | — | Lowest | ! | ! | Highest |

60. One Diagram to Remember Everything

WRITE THIS IN NOTES

TEXT



$$\begin{array}{c} | \\ LR(0) \end{array}$$

And separately memorize:

$$\boxed{LR(0) \subset SLR \subset LALR \subset CLR}$$

and:

$$\boxed{LL(1) \subset CLR(1)}$$

Do **not** insert $LL(1)$ into the LR chain.

MINI PRACTICE — PART 16

Try without looking above.

Q1

If G is $LR(0)$, which must be true?

A. G is $LL(1)$ B. G is $SLR(1)$ C. G is ambiguous D. G is not $CLR(1)$

Q2

If G is $LALR(1)$, which must be true?

A. G is $SLR(1)$ B. G is $LR(0)$ C. G is $CLR(1)$ D. G is $LL(1)$

Q3

If G is left recursive, which is impossible?

A. $LR(0)$ B. SLR C. $LALR$ D. $LL(1)$

Q4

If G is ambiguous, could it be $CLR(1)$?

A. Yes B. No

Q5 — Tricky

If G is $LL(1)$, which statement is guaranteed?

A. G is $LR(0)$ B. G is $SLR(1)$ C. G is $LALR(1)$ D. G is $LR(1)/CLR(1)$

SOLUTIONS

Q1

' B

$$LR(0) \subset SLR(1)$$

Q2

' C

$\mathbb{L} \text{ LALR}(1) \subseteq \text{CLR}(1) \mathbb{L}$

Q3

' D — LL(1)

Left recursion is incompatible with LL(1).

It can be fine for LR methods.

Q4

' B — No

A conflict-free CLR grammar is unambiguous.

Q5

' D

$\mathbb{L} \text{ LL}(1) \subseteq \text{LR}(1) \mathbb{L}$

Do not automatically infer SLR or LALR for the **same grammar**.

CUMULATIVE PARSING REVISION — PARTS 5–16

We have now finished the complete main parsing block.

CFG

$\mathbb{L} \text{ } G = (N, T, P, S) \mathbb{L}$

Derivations

TEXT

L L ! ' l e f t m o s t

L R ! ' r i g h t m o s t i n r e v e r s e

Ambiguity

$\mathbb{L} \text{ } \boxed{\text{One string + two parse trees}} \mathbb{L}$

Grammar Preparation

TEXT

L e f t r e c u r s i o n ! ' r e m o v e f o r L L
L e f t f a c t o r i n g ! ' c o m m o n p r e f i x

FIRST

Who can come first?

FOLLOW

Who can come after?

LL(1)

TEXT

FIRST/FIRST conflict
FIRST/FOLLOW conflict

No conflicts !' LL(1).

Bottom-Up

TEXT

Shift
Reduce
Handle
Viable prefix

LR(0)

TEXT

Items
Closure
GOTO
Canonical states

Completed item:

$\boxed{\text{reduce everywhere}}$

SLR

$\boxed{\text{reduce on FOLLOW}}$

CLR

Item:

$[A \rightarrow \alpha \cdot \beta, a]$

Closure lookahead:

$\boxed{\text{FIRST}(\beta a)}$

Reduction:

$\boxed{\text{only on item lookahead}}$

LALR

TEXT

same-core CLR states
!"
merge
!"
union lookaheads

Possible newly introduced:

$\boxed{R/R \text{ conflict}}$

30-SECOND REVISION

LR hierarchy

$\boxed{LR(0) < SLR < LALR < CLR}$

LL relation

$\boxed{LL(1) < CLR(1)}$

Do not put $LL(1)$ directly into the $LR(0) < SLR < LALR < CLR$.

All are unambiguous

TEXT

$LL(1)$
 $LR(0)$
SLR
LALR
CLR

Ambiguous grammar

$\boxed{\text{None of them}}$

Left recursive

$\boxed{\text{Not } LL(1)}$

but may be LR.

Reduction memory

TEXT

```
L R ( 0 ) !' A L L
S L R      !' F O L L O W
L A L R    !' m e r g e d   l o o k a h e a d
C L R      !' e x a c t   l o o k a h e a d
```

State-count memory

$\boxed{LR(0)=SLR=LALR \mid e CLR}$

Final parsing mantra

LL predicts. LR recognizes.

We have now completed the **entire parsing family block**. Next we move to the next official syllabus topic:

Syntax-Directed Translation

We will learn how semantic information such as **types, values, addresses and generated code** is attached to grammar symbols.

Completed: Parts 1–16/31 | Next: Part 17 — Syntax-Directed Definitions, Attributes & Dependency Graphs

Reply **NEXT** for **Part 17**.

USER

next

ASSISTANT

Compiler Design — Part 17: Syntax-Directed Definitions, Attributes & Dependency Graphs

Completed: Parts 1–16/31 | Current: Part 17 | Remaining: Parts 18–31

We have completed lexical analysis and parsing.

Now parsing tells us:

"This token sequence is grammatically correct."

But a compiler needs more than grammar structure.

For example, for:

```
c
x = a + b * c;
```

we may need to compute:

- expression value
- variable type
- generated intermediate code
- storage location
- postfix form

To attach such information to grammar symbols, compiler design uses:

$\boxed{\text{Syntax-Directed Translation}}$

The foundation is the:

$\boxed{\text{Syntax-Directed Definition (SDD)}}$

1. What Is Syntax-Directed Translation?

CONCEPT

Syntax-directed translation associates semantic information/actions with grammar productions.

The grammar determines the structure, while semantic rules determine what information should be computed.

Example grammar:

$E \rightarrow E_1 + T$

We may associate:

$E.val = E_1.val + T.val$

Grammar says:

How an expression is formed.

Semantic rule says:

How its value is calculated.

WRITE THIS IN NOTES

Syntax-directed translation: A method in which semantic processing is guided by the syntactic structure of the program.

2. Syntax-Directed Definition — SDD

An **SDD** consists of:

$\boxed{\text{CFG} + \text{Attributes} + \text{Semantic Rules}}$

So:

```
TEXT
Context-Free Grammar
+
Attributes attached to grammar symbols
+
Semantic rules attached to productions
!"
Syntax-Directed Definition
```

Memory Trick

SDD

Syntax + Data + Dependency

More formally, remember:

$\boxed{\text{SDD} = \text{Grammar} + \text{Attributes} + \text{Semantic Rules}}$

3. What Is an Attribute?

An **attribute** is a value/information associated with a grammar symbol.

Examples:

```
TEXT
value
type
name
location
code
address
width
```

Suppose:

$E \rightarrow E_1 + T$

We may associate:

```
TEXT
E.val
E • . v a l
T.val
```

These are attributes.

4. Example

Consider:

$E \rightarrow E_1 + T$

Semantic rule:

$\boxed{E.val = E_1.val + T.val}$

If:

$E_1.val = 5$

and:

$T.val = 3$

then:

$E.val = 5 + 3 = 8$

The syntax tree tells us **what combines with what**.

Attributes carry the semantic information.

5. Two Main Types of Attributes

This distinction is one of the most important GATE concepts.

Attributes are mainly:

1. Synthesized attributes
 2. Inherited attributes
-

6. Synthesized Attribute

CONCEPT

A synthesized attribute of a node is calculated from attributes of its:

- children, and possibly
- the node itself / constants associated with its production.

Information conceptually travels:

$\boxed{\text{Children} \rightarrow \text{Parent}}$

Example

Production:

$E \rightarrow E_1 + T$

Semantic rule:

$E.val = E_1.val + T.val$

Here:

- $E.val$ comes from child
- $T.val$ comes from child
- result $E.val$ is computed for parent

Thus:

$\boxed{E.val \text{ is synthesized}}$

Memory Trick

Synthesized = Sent UP.

TEXT

Children

!'

Parent

Or:

S = Son/children supply parent.

WRITE THIS IN NOTES

Synthesized attribute: An attribute whose value is computed using attributes of the node's children.

Direction:

$\boxed{\uparrow}$

7. Synthesized Example — Expression Evaluation

Grammar:

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow \text{num}$

Semantic rules:

$E.val = E_1.val + T.val$

$E.val = T.val$

$T.val = \text{num.lexval}$

For input:

TEXT

3 + 5

The information moves upward.

TEXT

```
      E.val = 8
     /  |  \
  E.val=3 + T.val=5
    |      |
  T.val=3  num=5
    |
  num=3
```

Result:

$\boxed{8}$

8. Lexical Attribute

When lexer recognizes:

TEXT

25

it may return:

$\langle \text{num}, 25 \rangle$

The lexical value 25 can be considered an attribute such as:

$\text{num.lexval} = 25$

Then semantic rules can use it:

$\text{T.val} = \text{num.lexval}$

So the lexer can supply initial attribute values used later by syntax-directed translation.

9. Inherited Attribute

Now consider information flowing from context.

CONCEPT

An inherited attribute of a grammar symbol is typically computed using attributes of:

- its parent
- its siblings

rather than its children.

Information conceptually flows:

$\boxed{\text{Parent/Sibling} \rightarrow \text{Child}}$

or sideways through the parse tree.

Memory Trick

Inherited = received from family/context.

A child **inherits** information from its parent or siblings.

WRITE THIS IN NOTES

Inherited attribute: An attribute whose value is obtained from attributes of the parent and/or siblings of a node.

Typical direction:

$\boxed{\downarrow \text{ or sideways}}$

10. Why Do We Need Inherited Attributes?

Consider declaration:

```
c
int a, b, c;
```

The type:

```
TEXT
int
```

must be associated with:

```
TEXT
a
b
c
```

The identifier list does not determine the type by itself.

It receives the type from the declaration context.

Conceptually:

```
TEXT
Declaration
type = int
    |
Identifier List !"
  /  |  \
 a  b  c
int int int
```

This is a natural use of inherited attributes.

11. Inherited Attribute Example

Grammar:

$\backslash [D \rightarrow TL]$

where:

TEXT

T = type

L = identifier list

Suppose:

$\backslash [T \rightarrow \text{int}]$

Semantic rule:

$\backslash [T.\text{type} = \text{int}]$

For:

$\backslash [D \rightarrow TL]$

we can write:

$\backslash [\boxed{L.in = T.type}]$

Here `L.in` is inherited from sibling `T` through the parent production.

12. Continue Declaration Example

Suppose:

$\backslash [L \rightarrow L_1, id]$

We can pass the type:

$\backslash [L_1.in = L.in]$

and assign:

$\backslash [id.type = L.in]$

Conceptually:

TEXT

int a, b, c

causes:

TEXT

a.type = int

b.type = int

c.type = int

The type information moves across/down the structure.

13. Synthesized vs Inherited — Master Table

| Synthesized | Inherited |
|--------------------------------|---|
| Usually computed from children | Usually computed from parent/siblings |
| Information moves upward | Information moves downward/sideways |
| Natural for expression values | Natural for contextual information |
| Example: E.val | Example: declaration type passed to IDs |

Memory Trick

TEXT

SYNTHESIZED

C h i l d r e n ! ' P a r e n t
U P !'

INHERITED

P a r e n t / S i b l i n g ! ' C h i l d
D O W N / S I D E ! " !'

14. GATE Trap — Inherited Doesn't Mean Only Parent

An inherited attribute does **not** have to come only from the parent.

It can be computed from:

- parent's attributes
- attributes of suitable siblings

depending on the SDD.

So:

"Inherited attributes are always copied directly from the parent."

'L False.

15. Semantic Rule

A **semantic rule** specifies how an attribute value is calculated.

Example:

Production:

$E \rightarrow E_1 + T$

Semantic rule:

$\boxed{E.val = E_1.val + T.val}$

Production:

$D \rightarrow TL$

Semantic rule:

$\boxed{L.in = T.type}$

16. Semantic Rules vs Grammar Productions

Grammar production tells:

syntactic relationship.

Semantic rule tells:

how semantic information is computed.

Example:

TEXT

Production:

$E \rightarrow E_1 + T$

Semantic rule:

$E.val = E_1.val + T.val$

They are related but different.

17. Attribute Instances

Suppose parse tree contains several occurrences of E .

Each occurrence can have its own instance of:

$E.val$

Example:

TEXT

$2 + 3 + 4$

Different E nodes may have:

TEXT

$E \bullet .val = 2$

$E , .val = 5$

$E f .val = 9$

Therefore attributes belong to **particular occurrences/nodes**, not merely to the grammar symbol name globally.

18. Dependency Between Attributes

Consider:

$E \rightarrow E_1 + T$

with rule:

$$\vee [E.val = E_1.val + T.val \vee]$$

Before computing:

$$\vee [E.val \vee]$$

we need:

$$\vee [E_1.val \vee]$$

and:

$$\vee [T.val \vee]$$

So:

$$\vee [E.val \vee]$$

depends on those attributes.

This dependency is represented using a:

$$\vee [\boxed{\text{Dependency Graph}} \vee]$$

19. Dependency Graph

CONCEPT

A dependency graph represents dependencies among attribute instances in a particular parse tree.

Nodes

Attribute instances.

Directed edge

If attribute $\vee (b) \vee$ is needed to compute attribute $\vee (a) \vee$, draw:

$$\vee [\boxed{b \rightarrow a} \vee]$$

WRITE THIS IN NOTES

Dependency graph: Directed graph whose nodes are attribute instances and whose edges represent semantic dependencies.

If:

$$\vee [x = f(y, z) \vee]$$

draw:

$$\vee [y \rightarrow x \vee]$$
$$\vee [z \rightarrow x \vee]$$

Memory Trick

Dependency arrow goes from "needed first" to "computed later."

20. Dependency Graph Example

Production:

$\{ E \rightarrow E_1 + T \}$

Rule:

$\{ E.val = E_1.val + T.val \}$

Dependencies:

$\{ E_1.val \rightarrow E.val \}$

$\{ T.val \rightarrow E.val \}$

Diagram:

TEXT

```
E 1 . v a l % % % %  
          % % % ! '   E . v a l  
T . v a l % % % % %
```

21. More Complex Example

Suppose semantic rules:

$\{ A.x = B.y + C.z \}$

$\{ B.q = A.p \}$

Then dependencies include:

$\{ B.y \rightarrow A.x \}$

$\{ C.z \rightarrow A.x \}$

and:

$\{ A.p \rightarrow B.q \}$

The graph determines what must be evaluated first.

22. Attribute Evaluation Order

Suppose:

$\{ x \}$

depends on:

$\{ y \}$

Then:

y

must be calculated before:

x

Thus the dependency graph determines a legal evaluation order.

23. Topological Ordering

If the dependency graph is a DAG — directed acyclic graph — its nodes can be evaluated according to a:

$\boxed{\text{Topological Order}}$

That means every attribute is computed only after all attributes it depends upon have already been computed.

WRITE THIS IN NOTES

$\boxed{\text{Legal attribute evaluation order} = \text{Topological order of dependency graph}}$

24. Simple Topological Example

Dependencies:

TEXT

| | | |
|---|---|---|
| a | ! | c |
| b | ! | c |
| c | ! | d |

Possible evaluation:

TEXT

a, b, c, d

or:

TEXT

b, a, c, d

But not:

TEXT

c, a, b, d

because **c** depends on **a** and **b**.

25. Can There Be Multiple Valid Evaluation Orders?

' Yes.

If two attributes do not depend on one another, either may be evaluated first.

Example:

TEXT

| | | |
|---|---|---|
| a | ! | c |
| b | ! | c |

Both:

TEXT

a, b, c

and:

TEXT

b, a, c

are legal.

So:

$\boxed{\text{Evaluation order need not be unique}}$

26. What If the Dependency Graph Has a Cycle?

Suppose:

$a \rightarrow b$

and:

$b \rightarrow a$

Then:

TEXT

| | | |
|---|----------|---|
| a | requires | b |
| b | requires | a |

Neither can be evaluated first using straightforward dependency ordering.

There is no topological ordering.

For standard syntax-directed attribute evaluation, such cyclic dependencies are problematic.

GATE USE

If dependency graph contains a directed cycle:

$\boxed{\text{No topological evaluation order exists}}$

27. Example of a Circular Dependency

Suppose rules say:

$\forall [A.x=B.y \]$

$\forall [B.y=A.x+1 \]$

Then:

$\forall [B.y \rightarrow A.x \]$

and:

$\forall [A.x \rightarrow B.y \]$

forming:

TEXT

A . x ! ' B . y
! '
% % % % % % % %

This is circular.

28. Dependency Graph Is Parse-Tree Specific

The grammar contains general semantic rules.

But a dependency graph is constructed for a **particular parse tree / particular input instance**.

This distinction matters.

TEXT

SDD
! ' g e n e r a l s p e c i f i c a t i o n

Dependency graph
! ' d e p e n d e n c i e s f o r p a r t i c u l a r p a r s e - t r e e a t t r i b u t e i n s t

29. Annotated Parse Tree

A parse tree where attribute values are shown at nodes is called an:

$\forall [\boxed{\text{Annotated Parse Tree}}]$

Example:

Input:

TEXT

3 + 5

Annotated structure:

TEXT

E.val=8
/ | \
E.val=3 + T.val=5

```

      |           |
T.val=3      num.lexval=5
      |
num.lexval=3

```

WRITE THIS IN NOTES

Annotated parse tree: Parse tree showing attribute values associated with grammar-symbol occurrences.

30. Example — Expression Evaluation Fully

Grammar:

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow \text{num}$

Rules:

For:

$E \rightarrow E_1 + T$

$E.\text{val} = E_1.\text{val} + T.\text{val}$

For:

$E \rightarrow T$

$E.\text{val} = T.\text{val}$

For:

$T \rightarrow \text{num}$

$T.\text{val} = \text{num.lexval}$

Input:

$2 + 4$

Step 1

Lexer supplies:

$\text{num}_1.\text{lexval} = 2$

$\text{num}_2.\text{lexval} = 4$

Step 2

Compute:

$T_1.\text{val} = 2$

$T_2.\text{val} = 4$

Step 3

Compute first E:

$\{ E_1.val = T_1.val = 2 \}$

Step 4

Compute final E:

$\{ E.val = E_1.val + T_2.val \}$

$\{ = 2 + 4 \}$

$\{ \boxed{6} \}$

All attributes here are synthesized.

31. Syntax-Directed Translation Can Generate Strings Too

Attributes don't have to be numbers.

For example, we can generate postfix notation.

Production:

$\{ E \rightarrow E_1 T \}$

Rule:

$\{ E.post = E_1.post \parallel T.post \parallel "+" \}$

where $\parallel$ means concatenation.

For:

$\{ a + b \}$

we obtain:

$\{ \boxed{ab+} \}$

So attributes can represent:

- numerical value
- string
- type
- code
- memory address
- syntax representation

32. Example — Postfix Translation

Grammar:

$\{ E \rightarrow E_1 T \}$

$\{ E \rightarrow T \}$

$\{ T \rightarrow id \}$

Rules:

$\vee [E.post = E_1.post \vee T.post \vee "+"]$

$\vee [E.post = T.post]$

$\vee [T.post = id.lexeme]$

Input:

TEXT

a+b

gives:

$\vee [E_1.post = a]$

$\vee [T.post = b]$

Therefore:

$\vee [E.post = ab+]$

Again, information flows upward.

So `post` is synthesized.

33. Attributes for Type Checking

Suppose expression:

c

a + b

Attributes may contain:

TEXT

a.type

b.type

E.type

A semantic rule could conceptually check:

TEXT

if a.type and b.type are compatible:

 E.type = resulting type

else:

 type error

Thus syntax-directed definitions can be used during semantic analysis.

34. Attributes for Intermediate Code

Later we may use attributes such as:

TEXT

E.place
E.code

Example:

$\backslash [E \rightarrow E_1 + E_2] \backslash$

Semantic rules might create:

TEXT

```
E.place = newtemp()  
E.code  = E1.code || E2.code ||  
         "E.place = E1.place + E2.place"
```

This connects SDDs to intermediate-code generation.

35. Semantic Rules vs Semantic Actions

These terms are related but should be distinguished.

Semantic Rule

Describes the relationship between attributes.

Example:

$\backslash [E.val = E_1.val + T.val] \backslash$

Semantic Action

An executable operation attached to a production during translation.

Example conceptually:

TEXT

```
print("+")
```

or:

TEXT

```
emit(t1 = a + b)
```

We'll discuss SDT schemes more in Part 18.

36. SDD vs SDT — Preview

SDD — Syntax-Directed Definition

Specifies:

TEXT

```
attributes  
+  
semantic equations/rules
```

It is more **declarative**:

What should be computed?

SDT — Syntax-Directed Translation scheme

Embeds semantic actions directly within production bodies.

Example:

```
\[ E \rightarrow E_1 + T \{ \text{print}("+") \} \]
```

More operational:

When should an action execute?

We will cover this properly in Part 18.

37. Synthesized Attribute Recognition Shortcut

Given:

```
\[ A \rightarrow XYZ \]
```

If a rule looks like:

```
\[ A.val = f(X.val, Y.val, Z.val) \]
```

then `A.val` is:

```
\[ \boxed{\text{Synthesized}} \]
```

because children determine parent.

38. Inherited Attribute Recognition Shortcut

Given:

```
\[ A \rightarrow XYZ \]
```

If rule is:

```
\[ Y.in = f(A.in, X.val) \]
```

then `Y.in` is:

```
\[ \boxed{\text{Inherited}} \]
```

because information comes from:

- parent A
- left sibling X

39. GATE-STYLE Dependency Example

Production:

```
\[ A \rightarrow BC \]
```

Rules:

$\{ B.i = A.i \}$

$\{ C.i = B.s \}$

$\{ A.s = C.s \}$

Dependencies:

Rule 1

$\{ A.i \rightarrow B.i \}$

Rule 2

$\{ B.s \rightarrow C.i \}$

Rule 3

$\{ C.s \rightarrow A.s \}$

So dependency graph contains:

TEXT

A . i ! ' B . i

B . s ! ' C . i

C . s ! ' A . s

40. Which Are Synthesized and Inherited?

For:

$\{ A \rightarrow BC \}$

B.i

Computed from parent A:

$\{ B.i = A.i \}$

So:

$\{ \boxed{B.i \text{ inherited}} \}$

C.i

Computed from sibling B:

$\{ C.i = B.s \}$

So:

$\{ \boxed{C.i \text{ inherited}} \}$

A.s

Computed from child C:

$\{ A.s = C.s \}$

So:

$\boxed{\text{A.s}\{\text{synthesized}\}}$

41. Attribute Type Is Determined by Direction of Definition

Do not classify an attribute based on its name.

`val`, `type`, `in`, `s`, etc. are arbitrary names.

What matters is:

How is the attribute instance defined relative to the parse-tree node?

So `type` can sometimes be synthesized and elsewhere an inherited attribute depending on the SDD design.

42. GATE TRAPS

Trap 1

A synthesized attribute is inherited from the parent.

'L False.

Usually:

$\boxed{\text{Children ! ' Parent}}$

Trap 2

Inherited attributes can only use the parent.

'L False.

They may use parent and appropriate sibling information.

Trap 3

Dependency graph nodes are grammar non-terminals only.

'L False.

Nodes represent:

$\boxed{\text{attribute instances}}$

Trap 4

If $x=f(y)$, dependency edge is:

$x \rightarrow y$

'L Backwards.

Correct:

$\boxed{y \rightarrow x}$

because y must be known before x.

Trap 5

Dependency graph always has exactly one topological order.

'L False.

There may be several valid orders.

Trap 6

A cyclic dependency graph has a topological order.

'L False.

A DAG is required.

Trap 7

An SDD is only a CFG.

'L False.

$\boxed{\text{CFG} + \text{Attributes} + \text{Semantic Rules}}$

Trap 8

Attributes can only store numeric values.

'L False.

They can represent:

- values
 - types
 - strings
 - locations
 - code
 - addresses
 - etc.
-

Trap 9

Parse tree and dependency graph are the same graph.

'L False.

Parse tree represents grammatical structure.

Dependency graph represents attribute-computation dependencies.

Trap 10

Every attribute in an SDD must be synthesized.

'L False.

An SDD may use synthesized and inherited attributes.

43. GATE-STYLE EXAMPLE 1

Production:

$\text{E} \rightarrow \text{E}_1 + \text{T}$

Rule:

$\text{E.val} = \text{E}_1.\text{val} + \text{T.val}$

What type of attribute is `E.val` ?

$\boxed{\text{Synthesized}}$

because it is calculated from child attributes.

44. GATE-STYLE EXAMPLE 2

Production:

$\text{D} \rightarrow \text{TL}$

Rule:

$\text{L.type} = \text{T.type}$

If `L.type` receives contextual information from sibling `T`, it functions as:

$\boxed{\text{Inherited attribute}}$

of `L`.

45. GATE-STYLE EXAMPLE 3

Rule:

$\text{X.a} = f(\text{Y.b}, \text{Z.c})$

Which dependency edges exist?

$\boxed{\text{Y.b} \rightarrow \text{X.a}}$

$\boxed{\text{Z.c} \rightarrow \text{X.a}}$

46. GATE-STYLE EXAMPLE 4

Dependency graph:

TEXT

| | | | |
|---|---|---|---|
| A | ! | ' | C |
| B | ! | ' | C |
| C | ! | ' | D |

Which evaluation order is legal?

A. C, A, B, D B. A, C, B, D C. B, A, C, D D. D, A, B, C

Only:

$\boxed{B, A, C, D}$

among these is legal.

(A, B, C, D would also be legal if offered.)

47. GATE-STYLE EXAMPLE 5

Dependency graph:

TEXT

| | | | |
|---|---|---|---|
| A | ! | ' | B |
| B | ! | ' | C |
| C | ! | ' | A |

Can attributes be evaluated by a topological ordering?

$\boxed{\text{No}}$

because the graph contains a cycle.

48. MINI PRACTICE — PART 17

Try these first.

Q1 — Easy

An SDD consists of:

A. DFA + stack B. CFG + attributes + semantic rules C. Tokens only D. LR items only

Q2 — Easy

An attribute computed from children's attributes is usually:

A. Inherited B. Synthesized C. Lexical error D. Handle

Q3 — Medium

Given:

$A \rightarrow BC$

and:

$A.x = B.y + C.z$

what dependency edges are required?

Q4 — Medium

Given:

$\{ A \rightarrow BC \}$

and:

$\{ B.i = A.i \}$

is **B.i** synthesized or inherited?

Q5 — Tricky

If the dependency graph contains a directed cycle, which statement is correct?

A. A topological evaluation order definitely exists. B. No topological ordering exists. C. Every attribute becomes synthesized. D. FOLLOW sets solve the cycle.

SOLUTIONS

Q1

' B

$\{ \boxed{\text{SDD} = \text{CFG} + \text{Attributes} + \text{Semantic Rules}} \}$

Q2

' B — Synthesized

Memory:

Synthesized goes UP.

Q3

Since:

$\{ A.x = B.y + C.z \}$

A.x depends on B.y and C.z.

Therefore:

$\{ \boxed{B.y \rightarrow A.x} \}$

$\{ \boxed{C.z \rightarrow A.x} \}$

Q4

$\boxed{B.i=A.i}$

B receives information from its parent A.

Therefore:

$\boxed{\text{Inherited}}$

Q5

' B

A directed cycle prevents topological ordering.

WRITE THIS IN NOTES — PART 17 MASTER SHEET

Syntax-Directed Definition

$\boxed{SDD=CFG+Attributes+Semantic\ Rules}$

Synthesized Attribute

$\boxed{\text{Children} \rightarrow \text{Parent}}$

Example:

$E.val=E_1.val+T.val$

Memory:

Synthesized = sent UP.

Inherited Attribute

$\boxed{\text{Parent/Sibling} \rightarrow \text{Child}}$

Example:

$L.in=T.type$

Memory:

Inherited = received from context/family.

Dependency Graph

Node:

$\boxed{\text{Attribute instance}}$

If:

$\forall x=f(y,z)$

edges:

$\boxed{y \rightarrow x}$

$\boxed{z \rightarrow x}$

Evaluation Order

$\boxed{\text{Topological order of dependency graph}}$

provided the graph is acyclic.

Annotated Parse Tree

$\boxed{\text{Parse tree + attribute values}}$

30-SECOND REVISION

SDD

TEXT

Grammar

+

Attributes

+

Semantic rules

Two attribute types

TEXT

S y n t h e s i z e d ! ' U P ! '
I n h e r i t e d ! ' D O W N / S I D E ! " ! '

Synthesized

Parent value computed from children.

Inherited

Child receives information from parent/siblings.

Dependency graph

If:

$\forall x=f(y)$

draw:

$\boxed{y \rightarrow x}$

Evaluation

Compute prerequisites first.

Legal evaluation order:

$\boxed{\text{Topological ordering}}$

Cycle?

$\boxed{\text{No topological order}}$

The next part completes syntax-directed translation and contains several favorite GATE distinctions:

- **S-attributed SDD**
- **L-attributed SDD**
- how they relate to bottom-up/top-down evaluation
- exact restrictions on inherited attributes
- semantic actions
- **SDD vs SDT**
- postfix translation examples

Completed: Parts 1–17/31 | Next: Part 18 — S-Attributed, L-Attributed & Syntax-Directed Translation Schemes

Reply **NEXT** for **Part 18**.

USER

NEXT

ASSISTANT

Compiler Design — Part 18: S-Attributed, L-Attributed & Syntax-Directed Translation Schemes

Completed: Parts 1–17/31 | Current: Part 18 | Remaining: Parts 19–31

In Part 17, we learned:

- synthesized attributes
- inherited attributes
- dependency graphs
- evaluation order

Now we answer a very important question:

Can these attributes be evaluated conveniently during parsing?

Two special classes of SDDs make this possible:

$\boxed{\text{S-Attributed SDD}}$

and

$\boxed{\text{L-Attributed SDD}}$

Then we'll distinguish:

$\boxed{\text{SDD vs SDT}}$

1. Quick Recall: Two Attribute Types

Synthesized

Information moves:

$\boxed{\text{Child} \rightarrow \text{Parent}}$

Example:

$E \rightarrow E_1 + T$

$E.val = E_1.val + T.val$

Inherited

Information usually moves:

$\boxed{\text{Parent/Left Context} \rightarrow \text{Child}}$

Example:

$D \rightarrow TL$

$L.in = T.type$

2. What Is an S-Attributed SDD?

CONCEPT

An SDD is **S-attributed** if it uses **only synthesized attributes**.

That's the entire definition.

$\boxed{S\text{-attributed} \iff \text{all attributes are synthesized}}$

Memory Trick

S-attributed = Synthesized only.

Both begin with:

$\boxed{S}$

WRITE THIS IN NOTES

S-attributed SDD: An SDD containing only synthesized attributes.

Information flows:

```
```text id="dv4as5" Leaves !' !' !' Root
```

More accurately:

```
\[
\boxed{\text{Bottom-up through the parse tree}}
\]
```

---

### # 3. Example of S-Attributed SDD

Grammar:

```
\[
E \rightarrow E_1 + T
\]
```

```
\[
E \rightarrow T
\]
```

```
\[
T \rightarrow \text{num}
\]
```

Semantic rules:

```
\[
E.val = E_1.val + T.val
\]
```

```
\[
E.val = T.val
\]
```

```
\[
T.val = \text{num.lexval}
\]
```

Every computed nonlexical attribute is synthesized.

Therefore:

```
\[
\boxed{\text{S-attributed}}
\]
```

---

### # 4. Why Is S-Attributed Natural for Bottom-Up Parsing?

Bottom-up parsing recognizes children/RHS before reducing them to their parent/LHS.

Suppose:

```
\[
E \rightarrow E_1 + T
\]
```

Before reducing:

```
\[
E_1+T\rightarrow E
\]
```

the parser has already recognized:

```
- \ (E_1\)
- `+`
- \ (T\)
```

So their synthesized values are already available.

Then it can calculate:

```
\[
E.val=E_1.val+T.val
\]
```

during reduction.

Therefore S-attributed definitions are particularly convenient for:

```
\[
\boxed{\text{Bottom-up / LR parsing}}
\]
```

---

# WRITE THIS IN NOTES

```
\[
\boxed{
S\text{-attributed SDDs are naturally evaluated bottom-up}
}
\]
```

and are convenient with:

```
\[
\boxed{LR\ parsers}
\]
```

---

# 5. Evaluation at Reduction Time

Suppose parser stack effectively contains:

```
```text id="cct6ax"
E1.val=5   +   T.val=4
```

and production:

```
\[ E\rightarrow E_1+T \]
```

is reduced.

At this point compute:

```
\[ E.val=5+4=9 \]
```

Then push the new **E** with:

`\[ E.val=9 \]`

Thus semantic computation fits naturally with the bottom-up reduction.

6. Postorder Evaluation

In a parse tree, synthesized attributes can generally be evaluated using a **postorder traversal**:

1. evaluate children
2. evaluate parent

Conceptually:

````text id="qrfqcs" Left child !" Right child !" Parent`

Therefore:

```
\[
\boxed{\text{S-attributed !' postorder-friendly}}
\]
```

---

# 7. Are Lexical Attributes Allowed?

Yes.

Suppose terminal:

```
```text id="bkx9r4"
num
```

has:

`\[ num.lexval \]`

supplied by the lexer.

The non-terminal values can then be synthesized upward.

The statement "S-attributed uses only synthesized attributes" concerns how attributes in the syntax-directed definition depend/evaluate; lexical token attributes can naturally provide leaf information.

8. What Is an L-Attributed SDD?

Now the more general and slightly harder concept.

An **L-attributed SDD** allows:

- synthesized attributes
- inherited attributes

but inherited attributes have a particular restriction designed to permit evaluation from **left to right**.

9. General Production

Consider:

$A \rightarrow X_1 X_2 X_3 \dots X_n$

Suppose (X_i) has an inherited attribute.

In an L-attributed definition, that inherited attribute may depend only on:

1. inherited attributes of the parent (A) , and
2. attributes of symbols $(X_1, \dots, X_{i-1})$ to the **left** of (X_i)

It must not depend on attributes of symbols to its right that are not yet known.

WRITE THIS IN NOTES

For:

$A \rightarrow X_1 X_2 \dots X_n$

an inherited attribute of (X_i) in an L-attributed SDD may depend on:

$\boxed{\text{attributes of } A}$

and:

$\boxed{\text{attributes of } X_1, \dots, X_{i-1}}$

subject to the usual dependency restrictions.

Core memory:

Inherited information can come from Parent + Left siblings.

10. Why the Name L-Attributed?

Because attributes can be evaluated in a traversal that respects a:

$\boxed{\text{Left-to-right}}$

order through the RHS.

Memory:

L-attributed = inherited information moves using Left context.

11. Simple L-Attributed Example

Grammar:

$D \rightarrow TL$

Suppose:

$T.type$

is synthesized.

Semantic rule:

$\boxed{L.in = T.type}$

For symbol **L**, its inherited attribute uses the attribute of **T**, which is its **left sibling**.
This is allowed.
Therefore this can be L-attributed.

12. Declaration Example

Input:

```
```c id="67ar7w" float x, y;
```

Grammar:

```
\[
D\rightarrow TL
\]
```

```
\[
T\rightarrow float
\]
```

```
\[
L\rightarrow id,L_1
\]
```

Semantic rules might include:

```
\[
T.type=float
\]
```

```
\[
L.in=T.type
\]
```

```
\[
id.type=L.in
\]
```

```
\[
L_1.in=L.in
\]
```

Information flows:

```
```text id="i8ou6p"
T.type = float
      !"
      /  \
    L.in  L1.in
    /  \
id.type L1.in
```

This is consistent with left-to-right contextual propagation.

13. Allowed Dependency Example

Production:

```
\[ A\rightarrow XYZ \]
```

Consider inherited attribute:

$$\{ Z.i = f(A.i, X.s, Y.s) \}$$

This is allowed in an L-attributed SDD because $Z.i$ depends on:

- parent $A.i$
- left sibling X
- left sibling Y

All information comes from parent or left context.

14. Another Allowed Example

For:

$$\{ A \rightarrow XYZ \}$$

rule:

$$\{ Y.i = f(A.i, X.s) \}$$

Allowed:

- A = parent
- X = left sibling

Therefore:

$$\boxed{\text{L-attributed compatible}}$$

15. Typically Forbidden Right-Sibling Dependency

Consider:

$$\{ A \rightarrow XYZ \}$$

with:

$$\{ Y.i = Z.s \}$$

Here Y 's inherited attribute depends on Z , which is to its **right**.

When processing left-to-right, $Z.s$ may not yet be available.

So this violates the normal L-attributed restriction.

$$\boxed{\text{Not L-attributed in the standard sense}}$$

Memory Trick

For an inherited attribute:

Look left, not right.

16. Important Technical Detail

The full textbook definition has a subtle allowance concerning attributes of (X_i) itself, provided dependencies do not create cycles.

But for GATE-style recognition, the crucial rule is:

$\boxed{\text{Parent + left siblings are allowed; future/right sibling dependence is not}}$

This is the safest practical test.

17. S-Attributed Is a Subclass of L-Attributed

An S-attributed SDD has:

- no inherited attributes

Therefore it automatically satisfies L-attributed restrictions.

Hence:

$\boxed{S\text{-attributed} \subseteq L\text{-attributed}}$

In the usual sense, S-attributed definitions form a special case of L-attributed definitions.

WRITE THIS IN NOTES

$\boxed{S\text{-attributed} \not\Rightarrow L\text{-attributed}}$

but:

$\boxed{L\text{-attributed} \not\Rightarrow S\text{-attributed}}$

because an L-attributed SDD may contain inherited attributes.

18. Example Showing the Difference

Suppose:

$A \rightarrow BC$

rules:

$B.i = A.i$

$A.s = C.s$

Here:

- $B.i$ is inherited
- $A.s$ is synthesized

Could this be S-attributed?

'L No, because inherited attribute exists.

Could it be L-attributed?

' Potentially yes, because $B.i$ depends on parent A and obeys left-to-right rules.

19. L-Attributed Evaluation

For a production:

$A \rightarrow X_1 X_2 \dots X_n$

conceptually:

Step 1

Use information from A to compute inherited attributes of (X_1) .

Step 2

Process (X_1) .

Step 3

Use parent + (X_1) information to compute inherited attributes of (X_2) .

Step 4

Process (X_2) .

Continue left-to-right.

Step 5

Once child information is available, synthesize information back toward A.

20. Tree-Walk Intuition

L-attributed evaluation fits naturally with a depth-first traversal:

Parent information !
left child !
obtain its synthesized info !
next child !
... !
parent synthesized result

Hence information can move:

- downward from parent
- left-to-right among siblings
- upward through synthesized attributes

21. L-Attributed and Top-Down Parsing

L-attributed definitions are especially useful with:

$\boxed{\text{Top-down parsing}}$

because a top-down parser processes symbols roughly left-to-right and can pass inherited information as it descends.

But don't turn this into an absolute statement that "only top-down parsers can evaluate them."

The important association is:

$\text{S - a t t r i b u t e d ! ' n a t u r a l l y b o t t o m - u p}$

22. Master Comparison

Feature	S-Attributed	L-Attributed
Synthesized attrs	Yes	Yes
Inherited attrs	No	Yes, restricted
Main flow	Upward	Parent/left !' child + upward
Natural evaluation	Bottom-up/postorder	Depth-first left-to-right
Convenient parser	LR/bottom-up	LL/top-down style
More general?	Less	More

23. GATE Recognition Shortcut

Given an SDD:

Question 1

Are **all** attributes synthesized?

Yes:

$\boxed{S \text{ text{-}attributed}}$

and therefore also L-attributed.

Question 2

Inherited attributes exist?

Then not S-attributed.

Check each inherited dependency.

Does $(X_i.in)$ depend only on:

- parent information
- left siblings / already available context?

If yes:

$\boxed{L \text{ text{-}attributed}}$

If it depends on future/right sibling information:

$\boxed{\text{Not L-attributed}}$

24. Worked Classification Example 1

Production:

$\text{A} \rightarrow \text{BC}$

Rules:

$\text{A.s} = \text{B.s} + \text{C.s}$

Only **A.s** is being synthesized from children.

Therefore:

$\boxed{\text{S-attributed}}$

and consequently:

$\boxed{\text{L-attributed too}}$

25. Worked Classification Example 2

Production:

$\text{A} \rightarrow \text{BC}$

Rules:

$\text{B.i} = \text{A.i}$

$\text{C.i} = \text{B.s}$

$\text{A.s} = \text{C.s}$

Classification:

B.i

parent !' B:

Allowed inherited.

C.i

left sibling B !' C:

Allowed inherited.

A.s

child C !' parent:

Synthesized.

Thus:

$\boxed{\text{L-attributed}}$

but:

$\boxed{\text{Not S-attributed}}$

26. Worked Classification Example 3

Production:

$\text{A} \rightarrow \text{BC}$

Rule:

$\text{B.i} = \text{C.s}$

B depends on a **right sibling C**.

Therefore:

$\boxed{\text{Not L-attributed}}$

and since an inherited attribute is involved:

$\boxed{\text{Not S-attributed either}}$

27. Now: SDD vs SDT

This distinction is another important GATE concept.

We already know:

SDD

$\boxed{\text{Syntax-Directed Definition}}$

It attaches:

- attributes
- semantic rules

to grammar productions.

28. What Is SDT?

SDT stands for:

$\boxed{\text{Syntax-Directed Translation Scheme}}$

In an SDT, **semantic actions are explicitly inserted into the RHS of grammar productions**.

Example:

$E \rightarrow E_1 + T \quad \{\text{print}("+")\}$

The code/action:

```
``text id="zyvho6" { print("+") }
```

is part of the translation scheme.

WRITE THIS IN NOTES

> **SDT:** CFG in which executable semantic actions are embedded at specific positions within production bodies.

29. SDD vs SDT

SDD asks:

> ****What semantic values depend on what?****

Example:

```
\[
E.post=E_1.post\ ||\ T.post\ || "+"
\]
```

SDT asks more operationally:

> ****When should an action be executed during parsing?****

Example:

```
\[
E\rightarrow E_1+T\{print("+")\}
\]
```

Master Comparison

```
| SDD | SDT |
|---|---|
| Attributes + semantic rules | Embedded semantic actions |
| More declarative | More operational |
| Says what is computed | Shows when actions execute |
| Dependency graph useful | Action position matters |
| Example: `E.val=...` | Example: `{print("+")}` |
```

Memory Trick

> ****Definition = relationships.****

> ****Translation scheme = actions in sequence.****

30. Position of a Semantic Action Matters

Consider:

```
\[
A\rightarrow X\ \{action\}\ Y
\]
```

The semantic action executes after processing `X` but before processing `Y`, according to the parsing/evaluation scheme.

If instead:

```
\[
A\rightarrow XY\ \{action\}
\]
```

the action executes after both X and Y have been handled.

Thus in SDT:

```
\[
```

```
\boxed{\text{Action position has operational meaning}}
\]
```

31. Postfix Translation Using SDT

Suppose grammar:

```
\[
E\rightarrow E+T
\]
```

```
\[
E\rightarrow T
\]
```

```
\[
T\rightarrow \text{num}
\]
```

We want infix:

```
\[
3+5
\]
```

to become postfix:

```
\[
35+
\]
```

An SDT can conceptually use:

```
\[
E\rightarrow E+T\{\text{print}("+")\}
\]
```

```
\[
T\rightarrow \text{num}\{\text{print}(\text{num.lexeme})\}
\]
```

32. Why Does It Produce Postfix?

Input:

```
```\text id="vnwhos"
3 + 5
```

## Process first number

Print:

```
```\text id="9ky9pr" 3
```

```
### Process second number
```

Print:

```
```\text id="nkujh6"
```

### After both operands

Execute:

```
```text id="qeasdb" print("+")
```

Output:

```
```text id="31usj0"
35+
```

Exactly postfix.

## 33. Postfix Memory Rule

For binary operator:

$\backslash [ a+b \backslash ]$

postfix is:

$\backslash [ \boxed{ab+} \backslash ]$

So semantic action for operator should execute:

$\backslash [ \boxed{\text{after both operands}} \backslash ]$

This is why the action appears at the end of that production.

## 34. Prefix Translation Intuition

For prefix:

$\backslash [ +a b \backslash ]$

the operator would need to be output **before** its operands.

So action placement would differ.

Again:

**SDT action position determines translation timing.**

## 35. Semantic Actions Can Do More Than Print

An action may:

- insert into symbol table
- check types
- allocate storage
- generate temporaries

- emit intermediate code
- calculate values
- update attributes

Example:

```
```text id="ps9p24" { t = newtemp(); emit(t = x + y); }
```

So SDT is not merely output formatting.

36. SDT With Actions at End

A particularly convenient form for bottom-up parsing places actions at the **right end** of productions.

Example:

```
\[
A\rightarrow XYZ\{action\}
\]
```

When bottom-up parser reduces:

```
\[
XYZ\rightarrow A
\]
```

all RHS components have been recognized.

Then execute the action.

This fits naturally with synthesized attributes.

37. S - A t t r i b u t e d S D D ! ' E a s y B o t t o m - U p S D T

Because an S-attributed definition computes parent attributes from already recognized children, its semantic rules can often be implemented at reduction time.

Thus:

```
\[
\boxed{
S\text{-attributed}
\rightarrow
\text{very convenient with bottom-up parsing}
}
\]
```

38. Actions in the Middle

Suppose:

```
\[
A\rightarrow X\{M\}Y
\]
```

The action `M` must execute after X but before Y.

For top-down parsing, this can often be handled naturally.

For bottom-up parsing, mid-production actions may require transformations such as introducing a marker non-terminal.

This is useful conceptual knowledge but not something to overfocus on for your GATE syllabus.

39. Marker Non-Terminal Intuition

An embedded action:

```
\[
A\rightarrow X\{action\}Y
\]
```

may conceptually be transformed using:

```
\[
M\rightarrow\epsilon
\]
```

so:

```
\[
A\rightarrow XMY
\]
```

and associate the action with:

```
\[
M\rightarrow\epsilon
\]
```

This makes the action execute at the desired parsing position.

Remember only the intuition unless a specific problem asks it.

40. Dependency Graph vs SDT Order

In SDD:

> Evaluation order can be obtained from attribute dependencies.

In SDT:

> Actions are explicitly positioned, so the translation scheme directly constrains when they execute.

Thus:

```
```text id="jz47tn"
S D D !' d e p e n d e n c y - d r i v e n

S D T !' p o s i t i o n / a c t i o n - d r i v e n
```
```

41. S-Attributed vs L-Attributed vs General SDD

Relationship:

In set notation:

```
\[
\boxed{
S\text{-attributed}
\subseteq
L\text{-attributed}
\subseteq
General\ SDD
}
\]
```

Not every general SDD is L-attributed because arbitrary dependency directions may occur.

42. Why L-Attributed Is Useful

General inherited attributes could depend on future/right sibling information:

```
\[
X_i.in=f(X_{i+1}.s)
\]
```

which makes straightforward left-to-right evaluation difficult.

L-attributed definitions restrict dependencies so values can be computed in a manageable traversal.

Thus L-attributed is a useful balance:

> more expressive than synthesized-only, but still efficiently evaluable.

43. GATE Pattern – Identify S-Attributed

Production:

```
\[
A\rightarrow BC
\]
```

Rules:

```
\[
B.s=f(...)
\]
```

```
\[
C.s=g(...)
\]
```

```
\[
A.s=h(B.s,C.s)
\]
```

If all attributes throughout the SDD are synthesized:

```
\[
\boxed{S\text{-attributed}}
\]
```

44. GATE Pattern – Identify L-Attributed

Production:

```
\[
A\rightarrow XYZ
\]
```

Rules:

```
\[
X.i=A.i
\]
```

```
\[
Y.i=f(A.i,X.s)
\]
```

```
\[
Z.i=g(X.s,Y.s)
\]
```

```
\[
A.s=h(Z.s)
\]
```

All inherited information travels from:

- parent
- left siblings

Thus:

```
\[
\boxed{\text{L-attributed}}
\]
```

Not S-attributed because inherited attributes exist.

45. GATE Pattern – Not L-Attributed

Production:

```
\[
A\rightarrow XYZ
\]
```

Rule:

```
\[
X.i=Z.s
\]
```

`Z` lies to the right of X.

Therefore:

```
\[
\boxed{\text{Not L-attributed}}
\]
```

46. GATE-STYLE Question – Relationship

Which statement is correct?

- A. Every L-attributed SDD is S-attributed.
- B. Every S-attributed SDD is L-attributed.
- C. They are unrelated.
- D. Every SDD is S-attributed.

Correct:

```
\[
\boxed{B}
\]
```

47. GATE TRAPS

Trap 1

S-attributed SDD may contain inherited attributes.

'L F a l s e .

```
\[
\boxed{\text{Synthesized only}}
\]
```

Trap 2

Every L-attributed definition is S-attributed.

'L F a l s e .

L-attributed may use inherited attributes.

Trap 3

Every S-attributed definition is L-attributed.

' T r u e .

No inherited dependency exists to violate L-attributed restrictions.

Trap 4

In an L-attributed SDD, inherited attribute of a symbol may freely depend on any right sibling.

'L F a l s e .

That's the key restriction.

Trap 5

L-attributed means only inherited attributes.

```
'L F a l s e .
```

It can contain both synthesized and inherited attributes.

```
---
```

```
## Trap 6
```

S-attributed definitions are naturally suitable for bottom-up evaluation.

```
' T r u e .
```

```
---
```

```
## Trap 7
```

L-attributed definitions are naturally associated with left-to-right/top-down evaluation.

```
' T r u e a s t h e s t a n d a r d a s s o c i a t i o n .
```

```
---
```

```
## Trap 8
```

SDD and SDT mean exactly the same representation.

```
'L F a l s e .
```

```
```text id="tkfpzx"
```

```
S D D ! ' s e m a n t i c r u l e s / a t t r i b u t e s
```

```
S D T ! ' e m b e d d e d s e m a n t i c a c t i o n s
```

---

## Trap 9

Position of a semantic action in an SDT does not matter.

'LFalse.

Position determines when it executes.

---

## Trap 10

Semantic actions can only print output.

'LFalse.

They can perform semantic analysis and code-generation tasks too.

---

## 48. GATE-STYLE EXAMPLE 1

Production:

$[ A \rightarrow BC ]$

Semantic rules:

$[ A.val = B.val + C.val ]$

What kind of attribute is `A.val` ?

$\boxed{\text{Synthesized}}$

If the whole SDD uses only such attributes:

$\boxed{\text{S-attributed}}$

---

## 49. GATE-STYLE EXAMPLE 2

Production:

$A \rightarrow BC$

Rules:

$B.i = A.i$

$C.i = B.s$

$A.s = C.s$

Classification?

Inherited:

````text id="3xfxsu" A !' B B !' C`

moves using parent/left context.

Synthesized:

````text id="6onccp"  
C !' A`

Therefore:

$\boxed{\text{L-attributed but not S-attributed}}$

---

## 50. GATE-STYLE EXAMPLE 3

Production:

$A \rightarrow BC$

Rule:

$B.i = C.s$

**B.i** depends on right sibling C.

Therefore:

$\boxed{\text{Not L-attributed}}$

---

## 51. GATE-STYLE Example 4 — Postfix

SDT:

$E \rightarrow E_1 + T \{ \text{print}("+") \}$

Input:

```
```text id="li4avq" a+b
```

Operands are emitted first.

Then action prints `+`.

Output:

```
\[
\boxed{ab+}
\]
```

52. MINI PRACTICE – PART 18

Try these before the answers.

Q1 – Easy

An S-attributed SDD contains:

- A. Only inherited attributes
- B. Only synthesized attributes
- C. No attributes
- D. Only lexical errors

Q2 – Easy

Which is true?

- A. Every L-attributed SDD is S-attributed.
- B. Every S-attributed SDD is L-attributed.
- C. Neither relationship holds.
- D. SDDs cannot be L-attributed.

Q3 – Medium

For:

```
\[
A\rightarrow XYZ
\]
```

is this compatible with L-attributed evaluation?

```
\[
Y.i=f(A.i,X.s)
\]
```

- A. Yes
- B. No

Q4 – Medium

For:

```
\[
A\rightarrow XYZ
\]
```

is:

```
\[
X.i=Z.s
\]
```

L-attributed?

- A. Yes
- B. No

Q5 – Tricky

Which statement best distinguishes SDD and SDT?

- A. SDD uses DFA, SDT uses PDA.
- B. SDD specifies semantic attributes/rules; SDT embeds semantic actions at specific positions in productions.
- C. They are unrelated to grammar.
- D. SDT can contain no actions.

SOLUTIONS

Q1

' * * B * '

```
\[
\boxed{S\text{-attributed}=Synthesized\ only}
\]
```

Q2

' * * B * '

```
\[
\boxed{
S\text{-attributed}
\subseteq
L\text{-attributed}
}
\]
```

Q3

' * * A * '

`Y.i` depends on:

- parent A
- left sibling X

So it satisfies the main L-attributed restriction.

```

---

## Q4

'   * * B   —   N o * *

X depends on right sibling Z.

That violates left-to-right dependency restrictions.

---

## Q5

'   * * B * *

### SDD

attributes + semantic rules.

### SDT

semantic actions explicitly embedded in productions.

---

# WRITE THIS IN NOTES — PART 18 MASTER SHEET

## S-Attributed

\[
\boxed{\text{Only synthesized attributes}}
\]

Information:

\[
\boxed{\text{Children} \rightarrow \text{Parent}}
\]

Natural evaluation:

\[
\boxed{\text{Bottom-up / postorder}}
\]

Natural parser association:

\[
\boxed{\text{LR}}
\]

---

## L-Attributed

For:

\[
A \rightarrow X_1 X_2 \cdots X_n
\]

an inherited attribute of  $(X_i)$  may use:

\[
\boxed{\text{parent } A}
\]

```



```

\]

and:

\[
\boxed{\text{left siblings } X_1, \ldots, X_{i-1}}
\]

Main restriction:

> **Don't depend on future/right siblings.**

Natural evaluation:

\[
\boxed{\text{Depth-first, left-to-right}}
\]

---

## Relationship

\[
\boxed{
\begin{array}{l}
S \text{ \text{-attributed} } \\
\subseteq \\
L \text{ \text{-attributed} } \\
\subseteq \\
\text{General \ SDD}
\end{array}
}
\]

---

## SDD

\[
\boxed{\text{CFG+Attributes+Semantic \ Rules}}
\]

asks:

> What should be computed?

---

## SDT

\[
\boxed{\text{Grammar + embedded semantic actions}}
\]

asks:

> When should the action execute?

---

# 30-SECOND REVISION

### S-attributed

> **S = Synthesized only.**

``text id="h0sbjp"
Children !' Parent

```

Best association:

$\boxed{\text{Bottom-up}}$

L-attributed

L = Left context allowed.

Inherited information can come from:

``text id="5uf3ba" Parent + Left siblings

Not future right siblings.

Relationship

$\boxed{S \rightarrow L}$

but:

$\boxed{L \not\rightarrow S}$

SDD vs SDT

``text id="ebvf7q"
SDD !' attributes + equations
SDT !' embedded executable actions

Postfix

For:

$a+b$

emit operands, then operator:

$\boxed{ab+}$

Best GATE classification method

``text id="j9jyfc" Only synthesized? !' S-attributed

Inherited exists? !' check parent + left sibling dependencies !' L-attributed if valid

We have now completed **Syntax-Directed Translation**.

Next, we move to the official syllabus topic:

Runtime Environments

This will cover:

- runtime storage organization
- static / stack / heap allocation
- activation records
- activation trees
- parameters
- local variables
- temporaries
- return address/value
- recursion and procedure calls

****Completed: Parts 1–18/31 | Next: Part 19 – Runtime Storage Organization & Activation Records****

Reply ****NEXT**** for ****Part 19****.

USER

NEXT

ASSISTANT

Compiler Design — Part 19: Runtime Storage Organization & Activation Records

Completed: Parts 1–18/31 | Current: Part 19 | Remaining: Parts 20–31

We now move from compile-time translation to what happens **when the generated program actually runs**.

A compiler must decide:

- where variables live in memory
- how function calls get their own storage
- where parameters and local variables are stored
- how recursion works
- where return addresses are kept
- which data goes to stack and which goes to heap

This is the job of the:

$\boxed{\text{Runtime Environment}}$

1. What Is a Runtime Environment?

CONCEPT

The **runtime environment** is the organization and management of memory and other resources required while a compiled program executes.

It deals with things such as:

- procedure/function calls
- local variables

- parameters
- temporary values
- dynamically allocated objects
- return addresses
- control information

WRITE THIS IN NOTES

Runtime environment: The memory organization and supporting mechanisms used to execute the target program generated by the compiler.

2. Typical Runtime Memory Layout

A running program is often conceptualized as:

TEXT

Higher addresses

[illegible]

Exact layout varies by system, but for compiler theory, remember the major regions:

1. Code
2. Static/global area
3. Stack
4. Heap

Memory Trick

CSSH

TEXT

Code
Static
Stack
Heap

Or simply remember:

Code + Static + Stack + Heap

3. Three Main Storage Allocation Strategies

Compiler runtime environments commonly discuss:

$\boxed{\text{Static Allocation}}$

$\boxed{\text{Stack Allocation}}$

$\boxed{\text{Heap Allocation}}$

These are extremely important.

4. Static Allocation

CONCEPT

In **static allocation**, storage addresses are determined before program execution and remain fixed throughout execution.

Typical examples:

- global variables
- static variables
- fixed-lifetime objects

WRITE THIS IN NOTES

Static allocation: Memory is allocated at compile/load time and remains allocated for the entire program execution.

5. Properties of Static Allocation

Lifetime

Usually:

$\boxed{\text{Entire program execution}}$

Address

Known in advance or fixed before normal execution.

Allocation/deallocation

No repeated runtime allocation for each function call.

6. Static Allocation and Recursion

Suppose function:

```
c
f() {
    int x;
    f();
}
```

If `x` had only one fixed static location, every recursive invocation would try to use the same `x`.

But recursive calls require separate copies of local variables.

Therefore purely static allocation is generally unsuitable for recursive procedures with per-call locals.

$\boxed{\text{Static allocation cannot naturally support arbitrary recursion}}$

GATE TRAP

Static allocation means a recursive function gets a fresh copy of local variables automatically.

'L False.

Fresh per-call storage is the job of stack allocation.

7. Stack Allocation

CONCEPT

In **stack allocation**, storage is allocated when a procedure/function is called and released when that call returns.

Each invocation gets its own storage block.

This block is called an:

$\boxed{\text{Activation Record}}$

or:

$\boxed{\text{Stack Frame}}$

WRITE THIS IN NOTES

Stack allocation: Each procedure activation gets an activation record pushed onto the runtime stack; the record is removed when the procedure returns.

8. Why a Stack Works

Function calls normally follow:

`\boxed{\text{LIFO}}`

Last function called is normally the first one to finish.

Example:

```
TEXT
main calls A
A calls B
B calls C
```

Return order:

```
TEXT
C returns
B returns
A returns
main continues
```

This is:

`\boxed{\text{Last In, First Out}}`

Exactly what a stack provides.

9. Example Runtime Stack

Suppose:

```
C
main() {
    A();
}

A() {
    B();
}
```

While **B** is running:

```
TEXT
Top of stack
% % % % % % % % % % % % % % %
%   B   f r a m e           %
% % % % % % % % % % % % % % %$
%   A   f r a m e           %
% % % % % % % % % % % % % % %$
%   m a i n   f r a m e     %
% % % % % % % % % % % % % % %
```

When B returns:

```
TEXT
Top
% % % % % % % % % % % % % % %
%   A   f r a m e           %
% % % % % % % % % % % % % % %$
```

```
%   m a i n   f r a m e       %  
% % % % % % % % % % % % % %
```

B's frame is removed.

10. Stack Allocation Supports Recursion

Consider:

```
c  
factorial(3)
```

Calls:

```
TEXT  
fact(3)  
!'  f a c t ( 2 )  
!'  f a c t ( 1 )
```

Each invocation needs its own:

- parameter n
- return address
- temporary information
- local variables

So the stack may look like:

```
TEXT  
Top  
% % % % % % % % % % % % % %  
%   f a c t ( 1 )               %  
% % % % % % % % % % % % % % $  
%   f a c t ( 2 )               %  
% % % % % % % % % % % % % % $  
%   f a c t ( 3 )               %  
% % % % % % % % % % % % % % $  
%   m a i n                     %  
% % % % % % % % % % % % % %
```

Each call is a separate activation.

Therefore:

[\boxed{\text{Stack allocation naturally supports recursion}}]

11. Heap Allocation

CONCEPT

The heap stores data whose lifetime cannot be handled conveniently by simple LIFO stack behavior.

Typical examples:

- dynamically allocated objects
- linked-list nodes
- objects created with `new`
- memory allocated with `malloc`
- objects that may outlive the function that created them

WRITE THIS IN NOTES

Heap allocation: Runtime allocation used for objects with dynamically determined size or lifetime that does not follow procedure-call LIFO order.

12. Why Not Put Everything on the Stack?

Suppose function creates an object that must remain alive even after the function returns.

If it were stored only inside that function's activation record, the memory would disappear when the function returns.

So such data may need heap storage.

13. Stack vs Heap

| Stack | Heap |
|---------------------------------|-----------------------------|
| Procedure activations | Dynamic objects |
| LIFO lifetime | Arbitrary lifetime |
| Fast allocation/deallocation | More flexible |
| Automatically removed on return | Requires runtime management |
| Recursion frames | Linked structures/objects |
| Structured lifetime | Unstructured lifetime |

Memory Trick

Stack = Calls

Heap = Objects with flexible lifetime

14. Static vs Stack vs Heap

| Property | Static | Stack | Heap |
|--------------------|-----------------------------|---------------------------|------------------------------------|
| Allocation time | Before/throughout execution | On call | Dynamically at runtime |
| Lifetime | Whole program | One activation | Arbitrary |
| Recursion-friendly | No, for per-call locals | Yes | Yes, but not primarily call frames |
| LIFO | No | Yes | No |
| Typical data | Globals/statics | Params/locals/return info | Dynamic objects |

15. What Is a Procedure Activation?

A procedure is the code itself.

An **activation** is one particular execution of that procedure.

Example:

TEXT

```
factorial(5)
factorial(4)
factorial(3)
```

These are three distinct activations of the same function.

WRITE THIS IN NOTES

Procedure activation: One execution instance of a procedure/function.

A recursive function can have many simultaneous activations.

16. What Is an Activation Record?

An **activation record** is the block of memory containing information needed for one procedure activation.

Also called:

$\boxed{\text{Stack Frame}}$

Typical contents include:

- actual/formal parameter information
- return value
- return address
- control/dynamic link
- access/static link
- saved machine status
- local variables
- temporaries

Not every implementation contains every field exactly this way, but these are the standard compiler-theory components.

WRITE THIS IN NOTES

$\boxed{\text{Activation Record} = \text{information needed for one procedure invocation}}$

17. Typical Activation Record Layout

A conceptual layout:

| TEXT | |
|----------------------|--|
| Actual Parameters | |
| Return Value | |
| Return Address | |
| Control/Dynamic Link | |
| Access/Static Link | |
| Saved Machine Status | |
| Local Variables | |
| Temporaries | |

Exact order varies.

GATE usually tests **purpose**, not exact physical ordering unless a specific layout is given.

18. Parameters

Parameters are values/information passed from caller to callee.

Example:

| C |
|------------|
| sum(5, 10) |

The activation of `sum` needs access to:

| TEXT |
|------|
| 5 |
| 10 |

These may be stored in:

- registers
- activation record
- some calling-convention-specific locations

In compiler theory, they are commonly shown as part of the activation record.

19. Return Value

Suppose:

```
c
x = sum(a, b);
```

`sum` must return a result.

The result may be placed in:

- a register
- designated memory
- part of activation-record/calling convention

Conceptually the activation record includes provision for:

$\boxed{\text{Return Value}}$

20. Return Address

When function B finishes, it must know:

Where in function A should execution continue?

That location is stored as the:

$\boxed{\text{Return Address}}$

Example:

```
TEXT
A:
  instruction 1
  call B
      i n s t r u c t i o n   3       ! •   c o n t i n u e   h e r e   a f t e r   B   r e t u r n s
```

The address of the continuation location is saved.

Memory Trick

Return address = Where to continue

Return value = What result to give back

Do not confuse them.

21. Local Variables

Consider:

```
C
void f() {
    int x;
    int y;
}
```

Each activation of `f` needs its own `x` and `y`.

Therefore these are commonly stored in the activation record.

During recursion:

```
TEXT
f  c a l l   1  !'   x 1 ,   y 1
f  c a l l   2  !'   x 2 ,   y 2
f  c a l l   3  !'   x 3 ,   y 3
```

Separate frames keep them separate.

22. Temporaries

A compiler may generate temporary values while evaluating expressions.

For:

```
C
x = a + b * c;
```

intermediate operations might involve:

```
TEXT
t1 = b * c
t2 = a + t1
```

Some temporaries may use registers.

Others may need slots in the activation record.

Thus:

$\boxed{\text{Temporaries}}$

are another possible activation-record component.

23. Saved Machine Status

When calling another function, the caller's execution state may need to be preserved.

Examples:

- register values
- program state

- frame pointer-related data

This is broadly represented as:

`\[ \boxed{\text{Saved Machine Status}} \]`

When callee returns, necessary state can be restored.

24. Control Link / Dynamic Link

This deserves special attention.

The **control link**, also called the:

`\[ \boxed{\text{Dynamic Link}} \]`

typically points to the activation record of the **caller**.

If:

```
TEXT
A calls B
```

then B's dynamic/control link points to A's frame.

WRITE THIS IN NOTES

Control/Dynamic Link: Points to the activation record of the calling procedure.

Memory:

Dynamic link follows the call chain.

25. Dynamic Link Example

Calls:

```
TEXT
main !' A !' B
```

Frames:

```
TEXT
B frame
%
% % %   d y n a m i c   l i n k   !'   A   f r a m e

A frame
%
% % %   d y n a m i c   l i n k   !'   m a i n   f r a m e
```

Thus dynamic links reflect:

$\boxed{\text{Runtime calling history}}$

26. Access Link / Static Link — Preview

The **access link**, also called:

$\boxed{\text{Static Link}}$

is used in languages with nested procedures and static/lexical scope.

It helps access variables in an enclosing lexical scope.

Important:

Dynamic link

follows:

$\boxed{\text{Who called me?}}$

Static link

follows:

$\boxed{\text{Who lexically encloses me?}}$

We will study this deeply in Part 20.

27. Dynamic vs Static Link Preview

Suppose:

TEXT

Procedure P

Procedure Q

Q is lexically nested inside P.

If some other legal call structure invokes Q, its:

- dynamic link depends on runtime caller
- static link points toward lexical parent P's relevant activation

So the two links need not point to the same frame.

28. Activation Tree

Procedure calls during program execution can be represented using an:

$\boxed{\text{Activation Tree}}$

Each node = one procedure activation.

An edge:

$A \rightarrow B$

means activation A directly called activation B.

29. Example Activation Tree

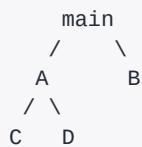
Suppose:

TEXT

```
main
  % % %   A
  %       % % %   C
  %       % % %   D
  % % %   B
```

Then the activation tree is:

TEXT



Important:

This is about runtime calls, not grammar parse trees.

30. Activation Tree Properties

If activation A calls B:

$\backslash$ A $\backslash$

is parent of:

$\backslash$ B $\backslash$

in activation tree.

The execution order has a useful property:

Calls

correspond roughly to preorder-style entry.

Returns

correspond to completing subtrees before returning to parent.

31. Relationship Between Activation Tree and Stack

At any instant, the active procedure calls correspond to a path from the activation-tree root to the currently executing activation.

Example tree:

TEXT

```
main
 |
A
 |
B
 |
C
```

while C is running, runtime stack contains:

TEXT

```
main
A
B
C
```

So:

$\boxed{\text{Runtime stack = current root-to-active-node path in activation tree}}$

This is a useful GATE fact.

32. Why Finished Sibling Calls Aren't on the Stack

Suppose:

TEXT

```
main
  % %    A
        % %    B
        % %    C
```

If B returns and then A calls C, while C runs:

Stack:

TEXT

```
main
A
C
```

B is no longer active.

The activation tree keeps historical structure, but runtime stack only keeps currently active calls.

33. Lifetime of an Activation Record

For ordinary stack-allocated function frames:

Created when:

$\boxed{\text{procedure is called}}$

Exists while:

$\boxed{\text{procedure activation is active}}$

Destroyed when:

$\boxed{\text{procedure returns}}$

34. Caller and Callee

Suppose:

TEXT

A calls B

Then:

A

is the:

$\boxed{\text{Caller}}$

B

is the:

$\boxed{\text{Callee}}$

You'll repeatedly encounter these terms.

35. Call Sequence and Return Sequence

The compiler/runtime performs certain operations around a procedure call.

Conceptually:

Call sequence

- evaluate arguments
- save needed caller information
- set up callee's activation record
- store return address
- transfer control

Return sequence

- produce return value
- restore saved state
- remove activation record
- transfer control back to caller

Exact responsibilities between caller and callee depend on calling convention.

36. Stack Pointer vs Frame Pointer — Useful Concept

Stack Pointer (SP)

Points near the current top of stack.

Changes as data is pushed/popped.

Frame Pointer (FP)

Often provides a stable reference point within the current activation record.

Then locals/parameters can be accessed by fixed offsets.

Example:

TEXT

```
FP + offset !' parameter
FP - offset !' local
```

Implementation details vary, but this conceptual distinction can help.

37. Why Fixed Offsets Are Useful

Suppose local `x` is always at:

`\[ FP-8 \]`

within a function's frame.

Even if the absolute memory address changes between calls, the compiler can generate code relative to the frame pointer.

Thus recursion still works:

TEXT

```
fact(5) frame: x at FP1 - 8
fact(4) frame: x at FP2 - 8
```

Same relative layout, different frame bases.

38. Static Allocation Example

C

```
int g;
```

`g` exists for the entire program.

So it naturally belongs in:

`\boxed{\text{Static/Global Area}} \]`

Not in a fresh stack frame for every function call.

39. Stack Allocation Example

```
c
int f(int n) {
    int x;
    ...
}
```

Typical:

- `n` !' parameter area/register/frame
- `x` !' activation record
- return address !' activation record/calling machinery

New invocation !' new frame.

40. Heap Allocation Example

```
c
Node *p = malloc(sizeof(Node));
```

The Node's lifetime may outlive the current function.

Therefore Node belongs to:

`\boxed{\text{Heap}}`

The pointer `p` itself, if local, may live in:

`\boxed{\text{Stack frame}}`

This is an important distinction.

GATE TRAP

Given:

```
c
Node *p = malloc(...);
```

Do not say both `p` and the allocated Node necessarily live in the heap.

Commonly:

```
TEXT
local pointer p !' stack
object allocated by malloc !' heap
```

41. Variable Lifetime vs Scope

These are different concepts.

Scope

Where in the program text a name can be referenced.

Lifetime

When during execution its storage exists.

Example:

A local variable may have limited lexical scope and one-call lifetime.

A global variable can have global/static lifetime.

We'll connect scope to access links in Part 20.

42. Activation Record Size

Can activation-record size always be known completely at compile time?

Not necessarily in every language/runtime.

For many conventional functions with fixed locals, much of the frame layout is known at compile time.

But variable-sized objects or dynamic features may require runtime handling.

GATE generally focuses on conceptual fields and allocation methods rather than platform-specific frame sizes.

43. Static Allocation Limitations

Static allocation works well when:

- number of activations is fixed/simple
- no recursion requiring distinct per-call locals
- lifetimes can be determined in advance

It is poor for:

- recursion
- dynamic data structures
- arbitrary object lifetimes

44. Stack Allocation Limitations

Stack allocation works when lifetimes are properly nested:

TEXT

```
A starts
  B starts
  B ends
A ends
```

But not naturally when an object created in B must survive after B returns.

Such objects need a different lifetime mechanism, usually heap.

45. Nested Lifetime Property

Stack allocation requires lifetimes resembling:

$\boxed{\text{Properly nested intervals}}$

If A's activation starts before B's and B is nested inside A:

TEXT

```
A start
  B start
  B end
A end
```

This naturally gives LIFO behavior.

46. Heap's Advantage

Heap objects can be:

- created in arbitrary order
- destroyed in arbitrary order

Example:

TEXT

```
create A
create B
destroy A
create C
destroy B
destroy C
```

This is not LIFO.

Hence heap management is required.

47. Heap Management

Memory may be reclaimed by:

- explicit deallocation, e.g. `free`
- automatic garbage collection

The exact method depends on language/runtime.

For GATE compiler design, remember mainly:

$\boxed{\text{Heap supports non-LIFO lifetimes}}$

48. GATE-STYLE Example 1

Which allocation strategy best supports recursive procedure activations?

A. Static allocation B. Stack allocation C. Code segment D. Symbol table

Answer:

$\boxed{B}$

49. GATE-STYLE Example 2

Which activation-record field tells the callee where execution should continue after return?

A. Local variable B. Return address C. Static variable D. Token attribute

Answer:

$\boxed{\text{Return Address}}$

50. GATE-STYLE Example 3

Function A calls B.

The dynamic/control link in B's frame usually points to:

$\boxed{\text{A's activation record}}$

because A is the caller.

51. GATE-STYLE Example 4

Which storage region is most appropriate for an object whose lifetime extends beyond the procedure that created it?

$\boxed{\text{Heap}}$

52. GATE-STYLE Example 5

During recursion:

TEXT

f (3) ! ' f (2) ! ' f (1)

How many simultaneous activations of f exist while f(1) is executing?

$\boxed{3}$

Each needs a separate activation record.

53. GATE-STYLE Example 6

Which statement is correct?

A. Procedure and activation are the same thing. B. One procedure can have multiple simultaneous activations. C. Recursion needs only one activation record. D. Stack allocation cannot support nested calls.

Correct:

$\boxed{B}$

54. GATE TRAPS

Trap 1

One procedure has exactly one activation record for the entire program.

'LFalse.

One **activation** gets one activation record.

Recursion can create many frames for the same procedure.

Trap 2

Static allocation naturally supports arbitrary recursion.

'LFalse.

Trap 3

Heap allocation follows strict LIFO order.

'LFalse.

Stack follows LIFO.

Trap 4

A return address and return value are the same thing.

'LFalse.

TEXT

```
R e t u r n   a d d r e s s   ! '   w h e r e   t o   c o n t i n u e
R e t u r n   v a l u e       ! '   r e s u l t   t o   g i v e   b a c k
```

Trap 5

Dynamic link points to lexical parent.

'LUsually false.

Dynamic/control link points to:

$\boxed{\text{caller}}$

Static/access link handles lexical nesting.

Trap 6

Every dynamically allocated object's pointer must also reside in heap.

'LFalse.

A local pointer can live on stack while pointing to heap data.

Trap 7

Runtime stack contains every procedure activation that has ever occurred.

'LFalse.

Only currently active, not-yet-returned activations are present.

Trap 8

Activation tree and parse tree are the same.

'LFalse.

Activation tree represents runtime calls.

Parse tree represents syntax.

Trap 9

Heap is primarily used for storing return addresses.

'LFalse.

Return/control information belongs to call-management structures, usually stack frames.

Trap 10

The exact field order in every activation record is universally fixed.

'LFalse.

Conceptual fields are standard; actual layout depends on implementation/calling convention.

55. MINI PRACTICE — PART 19

Try before checking.

Q1 — Easy

An activation record is created for:

A. Every grammar production B. Every procedure activation C. Every token D. Every global variable

Q2 — Easy

Which storage allocation naturally supports recursive calls?

A. Static B. Stack C. Text segment D. Symbol table

Q3 — Medium

If A calls B, B's dynamic link usually points to:

A. B itself B. A's activation record C. A's source code D. The heap

Q4 — Medium

Which is best suited to an object whose lifetime does not follow call/return nesting?

A. Runtime stack B. Heap C. Code segment D. FOLLOW set

Q5 — Tricky

While the call sequence is:

TEXT

m a i n ! ' P ! ' Q ! ' R

and R is currently executing, which activations are on the runtime stack?

SOLUTIONS

Q1

' B

\[\boxed{\text{One activation record per procedure activation}} \]

Q2

' B — Stack

Each recursive invocation gets a distinct frame.

Q3

' B

Dynamic/control link follows the runtime caller relationship:

\[B \rightarrow A \]

Q4

' B — Heap

Heap supports arbitrary/non-LIFO lifetime.

Q5

All four currently active activations:

TEXT

main
P
Q
R

with R at the top.

$\boxed{\text{active frames}}$

WRITE THIS IN NOTES — PART 19 MASTER SHEET

Runtime Memory

Main conceptual regions:

TEXT

Code
Static
Stack
Heap

Static Allocation

$\boxed{\text{Fixed / whole-program lifetime}}$

Typical:

- globals
- static data

Poor for recursive per-call locals.

Stack Allocation

$\boxed{\text{One activation record per active call}}$

LIFO.

Supports:

$\boxed{\text{Recursion}}$

Heap Allocation

$\boxed{\text{Dynamic arbitrary-lifetime objects}}$

Not restricted to LIFO.

Activation Record

Typical fields:

TEXT

Parameters
Return value
Return address
Dynamic/control link
Static/access link
Saved machine status
Local variables
Temporaries

Dynamic Link

$\boxed{\text{Points to caller's activation}}$

Static Link

$\boxed{\text{Used for lexical/enclosing-scope access}}$

Detailed next part.

Activation Tree

Each node:

$\boxed{\text{one procedure activation}}$

Parent calls child.

Current runtime stack corresponds to current root-to-active-node path.

30-SECOND REVISION

Three allocations

TEXT

STATIC

! ' whole - program / fixed lifetime

STACK

! ' calls / recursion / LIFO

HEAP

! ' dynamic objects / arbitrary lifetime

Activation

One execution of a procedure.

Activation record

Storage/data needed for one invocation.

Most important fields

TEXT

parameters
return value
return address
links
locals
temporaries
saved state

Return address

Where to go back.

Return value

What result to return.

Dynamic link

Who called me?

Static link

Who lexically encloses me?

Recursion

`\[ \boxed{\text{Each recursive call gets a separate frame}} \]`

Completed: Parts 1–19/31 | Next: Part 20 — Static vs Dynamic Links, Scope, Recursion & Parameter Passing

Reply **NEXT** for Part 20.

USER

NEXT

ASSISTANT

Compiler Design — Part 20: Static vs Dynamic Links, Scope, Recursion & Parameter Passing

Completed: Parts 1–19/31 | Current: Part 20 | Remaining: Parts 21–31

In Part 19, we introduced the activation record and two important links:

$\boxed{\text{Dynamic / Control Link}}$

and

$\boxed{\text{Static / Access Link}}$

These two are frequently confused in GATE.

The easiest starting memory is:

$\boxed{\text{Dynamic Link = Who called me?}}$

$\boxed{\text{Static Link = Who lexically encloses me?}}$

We will also cover:

- static vs dynamic scope
- nested procedures
- recursion
- parameter-passing methods
- call by value
- call by reference
- call by value-result
- call by name, conceptually

1. First: Scope vs Lifetime

These are different.

Scope

Scope tells:

Where in the source program can a name be referred to?

Lifetime

Lifetime tells:

During what period of execution does its storage exist?

WRITE THIS IN NOTES

$\boxed{\text{Scope = textual visibility}}$

$\boxed{\text{Lifetime = runtime existence}}$

Do not confuse them.

2. Example

```
c
int x;

void f() {
    int y;
}
```

x may have program-wide/static scope depending on the language rules.

y is visible only in its appropriate local scope.

Its stack storage normally exists only while **f** is active.

So:

```
TEXT
Scope      !'  source - code  concept
Lifetime  !'  runtime   concept
```

3. Static / Lexical Scope

CONCEPT

Under **static scope**, also called:

$\boxed{\text{Lexical Scope}}$

the declaration associated with a variable reference is determined from the **program's lexical nesting structure**.

In simple words:

Look at where the procedures/blocks are written inside one another.

Example

```
TEXT
P
{
    x

    Q
    {
        use x
    }
}
```

If Q does not declare its own **x**, its reference to **x** can use the **x** declared in enclosing procedure P.

This is determined from the source-code structure.

WRITE THIS IN NOTES

Static/Lexical scope: Binding of a non-local name is determined from the lexical nesting of declarations in the source program.

4. Dynamic Scope

Under **dynamic scope**, a non-local name is resolved using the **runtime sequence of procedure calls**.

Instead of asking:

Where is this function written?

we ask:

Which active caller most recently provides a declaration of this name?

Memory Trick

Static scope

Where am I written?

Dynamic scope

Who called me?

5. Static vs Dynamic Scope

| Static/Lexical Scope | Dynamic Scope |
|--|--|
| Based on source nesting | Based on runtime call chain |
| Usually determined largely at compile time | Depends on execution history |
| Uses lexical enclosing environment | Searches active caller chain |
| Associated with static/access links | Conceptually associated with dynamic chain |

6. Example Showing the Difference

Suppose:

```
TEXT
procedure P
  x = 10
```



```

    procedure Q
      print(x)

  procedure R
    x = 20
    call Q

```

Assume language permits this call setup for illustration.

When Q refers to `x`:

Static scope

Q is lexically inside P.

So:

$\boxed{x=10}$

Dynamic scope

Q was called through R.

So runtime search may find R's:

$\boxed{x=20}$

This illustrates why static and dynamic scope are fundamentally different.

7. Static / Access Link

Now connect scope to activation records.

Suppose procedures are nested:

```

TEXT
P
  % % %   Q
          % % %   R

```

R may need access to variables declared in Q or P.

An activation record may contain a:

$\boxed{\text{Static Link}}$

also called:

$\boxed{\text{Access Link}}$

which points toward the activation corresponding to the procedure's **lexically enclosing scope**.

WRITE THIS IN NOTES

Static/access link: Pointer used to access data in the lexically enclosing procedure's activation record.

8. Dynamic / Control Link

Recall:

Dynamic link points to the activation record of the caller.

Suppose runtime calls are:

TEXT

m a i n ! ' A ! ' B

Then:

TEXT

B d y n a m i c l i n k ! ' A f r a m e
A d y n a m i c l i n k ! ' m a i n f r a m e

Therefore dynamic links reflect:

$\boxed{\text{Actual runtime calling sequence}}$

9. Static Link vs Dynamic Link

This table is essential.

| Static / Access Link | Dynamic / Control Link |
|----------------------------------|--|
| Lexical relationship | Runtime caller relationship |
| Points toward enclosing scope | Points toward caller |
| Helps access non-local variables | Helps restore caller/control information |
| Based on program nesting | Based on call sequence |
| Can differ from dynamic link | Can differ from static link |

Memory Trick

Dynamic

$\boxed{\text{Call chain}}$

Static

$\boxed{\text{Scope chain}}$

10. When Can They Be the Same?

Suppose P lexically contains Q and P directly calls Q.

Then Q's:

- lexical parent = P
- runtime caller = P

So both links may point to P's activation.

But this is not guaranteed in general.

11. Why Can They Differ?

Suppose lexical structure:

```
TEXT
P
% % %      Q
% % %      R
```

and runtime:

```
TEXT
P calls Q
Q calls R
```

R is lexically nested inside P, not Q.

So R's:

Dynamic link

points to Q because:

$\backslash[Q\text{ called }R]\backslash$

Static link

points to P's appropriate activation because:

$\backslash[P\text{ lexically encloses }R]\backslash$

Thus:

$\backslash\boxed{\text{Static link}}\neq\text{Dynamic link}\backslash$

12. Full Example

Lexical nesting:

```
TEXT
P
% % %      Q
% % %      R
```

Runtime call sequence:

```
TEXT
P !' Q !' R
```

Frames:

TEXT

```
R frame
  % % %   d y n a m i c   l i n k   ! '   Q
  % % %   s t a t i c   l i n k   ! '   P

Q frame
  % % %   d y n a m i c   l i n k   ! '   P
  % % %   s t a t i c   l i n k   ! '   P
```

This is an excellent example to remember.

13. Lexical Level

Nested procedures can be assigned lexical nesting levels.

Example:

TEXT

```
Level 1: P
Level 2: Q
Level 3: R
```

If R accesses a variable declared in P, it may need to follow static links upward.

For:

TEXT

```
R !'   Q !'   P
```

it may traverse:

$\boxed{\text{static links}}$

depending on the exact nesting arrangement.

14. Accessing Non-Local Variables

Suppose:

TEXT

```
P
{
  int x;

  Q
  {
    R
    {
      use x;
    }
  }
}
```

R does not have `x`.

Q does not have `x`.

P does.

Under lexical scope, the runtime can follow:

TEXT

```
R   s t a t i c   l i n k   !'   Q
Q   s t a t i c   l i n k   !'   P
```

Then access P's `x`.

15. Static Chain

A sequence of static links is called the:

$\boxed{\text{Static Chain}}$

It corresponds to lexical nesting.

Likewise, following dynamic links gives a:

$\boxed{\text{Dynamic Chain}}$

which follows active callers.

WRITE THIS IN NOTES

TEXT

```
S t a t i c   c h a i n   !'   l e x i c a l   n e s t i n g
D y n a m i c   c h a i n   !'   r u n t i m e   c a l l i n g   s e q u e n c e
```

16. Access Link Count — GATE Pattern

Suppose:

- current procedure is at lexical level $\backslash(m\backslash)$
- variable is declared at lexical level $\backslash(n\backslash)$

where:

$\backslash m > n \backslash$

Then conceptually the number of enclosing-level steps is often:

$\boxed{m-n}$

Static links may be followed that many times, depending on the exact level convention.

Always check how levels are numbered in the question.

17. Another Technique: Display

There is another mechanism for accessing non-local variables called a:

$\boxed{\text{Display}}$

A display keeps direct references to active activation records at different lexical nesting levels.

Instead of walking several static links:

TEXT

R !' Q !' P

a display may allow direct access to the frame for lexical level P.

For GATE, remember only the high-level distinction unless the question explicitly asks.

18. Static Chain vs Display

Static chain

May require following several pointers.

Display

Keeps direct pointers indexed by lexical level.

Therefore display can make non-local access faster, at the cost of maintaining the display during calls/returns.

19. Recursion Revisited

A recursive procedure calls itself directly or indirectly.

Direct recursion

TEXT

A !' A

at runtime.

Example:

```
C
fact(n) {
  return n * fact(n-1);
}
```

Indirect recursion

TEXT

A calls B
B calls A

20. Why Recursion Requires Separate Frames

For:

TEXT

```
fact(3)
fact(2)
fact(1)
```

each activation has a different:

`\[ n \]`

and different:

- return address
- locals
- temporaries
- saved state

So the stack might contain:

TEXT

```
fact(1): n=1
fact(2): n=2
fact(3): n=3
main
```

Therefore:

`\[ \boxed{\text{One procedure can have multiple simultaneous activation records}} \]`

21. Activation Record and Recursion

The **code** for `fact` exists only once.

But its activation record can appear many times.

This distinction is frequently tested.

TEXT

```
P r o c e d u r e   c o d e   ! '   o n e   c o p y   c o n c e p t u a l l y
A c t i v a t i o n   r e c o r d   ! '   o n e   p e r   a c t i v e   i n v o c a t i o n
```

22. Tail Recursion — Useful Concept

A procedure is tail recursive when its recursive call is the final operation whose result is immediately returned.

Conceptually:

```
c
f(n) {
    ...
    return f(n-1);
}
```

Some compilers may optimize tail recursion by reusing a frame instead of creating a new one.

But:

Do not assume every language/compiler always performs tail-call optimization.

23. Parameter Passing

Now suppose:

```
c
f(x);
```

How does information associated with `x` move from caller to callee?

This is controlled by a:

`\boxed{\text{Parameter-Passing Mechanism}}`

Important methods include:

1. Call by value
2. Call by reference
3. Call by value-result
4. Call by name

24. Actual vs Formal Parameters

Suppose:

```
c
f(a);
```

Function declaration:

```
c
f(int x)
```

Then:

a

is the:

`\boxed{\text{Actual Parameter / Argument}}`

x

is the:

`\boxed{\text{Formal Parameter}}`

Memory Trick

Actual = supplied by caller

Formal = named by callee

25. Call by Value

CONCEPT

In call by value, the value of the actual argument is copied into the formal parameter.

Example:

TEXT

```
a = 10  
call f(a)
```

Callee receives something like:

TEXT

```
x = 10
```

`x` is a separate parameter variable.

Changes to `x` do not directly change `a`.

Example

```
c  
void f(int x) {  
    x = 50;  
}  
  
a = 10;  
f(a);
```

After return:

`\boxed{a=10}`

under ordinary call-by-value semantics.

WRITE THIS IN NOTES

Call by value: Copy actual value into formal parameter. Changes to formal do not directly alter the caller's variable.

Memory Trick

Value = Copy

26. Call by Reference

CONCEPT

In call by reference, the formal parameter refers to the caller's actual storage location.

Thus both effectively access the same variable.

Example:

TEXT

```
a = 10
```

`f` receives reference/address to `a`.

If formal parameter changes it to 50:

`\boxed{a=50}`

after return.

WRITE THIS IN NOTES

Call by reference: Formal parameter refers to the same storage location as the actual argument.

Memory Trick

Reference = Same room

27. Value vs Reference

| Call by Value | Call by Reference |
|--|--------------------------------|
| Copy value | Pass location/reference |
| Formal has independent storage conceptually | Formal aliases actual |
| Formal modification doesn't directly affect actual | Modification can affect actual |
| No aliasing through this parameter alone | Aliasing possible |

28. Aliasing

Two names are aliases if they refer to the same storage location.

Example under reference passing:

```
TEXT
f(a, a)
```

If both formals refer to `a`, then:

```
TEXT
x
y
```

may alias the same variable.

This can produce interesting GATE questions.

29. Call by Value-Result

Also called:

$\boxed{\text{Copy-In Copy-Out}}$

At function call:

$\boxed{\text{Actual value copied into formal}}$

At function return:

$\boxed{\text{Final formal value copied back into actual}}$

So it behaves like:

```
TEXT
CALL:
  a c t u a l ! '   f o r m a l

RETURN:
  f o r m a l ! '   a c t u a l
```

WRITE THIS IN NOTES

Call by value-result: Copy-in at call and copy-out at return.

30. Example — Value Result

Suppose:

TEXT

```
a = 10
```

Call:

TEXT

```
f(a)
```

Inside f:

TEXT

```
x initially = 10  
x = 50
```

On return:

TEXT

```
x copied back to a
```

Therefore:

$\boxed{a=50}$

It can resemble call by reference in simple examples, but the mechanisms differ.

31. Reference vs Value-Result Difference

Reference

Formal accesses actual variable directly throughout the call.

Value-result

Formal uses a separate local copy during execution, then copies result back at return.

This difference matters with:

- aliasing
- multiple parameters
- changes to shared/global data during call

32. Aliasing Trap With Value-Result

Suppose:

TEXT

```
f(a, a)
```

with two value-result parameters.

Each formal may get its own copy of `a`.

At return, both copies may be written back.

Then:

The final result may depend on copy-out order.

This is a classic conceptual problem with value-result semantics.

33. Call by Name

Call by name is more unusual.

The actual argument is conceptually re-evaluated in the caller's environment whenever the formal parameter is used.

It resembles textual substitution, but proper implementations preserve scoping using mechanisms historically called **thunks**.

For GATE, remember the intuition:

Call by name delays evaluation and re-evaluates the argument expression when used.

34. Example — Call by Name Intuition

Suppose:

```
TEXT
i = 2
f(A[i])
```

Inside `f`, if `i` changes before the formal is used again, call-by-name semantics can cause the expression:

`A[i]`

to be evaluated using the updated `i`.

This differs from simply copying the initial value.

35. Thunk — Concept Only

A **thunk** is a runtime mechanism representing delayed computation together with the environment needed to evaluate it.

It is associated historically with call by name.

You usually do not need implementation details for GATE compiler-design basics.

36. Parameter Passing Comparison

| Method | What is passed? | Actual updated during call? |
|-----------|-------------------|-----------------------------|
| Value | Copy of value | No direct effect |
| Reference | Address/reference | Yes |

| Method | What is passed? | Actual updated during call? |
|--------------|--------------------------------|-----------------------------|
| Value-result | Value copied in/out | On return |
| Name | Delayed expression/environment | Depends on expression/use |

37. Call by Value and Pointers — Important Caveat

Suppose a language uses call by value but passes a pointer as the value.

Example:

```
c
f(int *p)
```

The pointer value itself is copied.

But both caller and callee pointer values may point to the same object.

So the callee can modify:

```
c
*p
```

even though parameter passing is still call by value.

Do not confuse:

$\boxed{\text{passing a pointer by value}}$

with true language-level:

$\boxed{\text{call by reference}}$

38. Example

```
c
void f(int *p) {
    *p = 20;
}
```

Caller:

```
c
int x = 10;
f(&x);
```

The address `&x` is passed by value.

But it points to x's storage.

So:

$\boxed{x=20}$

after the call.

39. Parameter Evaluation and Activation Records

Formal parameters may be stored in:

- registers
- activation records
- referenced caller locations

depending on:

- parameter-passing method
- calling convention
- architecture

For compiler theory, think:

$\boxed{\text{Activation record contains/represents parameter information}}$

40. GATE-STYLE Example — Static vs Dynamic Link

Lexical structure:

TEXT

P

% % % Q

% % % R

Runtime:

TEXT

P !' Q !' R

While R executes:

Dynamic link of R

$\boxed{Q}$

because Q called R.

Static link of R

$\boxed{P}$

because R is lexically inside P.

41. GATE-STYLE Example — Static Chain

Suppose:

TEXT

```
P level 1
  Q level 2
    R level 3
```

R accesses a variable declared in P.

Number of lexical-level jumps:

$\backslash[3-1=2\backslash]$

So conceptually it may follow:

$\backslash[\boxed{2}\text{static links}]\backslash]$

42. GATE-STYLE Example — Call by Value

TEXT

```
a = 5

f(x):
  x = x + 10

f(a)
```

After return:

$\backslash[\boxed{a=5}]\backslash]$

because x is a copy.

43. GATE-STYLE Example — Call by Reference

Same program under reference passing:

TEXT

```
x aliases a
```

So:

$\backslash[a=5+10\backslash]$

$\backslash[\boxed{a=15}]\backslash]$

44. GATE-STYLE Example — Value-Result

TEXT

```
a = 5
f(a)

inside:
x = 15
```

At return:

`\[ x\rightarrow a \]`

Therefore:

`\[ \boxed{a=15} \]`

But remember this is copy-out, not direct aliasing throughout execution.

45. GATE-STYLE Example — Recursion

If:

TEXT

```
f(4)
! '  f ( 3 )
! '  f ( 2 )
! '  f ( 1 )
```

and none have returned yet, number of f activation records:

`\[ \boxed{4} \]`

46. GATE TRAPS

Trap 1

Static link always points to caller.

'L False.

That's dynamic/control link.

Static link follows lexical scope.

Trap 2

Dynamic link always points to lexical parent.

'L False.

It points toward the runtime caller.

Trap 3

Static and dynamic links can never be equal.

'L False.

If caller is also lexical parent, they may point to the same frame.

Trap 4

Static scope is determined from runtime calling sequence.

'LFalse.

That's dynamic scope.

Trap 5

Dynamic scope is determined solely from source-code nesting.

'LFalse.

It uses runtime call chain.

Trap 6

Call by value allows assignment to formal to automatically change actual.

'LFalse.

Trap 7

Call by reference uses a separate independent formal storage value.

'LGenerally false.

Formal aliases/references actual storage.

Trap 8

Call by value-result modifies the actual immediately whenever formal changes.

'LFalse.

Conceptually modification is copied back at return.

Trap 9

Call by value-result and call by reference are always observationally identical.

'LFalse.

Aliasing and copy-out order can make them differ.

Trap 10

Recursion creates a new copy of the function's machine code for every call.

'LFalse.

It creates a new activation/frame, not a new copy of source/function code.

47. MINI PRACTICE — PART 20

Try these first.

Q1 — Easy

The dynamic/control link of a procedure activation usually points to:

A. Lexical child B. Caller activation C. Heap object D. Symbol table

Q2 — Easy

The static/access link primarily supports:

A. Token recognition B. Lexically non-local variable access C. Heap deallocation D. Register allocation

Q3 — Medium

Lexical structure:

TEXT

P

% % % Q

% % % R

Runtime:

TEXT

P calls Q

Q calls R

What do R's dynamic and static links point to?

Q4 — Medium

Under call by value:

TEXT

a = 10

f(x):

x = 20

f(a)

What is a after return?

Q5 — Tricky

Which method is called **copy-in copy-out**?

A. Call by reference B. Call by value C. Call by value-result D. Call by name

SOLUTIONS

Q1

' B

$\boxed{\text{Dynamic link !' caller}}$

Q2

' B

$\boxed{\text{Static link !' lexical enclosing environment}}$

Q3

R was called by Q:

$\boxed{\text{Dynamic link of R !' Q}}$

R is lexically nested in P:

$\boxed{\text{Static link of R !' P}}$

Q4

Call by value copies:

$a=10 \rightarrow x=10$

Changing x does not directly change a.

Therefore:

$\boxed{a=10}$

Q5

' C

$\boxed{\text{Call by value-result = copy-in copy-out}}$

WRITE THIS IN NOTES — PART 20 MASTER SHEET

Scope vs Lifetime

TEXT

Scope = where name is visible

Lifetime = when storage exists

Static / Lexical Scope

$\boxed{\text{Based on program nesting}}$

Question:

Where am I written?

Dynamic Scope

`\boxed{\text{Based on runtime call chain}} \]`

Question:

Who called me?

Static / Access Link

`\boxed{\text{Points toward lexical enclosing activation}} \]`

Used for:

`\boxed{\text{non-local variable access}} \]`

Dynamic / Control Link

`\boxed{\text{Points toward caller activation}} \]`

Follows:

`\boxed{\text{runtime call chain}} \]`

Static vs Dynamic Memory Trick

TEXT

```
STATIC LINK    !'  SCOPE  chain
DYNAMIC LINK !'   CALL  chain
```

Recursion

`\boxed{\text{Each active recursive call gets a separate activation record}} \]`

Parameter Passing

Value

`\boxed{\text{Copy value}} \]`

Reference

`\boxed{\text{Share/reference actual location}} \]`

Value-Result

$\boxed{\text{Copy in} + \text{Copy out}}$

Name

$\boxed{\text{Delayed/repeated evaluation in caller context}}$

30-SECOND REVISION

Two links

TEXT

Dynamic link:
Who CALLED me?

Static link:
Who ENCLOSES me lexically?

Two scopes

TEXT

Static scope !' source nesting
Dynamic scope !' runtime calls

Recursion

TEXT

same procedure code
+
multiple active frames

Parameter memory

TEXT

VALUE
= Copy

REFERENCE
= Same location

VALUE-RESULT
= Copy in / Copy out

NAME
= Delayed expression

Biggest trap

$\boxed{\text{Dynamic link}} \neq \boxed{\text{Static link}}$

unless the runtime caller happens also to be the lexical parent.

We have now completed **Runtime Environments**.

Next we move to:

Intermediate Code Generation

We will cover:

- intermediate representations
- parse tree vs AST
- DAG
- three-address code
- temporary variables
- TAC forms
- expression translation

Completed: Parts 1–20/31 | Next: Part 21 — Intermediate Representations, AST, DAG & Three-Address Code

Reply **NEXT** for **Part 21**.

USER

NEXT

ASSISTANT

Compiler Design — Part 21: Intermediate Representations, AST, DAG & Three-Address Code

Completed: Parts 1–20/31 | Current: Part 21 | Remaining: Parts 22–31

We now move from syntax-directed translation and runtime environments to:

$\boxed{\text{Intermediate Code Generation}}$

The compiler usually does **not** directly convert every source-language statement into final machine code immediately.

Instead:

$\boxed{\text{Source Program} \rightarrow \text{Intermediate Representation} \rightarrow \text{Target Code}}$

This intermediate form makes:

- optimization easier
- machine independence easier
- code generation cleaner

The most important GATE representation here is:

$\boxed{\text{Three-Address Code (TAC)}}$

1. What Is an Intermediate Representation?

CONCEPT

An **Intermediate Representation (IR)** is an internal form of the source program used between the compiler front end and back end.

Example:

```
c
a = b + c * d;
```

may become:

```
TEXT
t1 = c * d
t2 = b + t1
a  = t2
```

This is much easier for a compiler to optimize than raw source code.

WRITE THIS IN NOTES

Intermediate Representation (IR): A machine-independent or partially machine-independent internal representation of the source program used for analysis, optimization and code generation.

2. Why Do Compilers Use IR?

Suppose we directly build a compiler from every programming language to every machine.

If there are:

- m source languages
- n target machines

we could need roughly:

$m \times n$

different direct translation combinations.

With a common IR:

```
TEXT
Source languages
      !"
      IR
      !"
Target machines
```

we can separate:

- front ends
- back ends

This improves compiler portability and reuse.

Memory Trick

Front end speaks SOURCE.

Back end speaks MACHINE.

IR is the common language between them.

3. Advantages of Intermediate Representation

IR helps with:

- machine-independent optimization
- code generation
- portability
- separating front end and back end
- program analysis
- reuse of optimizers

For GATE, the strongest idea is:

$\boxed{\text{IR makes compiler optimization and retargeting easier}}$

4. Common Intermediate Representations

Important representations include:

1. Parse tree
2. Abstract Syntax Tree (AST)
3. Directed Acyclic Graph (DAG)
4. Three-Address Code (TAC)

We already saw parse trees earlier.

Now let's distinguish all four properly.

5. Parse Tree

A parse tree represents the **complete grammatical derivation** according to the CFG.

Example expression:

$a+b*c$

Using expression grammar:

$E \rightarrow E+T$

$T \rightarrow T * F$

etc., the parse tree may include:

- E

- T
- F
- terminals
- parentheses grammar structure

It contains a lot of grammatical detail.

6. Abstract Syntax Tree

CONCEPT

An **Abstract Syntax Tree (AST)** removes unnecessary grammar details and keeps only the essential hierarchical structure of the program.

For:

$\backslash$ [a+b*c]

AST:

TEXT



Multiplication is deeper, representing its higher precedence.

WRITE THIS IN NOTES

AST: A compact tree representation of the essential syntactic structure of a program, omitting unnecessary grammar-specific nodes.

7. Parse Tree vs AST

Consider:

TEXT

a + b

A parse tree may contain:

TEXT

E

T

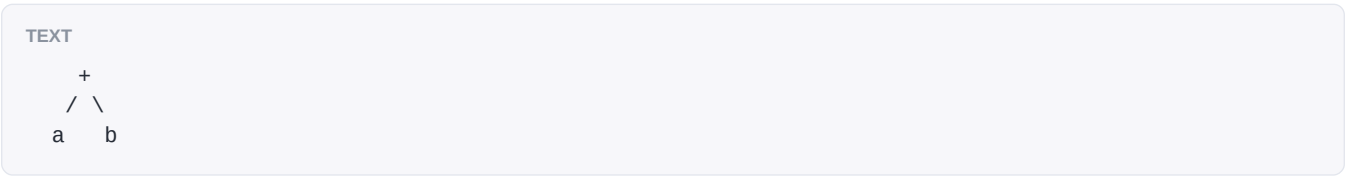
F

id

+

multiple grammar levels.

AST may simply be:



Thus:

$\boxed{\text{AST is usually more compact than parse tree}}$

8. Parse Tree vs AST — GATE Table

| Parse Tree | AST |
|---------------------------------|-----------------------------------|
| Contains detailed CFG structure | Keeps essential program structure |
| Includes many non-terminals | Removes unnecessary grammar nodes |
| Useful for syntax derivation | Useful for later compiler phases |
| Usually larger | Usually smaller |
| Closely tied to grammar | More abstract |

9. AST Leaves and Internal Nodes

For an arithmetic expression:

Leaves

typically represent:

- identifiers
- constants

Internal nodes

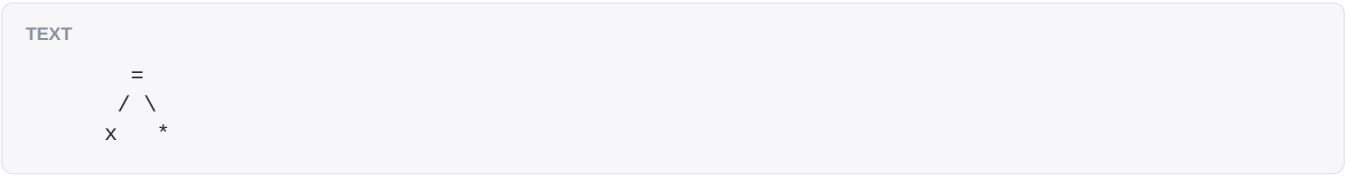
typically represent:

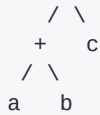
- operators
- statements
- language constructs

Example:

$x=(a+b)*c$

AST could be:





10. Syntax Tree and Operator Precedence

For:

$\backslash [a+b*c]$

correct tree:

TEXT



This corresponds to:

$\backslash [a+(b*c)]$

Not:

TEXT



which would mean:

$\backslash [(a+b)*c]$

So the tree structure captures precedence and associativity.

11. DAG — Directed Acyclic Graph

Now another important representation.

A **DAG** is like a syntax tree, except identical/common subexpressions can share the same node.

DAG means:

$\backslash \boxed{\text{Directed Acyclic Graph}} \backslash$

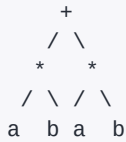
12. Why Use a DAG?

Consider:

$\backslash [ab+ab]$

An ordinary syntax tree may create two separate multiplication nodes:

TEXT



But:

$\backslash [a*b \backslash]$

is repeated.

A DAG can share it:

TEXT



Both operands of $+$ refer to the same $*$ node.

WRITE THIS IN NOTES

DAG: A syntax representation that can share nodes for identical subexpressions, making common subexpressions explicit.

13. Tree vs DAG

A tree requires:

Every non-root node has exactly one parent.

A DAG may allow:

A node to have multiple parents.

Therefore shared subexpressions are possible.

14. DAG Is Acyclic

The **A** in DAG means:

$\backslash \boxed{\text{Acyclic}}$

There is no directed cycle.

So you cannot start at a node, follow directed edges, and return to the same node.

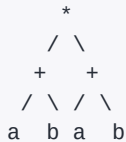
15. DAG and Common Subexpressions

Expression:

[(a+b)*(a+b)]

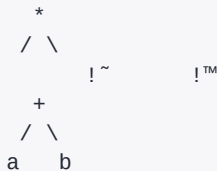
Tree:

TEXT



DAG:

TEXT



Thus:

[a+b]

can be calculated once and reused.

This connects directly to:

$\text{[Common Subexpression Elimination]}$

which we study later.

16. Important Caveat: Same Text "Always Same Computation

Suppose variables can be modified between two occurrences.

Example:

TEXT

```
x = a + b
a = 10
y = a + b
```

The two expressions:

[a+b]

are textually identical but do not necessarily compute the same value because `a` changed.

So common-subexpression sharing must respect data dependencies and definitions.

We'll study this in optimization.

17. Three-Address Code

Now the most important IR for GATE.

CONCEPT

Three-Address Code (TAC) consists of simple statements where the RHS typically contains at most one operator and the instruction involves at most three addresses.

Typical form:

$\boxed{x = y \text{ op } z}$

The three addresses are:

- destination `x`
- operand `y`
- operand `z`

WRITE THIS IN NOTES

Three-address code: A linear intermediate representation consisting of simple instructions containing at most three address references.

18. Why "Three Address"?

Example:

TEXT

`t1 = b * c`

Addresses:

1. `t1`
2. `b`
3. `c`

Hence:

$\boxed{\quad}$

But not every TAC instruction has exactly three addresses.

Some have:

- 2
- 1
- 0

The term means **at most three** in the standard conceptual instruction.

GATE TRAP

Every TAC instruction must contain exactly three addresses.

'L False.

$\boxed{\text{At most three}}$

19. Temporary Variables

Compilers introduce temporary variables such as:

TEXT

t1
t2
t3

to break complicated expressions into simple operations.

Example:

$a = b + c * d$

TAC:

TEXT

t1 = c * d
t2 = b + t1
a = t2

20. Correct Translation Order

For:

$a = b + c * d$

multiplication has higher precedence.

So first:

$c * d$

then:

$b + t1$

Therefore:

TEXT

t1 = c * d
t2 = b + t1
a = t2

21. GATE Shortcut — TAC Generation

Whenever you see a complex expression:

Step 1

Respect parentheses.

Step 2

Respect operator precedence.

Step 3

Respect associativity.

Step 4

Replace each intermediate operation with a temporary.

Example:

$\boxed{x = (a+b) * (c-d)}$

becomes:

TEXT

```
t1 = a + b
t2 = c - d
t3 = t1 * t2
x = t3
```

22. TAC for Unary Operation

Example:

$\boxed{x = -y}$

TAC:

TEXT

```
x = -y
```

This has fewer than three addresses.

General unary form:

$\boxed{x = op\ y}$

23. TAC for Copy Assignment

Example:

$\boxed{x = y}$

TAC:

TEXT

$x = y$

No operator needed.

This becomes important for:

$\boxed{\text{Copy Propagation}}$

later.

24. TAC for Array Access

Typical conceptual form:

TEXT

$x = y[i]$

meaning read array element.

Or:

TEXT

$x[i] = y$

meaning write array element.

The exact low-level translation may involve address arithmetic.

25. TAC for Address and Pointer Operations

Common conceptual TAC forms include:

TEXT

$x = \&y$

$x = *y$

$*x = y$

where:

- $\&y$ = address of y
- $*y$ = value stored at address y

These are useful when translating pointer-related constructs.

26. TAC for Jumps

Control flow also uses TAC instructions.

Examples:

TEXT

```
goto L1
```

and:

TEXT

```
if x < y goto L1
```

or:

TEXT

```
ifFalse x goto L2
```

These will become important in Part 23.

27. TAC for Procedure Calls

Common conceptual instructions:

TEXT

```
param x  
call p, n  
return y
```

where:

- `param x` prepares parameter
- `call p, n` calls `p` with `n` parameters
- `return y` returns a value

Exact IR syntax varies.

28. Common TAC Instruction Forms

You should recognize forms like:

TEXT

```
x = y op z  
x = op y  
x = y  
goto L  
if x relop y goto L  
if x goto L  
ifFalse x goto L  
param x  
call p, n  
return x  
x = y[i]  
x[i] = y  
x = &y  
x = *y  
*x = y
```

Don't memorize implementation syntax word-for-word. Understand the operation represented.

29. One Operator per TAC Instruction

Consider:

TEXT

```
x = a + b * c
```

This is generally **not** proper three-address decomposition as one instruction because RHS contains two operators:

- 
- 

Break it:

TEXT

```
t1 = b * c  
x = a + t1
```

30. Number of TAC Instructions for Expressions

A useful GATE pattern:

For a binary expression tree containing $\backslash(k\backslash)$ binary operator occurrences, evaluating the expression itself generally requires:

$\backslash[\boxed{k}\backslash]$

binary-operation TAC instructions.

If the final result also needs assignment to a variable, there may be one additional copy instruction.

Example

$\backslash[x=a+b*c-d \backslash]$

Operators:

- 
- 
- 

= 3 arithmetic operations.

TAC:

TEXT

```
t1 = b * c  
t2 = a + t1  
t3 = t2 - d  
x = t3
```

3 operation instructions + final assignment.

Sometimes compiler can directly use:

TEXT

```
x = t2 - d
```

and avoid the extra copy:

TEXT

```
t1 = b * c
t2 = a + t1
x = t2 - d
```

So exact temporary count can depend on representation/convention.

31. Minimum Temporaries vs Number of Operators

Do not blindly assume:

$\lceil \text{number of operators} \rceil = \lceil \text{number of temporaries} \rceil$

A compiler may reuse temporaries or store final operation directly into the destination.

GATE questions normally specify enough context if exact temporary count matters.

32. Example — Nested Expression

Translate:

$x = (a - b) + (c * d)$

Step 1

$a - b$

TEXT

```
t1 = a - b
```

Step 2

$c * d$

TEXT

```
t2 = c * d
```

Step 3

Add:

TEXT

```
t3 = t1 + t2
```

Step 4

Assign:

TEXT

$x = t3$

33. Associativity Matters

Expression:

$\backslash [a-b-c] \backslash$

Subtraction is normally left associative:

$\backslash [(a-b)-c] \backslash$

TAC:

TEXT

$t1 = a - b$
 $t2 = t1 - c$

Not:

TEXT

$t1 = b - c$
 $t2 = a - t1$

because that represents:

$\backslash [a-(b-c)] \backslash$

34. Parentheses Override Associativity

Expression:

$\backslash [a-(b-c)] \backslash$

TAC:

TEXT

$t1 = b - c$
 $t2 = a - t1$

So always inspect parentheses before using default precedence/associativity.

35. AST to TAC

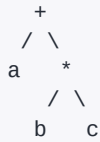
The AST makes TAC generation straightforward.

For:

$\backslash [a+b*c] \backslash$

AST:

TEXT



Use postorder-style processing:

Visit **b**, **c**, then *****

TEXT

```
t1 = b * c
```

Then **a**, **t1**, **+**

TEXT

```
t2 = a + t1
```

Thus:

$\boxed{\text{Postorder traversal of expression AST naturally supports TAC generation}}$

36. DAG to TAC

Suppose:

$(a+b)*(a+b)$

DAG contains one shared **a+b** node.

Efficient TAC:

TEXT

```
t1 = a + b
t2 = t1 * t1
```

Instead of computing **a+b** twice.

Thus DAG can directly support local common-subexpression elimination.

37. AST vs DAG — Master Difference

AST

Repeated expression can appear as repeated subtrees.

DAG

Repeated equivalent subexpression can share one node.

Therefore:

$\boxed{\text{DAG exposes common subexpressions}}$

38. Linear vs Tree IR

AST and DAG are graph/tree representations.

TAC is:

$\boxed{\text{Linear}}$

Example tree:

TEXT



Linear TAC:

TEXT

```
t1 = b * c
t2 = a + t1
```

Both represent the same essential computation differently.

39. High-Level vs Low-Level IR

IRs may exist at different abstraction levels.

High-level IR

Closer to source syntax.

Example:

- AST

Lower-level IR

Closer to machine operations.

Example:

- TAC

The exact classification depends on compiler design, but this intuition is useful.

40. Machine Independence

TAC usually avoids committing immediately to:

- physical registers
- exact machine instructions
- particular CPU opcodes

Example:

TEXT

```
t1 = a + b
```

can later become different target instructions on different machines.

This is why TAC is a useful intermediate form.

41. GATE-STYLE Example 1 — AST

Expression:

$\backslash [(a+b)*c]$

AST:

TEXT



Root is $*$, because the parenthesized addition is performed first and then multiplied.

42. GATE-STYLE Example 2 — TAC

Translate:

$\backslash [x=ab+cd]$

Correct TAC:

TEXT

```
t1 = a * b
t2 = c * d
t3 = t1 + t2
x = t3
```

43. GATE-STYLE Example 3 — Associativity

Translate:

$\backslash [x=a/b/c]$

assuming $/$ is left-associative.

$\backslash [(a/b)/c]$

TAC:

TEXT

```
t1 = a / b
t2 = t1 / c
x = t2
```

44. GATE-STYLE Example 4 — DAG

Expression:

$\lfloor ab+c+ab \rfloor$

If neither **a** nor **b** changes between occurrences, DAG can share:

$\lfloor a*b \rfloor$

So:

TEXT

```
t1 = a * b
t2 = t1 + c
t3 = t2 + t1
```

No second multiplication required.

45. GATE-STYLE Example 5 — Count Items

Expression:

$\lfloor a+b*c-d/e \rfloor$

Arithmetic operator occurrences:

- **+**
- *****
- **-**
- **/**

Total:

$\lfloor \boxed{4} \rfloor$

So expression evaluation needs four arithmetic operations.

Possible TAC:

TEXT

```
t1 = b * c
t2 = a + t1
t3 = d / e
t4 = t2 - t3
```

46. Syntax Tree vs Parse Tree Trap

Suppose grammar has:

$\lfloor E \rightarrow E+T \rfloor$

$\lfloor T \rightarrow id \rfloor$

The parse tree includes grammar variables such as E and T.

AST may simply contain:

TEXT

```
  +
 / \
a   b
```

Therefore:

$\boxed{\text{AST is not just another name for full parse tree}}$

47. DAG Trap

A DAG is not merely "a tree drawn differently."

The key additional property is:

$\boxed{\text{node sharing}}$

A DAG node can have multiple incoming edges.

A tree node cannot.

48. TAC Trap

Three-address code does **not** mean:

exactly three variables per source statement.

It means individual instructions are decomposed into simple forms containing at most three address references.

49. Address Means More Than Memory Address

In the term "three-address code," an address may denote:

- variable name
- temporary
- constant
- symbolic location

So don't interpret "address" only as a physical memory address.

50. Constant as an Operand

Example:

TEXT

```
t1 = x + 5
```

Here **5** is an operand in TAC.

It's fine even though it is a literal constant, not a named variable.

51. TAC and Optimization

TAC makes patterns obvious.

Example:

```
TEXT
t1 = 2 * 10
x  = t1 + 0
```

Optimizer can transform:

```
TEXT
t1 = 20
x  = t1
```

and perhaps further:

```
TEXT
x = 20
```

So TAC is convenient for:

- constant folding
- copy propagation
- dead-code elimination
- CSE
- data-flow analysis

These are later parts.

52. GATE TRAPS

Trap 1

AST contains more grammar detail than parse tree.

'LFalse.

Parse tree usually contains more grammatical detail.

Trap 2

A DAG cannot share common subexpressions.

'LFalse.

That's one of its main advantages.

Trap 3

Every node in a DAG has at most one parent.

'LFalse.

That's a tree property.

Trap 4

Three-address code requires exactly three addresses.

'LFalse.

$\boxed{\text{At most three}}$

Trap 5

$x = a + b * c$ is already decomposed TAC.

'LGenerally no.

Break:

TEXT

```
t1 = b * c
x  = a + t1
```

Trap 6

Operator precedence can be ignored during TAC generation.

'LFalse.

The IR must preserve source semantics.

Trap 7

Repeated textual expressions are always common subexpressions.

'LFalse.

Operands may have been redefined between occurrences.

Trap 8

TAC commits the compiler to one CPU architecture.

'LUsually false.

It is commonly machine-independent or machine-neutral enough for optimization.

Trap 9

AST and DAG are necessarily linear representations.

'LFalse.

TAC is linear; AST/DAG are structural graphs.

Trap 10

A temporary variable must correspond to a source-language variable.

'LFalse.

Compiler generates temporaries internally.

53. MINI PRACTICE — PART 21

Try before checking the solutions.

Q1 — Easy

Which representation generally removes unnecessary grammar nodes from a parse tree?

A. Symbol table B. AST C. Activation record D. FIRST set

Q2 — Easy

What is the main difference between an AST and a DAG?

A. DAG cannot contain operators B. DAG can share common subexpressions C. AST must have cycles D. DAG contains parsing-table states

Q3 — Medium

Generate TAC for:

$\backslash [x = (a + b) * d \]$

Q4 — Medium

For:

$\backslash [a - b - c \]$

assuming normal left associativity, which operation happens first?

A. $\backslash (b - c)$ B. $\backslash (a - b)$

Q5 — Tricky

Consider:

TEXT

```
t1 = a + b
a  = 10
t2 = a + b
```

Can the second `a+b` always be replaced by `t1`?

A. Yes B. No

SOLUTIONS

Q1

' B — AST

AST removes unnecessary grammar-specific nodes.

Q2

' B

A DAG can let multiple parents refer to a shared subexpression node.

Q3

$\backslash [x = (a+b)*d]$

TAC:

TEXT

```
t1 = a + b
t2 = t1 * d
x = t2
```

Or with final destination optimization:

TEXT

```
t1 = a + b
x = t1 * d
```

Both preserve the computation; exact convention may vary.

Q4

' B

Subtraction is left associative:

$\backslash [(a-b)-c]$

So:

$\backslash [a-b]$

first.

Q5

' B — No

Because:

TEXT

a = 10

changes an operand.

The second `a+b` may have a different value.

So the first result cannot automatically be reused.

WRITE THIS IN NOTES — PART 21 MASTER SHEET

Intermediate Representation

$\boxed{\text{Source} \rightarrow \text{IR} \rightarrow \text{Target}}$

Purpose:

- optimization
 - portability
 - cleaner code generation
-

Parse Tree

$\boxed{\text{Full grammatical structure}}$

AST

$\boxed{\text{Essential syntax only}}$

Usually smaller than parse tree.

DAG

$\boxed{\text{AST-like graph + shared common subexpressions}}$

Useful for:

$\boxed{\text{CSE}}$

Three-Address Code

Typical:

$\boxed{x = y \text{ op } z}$

At most three address references.

Complex:

$\boxed{a + b * c}$

becomes:

TEXT

```
t1 = b * c
t2 = a + t1
```

30-SECOND REVISION

IR

Compiler's internal middle language.

Parse tree

Full grammar.

AST

Remove grammar clutter.

DAG

Share repeated computation.

TAC

Break complex statements into simple operations.

Example

$\boxed{x = (a+b) * (c-d)}$

TEXT

```
t1 = a + b
t2 = c - d
t3 = t1 * t2
x = t3
```

Critical memories

TEXT

```
A S T ! '   t r e e
D A G ! '   s h a r e d   n o d e s
T A C ! '   l i n e a r   i n s t r u c t i o n s
```

and:

$\boxed{\text{Three-address means at most three, not exactly three}}$

Next we will learn the three classic ways of storing TAC:

$\boxed{\text{Quadruples}}$

$\boxed{\text{Triples}}$

$\boxed{\text{Indirect Triples}}$

This is a very common GATE comparison/counting topic.

Completed: Parts 1–21/31 | Next: Part 22 — Quadruples, Triples & Indirect Triples

Reply **NEXT** for **Part 22**.

USER

NEXT

ASSISTANT

Compiler Design — Part 22: Quadruples, Triples & Indirect Triples

Completed: Parts 1–21/31 | Current: Part 22 | Remaining: Parts 23–31

In Part 21, we learned **Three-Address Code (TAC)** such as:

TEXT

```
t1 = b * c
t2 = a + t1
x  = t2
```

Now the question is:

How does the compiler store these TAC instructions internally?

Three classic representations are:

$\boxed{\text{Quadruples}}$

$\boxed{\text{Triples}}$

$\boxed{\text{Indirect Triples}}$

This is a common GATE comparison topic.

1. Quick Recall — Three-Address Code

Typical TAC:

$x = y \text{ op } z$

Example:

TEXT

```
t1 = a * b
```

contains:

- operator `*`
- operand `a`
- operand `b`
- result `t1`

Notice there are actually **four pieces of information**.

That leads naturally to the quadruple representation.

2. Quadruples

CONCEPT

A **quadruple** stores each TAC instruction using four fields:

$\boxed{(op, \text{arg1}, \text{arg2}, \text{result})}$

The four fields are:

1. operator
2. first argument
3. second argument
4. result

WRITE THIS IN NOTES

$\boxed{\text{Quadruple} = (op, \text{arg1}, \text{arg2}, \text{result})}$

Memory Trick

Quad = 4 fields

TEXT

Operator
Argument 1
Argument 2
Result

3. Simple Quadruple Example

TAC:

TEXT

`t1 = a + b`

Quadruple:

$\boxed{(+, a, b, t1)}$

4. Another Example

TAC:

```
TEXT
t2 = t1 * c
```

Quadruple:

$\boxed{(*, t1, c, t2)}$

5. Complete Example

Expression:

$\boxed{x = a + b * c}$

TAC:

```
TEXT
t1 = b * c
t2 = a + t1
x = t2
```

Quadruples:

| No. | op | arg1 | arg2 | result |
|-----|----|------|------|--------|
| 0 | * | b | c | t1 |
| 1 | + | a | t1 | t2 |
| 2 | = | t2 | — | x |

6. Assignment in Quadruples

For:

```
TEXT
x = y
```

a typical quadruple is:

$\boxed{ (=, y, -, x) }$

The second argument is unused.

Different books may use:

- blank
- 
- 

for an unused field.

The meaning is what matters.

7. Unary Operation in Quadruples

For:

```
TEXT
x = -y
```

quadruple:

$\boxed{(\text{uminus}, y, -, x)}$

or conceptually:

$\boxed{(-, y, -, x)}$

if unary and binary minus are distinguishable from context.

8. Important Advantage of Quadruples

Every intermediate result gets an explicit name:

```
TEXT
t1
t2
t3
```

Therefore if instructions are moved during optimization, references to intermediate results remain relatively easy to maintain.

Example:

```
TEXT
t1 = a + b
t2 = c * d
t3 = t1 - t2
```

`t3` refers explicitly to `t1` and `t2`, not to instruction positions.

This makes quadruples convenient for:

$\boxed{\text{Code motion and optimization}}$

9. Main Disadvantage of Quadruples

They introduce many compiler-generated temporary names:

```
TEXT
t1
t2
t3
```

...

So compared with triples, quadruples require explicit temporary-result fields.

WRITE THIS IN NOTES

Quadruple Advantage

$\boxed{\text{Easy to rearrange instructions}}$

because results have explicit names.

Quadruple Disadvantage

$\boxed{\text{Many temporary names}}$

10. Triples

Now remove the explicit `result` field.

A **triple** generally has three fields:

$\boxed{(op, \text{arg1}, \text{arg2})}$

So how do we refer to an intermediate result?

By the **position/index of the instruction that produced it**.

WRITE THIS IN NOTES

$\boxed{\text{Triple} = (op, \text{arg1}, \text{arg2})}$

Intermediate results are referred to by:

$\boxed{\text{instruction number} / \text{position}}$

Memory Trick

Quadruple names results.

Triple numbers results.

11. Triple Example

TAC:

TEXT

```
t1 = b * c
t2 = a + t1
```

$x = t_2$

Triples can be represented as:

| Index | op | arg1 | arg2 |
|-------|----|------|------|
| 0 | * | b | c |
| 1 | + | a | (0) |
| 2 | = | (1) | x |

Here:

$\boxed{(0)}$

means:

result produced by instruction 0.

And:

$\boxed{(1)}$

means:

result produced by instruction 1.

12. Important Triple Idea

There is no explicit:

TEXT

t1

t2

Instead:

TEXT

result of instruction 0

result of instruction 1

act as the temporary values.

So:

$\boxed{\text{Instruction position itself identifies intermediate result}}$

13. Quadruple vs Triple Side by Side

Expression:

$a + b * c$

Quadruples

TEXT

```
0: (*, b, c, t1)
1: (+, a, t1, t2)
```

Triples

TEXT

```
0: (*, b, c)
1: (+, a, (0))
```

The result of instruction 0 is implicitly referenced as `(0)`.

14. Main Advantage of Triples

Triples do not require explicit temporary names such as:

TEXT

```
t1, t2, t3
```

Therefore representation can be more compact.

$\boxed{\text{No explicit result-temp field}}$

15. Main Disadvantage of Triples

Suppose instruction 0 produces a value referenced as:

$\boxed{(0)}$

If an optimization moves instruction 0 to another position, references based on instruction positions may need to be updated.

Thus code movement/reordering is more awkward.

WRITE THIS IN NOTES

Triple Advantage

$\boxed{\text{Fewer explicit temporary variables}}$

Triple Disadvantage

$\boxed{\text{Instruction reordering is inconvenient}}$

because results are identified by positions.

16. Example of the Reordering Problem

Original triples:

TEXT

0: (*, a, b)
1: (+, (0), c)

Instruction 1 depends on the result at position 0.

Suppose optimization rearranges instructions.

If position numbers change, the reference:

TEXT

(0)

may have to change as well.

Quadruples avoid much of this because an instruction can keep referring to:

TEXT

t1

regardless of where the instruction producing **t1** is located.

17. Indirect Triples

Indirect triples are designed to reduce this problem.

Instead of directly using the physical order of triples as the execution order, we maintain a separate **list of pointers** to triples.

So there are two structures:

1. Triple table

Stores the actual triples.

2. Pointer list

Specifies the order in which triples are considered/executed.

WRITE THIS IN NOTES

Indirect triples: Triples plus a separate list of pointers to triple entries, allowing instruction order to be changed by rearranging pointers rather than moving the triples themselves.

18. Basic Indirect Triple Structure

Suppose triple table is:

| Triple index | op | arg1 | arg2 |
|--------------|----|------|------|
| 0 | * | b | c |

| Triple index | op | arg1 | arg2 |
|--------------|----|------|------|
| 1 | + | a | (0) |
| 2 | = | (1) | x |

Then pointer table might initially be:

| Execution position | Points to triple |
|--------------------|------------------|
| 0 | 0 |
| 1 | 1 |
| 2 | 2 |

Conceptually:

TEXT

Pointer list

!"

```
0 !' Triple 0
1 !' Triple 1
2 !' Triple 2
```

19. Why Is This Helpful?

Suppose independent instructions should be reordered.

Instead of physically moving triple records and disturbing their indices, we can rearrange the pointer list.

That makes code motion easier than with ordinary triples.

Memory Trick

Indirect Triple = Triple + Pointer Table

20. Quadruple vs Triple vs Indirect Triple

| Feature | Quadruple | Triple | Indirect Triple |
|---------------------------------|---------------------|-------------------|--|
| Main fields | op,arg1,arg2,result | op,arg1,arg2 | triples + pointer list |
| Explicit result temp | Yes | No | No explicit temp like quadruple |
| Intermediate result referred by | name (t1) | instruction index | triple reference via pointer structure |
| Reordering instructions | Easier | Harder | Easier than triples |
| Extra pointer table | No | No | Yes |

| Feature | Quadruple | Triple | Indirect Triple |
|----------------|---------------|---------|----------------------------------|
| Representation | More explicit | Compact | Triple flexibility + indirection |

21. The Most Important Exam Comparison

Quadruple

Result has a name.

Triple

Result has a position.

Indirect Triple

Position/order is handled through pointers.

22. Full Worked Example

Convert:

$\backslash [x = (a+b) * (c-d)]$

into all three representations.

23. Step 1 — Generate TAC

$\backslash [a+b]$

TEXT

t1 = a + b

$\backslash [c-d]$

TEXT

t2 = c - d

Multiply:

TEXT

t3 = t1 * t2

Assign:

TEXT

x = t3

Complete TAC:

TEXT

t1 = a + b
t2 = c - d
t3 = t1 * t2
x = t3

24. Quadruple Representation

| No. | op | arg1 | arg2 | result |
|-----|----|------|------|--------|
| 0 | + | a | b | t1 |
| 1 | - | c | d | t2 |
| 2 | * | t1 | t2 | t3 |
| 3 | = | t3 | — | x |

25. Triple Representation

| Index | op | arg1 | arg2 |
|-------|----|------|------|
| 0 | + | a | b |
| 1 | - | c | d |
| 2 | * | (0) | (1) |
| 3 | = | (2) | x |

Notice:

TEXT

(0) = result of a+b
(1) = result of c-d
(2) = result of multiplication

26. Indirect Triple Representation

Triple table remains:

| Triple | op | arg1 | arg2 |
|--------|----|------|------|
| 0 | + | a | b |

| Triple | op | arg1 | arg2 |
|--------|----|------|------|
| 1 | - | c | d |
| 2 | * | (0) | (1) |
| 3 | = | (2) | x |

Pointer list:

| Position | Pointer |
|----------|---------|
| 0 | 0 |
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |

So execution order initially is:

TEXT

```
Triple 0
Triple 1
Triple 2
Triple 3
```

27. Reordering Independent Instructions

Here:

TEXT

```
t1 = a + b
t2 = c - d
```

are independent of each other.

In an indirect-triple representation, their order could be represented by pointer list:

TEXT

```
1
0
2
3
```

rather than physically moving the triple entries.

But of course dependencies must still be respected:

- triple 2 needs results of triples 0 and 1
- triple 3 needs result of triple 2

So not every arbitrary order is valid.

28. GATE Trap — Indirect Triples Remove Dependencies

'L False.

Indirection makes **reordering representation easier**.

It does not allow instructions to violate data dependencies.

29. Another Worked Example

Expression:

$\backslash [x = a * b + c \backslash]$

TAC:

TEXT

```
t1 = a * b
t2 = t1 + c
x  = t2
```

Quadruples

TEXT

```
(*, a, b, t1)
(+, t1, c, t2)
(=, t2, -, x)
```

Triples

TEXT

```
0: (*, a, b)
1: (+, (0), c)
2: (=, (1), x)
```

30. Number of Entries

Suppose TAC has $\backslash (n \backslash)$ instructions.

Quadruple representation

Usually:

$\backslash [\boxed{n \text{ quadruple entries}} \backslash]$

Triple representation

Usually:

$\backslash [\boxed{n \text{ triple entries}} \backslash]$

Indirect triples

Usually:

$\backslash [\boxed{n \text{ triples}} + n \text{ pointer entries} \backslash]$

assuming one pointer per represented instruction.

31. Don't Confuse "Fields" With "Instructions"

Quadruple does **not** mean four TAC instructions.

It means:

$\boxed{4 \text{ fields per record}}$

Triple means:

$\boxed{3 \text{ main fields per record}}$

32. Why Is Assignment Written Differently in Triple Tables?

For ordinary operation:

TEXT

t1 = a + b

the result is implicit in the row number.

But assignment:

TEXT

x = t1

needs to say that the value of the earlier expression goes to named destination **x**.

So one common triple notation is:

$\boxed{(\text{=}, 0, x)}$

Exact placement may differ among textbooks, but the logical meaning is:

assign result of triple 0 to x.

For GATE, follow the notation given in the question if it differs.

33. Array Example — Quadruple

TAC:

TEXT

t1 = a[i]

Possible quadruple:

$\boxed{([], a, i, t1)}$

Similarly:

TEXT

```
a[i] = x
```

could use a representation indicating array store.

Exact operator names vary.

Do not memorize punctuation; understand the fields.

34. Conditional Jump — Quadruple

For:

TEXT

```
if x < y goto L1
```

one possible quadruple representation:

$\boxed{\text{if } x < y, \text{ goto } L1}$

or it may be split into comparison + branch depending on IR design.

Again, exact syntax is implementation-specific.

35. Unconditional Jump

For:

TEXT

```
goto L1
```

quadruple might be:

$\boxed{\text{goto } -, -, L1}$

The unused argument fields are blank.

36. Pointer/reference to Results in Triples

For triple:

TEXT

```
0: (*, a, b)
1: (+, (0), c)
```

(0) is not the integer constant zero.

It means:

$\boxed{\text{result of triple number 0}}$

This is a classic reading trap.

37. Quadruples and Code Motion

Suppose:

TEXT

```
0: t1 = a + b
1: t2 = c + d
2: t3 = t1 * t2
```

If instructions 0 and 1 are exchanged:

TEXT

```
0: t2 = c + d
1: t1 = a + b
2: t3 = t1 * t2
```

references:

TEXT

```
t1
t2
```

still make sense.

That is why explicit result names help.

38. Triples and Code Motion

Original:

TEXT

```
0: (+, a, b)
1: (+, c, d)
2: (*, (0), (1))
```

If rows 0 and 1 are physically exchanged, row 2's positional references may need updating depending on how indices are managed.

Hence:

$\boxed{\text{ordinary triples make code motion less convenient}}$

39. Indirect Triples Solve This Conceptually

Keep the triples fixed.

Change only the pointer/order list.

So references to underlying triples stay stable while the logical execution sequence can be rearranged.

This is the key reason indirect triples exist.

40. Which Representation Uses More Explicit Temporary Names?

Among quadruples and triples:

$\boxed{\text{Quadruples}}$

because results are explicitly named:

TEXT

t1
t2
...

Triples use instruction positions instead.

41. Which Is More Convenient for Optimization?

This requires nuance.

Quadruples

Very convenient for code motion because explicit result names survive reordering.

Ordinary triples

Less convenient for code motion because result references depend on positions.

Indirect triples

Improve this by reordering pointers rather than actual triple records.

Thus in many textbook comparisons:

$\boxed{\text{Quadruples and indirect triples are more convenient for rearrangement than ordinary triples}}$

42. GATE-STYLE Example 1

TAC:

TEXT

t1 = a + b

Which is the correct quadruple?

A. (t1, a, +, b) B. (+, a, b, t1) C. (a, b, t1, +) D. (+, t1, a, b)

' B

43. GATE-STYLE Example 2

Triple:

TEXT

0: (*, a, b)
1: (+, (0), c)

What does (0) mean?

$\boxed{\text{Result of triple 0}}$

44. GATE-STYLE Example 3

Which representation has no explicit result field?

A. Quadruple B. Triple C. Both D. Neither

$\boxed{B}$

45. GATE-STYLE Example 4

Which representation adds a separate pointer list to facilitate code rearrangement?

$\boxed{\text{Indirect triples}}$

46. GATE-STYLE Example 5

Expression:

$x = a + b * c$

How many arithmetic-operation records are needed before the final assignment?

Operations:

- *
- +

Therefore:

$\boxed{2}$

If the explicit assignment $x = \dots$ is represented separately, total records may be 3.

Always check whether the question counts the final assignment.

47. GATE-STYLE Example 6

Which representation usually makes instruction rearrangement easiest because intermediate results have symbolic names?

$\boxed{\text{Quadruples}}$

48. GATE TRAPS

Trap 1

Quadruple means four operands.

'L False.

Fields are:

TEXT

op
arg1
arg2
result

Trap 2

Triple means every TAC instruction contains exactly three source variables.

'L False.

Triple refers to representation fields.

Trap 3

Triples explicitly store temporary result names.

'L False.

Results are referred to by instruction position/index.

Trap 4

Quadruples identify intermediate results only by instruction position.

'L False.

They explicitly name the result.

Trap 5

Indirect triples are completely unrelated to triples.

'L False.

They are:

$\boxed{\text{Triples + indirection/pointer list}}$

Trap 6

Instruction reordering is generally easier in ordinary triples than quadruples.

'L False.

Triples' positional references make reordering awkward.

Trap 7

Indirect triples allow dependent instructions to be executed in any order.

'L False.

Dependencies still have to be respected.

Trap 8

A triple reference (3) necessarily means numeric constant 3.

'LFalse.

In triple notation it usually means result of instruction 3.

Trap 9

Every TAC operator requires two operands.

'LFalse.

Examples:

TEXT

```
x = -y
goto L
return x
```

have different operand counts.

Trap 10

Exact punctuation for quadruples/triples is universal.

'LFalse.

Textbooks may vary.

Focus on:

- operator
 - operands
 - result/reference semantics.
-

49. MINI PRACTICE — PART 22

Try before checking.

Q1 — Easy

A quadruple has which fields?

A. op, arg1, arg2 B. op, arg1, arg2, result C. arg1, arg2, result only D. state, terminal, non-terminal, action

Q2 — Easy

Triples normally identify intermediate results using:

A. FOLLOW sets B. Temporary variable names only C. Instruction positions D. Activation records

Q3 — Medium

Convert:

$x = (a - b) * c$

into quadruples.

Q4 — Medium

Convert the same expression into triples.

Q5 — Tricky

Why are indirect triples preferable to ordinary triples when code must be rearranged?

A. They eliminate all temporary computations. B. They use a pointer list whose order can be changed without physically reordering the triple records. C. They remove data dependencies. D. They contain no operators.

SOLUTIONS

Q1

' B

$\boxed{(op, arg1, arg2, result)}$

Q2

' C — Instruction positions

Example:

(0)

means result of instruction 0.

Q3

Expression:

$x = (a - b) * c$

TAC:

TEXT

```
t1 = a - b
t2 = t1 * c
x = t2
```

Quadruples:

| No. | op | arg1 | arg2 | result |
|-----|----|------|------|--------|
| 0 | - | a | b | t1 |
| 1 | * | t1 | c | t2 |
| 2 | = | t2 | — | x |

Q4

Triples:

| Index | op | arg1 | arg2 |
|-------|----|------|------|
| 0 | - | a | b |
| 1 | * | (0) | c |
| 2 | = | (1) | x |

Q5

' B

Indirect triples separate:

- triple storage
- logical instruction ordering

So pointer order can be rearranged more conveniently.

WRITE THIS IN NOTES — PART 22 MASTER SHEET

Quadruple

$\boxed{\{op, arg1, arg2, result\}}$

Example:

```
TEXT
t1 = a + b
```

becomes:

$\boxed{\{+, a, b, t1\}}$

Advantage

$\boxed{\text{Easy code rearrangement}}$

Disadvantage

$\boxed{\text{Explicit temporary names}}$

Triple

$\boxed{\text{(op, arg1, arg2)}}$

No explicit result field.

Intermediate result:

$\boxed{\text{instruction index}}$

Example:

TEXT

0: (*, a, b)
1: (+, (0), c)

Advantage

$\boxed{\text{Fewer explicit temporaries}}$

Disadvantage

$\boxed{\text{Code motion/reordering harder}}$

Indirect Triple

$\boxed{\text{Triples + pointer list}}$

Main advantage:

$\boxed{\text{Reorder pointers instead of moving triple records}}$

30-SECOND REVISION

Quadruple

4 fields

TEXT

op | arg1 | arg2 | result

Triple

3 fields

TEXT

op | arg1 | arg2

Result is:

$\boxed{\text{row/index}}$

Indirect Triple

Triple + pointer table

Best memory

TEXT

```
Q u a d r u p l e ! '   N A M E   t h e   r e s u l t
T r i p l e   ! '   N U M B E R   t h e   r e s u l t
I n d i r e c t   ! '   P O I N T   t o   t h e   t r i p l e s
```

Code movement

TEXT

```
Q u a d r u p l e ! '   e a s y
T r i p l e ! '   a w k w a r d
I n d i r e c t   t r i p l e ! '   i m p r o v e d   t h r o u g h   p o i n t e r   o r d e r i n g
```

Next comes another important intermediate-code topic:

Boolean Expressions & Control Flow

We will cover:

- relational and Boolean expressions
- short-circuit evaluation
- conditional/unconditional jumps
- translating `if`, `if-else`, and loops
- true/false lists
- **backpatching**

Completed: Parts 1–22/31 | Next: Part 23 — Boolean Expressions, Control Flow & Backpatching

Reply **NEXT** for **Part 23**.

USER

NEXT

ASSISTANT

Compiler Design — Part 23: Boolean Expressions, Control Flow & Backpatching

Completed: Parts 1–22/31 | Current: Part 23 | Remaining: Parts 24–31

So far, our TAC mostly handled arithmetic expressions:

TEXT

```
t1 = b * c
t2 = a + t1
```

But real programs also contain:

```
c
if (a < b)
while (x != 0)
if (p && q)
```

These require the compiler to generate **control-flow instructions** such as conditional and unconditional jumps.

This part covers:

- Boolean expressions
- relational expressions
- short-circuit evaluation
- `if`
- `if-else`
- loops
- jump targets
- backpatching

1. Boolean Expressions in Intermediate Code

A Boolean expression produces a logical condition such as:

`\[ true \]`

or:

`\[ false \]`

Examples:

```
c
a < b
x == y
p && q
p || q
!p
```

A compiler can translate Boolean expressions in two major styles:

1. Compute a Boolean value such as `0` or `1`.
2. Generate **control flow directly** using jumps.

For conditions in `if` and loops, the second method is often more natural.

2. Numerical Method

Suppose:

```
C
x = (a < b);
```

The compiler could conceptually generate:

```
TEXT
if a < b goto L1
t1 = 0
goto L2
L1: t1 = 1
L2: x = t1
```

Here the Boolean expression actually produces a stored value.

3. Control-Flow Method

For:

```
C
if (a < b)
    x = 1;
```

there may be no need to create:

```
TEXT
t1 = true
```

Instead:

```
TEXT
if a < b goto Ltrue
goto Lend

Ltrue:
x = 1

Lend:
```

The condition directly controls execution.

This is often called:

`\[ \boxed{\text{Jumping code}} \]`

WRITE THIS IN NOTES

Boolean expressions can be translated either by computing explicit Boolean values or by generating jumps to true/false destinations.

4. Conditional Jump TAC

Common conceptual TAC forms:

TEXT

```
if x < y goto L1
```

or:

TEXT

```
if x goto L1
```

or:

TEXT

```
ifFalse x goto L2
```

Exact syntax differs between textbooks.

The idea is always:

Test a condition and change control flow if appropriate.

5. Unconditional Jump

An unconditional jump:

TEXT

```
goto L1
```

always transfers control to:

TEXT

```
L1
```

without testing a condition.

6. Relational Expressions

Common relational operators:

$\vee[< \vee]$

$\vee[\leq \vee]$

$\vee[$

$\vee]$

$\vee[\geq \vee]$

$\text{[} == \text{]}$

$\text{[} \neq \text{]}$

A relational expression can be translated as:

TEXT

```
if a relop b goto Ltrue
goto Lfalse
```

Example:

$\text{[} a < b \text{]}$

becomes:

TEXT

```
if a < b goto Ltrue
goto Lfalse
```

7. Translation of a Simple `if`

Source:

C

```
if (a < b)
    x = y + z;
```

Possible TAC:

TEXT

```
if a < b goto L1
goto L2
```

```
L1:
t1 = y + z
x = t1
```

```
L2:
```

8. Better Compact Form

We may invert the condition:

TEXT

```
ifFalse a < b goto L2
t1 = y + z
x = t1
```

```
L2:
```

This avoids one unconditional jump.

Both forms represent the same logical structure.

9. Translation of `if-else`

Source:

```
c
if (a < b)
    x = 1;
else
    x = 2;
```

Standard control flow:

```
TEXT
if a < b goto L1
goto L2

L1:
x = 1
goto L3

L2:
x = 2

L3:
```

10. Why Do We Need the Jump After the `then` Part?

Consider:

```
TEXT
L1:
x = 1
goto L3
```

Without:

```
TEXT
goto L3
```

execution would fall through into:

```
TEXT
L2:
x = 2
```

Then both branches would execute.

Therefore:

$\boxed{\text{Then branch must skip the else branch}}$

unless layout uses an equivalent inverted arrangement.

Memory Trick

For `if-else`:

TEXT

Condition

!™

!~

Then Else

\

/

Next

Both branches eventually join at a common continuation point.

11. Translation of `while`

Source:

```
C
while (a < b) {
    a = a + 1;
}
```

Possible TAC:

TEXT

L1:

if a < b goto L2

goto L3

L2:

a = a + 1

goto L1

L3:

12. Loop Structure

Every typical `while` translation has three conceptual areas:

TEXT

Ltest:

test condition

false !' exit

Lbody:

execute body

goto Ltest

Lexit:

WRITE THIS IN NOTES

For:

```
C
while (B)
    S;
```

conceptual flow:

```
TEXT
Lbegin:
    if B goto Lbody
    goto Lexit

Lbody:
    code(S)
    goto Lbegin

Lexit:
```

13. Translation of do-while

Source:

```
C
do {
    x = x + 1;
} while (x < 10);
```

Possible flow:

```
TEXT
L1:
x = x + 1
if x < 10 goto L1
```

The key difference:

Body executes at least once.

14. Boolean Operators

Common Boolean operators:

- AND:

\[&& \]

- OR:

\[

\]

- NOT:

[!]

The compiler must preserve their semantics, especially:

[\boxed{\text{Short-circuit evaluation}}]

15. Short-Circuit Evaluation

CONCEPT

With short-circuit evaluation, part of a Boolean expression may not be evaluated if the final result is already known.

Example:

```
C
A && B
```

If A is false:

[A and B=false]

regardless of B.

Therefore B need not be evaluated.

WRITE THIS IN NOTES

Short-circuit evaluation: Evaluate Boolean operands only as far as necessary to determine the final truth value.

16. AND Short-Circuit Rule

For:

[A and B]

If A is false

Immediately:

[\boxed{false}]

Do not evaluate B.

If A is true

Need to evaluate B.

So:

```
TEXT
A  f a l s e  ! '   f a l s e   d e s t i n a t i o n
A  t r u e    ! '   e v a l u a t e   B
```

Memory Trick

AND

False stops.

17. OR Short-Circuit Rule

For:

$\backslash [A \text{ or } B \backslash]$

If A is true

Immediately:

$\backslash [\boxed{\text{true}} \backslash]$

Do not evaluate B.

If A is false

Need B.

So:

TEXT

```
A   t r u e   ! '   t r u e   d e s t i n a t i o n
A   f a l s e   ! '   e v a l u a t e   B
```

Memory Trick

OR

True stops.

18. NOT Rule

For:

$\backslash [!A \backslash]$

true and false destinations simply exchange.

If A is true:

$\backslash [!A=\text{false} \backslash]$

If A is false:

$\backslash [!A=\text{true} \backslash]$

Thus:

$\backslash [\boxed{\text{NOT swaps TRUE and FALSE exits}} \backslash]$

19. Why Short-Circuit Evaluation Matters

It is not just an optimization.

It may affect program correctness.

Example:

```
C
x != 0 && y / x > 2
```

If:

$\neg [x=0]$

the second expression:

```
TEXT
y / x
```

must not be evaluated.

Short-circuit AND stops after:

```
TEXT
x != 0
```

is false.

This prevents division by zero.

20. Another Example

```
C
p != NULL && p->value > 10
```

If `p == NULL`, evaluating:

```
TEXT
p->value
```

could be invalid.

Short-circuit AND prevents it.

Therefore:

$\boxed{\text{Short-circuit semantics must be preserved by generated code}}$

21. TAC for AND

Condition:

```
c
if (a < b && c < d)
    x = 1;
```

Possible TAC:

```
TEXT
if a < b goto L1
goto Lend

L1:
if c < d goto L2
goto Lend

L2:
x = 1

Lend:
```

Notice:

- if first condition false !' skip second
- second evaluated only if first true

22. TAC for OR

Condition:

```
c
if (a < b || c < d)
    x = 1;
```

Possible TAC:

```
TEXT
if a < b goto Ltrue
goto L1

L1:
if c < d goto Ltrue
goto Lend

Ltrue:
x = 1

Lend:
```

If first condition is true, second is skipped.

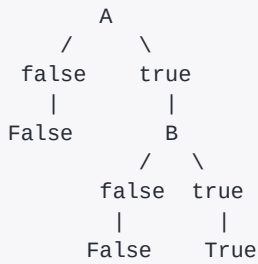
23. Boolean Expression as a Flow Graph

For:

$\neg [A \text{ and } B]$

think:

TEXT

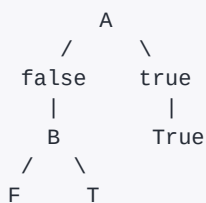


For:

$\lceil A \text{ or } B \rceil$

think:

TEXT



This visual is extremely useful.

24. The Problem With Unknown Jump Targets

While generating code in one pass, the compiler may know:

"I need a jump to the end of this if."

But it may not yet know the instruction number of that end.

Example:

TEXT

```
1: if a < b goto ____
2: goto ____
```

Where should these jumps go?

Their target instructions may be generated later.

This is where:

$\lceil \boxed{\text{Backpatching}} \rceil$

is used.

25. What Is Backpatching?

CONCEPT

Backpatching is a technique where jump instructions are generated with incomplete/unknown target addresses, and those targets are filled in later when the correct destination becomes known.

WRITE THIS IN NOTES

Backpatching: Filling in previously unknown jump targets after their destination addresses become known.

Memory Trick

Backpatch = Leave blank now, fill later.

26. Example

Generate:

TEXT

```
100: if a < b goto _  
101: goto _
```

Later we discover:

TEXT

```
true target = 120  
false target = 140
```

Backpatch:

TEXT

```
100: if a < b goto 120  
101: goto 140
```

27. Why Backpatching Is Useful

Without backpatching, we might need to know all label addresses in advance.

But during syntax-directed translation, code is often generated incrementally.

Backpatching lets the compiler:

1. emit jump now
2. remember which jumps are incomplete
3. generate more code
4. discover destination
5. fill target later

28. Important Backpatching Lists

Classic syntax-directed translation uses lists such as:

$\backslash \boxed{\text{truelist}} \backslash$

$\backslash \boxed{\text{falselist}} \backslash$

and for statements:

$\backslash \boxed{\text{nextlist}} \backslash$

29. truelist

For Boolean expression B:

$\backslash B.\text{truelist} \backslash$

contains instruction numbers of jumps that should eventually go to B's **true destination**.

Example:

TEXT

100: if a < b goto _

Then:

$\backslash B.\text{truelist}=\{100\} \backslash$

30. falselist

$\backslash B.\text{falselist} \backslash$

contains instruction numbers of jumps that should eventually go to B's **false destination**.

Example:

TEXT

101: goto _

Then:

$\backslash B.\text{falselist}=\{101\} \backslash$

31. nextlist

For a statement S:

$\backslash S.\text{nextlist} \backslash$

contains incomplete jumps whose target should be:

the instruction immediately following S.

This is especially useful for:

- `if`
- `if-else`
- sequences of statements
- loop exits

WRITE THIS IN NOTES

TEXT

```
t r u e l i s t   ! '   j u m p s   w a i t i n g   f o r   T R U E   t a r g e t
f a l s e l i s t   ! '   j u m p s   w a i t i n g   f o r   F A L S E   t a r g e t
n e x t l i s t   ! '   j u m p s   w a i t i n g   f o r   N E X T   s t a t e m e n t
```

32. Three Common Backpatching Functions

Classic compiler theory defines:

$\boxed{\text{makelist}(i)}$

$\boxed{\text{merge}(p_1, p_2)}$

$\boxed{\text{backpatch}(p, i)}$

These are worth remembering.

33. `makelist(i)`

Creates a new list containing instruction `i`.

$\boxed{\text{makelist}(i)=\{\backslash\}}$

Example:

TEXT

```
100: goto _
```

We may create:

$\boxed{\text{makelist}(100)}$

34. `merge(p1, p2)`

Combines two lists of incomplete jumps.

Example:

$\boxed{p_1=\{100, 104\}}$

$\boxed{p_2=\{110\}}$

Then:

$\boxed{\text{merge}(p_1, p_2)=\{100, 104, 110\}}$

35. backpatch(p, i)

Fills target `i` into every incomplete jump whose instruction number occurs in list `p`.

Example:

$\backslash$ `p={100,104}` $\backslash$

Call:

$\backslash$ `backpatch(p,120)` $\backslash$

Then jumps 100 and 104 get target 120.

Memory Trick

TEXT

```
m a k e l i s t      ! '   c r e a t e
m e r g e           ! '   c o m b i n e
b a c k p a t c h   ! '   f i l l
```

36. Boolean Leaf Translation

Suppose condition:

$\backslash$ `a < b` $\backslash$

At instruction number 100, generate:

TEXT

```
100: if a < b goto _
101: goto _
```

Then:

$\backslash$ `\boxed{B.truelist={100}}` $\backslash$

$\backslash$ `\boxed{B.falselist={101}}` $\backslash$

No destination yet.

37. AND With Backpatching

Consider:

$\backslash$ `B=B_1\land B_2` $\backslash$

Short-circuit rule:

If $\backslash$ (`B_1`) is true, evaluate $\backslash$ (`B_2`).

Therefore:

$\backslash B_1.\text{truelist} \backslash$

must be patched to:

$\backslash \boxed{\text{beginning of } B_2} \backslash$

If $\backslash(B_1)$ is false, the whole expression is false.

So false exits come from both operands.

38. AND Equations

For:

$\backslash B \rightarrow B_1 \&\& B_2 \backslash$

conceptually:

$\backslash \boxed{\text{backpatch}(B_1.\text{truelist}, B_2.\text{begin})} \backslash$

$\backslash \boxed{B.\text{truelist} = B_2.\text{truelist}} \backslash$

$\backslash \boxed{B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist})} \backslash$

Memory Trick for AND

TRUE

must pass through both:

$\backslash \boxed{\text{Final true list comes from } B_2} \backslash$

FALSE

either one can fail:

$\backslash \boxed{\text{Merge false lists}} \backslash$

39. OR With Backpatching

For:

$\backslash B = B_1 \vee B_2 \backslash$

Short circuit:

If $\backslash(B_1)$ is false, evaluate $\backslash(B_2)$.

Therefore patch:

$\backslash B_1.\text{falselist} \backslash$

to the beginning of $\backslash(B_2)$.

40. OR Equations

$\backslash \boxed{\text{backpatch}(B_1.\text{falselist}, B_2.\text{begin})} \backslash$

$\boxed{B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist})}$

$\boxed{B.\text{falselist} = B_2.\text{falselist}}$

Memory Trick for OR

TRUE

either operand can succeed:

$\boxed{\text{Merge true lists}}$

FALSE

must fail all the way through:

$\boxed{\text{Final false list from } B_2}$

41. NOT With Backpatching

For:

$B \neq B_1$

just swap:

$\boxed{B.\text{truelist} = B_1.\text{falselist}}$

$\boxed{B.\text{falselist} = B_1.\text{truelist}}$

This exactly matches Boolean NOT.

42. Master AND/OR/NOT Table

| Expression | What gets patched? | True list | False list | | |
|------------------|-------------------------------------|---------------------|--------------------------------------|------------|----------------------|
| $(B_1 \& B_2)$ | $(B_1.\text{true} \rightarrow B_2)$ | $(B_2.\text{true})$ | merge false | | |
| $(B_1 \vee B_2)$ | $\vee$ | B_2 | $(B_1.\text{false} \rightarrow B_2)$ | merge true | $(B_2.\text{false})$ |
| $\neg B_1$ | none | $B_1.\text{false}$ | $B_1.\text{true}$ | | |

This is a high-value GATE revision table.

43. Worked Backpatching Example — AND

Condition:

$a < b \wedge c < d$

Generate first condition:

TEXT

```
100: if a < b goto _  
101: goto _
```

So:

$\lceil B_1.true = \{100\} \rceil$

$\lceil B_1.false = \{101\} \rceil$

Second condition starts at 102.

Patch true list of $\lceil B_1 \rceil$:

$\lceil 100 \rightarrow 102 \rceil$

Generate:

TEXT

```
102: if c < d goto _  
103: goto _
```

Then overall:

$\lceil B.true = \{102\} \rceil$

$\lceil B.false = \{101, 103\} \rceil$

44. Later Patch Final Destinations

Suppose true body starts at:

$\lceil 120 \rceil$

and false continuation starts at:

$\lceil 140 \rceil$

Backpatch:

$\lceil B.true = \{102\} \rightarrow 120 \rceil$

$\lceil B.false = \{101, 103\} \rightarrow 140 \rceil$

Final:

TEXT

```
100: if a < b goto 102  
101: goto 140  
102: if c < d goto 120  
103: goto 140
```

Exactly short-circuit AND.

45. Worked Backpatching Example — OR

Condition:

$\lceil a < b \parallel c < d \rceil$

First condition:

TEXT

```
100: if a < b goto _  
101: goto _
```

Second condition begins 102.

For OR, first **false** exit continues to B2.

Patch:

`\[ 101\rightarrow102 \]`

Then:

TEXT

```
102: if c < d goto _  
103: goto _
```

Overall:

`\[ B.true=\{100,102\} \]`

`\[ B.false=\{103\} \]`

46. Final Patch for OR

Suppose true body begins:

`\[ 120 \]`

false continuation:

`\[ 140 \]`

Patch:

TEXT

```
1 0 0  !'  1 2 0  
1 0 2  !'  1 2 0  
1 0 3  !'  1 4 0
```

Final:

TEXT

```
100: if a < b goto 120  
101: goto 102  
102: if c < d goto 120  
103: goto 140
```

If first condition true, second is skipped.

47. Backpatching an `if`

Consider:

```

c
if (B)
    S;

```

Boolean expression B generates:

- B.trueList
- B.falseList

Suppose S begins at instruction i.

Patch:

$\boxed{B.trueList \rightarrow i}$

The false exits skip the body and should eventually go to the instruction after the whole if.

Therefore they become part of:

$\boxed{nextList}$

48. if-else Backpatching Intuition

For:

```

c
if (B)
    S1;
else
    S2;

```

Patch:

$\boxed{B.trueList}$

to beginning of (S_1) .

Patch:

$\boxed{B.falseList}$

to beginning of (S_2) .

Then both branch exits that need to continue afterward are merged into the whole statement's nextList.

49. While Backpatching Intuition

For:

```

c
while (B)
    S;

```

Suppose condition begins at label/instruction Lbegin.

- B true !' body S
- B false !' loop exit

- after body !' jump back to condition

So:

`\[ B.truelist\rightarrow S.begin \]`

and body's continuation gets sent back to:

`\[ Lbegin \]`

while:

`\[ B.falselist \]`

becomes loop's exit list.

50. Short-Circuit vs Explicit Boolean Value

Consider:

```
c
if (a < b && c < d)
```

Explicit value approach

Might compute:

```
TEXT
t1 = a < b
t2 = c < d
t3 = t1 && t2
```

But this may evaluate `c < d` even when `a < b` is false.

Jumping/short-circuit approach

Only evaluates second condition when needed.

Therefore short-circuit control-flow code is often more semantically faithful and efficient.

51. Boolean Operator Precedence

Typically:

`\[ ! \]`

has highest Boolean precedence,

then:

`\[ && \]`

then:

`\[`

`\]`

So:

```
c
!A || B && C
```

normally means:

$\neg (A) \vee (B \wedge C)$

unless parentheses override.

Generated control flow must preserve this structure.

52. Example

Condition:

```
c
a < b || c < d && e < f
```

Since `&&` has higher precedence:

$(a < b) \vee ((c < d) \wedge (e < f))$

So don't translate as:

$((a < b) \vee (c < d)) \wedge (e < f)$

53. Labels vs Instruction Numbers

Backpatching may use:

- symbolic labels:

`L1`, `L2`

- numeric instruction indices:

`100`, `101`

Both represent target locations.

GATE may use either notation.

54. Backpatching and One-Pass Translation

Backpatching is especially helpful when code is generated in a single traversal because forward destinations are not yet known.

Thus:

$\boxed{\text{Backpatching resolves forward jumps}}$

This is the key practical idea.

55. GATE-STYLE EXAMPLE 1

What does short-circuit evaluation of:

$\lceil A \& \& B \rceil$

do if A is false?

A. Evaluate B anyway B. Return true C. Skip B and conclude false D. Repeat A

' C

56. GATE-STYLE EXAMPLE 2

For:

$\lceil A \parallel B \rceil$

when is B evaluated?

$\lceil \boxed{\text{Only when A is false}} \rceil$

57. GATE-STYLE EXAMPLE 3

What is stored in:

$\lceil B.\text{truelist} \rceil$

?

$\lceil \boxed{\text{Incomplete jumps that should eventually reach B's true destination}} \rceil$

58. GATE-STYLE EXAMPLE 4

For:

$\lceil B = B_1 \& \& B_2 \rceil$

which list of $\lceil (B_1) \rceil$ is backpatched to beginning of $\lceil (B_2) \rceil$?

$\lceil \boxed{B_1.\text{truelist}} \rceil$

Because B2 is evaluated only if B1 is true.

59. GATE-STYLE EXAMPLE 5

For:

$\lceil B = B_1 \parallel B_2 \rceil$

which list of $\lceil (B_1) \rceil$ goes to B2?

$\lceil \boxed{B_1.\text{falselist}} \rceil$

because B2 is needed only when B1 is false.

60. GATE-STYLE EXAMPLE 6

For:

$\lceil B \neq B_1 \rceil$

what happens?

$\lceil \boxed{B.\text{true} = B_1.\text{false}} \rceil$

$\lceil \boxed{B.\text{false} = B_1.\text{true}} \rceil$

61. GATE TRAPS

Trap 1

Short-circuit AND evaluates the second operand even if first is false.

'L False.

$\lceil \boxed{\text{False stops AND}} \rceil$

Trap 2

Short-circuit OR evaluates second operand even if first is true.

'L False.

$\lceil \boxed{\text{True stops OR}} \rceil$

Trap 3

Backpatching means modifying source code after compilation.

'L False.

It means filling incomplete jump targets in generated intermediate code.

Trap 4

`trueList` stores all instructions that evaluate to integer 1.

'L False.

It stores unresolved jump instructions targeting the true branch.

Trap 5

For AND, patch `B1.false` to `B2`.

'L False.

For AND:

$\lceil \boxed{B_1.\text{true} \rightarrow B_2} \rceil$

Trap 6

For OR, patch B1.true to B2.

'LFalse.

For OR:

$\boxed{B_1.\text{false} \rightarrow B_2}$

Trap 7

NOT requires generating an entirely new set of Boolean tests.

'LUsually unnecessary.

Swap true/false lists.

Trap 8

An `if-else` needs no way for the then branch to skip the else branch.

'LFalse.

Control must avoid executing both branches.

Trap 9

A `while` body does not jump back to the condition.

'LFalse.

That's the loop-forming edge.

Trap 10

Backpatching is useful only for arithmetic expressions.

'LFalse.

It is mainly associated with forward jumps/control flow.

62. MINI PRACTICE — PART 23

Try before checking solutions.

Q1 — Easy

In short-circuit evaluation of:

$\boxed{A \& \& B}$

B is evaluated only when:

A. A is true B. A is false C. Always D. Never

Q2 — Easy

In:

$\backslash [A \parallel B] \backslash$

if A is true:

A. B must be evaluated B. B is skipped C. Result is false D. Program must terminate

Q3 — Medium

For:

$\backslash [B = B_1 \&\& B_2] \backslash$

which list should be patched to the beginning of $\backslash(B_2) \backslash$?

A. $\backslash(B_1.\text{falselist}) \backslash$ B. $\backslash(B_1.\text{truelist}) \backslash$ C. $\backslash(B_2.\text{truelist}) \backslash$ D. $\backslash(B_2.\text{falselist}) \backslash$

Q4 — Medium

For:

$\backslash [B = B_1 \parallel B_2] \backslash$

what is:

$\backslash [B.\text{truelist}] \backslash$

?

A. $\backslash(B_2.\text{truelist}) \backslash$ only B. $\backslash(B_1.\text{truelist}) \backslash$ only C. $\text{merge}(\backslash(B_1.\text{truelist}), \backslash(B_2.\text{truelist})) \backslash$ D. merge of false lists

Q5 — Tricky

What does:

$\backslash [\text{backpatch}(p, i)] \backslash$

do?

A. Creates a new symbol table B. Assigns destination i to incomplete jumps in list p C. Finds FIRST(p) D. Creates an activation record

SOLUTIONS

Q1

' A — A is true

AND needs B only if A hasn't already made the result false.

Q2

' B — B is skipped

Because:

$\lceil \text{true} \vee B = \text{true} \rceil$

Q3

' B

For AND:

$\lceil \boxed{B_1.\text{truelist} \rightarrow B_2.\text{begin}} \rceil$

Q4

' C

Either B1 or B2 can make OR true:

$\lceil \boxed{B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist})} \rceil$

Q5

' B

It fills target `i` into all unresolved jumps in list `p`.

WRITE THIS IN NOTES — PART 23 MASTER SHEET

Relational Expression

$\lceil a < b \rceil$

can generate:

TEXT

```
if a < b goto _  
goto _
```

Short-Circuit AND

$\lceil A \&\&B \rceil$

TEXT

```
A false !' FALSE  
A true !' evaluate B
```

Memory:

$\lceil \boxed{\text{False stops AND}} \rceil$

Short-Circuit OR

$\lceil A \parallel B \rceil$

TEXT

```
A   t r u e   !'   T R U E
A   f a l s e   !'   e v a l u a t e   B
```

Memory:

$\lceil \boxed{\text{True stops OR}} \rceil$

NOT

$\lceil !A \rceil$

$\lceil \boxed{\text{Swap true and false exits}} \rceil$

Backpatching

$\lceil \boxed{\text{Generate incomplete jump now; fill destination later}} \rceil$

Lists

TEXT

```
t r u e l i s t   !'   u n r e s o l v e d   T R U E   j u m p s
f a l s e l i s t   !'   u n r e s o l v e d   F A L S E   j u m p s
n e x t l i s t   !'   u n r e s o l v e d   j u m p s   t o   f o l l o w i n g   s t a t e m e n t
```

Functions

$\lceil \boxed{\text{makelist}(i)} \rceil$

create one-entry list.

$\lceil \boxed{\text{merge}(p_1, p_2)} \rceil$

combine lists.

$\lceil \boxed{\text{backpatch}(p, i)} \rceil$

fill target i .

30-SECOND REVISION

AND

$\lceil B_1 \&\& B_2 \rceil$

TEXT

```
patch B1.true !' B2
true = B2.true
false = merge(B1.false,B2.false)
```

OR

$\vee B_1 || B_2 \vee$

TEXT

```
patch B1.false !' B2
true = merge(B1.true,B2.true)
false = B2.false
```

NOT

TEXT

```
true !" false
```

if

TEXT

```
true !' body
false !' next
```

if-else

TEXT

```
true !' then
false !' else
both !' next
```

while

TEXT

```
condition true !' body
body !' condition
condition false !' exit
```

Backpatching mantra

Unknown jump target? Put it in a list and fill it later.

We have now completed the main **Intermediate Code Generation** block.

Next comes an extremely important bridge into optimization:

Basic Blocks & Control-Flow Graphs

You will learn:

- leader identification
- partitioning TAC into basic blocks
- CFG construction
- predecessor/successor
- entry/exit
- branch and fall-through edges

These concepts are required before data-flow analysis.

Completed: Parts 1–23/31 | Next: Part 24 — Basic Blocks, Leaders & Control-Flow Graphs

Reply **NEXT** for **Part 24**.

USER

Next

ASSISTANT

Compiler Design — Part 24: Basic Blocks, Leaders & Control-Flow Graphs

Completed: Parts 1–23/31 | Current: Part 24 | Remaining: Parts 25–31

We now move into the foundation required for **optimization and data-flow analysis**.

Suppose the compiler has generated TAC:

TEXT

```
1. t1 = a + b
2. t2 = t1 * c
3. if t2 > 10 goto 6
4. x = t2
5. goto 7
6. x = 10
7. y = x + 1
```

Before optimization, the compiler usually divides this code into:

$\boxed{\text{Basic Blocks}}$

and connects those blocks using a:

$\boxed{\text{Control-Flow Graph (CFG)}}$

These two ideas are extremely important for GATE.

1. What Is a Basic Block?

CONCEPT

A **basic block** is a maximal sequence of consecutive instructions such that:

1. control enters only at the beginning, and

2. once the block starts, all instructions execute sequentially until the end without branching except possibly at the last instruction.

WRITE THIS IN NOTES

Basic block: A maximal straight-line sequence of instructions having one entry and one exit.

2. Simplest Intuition

Inside a basic block:

If the first instruction executes, all later instructions in that block execute in sequence.

There is no jump into the middle.

There is no jump out from the middle.

Example:

TEXT

```
t1 = a + b
t2 = t1 * c
x  = t2 - d
```

This can be one basic block.

Memory Trick

Basic Block = No surprise in the middle.

Once you enter:

TEXT

```
Start
! "
I1
! "
I2
! "
I3
! "
End
```

You go straight through.

3. Why "Maximal"?

Suppose:

TEXT

```
1. a = b + c
2. d = a * 2
3. e = d - 1
```

All three can be one block.

We should not unnecessarily create:

TEXT

```
B1 = instruction 1
B2 = instruction 2
B3 = instruction 3
```

because they can be combined safely.

Thus a basic block is:

$\boxed{\text{maximal straight-line code}}$

4. Why Basic Blocks Matter

Optimizations often work:

- inside one basic block
- or across a CFG containing multiple blocks

Examples:

- constant folding
- common subexpression elimination
- dead-code elimination
- liveness analysis
- reaching definitions
- available expressions

So:

$\boxed{\text{Basic blocks are the units of control-flow analysis}}$

5. How Do We Find Basic Blocks?

We first identify special instructions called:

$\boxed{\text{Leaders}}$

Each leader begins a new basic block.

This is one of the most important GATE procedures.

6. Rules for Finding Leaders

For a sequence of TAC instructions:

Rule 1

The **first instruction** is a leader.

Rule 2

Any instruction that is the **target of a conditional or unconditional jump** is a leader.

Rule 3

Any instruction **immediately following a conditional or unconditional jump** is a leader.

WRITE THIS IN NOTES

Leader Rules — F T A

F

$\boxed{\text{First instruction}}$

T

$\boxed{\text{Target of jump}}$

A

$\boxed{\text{After a jump}}$

Memory:

FTA = First, Target, After-jump

7. Why Is the First Instruction a Leader?

Program execution begins there.

Therefore it starts the first basic block.

Simple.

8. Why Is a Jump Target a Leader?

Suppose:

TEXT

```
5. if x < y goto 10
...
10. a = b + c
```

Execution may arrive directly at instruction 10 from instruction 5.

Therefore instruction 10 must begin a basic block.

Otherwise control would enter into the middle of a block, violating the basic-block definition.

9. Why Is the Instruction After a Jump a Leader?

Consider:

TEXT

```
5. if x < y goto 10
6. a = 1
```

If condition is false, execution may continue at instruction 6.

Therefore instruction 6 begins a different possible control-flow path.

So it is a leader.

10. What About an Unconditional `goto`?

Example:

TEXT

```
5. goto 10
6. x = 1
```

Instruction 6 is still marked a leader by the standard leader rule:

instruction immediately following a jump is a leader.

Even if instruction 6 later turns out to be unreachable, leader identification still separates it structurally.

11. Basic-Block Construction Algorithm

After identifying all leaders:

Step 1

Sort leaders by instruction number.

Step 2

For each leader:

Start a new basic block.

Step 3

Include instructions from that leader up to:

the instruction immediately before the next leader.

The last leader's block extends to the end of the code.

WRITE THIS IN NOTES

If leaders are:

$\{L_1, L_2, \dots, L_k\}$

then:

$B_i = \text{instructions from } L_i \text{ to just before } L_{i+1}$

12. Full Worked Example

Consider:

TEXT

```
1. t1 = a + b
2. t2 = t1 * c
3. if t2 > 10 goto 6
4. x = t2
5. goto 7
6. x = 10
7. y = x + 1
8. z = y * 2
```

Find the basic blocks.

13. Step 1 — First Instruction

Instruction:

$\{1\}$

is a leader.

Leaders:

$\{1\}$

14. Step 2 — Jump Targets

Instruction 3:

TEXT

```
if t2 > 10 goto 6
```

Target:

$\{6\}$

So 6 is a leader.

Instruction 5:

TEXT

```
goto 7
```

Target:

$\{7\}$

So 7 is a leader.

Current leaders:

$\{1, 6, 7\}$

15. Step 3 — Instructions After Jumps

After instruction 3:

4

is a leader.

After instruction 5:

6

is a leader, already present.

Therefore:

$\boxed{\text{Leaders} = \{1, 4, 6, 7\}}$

16. Construct Blocks

Block (B_1)

Leader 1 up to before leader 4:

TEXT

1. $t1 = a + b$
2. $t2 = t1 * c$
3. if $t2 > 10$ goto 6

Block (B_2)

Leader 4 up to before leader 6:

TEXT

4. $x = t2$
5. goto 7

Block (B_3)

Leader 6 up to before leader 7:

TEXT

6. $x = 10$

Block (B_4)

Leader 7 to end:

TEXT

7. $y = x + 1$
8. $z = y * 2$

Therefore:

$\boxed{\text{basic blocks}}$

17. GATE Shortcut — Number of Basic Blocks

Once leaders are correctly identified:

$\boxed{\text{Number of basic blocks} = \text{Number of leaders}}$

assuming the entire listed instruction sequence is being partitioned by the standard algorithm.

This is a useful counting shortcut.

18. But Don't Count Duplicate Leaders Twice

Suppose:

- jump target = 8
- instruction after another jump = 8

Instruction 8 is still just one leader.

So use a **set** of leaders.

19. What Is a Control-Flow Graph?

After creating basic blocks, we connect them using possible execution paths.

The resulting graph is called:

$\boxed{\text{Control-Flow Graph}}$

or:

$\boxed{\text{Flow Graph / CFG}}$

Be careful:

Earlier CFG meant:

$\boxed{\text{Context-Free Grammar}}$

Here CFG means:

$\boxed{\text{Control-Flow Graph}}$

Context determines the meaning.

WRITE THIS IN NOTES

Control-Flow Graph: A directed graph whose nodes are basic blocks and whose edges represent possible transfers of control between blocks.

20. CFG Nodes

Each node represents:

$\boxed{\text{one basic block}}$

Example:

TEXT

B1
B2
B3
B4

21. CFG Edges

An edge:

$B_i \rightarrow B_j$

exists if control can move directly from the end of (B_i) to the beginning of (B_j) .

There are two common reasons:

1. explicit jump
2. fall-through

22. Explicit Jump Edge

Suppose (B_1) ends:

TEXT

if $x > 0$ goto B3

Then:

$\boxed{B_1 \rightarrow B_3}$

is an edge.

23. Fall-Through Edge

Suppose a block does not unconditionally jump away.

Then control may naturally continue into the next block.

Example:

TEXT

```
B1:  
if x > 0 goto B3  
  
B2:  
...
```

If the condition is false:

$\lceil B_1 \rightarrow B_2 \rceil$

by fall-through.

24. Conditional Jump Usually Gives Two Successors

Suppose:

TEXT

```
if x < y goto L1
```

At end of block.

There are usually two possible paths:

Condition true

$\lceil \rightarrow L1 \rceil$

Condition false

$\lceil \rightarrow \text{next block} \rceil$

Therefore:

$\lceil \boxed{\text{Conditional branch often gives two CFG successors}} \rceil$

25. Unconditional goto

If block ends:

TEXT

```
goto L1
```

control goes only to L1.

Normally there is no fall-through edge to the next textual block.

Thus:

$\lceil \boxed{\text{Unconditional goto usually gives one successor}} \rceil$

26. Return Statement

If a block ends with:

TEXT

```
return x
```

there is no ordinary successor inside the procedure.

It may connect conceptually to an:

$\boxed{\text{EXIT}}$

node.

27. Entry and Exit Nodes

CFGs are often shown with special nodes:

$\boxed{\text{ENTRY}}$

and:

$\boxed{\text{EXIT}}$

ENTRY represents entry to the procedure.

EXIT represents leaving the procedure.

Conceptually:

TEXT

ENTRY

|

v

B1

...

|

v

EXIT

28. Build CFG for Our Example

Recall:

$\boxed{\text{B}_1}$

TEXT

```
1. t1 = a + b
2. t2 = t1 * c
3. if t2 > 10 goto 6
```

$\boxed{\text{B}_2}$

TEXT

```
4. x = t2
5. goto 7
```

$\boxed{\text{B}_3}$

TEXT

6. $x = 10$

$\backslash(B_4)$

TEXT

7. $y = x + 1$

8. $z = y * 2$

29. Edges From $\backslash(B_1)$

At the end:

TEXT

if $t2 > 10$ goto 6

Instruction 6 begins:

$\backslash B_3 \backslash$

So true edge:

$\backslash \boxed{B_1 \rightarrow B_3} \backslash$

If condition false, execution falls through to instruction 4:

$\backslash B_2 \backslash$

So:

$\backslash \boxed{B_1 \rightarrow B_2} \backslash$

30. Edge From $\backslash(B_2)$

Ends:

TEXT

goto 7

Instruction 7 begins:

$\backslash B_4 \backslash$

Therefore:

$\backslash \boxed{B_2 \rightarrow B_4} \backslash$

No fall-through to B3 because the `goto` is unconditional.

31. Edge From $\backslash(B_3)$

B3:

TEXT

6. $x = 10$

No jump at end.

Next block is B4.

So fall-through:

$\boxed{B_3 \rightarrow B_4}$

32. (B_4)

It's the final block.

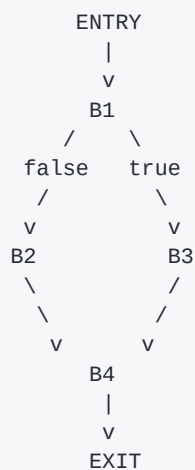
So:

$B_4 \rightarrow \text{EXIT}$

conceptually.

33. Final CFG

TEXT



Edges:

$\boxed{B_1 \rightarrow B_2}$

$\boxed{B_1 \rightarrow B_3}$

$\boxed{B_2 \rightarrow B_4}$

$\boxed{B_3 \rightarrow B_4}$

34. Successor

If there is an edge:

$B_i \rightarrow B_j$

then (B_j) is a:

$\boxed{\text{Successor}}$

of (B_i) .

Notation often:

$\text{Succ}(B_i)$

35. Predecessor

If:

$B_i \rightarrow B_j$

then (B_i) is a:

$\boxed{\text{Predecessor}}$

of (B_j) .

Notation:

$\text{Pred}(B_j)$

Memory Trick

Predecessor = comes before me

Successor = can come after me

36. Example

From our graph:

$B_1 \rightarrow B_2$

$B_1 \rightarrow B_3$

Therefore:

$\text{Succ}(B_1) = \{B_2, B_3\}$

For B_4 :

Incoming from B_2 and B_3 :

$\text{Pred}(B_4) = \{B_2, B_3\}$

37. Predecessor/Successor Are Local Graph Relations

A block may eventually reach another block through several edges without being its direct predecessor.

Example:

$\text{\textbackslash}[B_1\text{\textbackslashrightarrow} B_2\text{\textbackslashrightarrow} B_3 \text{\textbackslash}]$

B1 can reach B3.

But B1 is not necessarily a direct predecessor of B3.

So distinguish:

$\text{\textbackslash}[\text{\textbackslashboxed{\text{\textbackslashtext{Predecessor}}}} \text{\textbackslash}]$

from:

$\text{\textbackslash}[\text{\textbackslashboxed{\text{\textbackslashtext{Reachability}}}} \text{\textbackslash}]$

38. What Is a Path?

A CFG path is a sequence:

$\text{\textbackslash}[B_1, B_2, \text{\textbackslashldots}, B_k \text{\textbackslash}]$

such that each consecutive pair has an edge.

Example:

$\text{\textbackslash}[B_1\text{\textbackslashrightarrow} B_2\text{\textbackslashrightarrow} B_4 \text{\textbackslash}]$

is a path.

39. What Is a Cycle?

If control can follow a path and return to the same block:

$\text{\textbackslash}[B_1\text{\textbackslashrightarrow} B_2\text{\textbackslashrightarrow} B_3\text{\textbackslashrightarrow} B_1 \text{\textbackslash}]$

we have a cycle.

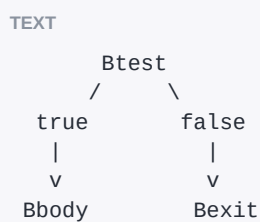
Loops naturally produce CFG cycles.

40. While-Loop CFG

Source:

```
c
while (x < 10) {
    x = x + 1;
}
```

Conceptually:



```
|
% % % % % % % % !'   B t e s t
```

The back edge:

$\backslash [\text{Bbody} \rightarrow \text{Btest}]$

creates a cycle.

41. Why CFG Is Required for Data-Flow Analysis

Suppose a variable `x` has different definitions on different branches:

```
TEXT
if (...)
    x = 1;
else
    x = 2;

y = x + 3;
```

At the block containing:

```
TEXT
y = x + 3
```

information may arrive from both predecessors.

Data-flow analysis must combine information from:

$\backslash [\boxed{\text{multiple CFG paths}}]$

That's why we need predecessors and successors before studying liveness, constant propagation, etc.

42. Basic Block vs CFG

Basic block

Straight-line code **inside one node**.

CFG

Relationships **between nodes**.

Memory:

```
TEXT
Basic block
= local straight-line region

CFG
= global control structure
```

43. Local Optimization

An optimization done entirely within one basic block is called:

$\boxed{\text{Local Optimization}}$

Examples:

- local common-subexpression elimination
- constant folding
- algebraic simplification
- local dead-code elimination

This is Part 25.

44. Global Analysis

When information must cross basic-block boundaries using CFG edges, we move toward:

$\boxed{\text{Global / Data-Flow Analysis}}$

Examples:

- liveness
 - reaching definitions
 - available expressions
 - global constant propagation
-

45. GATE-STYLE Leader Example

Given:

TEXT

```
1. a = b + c
2. if a > 0 goto 5
3. d = a + 1
4. goto 6
5. d = a - 1
6. e = d * 2
```

Find leaders.

46. Leader Rule 1

First:

$\boxed{1}$

47. Jump Targets

From instruction 2:

$\boxed{\text{goto } 5}$

so:

$\backslash [5 \backslash]$

leader.

From instruction 4:

$\backslash [\text{goto} \backslash 6 \backslash]$

so:

$\backslash [6 \backslash]$

leader.

48. After Jumps

After instruction 2:

$\backslash [3 \backslash]$

leader.

After instruction 4:

$\backslash [5 \backslash]$

already leader.

Therefore:

$\backslash [\boxed{\{1,3,5,6\}} \backslash]$

49. Basic Blocks

B1

TEXT

1. $a = b + c$
2. if $a > 0$ goto 5

B2

TEXT

3. $d = a + 1$
4. goto 6

B3

TEXT

5. $d = a - 1$

B4

50. CFG Edges

From B1:

- true !' B3
- false !' B2

$\backslash B_1 \rightarrow B_3 \backslash$

$\backslash B_1 \rightarrow B_2 \backslash$

From B2:

$\backslash B_2 \rightarrow B_4 \backslash$

From B3 fall-through:

$\backslash B_3 \rightarrow B_4 \backslash$

Same diamond pattern.

51. What If a Block Ends With No Jump?

If it is not the final block and no return occurs:

$\backslash \boxed{\text{fall-through edge to next block}} \backslash$

This is one of the most commonly forgotten CFG edges.

52. What If a Conditional Jump Is the Last Instruction?

Usually add:

1. edge to jump target
2. edge to next block

unless the instruction semantics specify otherwise.

53. What If an Unconditional Jump Is the Last Instruction?

Add only:

$\backslash \boxed{\text{edge to jump target}} \backslash$

No normal fall-through.

54. What If Last Instruction Is `return`?

No normal intra-procedural successor.

Conceptually:

$\boxed{\text{B} \rightarrow \text{EXIT}}$

55. Unreachable Basic Blocks

Consider:

TEXT

```
1. goto 4
2. x = 10
3. y = x + 1
4. z = 20
```

Leader rules identify:

- 1: first
- 2: after jump
- 4: jump target

So blocks:

TEXT

```
B1: 1
B2: 2-3
B3: 4
```

But:

$\boxed{B2}$

has no incoming edge from reachable code.

Therefore it is:

$\boxed{\text{Unreachable}}$

and may later be removed as dead/unreachable code.

56. Important Distinction

"Leader" does not automatically mean "reachable."

Leader rules partition code structurally.

Reachability is determined from the CFG starting at ENTRY.

57. Entry Block

The block containing the procedure's first executable instruction is the:

$\boxed{\text{Entry Basic Block}}$

Typically:

$\boxed{B_1}$

58. Exit Block

A procedure may have:

- one return block
- multiple return blocks

For analysis, a special synthetic:

$\boxed{\text{EXIT}}$

node can combine these exits.

59. GATE Trap — Every Jump Instruction Is a Leader

Not necessarily.

The **jump instruction itself** is not automatically a leader merely because it is a jump.

The rules are:

- first instruction
- jump target
- instruction after jump

A jump instruction could independently be a leader if it satisfies one of those conditions.

60. Example

TEXT

1. a = 1
2. b = 2
3. goto 6

Instruction 3 is not a leader merely because it is `goto`.

If no jump targets 3 and it isn't first/after another jump, it belongs in block beginning at 1.

So:

TEXT

B1:
1
2
3

is valid.

61. GATE Trap — Target Means Only Conditional Target

'LFalse.

Targets of both:

- conditional jumps
- unconditional jumps

are leaders.

62. GATE Trap — Instruction After Only Conditional Jumps Is a Leader

'LFalse.

Standard rule includes instruction following **any jump**, conditional or unconditional.

63. GATE Trap — Every Instruction Starts a Basic Block

'LFalse.

Only leaders do.

64. GATE Trap — Basic Block Can Have Two Entry Points

'LFalse by definition.

A basic block has:

$\boxed{\text{single entry}}$

65. GATE Trap — Branch Can Occur in the Middle of a Basic Block

A branch that can transfer control away would terminate that block.

So the actual control-transfer instruction is normally the final instruction of the block.

66. GATE Trap — A Conditional Branch Has Only One CFG Successor

Usually false.

It typically has:

$\boxed{2}$

- branch target
 - fall-through
-

67. GATE Trap — Unconditional Goto Has Fall-Through Edge

'LNormally false.

It jumps directly to its target.

68. GATE Trap — CFG Nodes Are Individual Source Variables

'LFalse.

Nodes represent:

$\boxed{\text{Basic blocks}}$

69. GATE Trap — CFG Edges Represent Data Dependence

'LNot primarily.

CFG edges represent:

$\boxed{\text{possible transfer of control}}$

Data dependence is a different concept.

70. GATE-STYLE Example — Count Leaders Quickly

Instructions:

TEXT

```
1. a = 1
2. if a > b goto 7
3. c = a + b
4. if c < 10 goto 6
5. goto 8
6. d = 10
7. d = 20
8. e = d + 1
```

Find leaders.

First

$\boxed{1}$

Targets

From 2 !' 7

From 4 !' 6

From 5 !' 8

So:

$\boxed{6,7,8}$

After jumps

After 2 !' 3

After 4 !' 5

After 5 !' 6

Therefore:

$\boxed{\{1,3,5,6,7,8\}}$

Number of basic blocks:

$\boxed{6}$

71. Build Those Blocks

B1

TEXT

```
1. a = 1
2. if a > b goto 7
```

B2

TEXT

```
3. c = a + b
4. if c < 10 goto 6
```

B3

TEXT

```
5. goto 8
```

B4

TEXT

```
6. d = 10
```

B5

TEXT

```
7. d = 20
```

B6

TEXT

```
8. e = d + 1
```

72. CFG Edges for That Example

B1:

TEXT

```
if a > b goto 7
```

target 7 = B5.

false fall-through = B2.

$B1 \rightarrow B5$

$B1 \rightarrow B2$

B2:

TEXT

```
if c < 10 goto 6
```

target 6 = B4.

false !' B3.

$\backslash [B2 \rightarrow B4 \backslash]$

$\backslash [B2 \rightarrow B3 \backslash]$

B3:

TEXT

```
goto 8
```

$\backslash [B3 \rightarrow B6 \backslash]$

B4 has no branch:

next B5.

$\backslash [B4 \rightarrow B5 \backslash]$

B5 fall-through:

$\backslash [B5 \rightarrow B6 \backslash]$

73. Predecessor Example

For B6:

Incoming:

$\backslash [B3 \rightarrow B6 \backslash]$

and:

$\backslash [B5 \rightarrow B6 \backslash]$

So:

$\backslash [\boxed{\text{Pred}(B6) = \{B3, B5\}} \backslash]$

74. Successor Example

For B2:

$\backslash [B2 \rightarrow B4 \backslash]$

$\backslash [B2 \rightarrow B3 \backslash]$

Therefore:

$\boxed{\text{Succ}(B2)=\{B3,B4\}}$

Order doesn't matter because this is a set.

75. Why Pred/Succ Matter Later

For forward data-flow analyses:

information commonly comes from:

$\boxed{\text{Pred}(B)}$

For backward analyses:

information commonly comes from:

$\boxed{\text{Succ}(B)}$

Preview:

Constant propagation

often forward.

Liveness

backward.

Available expressions

forward.

You'll see these equations soon.

76. Local vs Global View

Inside:

B

we can analyze instructions sequentially.

Across blocks:

$B_1 \rightarrow B_2$

we need CFG-based equations.

So think:

TEXT

BASIC BLOCK

! ' l o c a l w o r l d

CONTROL-FLOW GRAPH

! ' g l o b a l w o r l d

77. MINI PRACTICE — PART 24

Try first.

Q1 — Easy

Which instruction is always a leader?

A. Last instruction B. First instruction C. Every assignment D. Every arithmetic instruction

Q2 — Easy

If instruction 5 says:

TEXT

```
goto 10
```

which are leaders by the standard rules?

A. Only 5 B. 10 only C. 6 and 10 D. 5 and 10 only

Q3 — Medium

Given:

TEXT

```
1. a = b + c
2. if a > 0 goto 5
3. x = a
4. goto 6
5. x = 0
6. y = x + 1
```

Find all leaders.

Q4 — Medium

How many basic blocks does the previous example contain?

Q5 — Tricky

If a basic block ends in:

TEXT

```
if x < y goto B4
```

and the next textual block is B2, what are its normal successors?

A. B4 only B. B2 only C. B2 and B4 D. None

SOLUTIONS

Q1

' B — First instruction

Rule 1:

$\boxed{\text{First instruction is always leader}}$

Q2

Instruction 10 is jump target.

Instruction 6 follows a jump.

Therefore:

$\boxed{C}$

5 itself is not automatically a leader merely because it contains a jump.

Q3

First:

$\boxed{1}$

Jump targets:

TEXT

| | | |
|---|---|---|
| 2 | ! | 5 |
| 4 | ! | 6 |

so:

$\boxed{5,6}$

After jumps:

after 2:

$\boxed{3}$

after 4:

$\boxed{5}$

already included.

Thus:

$\boxed{\{1,3,5,6\}}$

Q4

Number of leaders:

$\boxed{4}$

Therefore:

$\boxed{4\text{ basic blocks}}$

Q5

Conditional branch:

True

$\lceil \rightarrow B4 \rceil$

False

fall-through:

$\lceil \rightarrow B2 \rceil$

Therefore:

$\lceil \boxed{C} \rceil$

WRITE THIS IN NOTES — PART 24 MASTER SHEET

Basic Block

$\lceil \boxed{\text{Maximal straight-line sequence}} \rceil$

Properties:

- one entry
 - control proceeds sequentially
 - branch only at end
 - no jump into middle
-

Leader Rules

FTA

$\lceil \boxed{\text{F = First instruction}} \rceil$

$\lceil \boxed{\text{T = Target of jump}} \rceil$

$\lceil \boxed{\text{A = After a jump}} \rceil$

Each leader begins a block.

Number of Blocks

$\lceil \boxed{\text{Number of leaders = Number of basic blocks}} \rceil$

after removing duplicate leader numbers.

Control-Flow Graph

Nodes:

$\lceil \boxed{\text{Basic blocks}} \rceil$

Edges:

$\boxed{\text{Possible control transfers}}$

Edges

Conditional branch

Usually:

$\boxed{2 \text{ successors}}$

- target
- fall-through

Unconditional goto

Usually:

$\boxed{1 \text{ successor}}$

- target

No branch

$\boxed{\text{Fall-through to next block}}$

Return

$\boxed{\rightarrow \text{EXIT}}$

Pred/Succ

If:

$B_i \rightarrow B_j$

then:

$\boxed{B_i \in \text{Pred}(B_j)}$

and:

$\boxed{B_j \in \text{Succ}(B_i)}$

30-SECOND REVISION

Basic block

One-entry straight road.

Leaders

Remember:

$\boxed{\text{FTA}}$

TEXT

First
Target
After jump

Construct block

Leader !' until just before next leader.

CFG

TEXT

Node = basic block
Edge = possible control flow

Conditional branch

TEXT

true !' target
false !' next block

Unconditional goto

TEXT

!' target only

No jump

TEXT

!' fall-through

Pred/Succ

TEXT

Pred = who can enter me?
Succ = where can I go next?

We now have the structure needed for optimization. Next comes:

Local Optimisation

We will cover and clearly distinguish:

- constant folding
- algebraic simplification
- strength reduction
- copy propagation
- dead-code elimination
- local common-subexpression elimination

- DAG-based optimization
- common GATE confusion pairs

Completed: Parts 1–24/31 | Next: Part 25 — Local Optimisation Techniques

Reply **NEXT** for **Part 25**.

USER

Next

ASSISTANT

Compiler Design — Part 25: Local Optimisation Techniques

Completed: Parts 1–24/31 | Current: Part 25 | Remaining: Parts 26–31

We now begin the official syllabus topic:

$\boxed{\text{Local Optimisation}}$

A **local optimization** works mainly inside a single:

$\boxed{\text{Basic Block}}$

Because a basic block has straight-line control, the compiler can reason about statements without worrying about internal branches.

The most important techniques are:

- constant folding
- algebraic simplification
- strength reduction
- copy propagation
- dead-code elimination
- common subexpression elimination
- DAG-based local optimization

1. What Is Optimization?

Compiler optimization transforms code into an equivalent form that is usually:

- faster
- smaller
- less memory-intensive
- less redundant

while preserving the program's observable meaning.

WRITE THIS IN NOTES

Code optimization: Transformation of intermediate code into an equivalent form with improved execution/resource characteristics.

The most important condition is:

$\boxed{\text{Program semantics must be preserved}}$

2. What Is Local Optimization?

A local optimization operates within one basic block.

Example:

TEXT

```
t1 = a + b
t2 = t1 * 2
x  = t2
```

Since there is no branch inside the block, statements execute sequentially.

Therefore information derived earlier in the block can often be safely used later.

Memory Trick

Local = one basic block.

3. Constant Folding

CONCEPT

If an expression contains constants whose value can be calculated at compile time, calculate it immediately.

Example:

TEXT

```
x = 5 * 10
```

becomes:

TEXT

```
x = 50
```

This is:

$\boxed{\text{Constant Folding}}$

WRITE THIS IN NOTES

Constant folding: Evaluate constant expressions during compilation instead of at runtime.

4. More Examples

TEXT

$t1 = 20 + 30$

becomes:

TEXT

$t1 = 50$

TEXT

$x = 8 / 2$

becomes:

TEXT

$x = 4$

subject to language arithmetic semantics.

TEXT

$b = 3 < 7$

may become:

TEXT

$b = \text{true}$

Memory Trick

Folding = Do the arithmetic now.

5. Constant Folding vs Constant Propagation

This is a major GATE confusion.

Constant Folding

Evaluate:

TEXT

$x = 5 + 10$

into:

TEXT

$x = 15$

Constant Propagation

If:

TEXT

$a = 10$

$b = a + 5$

replace known `a` :

TEXT

$b = 10 + 5$

Then folding may further produce:

TEXT

$b = 15$

So:

$\boxed{\text{Propagation substitutes}}$

$\boxed{\text{Folding evaluates}}$

6. Combined Example

Original:

TEXT

$a = 4$

$b = a * 5$

Constant propagation

TEXT

$b = 4 * 5$

Constant folding

TEXT

$b = 20$

These are two distinct optimizations.

7. Algebraic Simplification

CONCEPT

Use mathematical identities to replace an expression with a simpler equivalent expression.

Examples:

$$[x + 0 = x]$$

$$[x - 0 = x]$$

$$[x \times 1 = x]$$

$$[x / 1 = x]$$

So:

TEXT

$$t1 = x + 0$$

becomes:

TEXT

$$t1 = x$$

WRITE THIS IN NOTES

Algebraic simplification: Use valid algebraic identities to simplify expressions without changing their meaning.

8. Common Algebraic Identities

Useful conceptual examples:

$$[x + 0 \rightarrow x]$$

$$[0 + x \rightarrow x]$$

$$[x - 0 \rightarrow x]$$

$$[x \times 1 \rightarrow x]$$

$$[1 \times x \rightarrow x]$$

$$[x / 1 \rightarrow x]$$

Some compilers may also simplify:

$$[x \times 0 \rightarrow 0]$$

but real language semantics can make apparently obvious identities unsafe in special cases involving floating-point values, side effects, overflow rules, etc.

For GATE conceptual TAC questions, basic identities are normally assumed valid unless stated otherwise.

9. Folding vs Algebraic Simplification

Constant Folding

TEXT

$3 * 5$

becomes:

TEXT

15

Both operands are constants.

Algebraic Simplification

TEXT

$x * 1$

becomes:

TEXT

x

x may not be known.

The transformation comes from an identity.

10. Strength Reduction

CONCEPT

Replace an expensive operation with a cheaper equivalent operation.

Classic example:

$\lfloor x \times 2 \rfloor$

may be replaced by:

$\lfloor x + x \rfloor$

or a shift:

$\lfloor x \ll 1 \rfloor$

when language and target-machine semantics make the transformation valid.

This is:

$\boxed{\text{Strength Reduction}}$

WRITE THIS IN NOTES

Strength reduction: Replace a relatively expensive operation by an equivalent cheaper operation.

Memory Trick

Strong operation != weaker/cheaper operation.

11. Classic Examples

Conceptually:

$x \times 2 \rightarrow x \ll 1$

$x \times 8 \rightarrow x \ll 3$

for suitable integer arithmetic.

Another important form occurs in loops.

Original:

TEXT

```
address = base + i * 4
```

If `i` increases by 1 each iteration, instead of repeatedly multiplying by 4, the address may be incremented by 4.

That is an important strength-reduction idea.

12. Strength Reduction vs Constant Folding

Consider:

TEXT

```
x = 8 * 4
```

Since both are constants:

$\boxed{\text{Constant folding}}$

gives:

TEXT

```
x = 32
```

But:

TEXT

```
x = y * 8
```

might become:

TEXT

```
x = y << 3
```

That is:

$\boxed{\text{Strength reduction}}$

13. Copy Statement

Consider:

TEXT

```
x = y
```

This is called a:

$\boxed{\text{Copy}}$

It says `x` currently receives the same value as `y`.

This enables:

$\boxed{\text{Copy Propagation}}$

14. Copy Propagation

CONCEPT

After:

TEXT

```
x = y
```

replace later uses of `x` by `y` while the copy remains valid.

Example:

TEXT

```
x = y
z = x + 1
```

becomes:

TEXT

```
x = y
z = y + 1
```

If `x` is no longer needed afterward, the original copy may later become dead.

WRITE THIS IN NOTES

Copy propagation: Replace uses of a copied variable by the original variable when the copy remains valid.

Memory Trick

$x = y$!' use y directly.

15. When Is Copy Propagation Unsafe?

Original:

TEXT

```
x = y
y = 10
z = x + 1
```

Can we replace x in line 3 with y ?

'LNo.

x contains the **old value** of y .

After:

TEXT

```
y = 10
```

the relationship:

$\{ x=y \}$

is no longer guaranteed.

So copy propagation requires checking intervening definitions.

16. Another Copy Trap

TEXT

```
x = y
x = 20
z = x + 1
```

The first copy is killed by:

TEXT

```
x = 20
```

So line 3 obviously cannot use y .

17. Dead Code

CONCEPT

Code is **dead** if removing it does not affect the program's observable result.

A common example is an assignment whose value is never used before being overwritten.

Example:

TEXT

```
x = 10
x = 20
print(x)
```

The first:

TEXT

```
x = 10
```

is dead.

Because `x` is overwritten before its value is used.

18. Dead-Code Elimination

Remove such unnecessary code:

Original:

TEXT

```
x = 10
x = 20
print(x)
```

Optimized:

TEXT

```
x = 20
print(x)
```

This is:

`\boxed{\text{Dead-Code Elimination}}`

WRITE THIS IN NOTES

Dead-code elimination: Remove computations whose results cannot affect subsequent observable program behavior.

19. Dead Assignment Example

TEXT

```
t1 = a + b
t1 = c + d
x = t1
```

The result:

`[ a+b ]`

stored into `t1` is overwritten before it is used.

Therefore:

TEXT

```
t1 = a + b
```

is dead, provided the computation has no side effects/traps relevant under the language model.

Optimized:

TEXT

```
t1 = c + d
x = t1
```

20. Dead Code vs Unreachable Code

These concepts are related but not identical.

Dead Code

May be executed but its result is useless.

Example:

TEXT

```
x = 5
x = 10
```

first assignment dead.

Unreachable Code

Cannot be executed at all.

Example:

TEXT

```
goto L1
x = 10
L1:
```

`x = 10` is unreachable.

WRITE THIS IN NOTES

TEXT

Dead code
! ' may execute , but useless

Unreachable code
! ' cannot execute

21. Common Subexpression

Suppose:

TEXT

t1 = a + b
...
t2 = a + b

If:

- neither **a** nor **b** has changed,
- and the earlier result is still available,

then the second:

$\boxed{a+b}$

is a:

$\boxed{\text{Common Subexpression}}$

22. Common Subexpression Elimination — CSE

Instead of recalculating:

TEXT

t2 = a + b

reuse the previous result:

TEXT

t2 = t1

or directly use **t1** later.

Original:

TEXT

t1 = a + b
t2 = c * d
t3 = a + b
x = t3 * 2

Optimized:

TEXT

```
t1 = a + b
t2 = c * d
x  = t1 * 2
```

if `t2` is also unnecessary, further optimization may remove it.

WRITE THIS IN NOTES

Common subexpression elimination: Avoid recomputing an expression whose operands have not changed since a previous computation of that expression.

Memory Trick

Computed before + operands unchanged = reuse it.

23. CSE Validity Condition

Original:

TEXT

```
t1 = a + b
a  = 10
t2 = a + b
```

Can `t2` reuse `t1`?

'LNo.

Because operand:

`[ a ]`

was redefined.

Thus the previous expression value is no longer guaranteed to be valid.

24. Another CSE Trap

TEXT

```
t1 = a + b
b  = b + 1
t2 = a + b
```

Again:

`\boxed{\text{Not a reusable common subexpression}}`

because b changed.

25. Local CSE

If both occurrences lie inside the same basic block, we call the optimization:

$\boxed{\text{Local Common Subexpression Elimination}}$

Across multiple basic blocks, more global data-flow information is required.

Later, available-expression analysis handles that.

26. DAG-Based Local Optimization

Recall Part 21:

A DAG can share identical expression nodes.

Suppose block:

TEXT

```
t1 = a + b
t2 = t1 - c
t3 = a + b
t4 = t3 * d
```

The DAG needs only one node for:

$a+b$

Both `t1` and `t3` can refer to that value.

Thus the DAG naturally reveals common subexpressions.

27. DAG Leaf Nodes

For a basic block, DAG leaves commonly represent initial values of variables such as:

TEXT

```
a
b
c
d
```

Internal nodes represent operations:

TEXT

```
+
-
*
/
```

Labels on nodes may represent variables holding that computed value.

28. DAG Example

Basic block:

TEXT

```
a = b + c
d = b + c
e = a + d
```

The expression:

$\backslash$ b+c $\backslash$

is common.

Conceptual DAG:

TEXT

```
      +
     / \
    b   c
     \
    shared node
    labels: a,d
```

Then:

$\backslash$ e=a+d $\backslash$

uses that same computed value twice.

29. DAG and Copy Relationships

If both:

TEXT

a

and:

TEXT

d

label the same DAG node, they currently represent the same computed value.

This can expose:

- copies
- common expressions
- unnecessary names

30. DAG and Dead Code

Suppose:

TEXT

```
a = b + c
a = d + e
x = a
```

The first value assigned to `a` is overwritten and never used.

A DAG-based representation can help reveal that the first computation does not contribute to any live result from the block.

So DAGs can support local dead-code elimination too.

31. Optimization Sequence Matters

Consider:

TEXT

```
a = 5
b = a + 0
c = b * 2
```

Constant propagation

TEXT

```
b = 5 + 0
c = b * 2
```

Algebraic simplification / folding

TEXT

```
b = 5
```

Then propagate again:

TEXT

```
c = 5 * 2
```

Then fold:

TEXT

```
c = 10
```

Then if `a` and `b` have no later use:

TEXT

```
c = 10
```

So optimizations often enable further optimizations.

32. Optimization Is Often Iterative

One transformation can reveal another.

Example:

TEXT

```
x = y
z = x + 0
```

Copy propagation

TEXT

```
z = y + 0
```

Algebraic simplification

TEXT

```
z = y
```

Potentially more copy propagation follows.

33. Constant Propagation — Local Preview

Although full **constant propagation data-flow analysis** is Part 27, locally the idea is simple.

Example:

TEXT

```
x = 5
y = x + 3
```

Replace:

TEXT

```
x
```

by:

TEXT

```
5
```

giving:

TEXT

```
y = 5 + 3
```

then fold:

TEXT

```
y = 8
```

Across branches, however, deciding whether x is still a constant becomes harder.

That requires CFG/data-flow analysis.

34. Local vs Global Constant Propagation

Local

Inside one basic block:

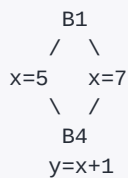
TEXT

```
x = 5
y = x + 1
```

Easy.

Across CFG

TEXT



What is x at B4?

It may be 5 or 7.

So x is not one single constant there.

This motivates Part 27.

35. Optimization and Side Effects

Consider:

TEXT

```
x = f()
x = 10
```

Can we always delete:

TEXT

```
x = f()
```

because x is overwritten?

'LNo.

Function `f()` may:

- print something
- modify global state
- perform I/O

- throw an exception
- change memory

Therefore dead-code elimination must consider **side effects**.

For GATE's simple TAC, expressions are often assumed side-effect-free unless specified.

36. Side-Effect-Free Expression

Expressions such as:

TEXT
a + b

using ordinary scalar variables are typically treated as side-effect-free in textbook examples.

But:

TEXT
f()

or certain memory operations may have effects.

So optimization cannot be based only on whether the assigned result is used.

37. Algebraic Identity Trap

Can:

$\forall x-x$

always become:

$\forall 0$

In pure mathematical algebra, yes.

In real programming languages, special floating-point values and side effects can complicate such transformations.

For standard GATE compiler-optimization questions involving integer TAC, expected algebraic simplifications usually follow conventional arithmetic assumptions.

Use the semantics specified by the question.

38. Strength Reduction Trap

Is:

$\forall x^2 \rightarrow x+x$

always faster?

Not necessarily on every modern processor.

"Strength reduction" is a compiler concept based on replacing expensive operations with cheaper equivalents under a particular cost model.

GATE usually tests the principle, not modern microarchitecture details.

39. Peephole Optimization — Useful Context

A **peephole optimizer** examines a small window of consecutive instructions and replaces inefficient patterns.

Examples may include:

- redundant instructions
- unnecessary jumps
- algebraic simplification
- machine idioms

This is related to local optimization, though your official syllabus specifically names local optimization rather than requiring deep peephole-optimizer study.

Remember the idea only.

40. Redundant Copy Example

Original:

TEXT

```
x = y
z = x
```

After copy propagation:

TEXT

```
x = y
z = y
```

If `x` has no other use:

TEXT

```
z = y
```

The first copy can be eliminated.

41. Redundant Assignment Example

TEXT

```
x = x
```

This assignment usually does nothing and can be removed under ordinary scalar semantics.

$\boxed{\text{Redundant operation}}$

42. Common GATE Confusion Table

| Code | Optimization |
|--|--------------------------|
| <code>x = 3 + 5 !' x = 8</code> | Constant folding |
| <code>a = 5 ; x = a + 1 !' x = 5 + 1</code> | Constant propagation |
| <code>x = y + 0 !' x = y</code> | Algebraic simplification |
| <code>x = y * 8 !' x = y << 3</code> | Strength reduction |
| <code>x = y ; z = x + 1 !' z = y + 1</code> | Copy propagation |
| Remove <code>x=5</code> before <code>x=10</code> if unused | Dead-code elimination |
| Compute <code>a+b</code> once and reuse | CSE |

This table is high-value for GATE.

43. Fast Identification Trick

Ask:

Are all operands constants?

$\boxed{\text{Constant Folding}}$

Is a known constant replacing a variable?

$\boxed{\text{Constant Propagation}}$

Is `+0`, `*1`, etc. disappearing?

$\boxed{\text{Algebraic Simplification}}$

Is an expensive operator replaced by cheaper one?

$\boxed{\text{Strength Reduction}}$

Is `x=y` being substituted into later code?

$\boxed{\text{Copy Propagation}}$

Is useless code being removed?

$\boxed{\text{Dead Code Elimination}}$

Is an earlier expression result being reused?

$\boxed{\text{Common Subexpression Elimination}}$

44. GATE-STYLE Example 1

Original:

TEXT

`x = 10 * 20`

Optimized:

TEXT

$x = 200$

Technique:

$\boxed{\text{Constant Folding}}$

45. GATE-STYLE Example 2

Original:

TEXT

$x = 20$

$y = x + z$

becomes:

TEXT

$y = 20 + z$

Technique:

$\boxed{\text{Constant Propagation}}$

46. GATE-STYLE Example 3

Original:

TEXT

$x = y * 1$

becomes:

TEXT

$x = y$

Technique:

$\boxed{\text{Algebraic Simplification}}$

47. GATE-STYLE Example 4

Original:

TEXT

$x = y * 8$

becomes conceptually:

TEXT

```
x = y << 3
```

Technique:

$\boxed{\text{Strength Reduction}}$

48. GATE-STYLE Example 5

Original:

TEXT

```
x = y
z = x + 5
```

becomes:

TEXT

```
z = y + 5
```

Technique:

$\boxed{\text{Copy Propagation}}$

provided the copy remains valid.

49. GATE-STYLE Example 6

Original:

TEXT

```
x = a + b
x = c + d
y = x
```

Which computation can be eliminated?

$\boxed{x=a+b}$

because that value is overwritten before use, assuming no side effects.

Technique:

$\boxed{\text{Dead-Code Elimination}}$

50. GATE-STYLE Example 7

Original:

TEXT

```
t1 = a * b
x = t1 + c
t2 = a * b
```

```
y = t2 + d
```

If a and b unchanged:

replace second multiplication with earlier result.

Optimized:

```
TEXT
t1 = a * b
x  = t1 + c
y  = t1 + d
```

Technique:

$\boxed{\text{Common Subexpression Elimination}}$

51. GATE-STYLE Example 8 — Not CSE

```
TEXT
t1 = a + b
a  = a + 1
t2 = a + b
```

Can second expression be eliminated?

$\boxed{\text{No}}$

because operand `a` changed.

52. GATE-STYLE Example 9 — Multiple Optimizations

Original:

```
TEXT
a = 10
b = a + 0
c = b * 2
```

Possible sequence:

Constant propagation

```
TEXT
b = 10 + 0
```

Fold/simplify

```
TEXT
b = 10
```

Propagate

TEXT

```
c = 10 * 2
```

Fold

TEXT

```
c = 20
```

If a,b are dead afterward:

TEXT

```
c = 20
```

This shows compiler optimizations interact.

53. GATE TRAPS

Trap 1

Constant folding and propagation are the same operation.

'LFalse.

TEXT

```
Propagation !' substitute known constant  
Folding      !' evaluate constant expression
```

Trap 2

Copy propagation is always valid after `x = y`.

'LFalse.

A later redefinition of x or y may invalidate the copy relation.

Trap 3

Two syntactically identical expressions are always common subexpressions.

'LFalse.

Their operands must not have changed in between.

Trap 4

Dead code means only unreachable code.

'LFalse.

Executed but useless computations can also be dead.

Trap 5

Every overwritten assignment can always be deleted.

'L False if its RHS has observable side effects.

Trap 6

Strength reduction means replacing constants by variables.

'L False.

It replaces expensive operations with cheaper equivalent operations.

Trap 7

DAGs cannot help identify common subexpressions.

'L False.

That's one of their major local-optimization uses.

Trap 8

Local optimization requires information from all procedure CFG paths.

'L Not necessarily.

Local optimization primarily works within one basic block.

Trap 9

$x = y + 0$ is constant folding.

'L Usually classified as:

$\boxed{\text{Algebraic Simplification}}$

Trap 10

$x = 4 * 5$ is strength reduction.

'L It is:

$\boxed{\text{Constant Folding}}$

54. MINI PRACTICE — PART 25

Try before checking.

Q1

TEXT

```
x = 15 + 5
```

becomes:

TEXT

```
x = 20
```

Which optimization?

A. Copy propagation B. Constant folding C. CSE D. Dead-code elimination

Q2

TEXT

```
a = 7  
x = a + b
```

becomes:

TEXT

```
x = 7 + b
```

Which technique?

Q3

TEXT

```
x = y  
z = x * 2
```

becomes:

TEXT

```
z = y * 2
```

Which technique?

Q4

TEXT

```
t1 = a + b  
c = t1 * 2  
t2 = a + b
```

Assume a and b are unchanged. Which optimization can remove the second `a+b` ?

Q5 — Tricky

TEXT

```
t1 = a + b
a  = 5
t2 = a + b
```

Can `t2 = a+b` reuse `t1`?

A. Yes B. No

SOLUTIONS

Q1

' B — Constant folding

`\[ 15+5 \]`

is calculated during compilation.

Q2

`\[ \boxed{\text{Constant Propagation}} \]`

Known value 7 replaces `a`.

Q3

`\[ \boxed{\text{Copy Propagation}} \]`

because:

TEXT

```
x = y
```

allows later use of `x` to be replaced by `y` while valid.

Q4

`\[ \boxed{\text{Common Subexpression Elimination}} \]`

because `a+b` was already computed and neither operand changed.

Q5

' B — No

`a` was redefined.

Therefore the two computations of:

`\[ a+b \]`

may have different values.

55. WRITE THIS IN NOTES — PART 25 MASTER SHEET

Local Optimization

$\boxed{\text{Optimization within a basic block}}$

Constant Folding

TEXT

5 + 10 !' 15

$\boxed{\text{Evaluate constants now}}$

Constant Propagation

TEXT

a = 5
x = a + b

becomes:

TEXT

x = 5 + b

$\boxed{\text{Substitute known constant}}$

Algebraic Simplification

TEXT

x + 0 !' x
x * 1 !' x

$\boxed{\text{Use identities}}$

Strength Reduction

TEXT

x * 8 !' x << 3

conceptually.

$\boxed{\text{Expensive !' cheaper operation}}$

Copy Propagation

TEXT

```
x = y
z = x + 1
```

becomes:

TEXT

```
z = y + 1
```

$\boxed{\text{Use original value directly}}$

Dead-Code Elimination

Remove:

TEXT

```
x = 10
```

from:

TEXT

```
x = 10
x = 20
use x
```

if the first result is unused and side-effect-free.

Common Subexpression Elimination

TEXT

```
t1 = a + b
...
t2 = a + b
```

Reuse `t1` if:

$\boxed{\text{a and b were not changed}}$

30-SECOND REVISION

Memorize this:

TEXT

```
3 + 4 != 7
= CONSTANT FOLDING
```

```
a = 3 ; x = a + b != x = 3 + b
= CONSTANT PROPAGATION
```

```

x + 0  !'  x
= ALGEBRAIC SIMPLIFICATION

x * 8  !'  x < 3
= STRENGTH REDUCTION

x = y ;  z = x + 1  !'  z = y + 1
= COPY PROPAGATION

u s e l e s s   r e s u l t  !'   r e m o v e   s t a t e m e n t
= DEAD-CODE ELIMINATION

same expression + operands unchanged
= COMMON SUBEXPRESSION ELIMINATION

```

Most important distinction

$\boxed{\text{Propagation substitutes; Folding calculates}}$

CSE rule

Same expression is reusable only if none of its relevant operands have been redefined.

Local

$\boxed{\text{One basic block}}$

We now have all prerequisites for the final major technical section of the syllabus:

Data-Flow Analysis

The next part is crucial because it gives the general framework behind:

- constant propagation
- liveness analysis
- available expressions/CSE

We will learn:

- **IN** and **OUT**
- **GEN** and **KILL**
- transfer functions
- forward vs backward analysis
- may vs must analysis
- union vs intersection
- fixed-point iteration

Completed: Parts 1–25/31 | Next: Part 26 — Data-Flow Analysis Foundations

Reply **NEXT** for **Part 26**.

USER

NEXT

Compiler Design — Part 26: Data-Flow Analysis Foundations

Completed: Parts 1–25/31 | Current: Part 26 | Remaining: Parts 27–31

We now enter the most equation-oriented part of the syllabus:

$\boxed{\text{Data-Flow Analysis}}$

Parts 27–29 will use this framework for:

- constant propagation
- liveness analysis
- common subexpression elimination / available expressions

The biggest goal of this part is to make these symbols completely comfortable:

$\boxed{\text{IN, OUT, GEN, KILL}}$

and understand:

$\boxed{\text{Forward vs Backward}}$

$\boxed{\text{May vs Must}}$

$\boxed{\text{Union vs Intersection}}$

$\boxed{\text{Fixed Point}}$

1. Why Do We Need Data-Flow Analysis?

Inside one basic block, execution is straight-line.

Example:

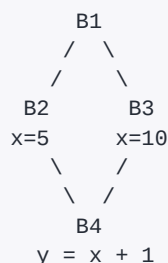
TEXT

```
x = 5
y = x + 1
```

It is easy to know that `x` is 5 when line 2 executes.

But consider:

TEXT



At B4, information may arrive from **multiple paths**.

We therefore need a systematic way to collect and combine information across the:

$\boxed{\text{Control-Flow Graph}}$

That systematic method is data-flow analysis.

WRITE THIS IN NOTES

Data-flow analysis: A compile-time technique for collecting information about how program values/properties flow through the control-flow graph.

2. What Kind of Information?

Different analyses track different properties.

Examples:

Constant propagation

Does variable x definitely have a known constant value here?

Liveness

Will the current value of x possibly be used later?

Available expressions

Has expression $a+b$ definitely already been computed and remained valid on every incoming path?

The framework is similar, but:

- direction
- equations
- merge operation

can differ.

3. Basic Data-Flow Setup

For each basic block B , we generally associate:

$\boxed{IN[B]}$

and:

$\boxed{OUT[B]}$

4. What Is $IN[B]$?

$IN[B]$

means:

Data-flow information valid immediately before entering block B.

5. What Is $OUT[B]$?

$\boxed{OUT[B]}$

means:

Data-flow information valid immediately after leaving block B.

Memory Trick

```
TEXT
IN[B]
! "
% % % % % % % %
%      B      %
% % % % % % % %
! "
OUT[B]
```

So:

$\boxed{IN = \text{text\{before block\}}}$

$\boxed{OUT = \text{text\{after block\}}}$

6. Transfer Function

A basic block transforms incoming information into outgoing information.

For a forward analysis, conceptually:

$\boxed{OUT[B] = f_B(IN[B])}$

where f_B is the:

$\boxed{\text{Transfer Function}}$

It describes:

What does block B do to the information entering it?

WRITE THIS IN NOTES

Transfer function: Function describing how a basic block transforms data-flow information.

Forward form:

$$\boxed{OUT[B] = f_B(IN[B])}$$

Backward analyses will reverse this relationship.

7. GEN and KILL Intuition

Many analyses summarize a block using:

$$\boxed{GEN[B]}$$

and:

$$\boxed{KILL[B]}$$

GEN

Information produced/generated by the block.

KILL

Previously valid information invalidated/destroyed by the block.

Memory Trick

GEN = Give new information

KILL = Destroy old information

8. Generic GEN/KILL Equation

A very common forward-analysis form is:

$$\boxed{OUT[B] = GEN[B] \cup (IN[B] - KILL[B])}$$

Read this as:

1. Start with information entering B.
2. Remove whatever B kills.
3. Add whatever B generates.

9. Step-by-Step Interpretation

Suppose:

$$IN[B] = \{a, b, c, d\}$$

$$KILL[B] = \{b, d\}$$

$$GEN[B] = \{x, y\}$$

First:

$$IN[B] - KILL[B] = \{a, c\}$$

Then:

$$\text{OUT}[B] = \{x, y\} \cup \{a, c\}$$

Therefore:

$$\boxed{\text{OUT}[B] = \{a, c, x, y\}}$$

10. Important Caution

Not every data-flow analysis literally uses the exact same GEN/KILL equation.

For example:

- liveness uses USE/DEF equations
- constant propagation usually works over a value lattice rather than simple sets
- available expressions has its own GEN/KILL meaning

But the general idea remains:

$$\boxed{\text{Block generates some facts and invalidates others}}$$

11. Forward Data-Flow Analysis

In a **forward analysis**, information flows in the same direction as program execution.

```
TEXT
ENTRY
  ! "
B1
  ! "
B2
  ! "
B3
```

Information moves:

$$\boxed{\text{ENTRY} \rightarrow \text{EXIT}}$$

Examples include:

- constant propagation
- available expressions
- reaching-definitions-type reasoning

WRITE THIS IN NOTES

Forward analysis: Information propagates from predecessors toward successors.

12. Forward Merge Equation

For a block B with predecessors:

$$\text{Pred}(B) = \{P_1, P_2, \dots, P_k\}$$

its incoming information is obtained by combining predecessor OUT values:

$$\text{IN}[B] = \bigvee_{P \in \text{Pred}(B)} \text{OUT}[P]$$

Here $\bigvee$ means the appropriate **meet/merge operator**, which might be:

- union
- intersection
- lattice merge

depending on the analysis.

13. Practical Forward Pattern

Remember:

$$\text{Pred} \rightarrow \text{IN} \rightarrow \text{Block} \rightarrow \text{OUT}$$

That one line will help you enormously.

14. Backward Data-Flow Analysis

Some properties depend on the **future**.

Example:

Is x live now?

To answer that, we need to know whether x will be used **after** this point.

Therefore information flows backward:

$$\text{EXIT} \rightarrow \text{ENTRY}$$

The classic example is:

$$\text{Liveness Analysis}$$

WRITE THIS IN NOTES

Backward analysis: Information propagates from successors toward predecessors.

15. Backward Merge Equation

For backward analysis:

$$\text{OUT}[B]$$

is typically obtained by combining information at the IN sets of successors:

$$\text{OUT}[B] = \bigvee_{S \in \text{Succ}(B)} \text{IN}[S]$$

Then the block's transfer function determines:

$$\boxed{IN[B]=f_B(OUT[B])}$$

16. Backward Pattern

Remember:

$$\boxed{ Succ \rightarrow OUT \rightarrow Block \text{ backward} \rightarrow IN }$$

Compare:

Forward

$$Pred \rightarrow IN \rightarrow OUT$$

Backward

$$Succ \rightarrow OUT \rightarrow IN$$

17. Forward vs Backward — Master Table

| Forward | Backward |
|---------------------------------------|---------------------------------------|
| Information follows execution | Information flows opposite execution |
| Uses predecessors to compute IN | Uses successors to compute OUT |
| Transfer usually $IN \rightarrow OUT$ | Transfer usually $OUT \rightarrow IN$ |
| Constant propagation | Liveness |
| Available expressions | — |

Memory Trick

Forward

Past tells present.

Backward

Future tells present.

18. Example — Why Constant Propagation Is Forward

Code:

TEXT

```
x = 5
...
y = x + 1
```

The fact:

$\boxed{x=5}$

comes from an earlier definition and moves toward later statements.

So constant information flows:

$\boxed{\text{forward}}$

19. Why Liveness Is Backward

Suppose:

TEXT

```
1. x = a + b
2. ...
3. print(x)
```

Whether x is live at line 1 depends on the future use:

TEXT

```
print(x)
```

So information must flow backward from that use.

$\boxed{\text{Liveness is backward}}$

20. Meet / Merge Operator

At CFG joins, multiple paths may reach the same block.

We must combine the information.

The operation used to combine path information is often called the:

$\boxed{\text{Meet Operator}}$

Depending on the problem, it may be:

$\boxed{\cup}$

or:

$\boxed{\cap}$

or a more general lattice operation.

21. Why Union Sometimes?

Suppose the question is:

Could a definition reach this block along at least one path?

If yes on any incoming path, include it.

That is a:

$\boxed{\text{MAY property}}$

Typically combined with:

$\boxed{\cup}$

22. Why Intersection Sometimes?

Suppose the question is:

Is expression $a+b$ available along every incoming path?

It must hold on all paths.

That is a:

$\boxed{\text{MUST property}}$

Typically combined with:

$\boxed{\cap}$

23. MAY Analysis

A MAY analysis asks:

Could this fact hold on at least one relevant path?

If one predecessor says yes, the fact may be true.

Thus:

$\boxed{\text{MAY often uses UNION}}$

Memory Trick

MAY = ANY path

$\boxed{\text{ANY} \rightarrow \text{UNION}}$

24. MUST Analysis

A MUST analysis asks:

Must this fact hold on every relevant path?

A fact survives a join only if all incoming paths agree it holds.

Thus:

$\boxed{\text{MUST often uses INTERSECTION}}$

Memory Trick

MUST = ALL paths

$\boxed{ALL \rightarrow INTERSECTION}$

25. Example — MAY

Suppose B has predecessors P1 and P2:

$OUT[P_1] = \{a, b\}$

$OUT[P_2] = \{b, c\}$

If analysis asks which facts **may** reach B:

$IN[B] = OUT[P_1] \cup OUT[P_2]$

$= \{a, b, c\}$

26. Example — MUST

Same predecessor sets:

$OUT[P_1] = \{a, b\}$

$OUT[P_2] = \{b, c\}$

If facts must hold along **all** paths:

$IN[B] = OUT[P_1] \cap OUT[P_2]$

$= \{b\}$

Only **b** is common to both.

27. High-Value GATE Memory

$\boxed{MAY \rightarrow UNION}$

$\boxed{MUST \rightarrow INTERSECTION}$

But remember this is the common set-analysis pattern, not a universal substitute for understanding the analysis.

Constant propagation, for example, uses a lattice-style merge.

28. Analysis Classification Preview

Liveness

Question:

Might the variable's current value be used on some future path?

This is a:

$\boxed{\text{MAY analysis}}$

and commonly uses:

$\boxed{\text{cup}}$

It is:

$\boxed{\text{Backward}}$

29. Available Expressions

Question:

Has expression E been computed and remained valid along every path to this point?

This is:

$\boxed{\text{MUST analysis}}$

Uses:

$\boxed{\text{cap}}$

and is:

$\boxed{\text{Forward}}$

30. Constant Propagation

Constant propagation is:

$\boxed{\text{Forward}}$

At a merge, we need agreement about values.

Example:

TEXT

Path 1: $x = 5$

Path 2: $x = 5$

Then at join:

$\boxed{x=5}$

But:

TEXT

Path 1: $x = 5$
Path 2: $x = 10$

Then:

$\{ \text{is not one definite constant} \}$

This is handled by a value lattice/merge rather than ordinary set union/intersection.

We do this fully in Part 27.

31. GEN and KILL Example Intuition

Suppose we are tracking whether expression:

$a+b$

is available.

Block:

TEXT

$t = a + b$
 $a = 10$

The first statement computes $a+b$.

So it can:

$\text{GEN}(a+b)$

But the later assignment:

TEXT

$a = 10$

invalidates any expression containing a .

So it:

$\text{KILL}(a+b)$

By the end of the block, $a+b$ is **not** available from that original computation.

This shows why block-level GEN must consider the order of statements.

32. Important: GEN Is Not Simply "Everything Generated"

Suppose:

TEXT

1. $t = a + b$
2. $a = 5$

Line 1 generates $a+b$.

Line 2 kills it.

Therefore the block's final GEN set should not blindly include `a+b` as a fact surviving to block exit.

GEN typically means information generated by the block **and still valid at the relevant boundary**.

33. Similarly With Definitions

Consider:

TEXT

1. $x = 5$
2. $x = 10$

The first definition is overwritten by the second.

If tracking definitions that leave the block, the second may survive while the first is killed.

Order matters.

34. Boundary Conditions

Data-flow analysis needs a starting value.

Examples:

Forward analysis

We often initialize information at:

$\boxed{\text{ENTRY}}$

Backward analysis

We often initialize information at:

$\boxed{\text{EXIT}}$

These initial values are called:

$\boxed{\text{Boundary Conditions}}$

WRITE THIS IN NOTES

Boundary condition: Initial data-flow value assigned at the entry or exit boundary to start the analysis.

35. Interior Initialization

Besides boundary values, the algorithm initializes data-flow sets for ordinary blocks.

The correct initialization depends on whether the analysis is:

- may
- must

- lattice-based

For example:

MAY analyses

often start interior sets with:

$\boxed{\emptyset}$

MUST analyses

often start with the:

$\boxed{\text{universal set}}$

because iterative refinement moves downward by intersection.

But always follow the specific analysis definition.

36. Why Initialization Matters

Suppose a MUST analysis uses intersection.

If you incorrectly initialize every block to:

$\emptyset$

then:

$X \cap \emptyset = \emptyset$

and you may destroy all information immediately.

So MUST analyses often start optimistically with "everything possible" and remove facts until stable.

37. Fixed-Point Iteration

CFGs can contain loops.

Example:

```
TEXT
B1
! "
  B 2  % % ! '   B 3
! '           %
% % % % % % %
```

Information can circulate around the loop.

A single pass through blocks may therefore be insufficient.

We repeatedly apply equations until nothing changes.

The stable solution is called a:

$\boxed{\text{Fixed Point}}$

WRITE THIS IN NOTES

Fixed point: A state of data-flow analysis where another iteration produces no changes in any IN/OUT value.

Memory Trick

Repeat until stable.

38. Fixed-Point Example

Suppose initially:

TEXT

```
OUT[B1] = {}  
OUT[B2] = {}  
OUT[B3] = {}
```

After first iteration:

TEXT

```
OUT[B1] = {a}  
OUT[B2] = {a, b}  
OUT[B3] = {a, b, c}
```

Because B3 loops back to B2, B2 may gain additional information in the next iteration.

Eventually suppose:

TEXT

```
OUT[B1] = {a}  
OUT[B2] = {a, b, c}  
OUT[B3] = {a, b, c}
```

and another iteration gives exactly the same sets.

Then:

$\boxed{\text{fixed point reached}}$

39. Worklist Algorithm — Concept

Instead of blindly recomputing every block forever, compilers often use a:

$\boxed{\text{Worklist}}$

Conceptually:

1. Put blocks into a worklist.
2. Recompute a block.
3. If its data-flow value changes, neighboring affected blocks may need reprocessing.

4. Continue until worklist becomes empty.

For GATE, the key idea is:

$\boxed{\text{Iterate until no values change}}$

40. Monotonicity — Useful Intuition

Well-designed data-flow frameworks usually update information monotonically toward a stable solution.

For finite lattices/finite sets, this ensures the iterative process terminates.

You do not need heavy lattice theory for this syllabus.

Remember:

Proper data-flow equations repeatedly refine facts until convergence.

41. What Is a Data-Flow Lattice?

For analyses like constant propagation, information values are more than ordinary sets.

For a variable x , we might have states such as:

- unknown/not yet determined
- constant 5
- constant 10
- not a constant

These values form an ordered structure called a:

$\boxed{\text{Lattice}}$

The merge operator combines information arriving from paths.

We will use only the necessary intuition in Part 27.

42. Statement-Level vs Block-Level Analysis

We can define data-flow information:

- before/after individual statements
- before/after entire basic blocks

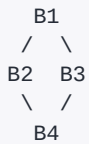
GATE often uses block-level equations because CFG nodes are basic blocks.

Within a block, instructions can then be processed sequentially to get finer statement-level information.

43. Example CFG Merge

Suppose:

TEXT



For a **forward** analysis:

$\backslash \text{IN}[B_4] \backslash$

depends on:

$\backslash \text{OUT}[B_2] \backslash$

and:

$\backslash \text{OUT}[B_3] \backslash$

Thus:

$\backslash \boxed{\text{IN}[B_4] = \text{OUT}[B_2] \backslash \text{meet} \backslash \text{OUT}[B_3] } \backslash$

44. Same CFG for Backward Analysis

For B1's successors B2 and B3:

$\backslash \text{OUT}[B_1] \backslash$

depends on:

$\backslash \text{IN}[B_2] \backslash$

and:

$\backslash \text{IN}[B_3] \backslash$

Thus:

$\backslash \boxed{\text{OUT}[B_1] = \text{IN}[B_2] \backslash \text{meet} \backslash \text{IN}[B_3] } \backslash$

This direction reversal is a favorite source of mistakes.

45. Forward Equation Pattern

Lock this:

$\backslash \boxed{\text{IN}[B] = \text{meet}_{\{P \in \text{Pred}(B)\}} \text{OUT}[P] } \backslash$

then:

$\backslash \boxed{\text{OUT}[B] = f_B(\text{IN}[B]) } \backslash$

46. Backward Equation Pattern

Lock this:

$\backslash \boxed{\text{OUT}[B] = \text{meet}_{\{S \in \text{Succ}(B)\}} \text{IN}[S] } \backslash$

then:

$\boxed{IN[B] = f_B(OUT[B])}$

47. GATE Shortcut: Direction Test

Ask:

Does the required information originate in the past or future?

From previous definitions/computations?

$\boxed{\text{Forward}}$

From future uses?

$\boxed{\text{Backward}}$

48. GATE Shortcut: MAY/MUST Test

Ask:

Is one path enough?

Then:

$\boxed{\text{MAY}}$

Usually union.

Ask:

Must it be true on every path?

Then:

$\boxed{\text{MUST}}$

Usually intersection.

49. Classification Table for Our Syllabus

| Analysis | Direction | Nature | Typical merge |
|-----------------------------|-----------|-----------------|---------------------|
| Constant propagation | Forward | value agreement | lattice/value merge |
| Liveness | Backward | MAY | Union |
| Available expressions / CSE | Forward | MUST | Intersection |

Memorize this table.

50. Why Liveness Is MAY

Suppose at a branch:

TEXT



Variable `x` is used later on path P but not path Q.

Is `x` live before the branch?

Yes.

Because there exists a future path on which its value is used.

Hence:

$\boxed{\text{Liveness} = \text{MAY}}$

51. Why Available Expressions Is MUST

Suppose expression:

$\boxed{a+b}$

was computed on path P, but not on Q.

At their join, can we safely say:

`a+b` has already been computed regardless of how we arrived here?

No.

It must be available on:

$\boxed{\text{every incoming path}}$

Hence:

$\boxed{\text{Available expressions} = \text{MUST}}$

52. Why Constant Propagation Needs Agreement

Suppose:

TEXT

Path 1: `x = 5`
Path 2: `x = 5`

At join:

$\boxed{x=5}$

safe.

But:

TEXT

Path 1: $x = 5$
Path 2: $x = 7$

No single constant is guaranteed.

So the merge says roughly:

$\lceil 5 \rceil \text{ meet } 7 = \text{not constant} \rceil$

This is why simple union doesn't capture constant propagation correctly.

53. Data-Flow Equation Solving Order

For acyclic CFGs, a suitable topological traversal may solve many analyses quickly.

But loops require iterative re-evaluation.

For GATE:

$\lceil \boxed{\text{Loops ! ' usually iterate to fixed point' }} \rceil$

54. Does Iteration Order Affect Final Answer?

For standard monotone data-flow frameworks, different reasonable processing orders should converge to the same intended fixed-point solution.

But some orders may converge faster.

Example:

Forward analysis

processing blocks roughly in forward/reverse-postorder often helps.

Backward analysis

reverse direction can help.

You do not need optimization of iteration order for basic GATE questions.

55. Entry/Exit Boundary Example

For liveness:

At procedure exit, if there are no special live variables required after return:

$\lceil \boxed{\text{OUT[EXIT]}} \text{ or equivalent boundary may be } \emptyset \rceil$

depending on formulation.

For available expressions:

At ENTRY, no expression can be assumed to have been computed before program entry:

$\lceil \boxed{\text{IN[ENTRY]} = \emptyset} \rceil$

These exact boundary values will be specified in upcoming parts.

56. Transfer Functions Must Be Conservative

Compiler analysis must avoid unsafe assumptions.

Suppose two paths disagree about whether x is constant.

The optimizer cannot simply pick one path's value.

It must conservatively represent:

$\boxed{\text{text}\{x \text{ is not known to be one constant}\}}$

Why?

Because optimization must preserve correctness for every possible execution.

57. Conservative Analysis

In compiler theory:

If uncertain, keep enough possibilities so optimization remains safe.

For example:

Liveness

If x might be used later:

$\boxed{\text{text}\{\text{Treat } x \text{ as live}\}}$

Available expression

If an expression may not have been computed on one incoming path:

$\boxed{\text{text}\{\text{Do not treat it as definitely available}\}}$

This explains MAY vs MUST behavior.

58. GATE-STYLE Example 1

Which is a forward analysis?

A. Liveness B. Constant propagation C. Neither D. Both

' B

59. GATE-STYLE Example 2

Which is a backward analysis?

$\boxed{\text{text}\{\text{Liveness}\}}$

because future use determines present liveness.

60. GATE-STYLE Example 3

A property should be included if it holds on **at least one** incoming path.

Which merge is typically used for set-based analysis?

$\boxed{\cup}$

because this is a MAY property.

61. GATE-STYLE Example 4

A property must hold on **every** incoming path.

Use:

$\boxed{\cap}$

for the usual set-based MUST analysis.

62. GATE-STYLE Example 5

For forward analysis:

$\text{Pred}(B) = \{P, Q\}$

Which information is combined to obtain $\text{IN}[B]$?

$\boxed{\text{OUT}[P] \text{ and } \text{OUT}[Q]}$

63. GATE-STYLE Example 6

For backward analysis:

$\text{Succ}(B) = \{S_1, S_2\}$

Which values are combined to obtain $\text{OUT}[B]$?

$\boxed{\text{IN}[S_1] \text{ and } \text{IN}[S_2]}$

64. GATE-STYLE Example 7

Suppose:

$\text{OUT}[P_1] = \{a, b\}$

$\text{OUT}[P_2] = \{b, c\}$

For a MAY analysis:

$\text{IN}[B] = ?$

$\{a, b\} \cup \{b, c\} = \boxed{\{a, b, c\}}$

65. GATE-STYLE Example 8

Same sets, but MUST analysis:

$IN[B] = \{a,b\} \cap \{b,c\}$

$\boxed{\{b\}}$

66. GATE-STYLE Example 9

What does fixed point mean?

A. Every block has one successor. B. IN/OUT sets no longer change after another iteration. C. CFG contains no loops. D. Every variable is constant.

' B

67. GATE TRAPS

Trap 1

Every data-flow analysis moves forward.

'L False.

Liveness is backward.

Trap 2

In a forward analysis, $IN[B]$ is computed from successors.

'L False.

Use:

$\boxed{Pred(B)}$

Trap 3

In backward analysis, $OUT[B]$ is computed from predecessors.

'L False.

Use:

$\boxed{Succ(B)}$

Trap 4

MAY analysis usually requires the fact on every path.

'L False.

One path is enough.

Trap 5

MUST analysis commonly uses union.

'L Usually false for set-based frameworks.

It commonly uses:

$\boxed{\cap}$

Trap 6

Available expressions is a MAY analysis.

'L False.

It is:

$\boxed{\text{MUST}}$

Trap 7

Liveness is a MUST analysis.

'L False.

It is:

$\boxed{\text{MAY}}$

Trap 8

Constant propagation can always merge path values using ordinary set union.

'L False.

It needs a value/lattice-style merge.

Trap 9

Loops can always be solved correctly with one pass.

'L False.

Usually iterate until fixed point.

Trap 10

GEN means every fact that appears anywhere in a block even if later destroyed.

'L False.

Block-level GEN must reflect facts generated and surviving according to that analysis's definition.

68. MINI PRACTICE — PART 26

Try first.

Q1

Which direction does liveness analysis use?

A. Forward B. Backward C. Neither D. Random

Q2

Which direction does available-expressions analysis use?

A. Forward B. Backward

Q3

For a forward analysis:

$\bigcup \text{IN}[B]$

is generally computed from:

A. successors' OUT B. predecessors' OUT C. successors' IN D. only GEN[B]

Q4

A set-based analysis asks:

Which facts hold on at least one path?

Which operator is normally appropriate?

A. Intersection B. Union

Q5

Available-expression analysis requires an expression to be valid on all incoming paths. It is therefore:

A. MAY B. MUST

Q6 — Tricky

What indicates that iterative data-flow solving is finished?

A. Every block has been visited once. B. Every variable is dead. C. Another iteration makes no IN/OUT changes. D. All GEN sets become empty.

SOLUTIONS

Q1

' B — Backward

Because liveness depends on future uses.

Q2

' A — Forward

Information about previously computed expressions moves in execution direction.

Q3

' B

$\boxed{IN[B] = \text{meet}_{\{P \in \text{Pred}(B)\}} OUT[P]}$

Q4

' B — Union

"At least one path" means:

$\boxed{MAY}$

Q5

' B — MUST

It has to hold on every incoming path.

Q6

' C

That is the:

$\boxed{\text{Fixed Point}}$

WRITE THIS IN NOTES — PART 26 MASTER SHEET

IN / OUT

$\boxed{IN[B] = \text{information before } B}$

$\boxed{OUT[B] = \text{information after } B}$

Forward Analysis

$\boxed{IN[B] = \text{meet}_{\{P \in \text{Pred}(B)\}} OUT[P]}$

$\boxed{OUT[B] = f_B(IN[B])}$

Memory:

Predecessors feed IN.

Backward Analysis

$$\boxed{OUT[B] = meet_{S \in Succ(B)} IN[S]}$$

$$\boxed{IN[B] = f_B(OUT[B])}$$

Memory:

Successors feed OUT.

GEN / KILL

$$\boxed{GEN = \text{facts generated}}$$

$$\boxed{KILL = \text{facts invalidated}}$$

Common form:

$$\boxed{OUT = GEN \cup (IN - KILL)}$$

when applicable.

MAY vs MUST

$$\boxed{MAY = \text{at least one path}}$$

Usually:

$$\boxed{\cup}$$

$$\boxed{MUST = \text{every path}}$$

Usually:

$$\boxed{\cap}$$

Fixed Point

$$\boxed{\text{Repeat equations until no IN/OUT value changes}}$$

Especially important with loops.

30-SECOND REVISION

Memorize this entire box:

TEXT

FORWARD

P r e d ! ' I N ! ' B l o c k ! ' O U T

BACKWARD

S u c c ! ' O U T ! ' B l o c k ! ' I N

MAY

ANY path

$\boxed{\text{cup}}$

MUST

ALL paths

$\boxed{\text{cap}}$

Our syllabus

TEXT

Constant propagation

! ' F O R W A R D

! ' v a l u e / l a t t i c e m e r g e

Liveness

! ' B A C K W A R D

! ' M A Y

! ' U N I O N

Available expressions

! ' F O R W A R D

! ' M U S T

! ' I N T E R S E C T I O N

GEN/KILL

GEN adds. KILL removes.

Fixed point

Repeat until nothing changes.

The framework is now ready. Next we'll apply it to the first official data-flow problem:

Constant Propagation

We will cover:

- UNDEF , constant and NAC
- constant-propagation lattice
- meet rules

- transfer through assignments
- CFG joins
- iterative solving
- difference between constant folding and constant propagation
- GATE-style path examples

Completed: Parts 1–26/31 | Next: Part 27 — Constant Propagation Data-Flow Analysis

Reply **NEXT** for **Part 27**.

USER

NEXT

ASSISTANT

Compiler Design — Part 27: Constant Propagation Data-Flow Analysis

Completed: Parts 1–26/31 | Current: Part 27 | Remaining: Parts 28–31

We already saw the local idea:

TEXT

```
x = 5
y = x + 2
```

Since `x` is known to be `5`, we can propagate:

TEXT

```
y = 5 + 2
```

and then fold:

TEXT

```
y = 7
```

But across branches and loops, constant propagation becomes a **data-flow problem**.

The compiler must determine:

At this program point, does a variable definitely have one known constant value on every relevant path?

1. What Is Constant Propagation?

CONCEPT

Constant propagation determines places where a variable has a known constant value and substitutes that constant for the variable.

Example:

TEXT

```
x = 10
y = x + 5
```

becomes:

TEXT

```
x = 10
y = 10 + 5
```

Then constant folding may produce:

TEXT

```
y = 15
```

WRITE THIS IN NOTES

Constant propagation: A forward data-flow analysis that determines whether variables have known constant values at program points and substitutes those constants where safe.

2. Direction

Suppose:

TEXT

```
x = 5
...
y = x + 1
```

The information:

$\backslash [x=5]$

comes from the past and moves toward later statements.

Therefore:

$\backslash [\boxed{\text{Constant propagation is FORWARD}}]$

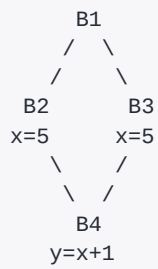
Memory Trick

Constant definitions flow forward to their uses.

3. Why Local Reasoning Is Not Enough

Consider:

TEXT



Both paths agree:

$\forall x=5$

So at B4:

$\forall \boxed{x=5}$

and:

TEXT

$y = x + 1$

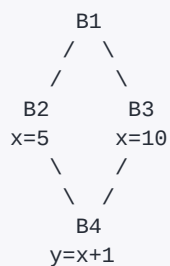
can become:

TEXT

$y = 6$

4. But What If Paths Disagree?

TEXT



At B4, x may be:

$\forall 5$

or:

$\forall 10$

Therefore:

$\forall \boxed{x \text{ is not one known constant}}$

We cannot safely replace x by either 5 or 10.

5. Constant-Propagation Abstract Values

For each variable, we conceptually track one of three kinds of information:

1. UNDEF
2. a particular constant
3. NAC

6. UNDEF

$\boxed{\text{UNDEF}}$

means roughly:

We do not yet have a defining constant value for this variable along the information currently considered.

It is often the initial state before useful information reaches a point.

7. Constant Value

A variable may have a known constant:

$\boxed{5}$

$\boxed{10}$

$\boxed{-3}$

etc.

Example:

TEXT

```
x = 5
```

makes x's abstract value:

$\boxed{5}$

8. NAC

$\boxed{\text{NAC}}$

means:

$\boxed{\text{Not A Constant}}$

It tells us that the analysis cannot represent the variable as one definite constant.

Example:

TEXT

```
x = input()
```

gives:

$\boxed{x=NAC}$

because x may take different runtime values.

WRITE THIS IN NOTES

TEXT

UNDEF

! ' no useful constant information yet

c

! ' definitely constant c

NAC

! ' not one definite constant

9. Memory Trick

U

Unknown so far

Constant

Known exactly

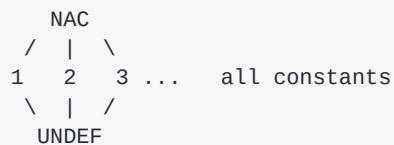
NAC

Not A Constant

10. Constant-Propagation Lattice Intuition

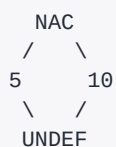
Conceptually:

TEXT



For example:

TEXT



Do not over-focus on lattice-theory terminology.

For GATE, the most important thing is the **merge table**.

11. Merge Rule — Same Constants

If two incoming paths say:

$x=5$

and:

$x=5$

then the join says:

$\boxed{x=5}$

So:

$\boxed{5 \text{ merge } 5=5}$

12. Merge Rule — Different Constants

If:

$x=5$

on one path and:

$x=10$

on another:

$\boxed{5 \text{ merge } 10=NAC}$

because x is not guaranteed to be one single constant.

13. Merge With NAC

If any incoming path says x is already not a constant:

NAC

and another says:

5

then overall:

$\boxed{NAC \text{ merge } 5=NAC}$

Similarly:

$\boxed{NAC \text{ merge } NAC=NAC}$

14. Merge With UNDEF

Operationally, UNDEF contributes no conflicting known value yet.

Thus:

$\boxed{\text{UNDEF} \text{ merge } 5=5}$

$\boxed{\text{UNDEF} \text{ merge } 10=10}$

and:

$\boxed{\text{UNDEF} \text{ merge } \text{NAC}=\text{NAC}}$

Also:

$\boxed{\text{UNDEF} \text{ merge } \text{UNDEF}=\text{UNDEF}}$

15. Master Merge Table

| Value 1 | Value 2 | Result |
|---------|-----------------------|--------|
| UNDEF | UNDEF | UNDEF |
| UNDEF | constant c | c |
| c | UNDEF | c |
| c | same c | c |
| c1 | different c2 | NAC |
| NAC | anything except UNDEF | NAC |
| NAC | UNDEF | NAC |

The core memory is:

Same constants survive. Different constants collapse to NAC.

16. The Most Important Rule

$\boxed{5 \text{ merge } 5=5}$

but:

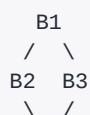
$\boxed{5 \text{ merge } 10=\text{NAC}}$

This single distinction solves many GATE questions.

17. Constant Propagation at a CFG Join

Suppose:

TEXT



Because this is forward analysis:

$\{ \text{IN}[B_4] \}$

is obtained by merging:

$\{ \text{OUT}[B_2] \}$

and:

$\{ \text{OUT}[B_3] \}$

variable by variable.

18. Example 1 — Paths Agree

Suppose:

$\{ \text{OUT}[B_2]: x=5, y=10 \}$

and:

$\{ \text{OUT}[B_3]: x=5, y=10 \}$

Then:

$\{ \text{IN}[B_4]: \boxed{x=5, y=10} \}$

Everything agrees.

19. Example 2 — Partial Agreement

Suppose:

$\{ \text{OUT}[B_2]: x=5, y=10 \}$

$\{ \text{OUT}[B_3]: x=5, y=20 \}$

Then:

x

$\{ 5 \}$ merge $5=5$

y

$\{ 10 \}$ merge $20=\text{NAC}$

Therefore:

$\{ \boxed{\text{IN}[B_4]: x=5, y=\text{NAC}} \}$

x can still be propagated.

y cannot.

20. Transfer Through Assignment $x = \text{constant}$

Suppose statement:

TEXT

$x = 7$

Then after the statement:

$\boxed{x=7}$

regardless of x's old value.

Example:

Before:

$x=NAC$

After:

TEXT

$x = 7$

we get:

$\boxed{x=7}$

because this statement gives x a new definite constant.

21. Transfer Through Copy $x = y$

Suppose:

TEXT

$x = y$

Then x receives y's abstract value.

If

$y=5$

then:

$\boxed{x=5}$

If

$y=NAC$

then:

$\boxed{x=NAC}$

If

$y=UNDEF$

then:

$\boxed{x=UNDEF}$

conceptually.

22. Transfer Through Arithmetic

Suppose:

TEXT

$$x = y + z$$

Case 1 — Both operands known constants

If:

$$\backslash[y=5, \backslash z=3 \backslash]$$

then:

$$\backslash[x=8 \backslash]$$

So:

$$\backslash[\boxed{x=8} \backslash]$$

23. Case 2 — An Operand Is NAC

If:

$$\backslash[y=NAC \backslash]$$

and:

$$\backslash[z=3 \backslash]$$

then normally:

$$\backslash[\boxed{x=NAC} \backslash]$$

because the result cannot be determined as one constant.

24. Case 3 — UNDEF Operand

If:

$$\backslash[y=UNDEF \backslash]$$

and:

$$\backslash[z=3 \backslash]$$

then x cannot yet be evaluated from current information.

Conceptually:

$$\backslash[\boxed{x=UNDEF} \backslash]$$

until additional information becomes available through iteration.

25. General Arithmetic Transfer Rule

For:

$\boxed{x=y \text{ op } z}$

If y and z are known constants:

Evaluate:

$\boxed{x=y \text{ op } z}$

If either is NAC:

Usually:

$\boxed{x=\text{NAC}}$

Otherwise if needed information is still UNDEF:

$\boxed{x=\text{UNDEF}}$

This is the useful GATE rule.

26. Example

Before statement:

$y=10$

$z=2$

Statement:

TEXT

$x = y * z$

After:

$\boxed{x=20}$

The analysis itself effectively performs constant evaluation.

27. Assignment From Unknown Runtime Input

TEXT

$x = \text{read}()$

or conceptually:

TEXT

$x = \text{input}$

If input is not compile-time known:

$\boxed{x=\text{NAC}}$

This kills any previous constant value of x.

28. Example

Before:

$\boxed{x=5}$

Then:

TEXT

```
x = input()
```

After:

$\boxed{x=NAC}$

The old `x=5` no longer matters.

29. Full Straight-Line Example

Code:

TEXT

```
1. a = 5
2. b = 10
3. c = a + b
4. d = c * 2
```

After line 1

$\boxed{a=5}$

After line 2

$\boxed{a=5, b=10}$

After line 3

$\boxed{c=15}$

After line 4

$\boxed{d=30}$

So the compiler can infer:

$\boxed{a=5, b=10, c=15, d=30}$

30. Constant Propagation + Folding

Original:

TEXT

```
a = 5
b = 10
c = a + b
```

$d = c * 2$

After propagation:

TEXT

$c = 5 + 10$
 $d = 15 * 2$

After folding:

TEXT

$c = 15$
 $d = 30$

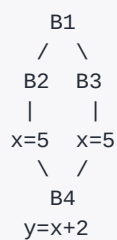
Again remember:

$\boxed{\text{Propagation substitutes; folding evaluates}}$

31. Full Branch Example

Consider:

TEXT



OUT[B2]

$\boxed{x=5}$

OUT[B3]

$\boxed{x=5}$

Merge:

$\boxed{5 \text{ merge } 5=5}$

Therefore:

$\boxed{\text{IN}[B4]:x=5}$

Then:

$\boxed{y=5+2=7}$

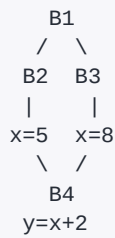
Thus:

$\boxed{y=7}$

32. Branch Example With Disagreement

Now:

TEXT



At B4:

$\{ \text{5} \setminus \text{merge8} = \text{NAC} \}$

Therefore:

$\{ x = \text{NAC} \}$

and:

$\{ y = \text{NAC} \}$

from the perspective of constant propagation.

We cannot replace `x` by one constant.

33. Important: NAC Does Not Mean Variable Has No Value

`NAC` does **not** mean runtime value is undefined.

It means:

The compiler cannot represent this variable as one definite compile-time constant.

At runtime `x` may have a perfectly valid value.

34. UNDEF vs NAC

This distinction is important.

UNDEF

Constant information has not yet been established / value not determined in the analysis state.

NAC

The compiler has determined that no single constant can safely represent the variable here.

So NAC contains **more negative information** than UNDEF.

35. GATE Trap

UNDEF and NAC mean the same thing.

'L False.

TEXT

```
U N D E F ! '   n o   u s e f u l   v a l u e   k n o w n   y e t
N A C ! '   k n o w n   n o t   t o   b e   o n e   c o n s t a n t
```

36. Loop Example

Consider:

TEXT

```
B1:
x = 0

B2:
x = x + 1
if (...) goto B2
```

Initially after B1:

$\{x=0\}$

But B2 has a back edge.

At later visits:

$\{x=1, 2, 3, \dots\}$

So at the loop header, x cannot remain one known constant in the fixed-point solution.

Eventually:

$\{x=\text{NAC}\}$

37. Why We Need Iteration

On the first pass, we may see only:

$\{x=0\}$

entering B2.

Then B2 computes:

$\{x=1\}$

and sends information around the back edge.

Now loop header receives:

- 0 from entry
- 1 from back edge

Merge:

$\{ \text{merge1} = \text{NAC} \}$

Then another iteration propagates NAC through the body.

Eventually values stop changing.

That's the fixed point.

38. Fixed-Point Loop Illustration

Initially:

$\{ \text{IN}[B2] = 0 \}$

After processing back edge:

$\{ \text{IN}[B2] = 0 \mid \text{merge1} = \text{NAC} \}$

Then:

TEXT

```
x = x + 1
```

with:

$\{ x = \text{NAC} \}$

gives:

$\{ x = \text{NAC} \}$

Next iteration remains NAC.

So:

$\{ \boxed{\text{fixed point reached}} \}$

39. But Some Loop Variables Can Stay Constant

Consider:

TEXT

```
x = 5
```

```
i = 0
```

```
while (...) {  
    i = i + 1  
    y = x + 2  
}
```

If `x` is never redefined:

$\{ \boxed{x=5} \}$

can remain constant throughout the loop.

So loops do not automatically make every variable NAC.

Only changing/uncertain values lose constant status.

40. Path Merge Example With UNDEF

Suppose one incoming path has:

$\{x = \text{UNDEF}\}$

and another has:

$\{x = 5\}$

Then operationally:

$\{\boxed{\text{UNDEF} \text{ merge } 5 = 5}\}$

This allows information discovered on one meaningful path to propagate during iterative solving.

41. Conditional Constant Branches

Suppose the compiler infers:

TEXT

```
x = 5
if (x < 10) goto L1
```

Then condition:

$\{5 < 10\}$

is always true.

This can enable:

- constant folding of condition
- unreachable-code elimination
- CFG simplification

So constant propagation often unlocks other optimizations.

42. Example

Original:

TEXT

```
x = 5
if x > 10 goto L1
y = 3
```

Propagate:

TEXT

```
if 5 > 10 goto L1
```

Fold:

$\{5 > 10 = \text{false}\}$

So branch is never taken.

Potentially remove it.

This demonstrates optimization interaction.

43. Constant Propagation Is Conservative

Suppose:

```
TEXT
if (...)
    x = 5
else
    x = 10

y = x + 1
```

Compiler cannot choose 5 or 10 without knowing which branch executes.

So:

$\boxed{x = \text{NAC}}$

at the join.

Safety is more important than guessing.

44. Constant Propagation vs Copy Propagation

Constant propagation

```
TEXT
x = 5
y = x + 1
```

Replace x with:

$\boxed{5}$

Copy propagation

```
TEXT
x = y
z = x + 1
```

Replace x with:

$\boxed{y}$

Memory:

```
TEXT
constant propagation != variable != constant
copy propagation != copied variable != source variable
```

45. Constant Propagation vs Constant Folding

Again:

Propagation

```
TEXT
x=5
y=x+2
```

becomes:

```
TEXT
y=5+2
```

Folding

```
TEXT
5+2
```

becomes:

```
TEXT
7
```

46. Constant Propagation vs CSE

Constant propagation tracks:

$\boxed{\text{Values of variables}}$

CSE/available expressions tracks:

$\boxed{\text{Whether expressions were already computed and remain valid}}$

Different problems.

47. Block-Level Constant Environment

Instead of sets, we can think of IN/OUT as maps:

$\text{IN}[B]: \{x \mapsto 5, y \mapsto \text{NAC}, z \mapsto 2\}$

Then the block executes statements and transforms the map into:

$\text{OUT}[B]$

48. Example Transfer Through a Block

Suppose:

$\{ \text{IN[B]: } a=5, b=3, c=\text{NAC} \}$

Block:

TEXT

```
x = a + b
y = x * 2
c = 10
```

x = a+b

$\{ x=8 \}$

y=x*2

$\{ y=16 \}$

c=10

old NAC is overwritten:

$\{ c=10 \}$

Therefore:

$\{ \boxed{\text{OUT[B]: } a=5, b=3, c=10, x=8, y=16} \}$

49. Another Block Example

$\{ \text{IN[B]: } a=5, b=\text{NAC} \}$

Block:

TEXT

```
x = a + b
y = 3
```

x :

$\{ 5+\text{NAC} \rightarrow \text{NAC} \}$

y :

$\{ 3 \}$

Therefore:

$\{ \boxed{x=\text{NAC}, y=3} \}$

50. Statement Redefinition

Suppose:

$\{ \text{IN[B]: } x=5 \}$

Block:

TEXT

$x = 7$

Then:

$\text{OUT}[B]: x=7$

The old constant is killed by the new definition.

So constant propagation still follows the general:

$\boxed{\text{new definition replaces old information}}$

idea.

51. GATE-STYLE Example 1

Two predecessors give:

$x=4$

and:

$x=4$

What is x at the join?

$\boxed{4}$

52. GATE-STYLE Example 2

Incoming values:

$x=4$

and:

$x=9$

Result:

$\boxed{\text{NAC}}$

53. GATE-STYLE Example 3

Incoming:

$x=\text{UNDEF}$

and:

$x=7$

Result:

$\boxed{7}$

54. GATE-STYLE Example 4

Incoming:

$\backslash[x=\text{NAC} \backslash]$

and:

$\backslash[x=7 \backslash]$

Result:

$\backslash[\boxed{\text{NAC}} \backslash]$

55. GATE-STYLE Example 5

Before:

$\backslash[a=3, \backslash b=4 \backslash]$

Statement:

TEXT

$x = a * b$

After:

$\backslash[\boxed{x=12} \backslash]$

56. GATE-STYLE Example 6

Before:

$\backslash[a=\text{NAC}, \backslash b=4 \backslash]$

Statement:

TEXT

$x = a * b$

Normally:

$\backslash[\boxed{x=\text{NAC}} \backslash]$

57. GATE-STYLE Example 7

Code:

TEXT

$x = 2$
 $y = x + 3$
 $z = y * 4$

Analysis:

$\backslash[x=2 \backslash]$

$\lfloor y=5 \rfloor$

$\lfloor z=20 \rfloor$

Therefore all three are constants at the end.

58. GATE-STYLE Example 8 — Branch

TEXT

```
if (...)
  x = 3
else
  x = 3

y = x + 1
```

Both paths:

$\lfloor x=3 \rfloor$

Therefore at join:

$\lfloor x=3 \rfloor$

and:

$\lfloor \boxed{y=4} \rfloor$

59. GATE-STYLE Example 9 — Branch Disagreement

TEXT

```
if (...)
  x = 3
else
  x = 4

y = x + 1
```

At join:

$\lfloor 3 \backslash \text{merge4}=\text{NAC} \rfloor$

So:

$\lfloor \boxed{x=\text{NAC}} \rfloor$

and no single constant for y can be inferred.

60. GATE-STYLE Example 10 — Loop

TEXT

```
x = 0
while (...) {
  x = x + 1
}
```


At repeated loop header visits, x can take different values.

Fixed-point analysis ultimately treats:

$\boxed{x = \text{NAC}}$

at the loop header/body in the general case.

61. GATE TRAPS

Trap 1

Constant propagation is backward analysis.

'L False.

$\boxed{\text{Forward}}$

Trap 2

If two paths give $x=5$ and $x=10$, choose whichever path is more likely.

'L False.

Compiler must remain correct for all paths.

$\boxed{x = \text{NAC}}$

Trap 3

Two paths both give $x=5$, so x becomes NAC.

'L False.

Agreement preserves:

$\boxed{x = 5}$

Trap 4

NAC means x has no runtime value.

'L False.

It means x is not known to be one compile-time constant.

Trap 5

UNDEF and NAC are identical.

'L False.

Trap 6

`x = 5 ; y = x` is only one optimization.

More precisely it combines:

1. constant propagation
2. constant folding

Trap 7

Every loop variable is NAC.

'LFalse.

A variable not modified and known constant may remain constant.

Trap 8

Once a variable becomes NAC, assigning `x=5` cannot make it constant again.

'LFalse.

A new definite assignment:

TEXT

`x = 5`

sets:

$\boxed{x=5}$

at that point.

Trap 9

A join of different constants can be represented by keeping both constants and choosing later.

Not for ordinary constant propagation.

The safe abstract value is:

$\boxed{NAC}$

Trap 10

Constant propagation uses ordinary union like liveness.

'LFalse.

It uses a value/lattice-style merge.

62. MINI PRACTICE — PART 27

Try these first.

Q1 — Easy

Constant propagation is:

A. Forward B. Backward

Q2 — Easy

Merge:

$\backslash$ 5\ merge5 $\backslash$

equals?

Q3 — Easy

Merge:

$\backslash$ 5\ merge10 $\backslash$

equals?

Q4 — Medium

Merge:

$\backslash$ UNDEF\ merge7 $\backslash$

equals?

Q5 — Medium

Merge:

$\backslash$ NAC\ merge7 $\backslash$

equals?

Q6 — Tricky

Before a block:

$\backslash$ a=4,\ b=5 $\backslash$

Block:

TEXT

```
x = a + b
y = x * 2
```

Find x and y at block exit.

SOLUTIONS

Q1

' A — Forward

Earlier definitions flow toward later uses.

Q2

$\boxed{5}$

Same constant on both paths remains known.

Q3

$\boxed{\text{NAC}}$

Different constants mean no single constant is guaranteed.

Q4

$\boxed{7}$

UNDEF contributes no conflicting known value in the standard iterative interpretation.

Q5

$\boxed{\text{NAC}}$

One path is already not a definite constant.

Q6

First:

$x=4+5=9$

Then:

$y=9*2=18$

Therefore:

$\boxed{x=9, y=18}$

WRITE THIS IN NOTES — PART 27 MASTER SHEET

Constant Propagation

$\boxed{\text{Forward data-flow analysis}}$

Tracks each variable as:

TEXT

UNDEF

constant c

Meanings

$\boxed{\text{UNDEF}=\text{not established yet}}$

$\boxed{c=\text{definitely constant } c}$

$\boxed{\text{NAC}=\text{not one definite constant}}$

Merge Rules

$\boxed{c \text{ merge } c=c}$

$\boxed{c_1 \text{ merge } c_2=\text{NAC} \text{ if } c_1 \neq c_2}$

$\boxed{\text{UNDEF} \text{ merge } c=c}$

$\boxed{\text{NAC} \text{ merge } c=\text{NAC}}$

Transfer Rules

x = c

$\boxed{x=c}$

x = y

$\boxed{x=\text{value}(y)}$

x = y op z

Both constants:

$\boxed{x=\text{evaluate}(y \text{ op } z)}$

Any operand NAC:

$\boxed{x=\text{NAC}}$

Required operand still UNDEF:

$\boxed{x=\text{UNDEF}}$

CFG Join

For forward analysis:

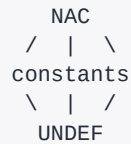
$\boxed{\text{IN}[B]=\text{merge of predecessor OUTs}}$

performed variable by variable.

30-SECOND REVISION

Remember the ladder:

TEXT



Same constants

$\boxed{5+5 \rightarrow 5}$

Different constants

$\boxed{5+10 \rightarrow \text{NAC}}$

Unknown + constant

$\boxed{\text{UNDEF}+5 \rightarrow 5}$

NAC + constant

$\boxed{\text{NAC}+5 \rightarrow \text{NAC}}$

Direction

$\boxed{\text{text}\{\text{FORWARD}\}}$

Best memory

All paths agree on the same constant !' propagate it.

Paths disagree !' NAC.

And never confuse:

TEXT

Propagation !' replace variable with constant
Folding !' calculate constant expression

Next comes one of the most frequently tested compiler data-flow analyses:

Liveness Analysis

We will cover:

- what "live" and "dead" really mean
- **USE** and **DEF**
- exact equations

$\text{OUT}[B] = \bigcup \text{IN}[\text{successors}]$

$\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] - \text{DEF}[B])$

- backward iteration
- statement-by-statement liveness
- connection to dead-code elimination and register allocation

Reply **NEXT** for **Part 28**.

USER

NEXT

ASSISTANT

Compiler Design — Part 28: Liveness Analysis

Completed: Parts 1–27/31 | Current: Part 28 | Remaining: Parts 29–31

We now study one of the most important GATE data-flow analyses:

$\boxed{\text{Liveness Analysis}}$

The central question is:

Will the current value of a variable be needed later?

This is used heavily in:

- register allocation
- dead-code elimination
- optimization

The two equations you must eventually know are:

$\boxed{OUT[B] = \bigcup_{S \in \text{Succ}(B)} IN[S]}$

and:

$\boxed{IN[B] = USE[B] \cup (OUT[B] - DEF[B])}$

We will understand them instead of just memorizing them.

1. What Does "Live" Mean?

A variable is **live at a program point** if its **current value may be used later before being overwritten**.

Example:

TEXT

```
1. x = 10
2. y = x + 5
```

Immediately after line 1:

$\boxed{x \text{ is live}}$

because the value `10` stored in `x` will be used in line 2.

WRITE THIS IN NOTES

A variable is live at a point if there exists a path from that point along which its current value is used before the variable is redefined.

The phrase:

`\[ \boxed{\text{used before redefined}} \]`

is extremely important.

2. What Does "Dead" Mean?

A variable's current value is **dead** if it will not be used before another definition replaces it.

Example:

TEXT

```
1. x = 10
2. x = 20
3. print(x)
```

After line 1, the value `10` is dead because line 2 overwrites `x` before any use.

Therefore line 1 may be removable if its RHS has no side effects.

3. Variable vs Value

Be precise:

Liveness refers to the current value of a variable, not merely whether the variable's name appears somewhere later.

Example:

TEXT

```
x = 10
x = 20
print(x)
```

`x` certainly appears later.

But the value produced by:

TEXT

```
x = 10
```

is dead.

Why?

Because:

TEXT

```
x = 20
```

kills that old value.

4. Why Is Liveness a Backward Analysis?

Consider:

TEXT

```
x = a + b
...
print(x)
```

To know whether x is needed **now**, we must inspect what happens **later**.

Therefore information flows:

$\boxed{\text{from future to past}}$

Hence:

$\boxed{\text{Liveness is a BACKWARD data-flow analysis}}$

Memory Trick

Live? Look Later.

Because liveness asks about future uses.

5. Why Is Liveness a MAY Analysis?

Suppose:

TEXT

```
    B
   / \
  P   Q
```

P: use x

Q: do not use x

At B, is x live?

Yes.

There exists **one possible future path** where x is used.

Therefore:

$\boxed{\text{Liveness is a MAY analysis}}$

and the merge operator is:

$\boxed{\text{cup}}$

WRITE THIS IN NOTES

$\boxed{\text{Liveness} = \text{Backward} + \text{MAY} + \text{Union}}$

This single line is worth memorizing.

6. Two Sets: USE and DEF

For every basic block B , define:

$\boxed{\text{USE}[B]}$

and:

$\boxed{\text{DEF}[B]}$

These summarize how variables are used and defined inside the block.

7. What Is DEF[B]?

$\text{DEF}[B]$

contains variables assigned/defined in B **before any later need for their incoming value matters**.

For standard liveness block equations, think:

Variables whose values are defined in the block.

Example:

TEXT

```
x = a + b
y = x + 1
```

Definitions:

$\boxed{\text{DEF}[B] = \{x, y\}}$

8. What Is USE[B]?

This is slightly more subtle.

$\text{USE}[B]$

contains variables whose values are **used in the block before any definition of that variable in the block**.

This "before definition" condition is essential.

WRITE THIS IN NOTES

USE[B]: Variables used before being defined within B.

DEF[B]: Variables defined within B.

For liveness, USE is often also called:

$\boxed{\text{UEVAR}}$

upward-exposed variables.

9. Simple USE/DEF Example

Block:

TEXT

1. $x = a + b$
2. $y = x + c$

Scan top to bottom.

Line 1

Uses:

$\{a, b\}$

Neither has been defined earlier in this block.

So:

$\{a, b\} \in \text{USE}[B]$

Defines:

$\{x\}$

So:

$x \in \text{DEF}[B]$

Line 2

Uses:

$\{x, c\}$

But x was already defined in this block.

Therefore x does **not** enter $\text{USE}[B]$.

c has not been defined.

So:

$c \in \text{USE}[B]$

Defines y .

Final:

$\boxed{\text{USE}[B] = \{a, b, c\}}$

$\boxed{\text{DEF}[B] = \{x, y\}}$

10. Critical USE Trap

Consider:

TEXT

1. $x = 10$
2. $y = x + a$

Is x in $\text{USE}[B]$?

'L No.

Because x is defined in line 1 before its use in line 2.

Its incoming value from before the block is irrelevant.

So:

$\boxed{\text{USE}[B] = \{a\}}$

and:

$\boxed{\text{DEF}[B] = \{x, y\}}$

11. Opposite Example

TEXT

1. $y = x + a$
2. $x = 10$

Now x is used before its definition.

Therefore:

$x \in \text{USE}[B]$

and a also is used.

So:

$\boxed{\text{USE}[B] = \{x, a\}}$

Definitions:

$\boxed{\text{DEF}[B] = \{y, x\}}$

Order matters.

12. Fast USE/DEF Algorithm

Scan statements **top to bottom**.

For every used variable v :

If v is not already in DEF, add v to USE.

For every defined variable v :

Add v to DEF.

Memory Trick

USE only cares about uses before first definition.

13. Liveness OUT Equation

Because liveness flows backward, $OUT[B]$ depends on successors.

If:

$$\forall Succ(B) = \{S_1, S_2, \dots, S_k\}$$

then:

$$\forall \boxed{OUT[B] = \bigcup_{S \in Succ(B)} IN[S]}$$

Why union?

Because if a variable is live into **any successor**, it may be needed after B.

14. Example

Suppose:

$$\forall IN[B_2] = \{x, y\}$$

and:

$$\forall IN[B_3] = \{y, z\}$$

and both are successors of B1.

Then:

$$\forall OUT[B_1] = IN[B_2] \cup IN[B_3]$$

$$\forall = \{x, y, z\}$$

Thus:

$$\forall \boxed{OUT[B_1] = \{x, y, z\}}$$

15. Liveness IN Equation

The key equation is:

$$\forall \boxed{IN[B] = USE[B] \cup (OUT[B] - DEF[B])}$$

Let's understand each piece.

16. Why Include $USE[B]$?

If B uses variable x before redefining it, then x must already have a meaningful value when entering B.

Therefore:

$$\forall x \in USE[B] \rightarrow x \in IN[B]$$

Hence:

$$\forall USE[B]$$

is automatically included.

17. Why $OUT - DEF$?

Suppose x is live after B.

Normally that means x should be live before B too.

But what if B defines x?

Example:

TEXT

B:
x = 10

and x is used after B.

The value needed after B is the **new value 10**, not x's incoming old value.

So x does not need to be live before B merely because it is live afterward.

Thus definitions in B kill the incoming value's liveness:

$$\forall \boxed{OUT[B] - DEF[B]}$$

18. Full Intuition

$$\forall IN[B] = USE[B] \cup (OUT[B] - DEF[B])$$

Read:

A variable is live before B if either:

1. B itself needs its incoming value, or
2. it is live after B and B does not redefine it.

That's the whole equation.

Memory Trick

19. Equation Pair — Must Memorize

$$\boxed{OUT[B] = \bigcup_{S \in Succ(B)} IN[S]}$$

$$\boxed{IN[B] = USE[B] \cup (OUT[B] - DEF[B])}$$

20. Straight-Line Block Example

Suppose:

$$USE[B] = \{a, b\}$$

$$DEF[B] = \{x\}$$

and:

$$OUT[B] = \{x, c\}$$

Then:

$$OUT[B] - DEF[B] = \{x, c\} - \{x\} = \{c\}$$

Thus:

$$IN[B] = \{a, b\} \cup \{c\}$$

$$\boxed{IN[B] = \{a, b, c\}}$$

Notice x is not live before the block because B creates the x value needed afterward.

21. Statement-Level Liveness

We can also perform liveness for individual instructions.

For instruction:

$$x = y + z$$

we have:

USE

$$\{y, z\}$$

DEF

$$\{x\}$$

Then:

$$\boxed{IN = USE \cup (OUT - DEF)}$$

The same equation applies.

22. Statement-Level Example

Instruction:

TEXT

$x = y + z$

Suppose after the instruction:

$\text{OUT} = \{x, w\}$

Then:

$\text{USE} = \{y, z\}$

$\text{DEF} = \{x\}$

So:

$\text{IN} = \{y, z\} \cup (\{x, w\} - \{x\})$

$= \{y, z\} \cup \{w\}$

$\boxed{\text{IN} = \{y, z, w\}}$

23. Why x Disappears From IN

The x needed after the statement is produced **by this statement**.

Its older incoming value is irrelevant.

Thus x is removed by:

$-\text{DEF}$

24. Statement $x = y$

For:

TEXT

$x = y$

$\text{USE} = \{y\}$

$\text{DEF} = \{x\}$

So if:

$\text{OUT} = \{x, z\}$

then:

$\text{IN} = \{y\} \cup \{z\}$

$\boxed{\text{IN} = \{y, z\}}$

25. Statement $\text{print}(x)$

If:

TEXT

```
print(x)
```

does not define any variable:

$$\mathbb{U} \text{ USE} = \{x\}$$
$$\mathbb{U} \text{ DEF} = \emptyset$$

Therefore:

$$\mathbb{U} \text{ IN} = \{x\} \cup \text{OUT}$$

So x definitely becomes live before the print.

26. Statement $x = 5$

Here:

$$\mathbb{U} \text{ USE} = \emptyset$$
$$\mathbb{U} \text{ DEF} = \{x\}$$

If:

$$\mathbb{U} \text{ OUT} = \{x, y\}$$

then:

$$\mathbb{U} \text{ IN} = \emptyset \cup (\{x, y\} - \{x\})$$
$$\boxed{\mathbb{U} \text{ IN} = \{y\}}$$

The incoming old value of x is dead.

27. Full Basic-Block Example

Consider:

TEXT

```
1. x = a + b
2. y = x + c
3. z = y + d
```

Assume:

$$\mathbb{U} \text{ OUT}[B] = \{z, w\}$$

First compute block USE/DEF.

28. USE/DEF

Line 1:

Uses a, b before definitions:

$\backslash[\text{USE}=\{a,b\} \backslash]$

Define x:

$\backslash[\text{DEF}=\{x\} \backslash]$

Line 2:

x already defined !' not USE.

c not defined:

$\backslash[\text{USE}=\{a,b,c\} \backslash]$

Define y.

Line 3:

y already defined.

d not defined:

$\backslash[\text{USE}=\{a,b,c,d\} \backslash]$

Define z.

Therefore:

$\backslash[\boxed{\text{USE}[B]=\{a,b,c,d\}} \backslash]$

$\backslash[\boxed{\text{DEF}[B]=\{x,y,z\}} \backslash]$

29. Compute IN

$\backslash[\text{OUT}[B]=\{z,w\} \backslash]$

Remove DEF:

$\backslash[\text{OUT-DEF} = \{z,w\} - \{x,y,z\} = \{w\} \backslash]$

Then:

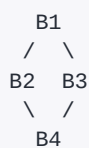
$\backslash[\text{IN} = \{a,b,c,d\} \cup \{w\} \backslash]$

$\backslash[\boxed{\text{IN}[B]=\{a,b,c,d,w\}} \backslash]$

30. CFG Example

Consider:

TEXT



Suppose:

$\backslash[\text{IN}[B_2]=\{x,y\} \backslash]$

$\backslash[\text{IN}[B_3]=\{y,z\} \backslash]$

Then:

$$[\text{OUT}[B_1] = \{x,y\} \cup \{y,z\}]$$

$$[\boxed{\text{OUT}[B_1] = \{x,y,z\}}]$$

Again:

$$[\boxed{\text{union because liveness is MAY}}]$$

31. Exit Boundary Condition

If nothing is assumed live after procedure exit, we commonly use:

$$[\boxed{\text{OUT}[\text{EXIT}] = \text{emptyset}}]$$

or for final block:

$$[\boxed{\text{OUT}[B_{\text{final}}] = \text{emptyset}}]$$

unless the problem specifies things like:

- return variables
- globals
- output parameters

that must be considered live.

Always use the boundary condition given in the question.

32. Iterative Liveness Algorithm

Because CFGs can contain loops:

Step 1

Initialize IN and OUT, often to empty sets.

Step 2

For every block, typically process backward:

$$[\text{OUT}[B] = \bigcup \text{IN}[\text{successors}]]$$

Step 3

Then:

$$[\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] - \text{DEF}[B])]$$

Step 4

Repeat until no IN/OUT set changes.

That is the:

$$[\boxed{\text{fixed point}}]$$

33. Why Process Backward?

Since successors determine OUT, it is often efficient to process blocks roughly from EXIT toward ENTRY.
But with loops, multiple iterations may still be necessary.

34. Loop Example

CFG:

```
TEXT
B 1  !'   B 2  !'   B 3
      !'           |
      % % % % % % %
```

Suppose B3 uses x.
Then x becomes live into B3.
That information propagates backward into B2.
Because B3 loops to B2, it may affect B3 again.
So iteration is required until stable.

35. Full Tiny Loop Example

Blocks:

B1

```
TEXT
x = 0
```

B2

```
TEXT
y = x + 1
```

B3

```
TEXT
x = y
if (...) goto B2
```

Assume exit after B3 otherwise.
The liveness of x/y propagates around the back edge.
A single pass may miss variables that become live due to the loop.
Thus:
 $\boxed{\text{repeat to fixed point}}$

36. Liveness and Dead-Code Elimination

Consider:

TEXT

```
x = a + b
```

If x is **not live after this statement**, then its computed value is never used.

Assuming RHS has no side effects:

$\boxed{\text{assignment may be dead}}$

and can potentially be removed.

WRITE THIS IN NOTES

For assignment:

$\boxed{x = \text{expr}}$

if:

$\boxed{x \notin \text{OUT}}$

and `expr` has no required side effects:

$\boxed{\text{statement is a dead assignment}}$

37. Example

TEXT

```
1. x = a + b
2. y = c + d
3. print(y)
```

After line 1, x is never used.

So:

$\boxed{x \notin \text{OUT}[1]}$

Therefore line 1 may be dead.

38. But Be Careful With Side Effects

TEXT

```
x = f()
```

Even if x itself is dead, `f()` might:

- print
- modify memory
- perform I/O

So deleting the whole statement may be unsafe.

Liveness of destination alone is not sufficient when RHS has side effects.

39. Liveness and Register Allocation

Suppose two variables are simultaneously live.

Then both values must be preserved at the same time.

They generally cannot occupy the same register at that point.

This leads to:

$\boxed{\text{Interference}}$

between live variables.

Register allocators often use liveness information to build:

$\boxed{\text{Interference Graphs}}$

40. Interference Intuition

If x and y are live at the same program point:

$\boxed{x \text{ and } y \text{ interfere}}$

because one register cannot safely hold both distinct values simultaneously.

Thus liveness is central to register allocation.

41. Live Range

The **live range** of a value is the portion of the program during which that value is live.

Example:

TEXT

```
1. x = 5
2. y = x + 1
3. z = x + 2
4. x = 10
```

The value `x=5` is live from after line 1 through its last use at line 3.

The new value `x=10` has a different live range.

42. Definition Kills Old Liveness

Suppose:

TEXT

```
x = 5
```

This definition creates a new value for x.

Therefore any old value of x coming from before the statement is no longer needed merely because the new x may be used later.

This explains:

$\backslash [\text{OUT-DEF}]$

again.

43. Use Generates Liveness

Suppose:

TEXT

$y = x + 1$

Even if x was not considered live after the statement, it must be live before it because the statement itself uses x.

This explains:

$\backslash [\text{USE}]$

being added.

44. GEN/KILL Analogy

For liveness, instead of GEN/KILL, we usually write:

$\backslash [\boxed{\text{USE}}]$

and:

$\backslash [\boxed{\text{DEF}}]$

Conceptually:

- USE **generates** liveness backward.
- DEF **kills** incoming liveness of the old value.

So:

$\backslash [\text{IN} = \text{USE} \cup (\text{OUT-DEF})]$

matches the GEN/KILL idea.

45. Backward Statement-by-Statement Method

Given a basic block, you can find liveness at each statement very quickly by scanning from the bottom upward.

Start with the block's OUT set.

For each statement, moving upward:

$\backslash [\boxed{\text{IN} = \text{USE} \cup (\text{OUT-DEF})}]$

Then that IN becomes the OUT of the previous statement.

46. Worked Statement-Level Example

Code:

TEXT

1. $a = b + c$
2. $d = a + e$
3. $f = d + c$

Assume at block exit:

$\{ \text{OUT}_3 = \{f\} \}$

Let's work backward.

47. Line 3

TEXT

$f = d + c$

$\{ \text{USE}_3 = \{d, c\} \}$

$\{ \text{DEF}_3 = \{f\} \}$

$\{ \text{OUT}_3 = \{f\} \}$

Therefore:

$\{ \text{IN}_3 = \{d, c\} \cup \{f\} - \{f\} \}$

$\{ \boxed{\text{IN}_3 = \{d, c\}} \}$

48. Line 2

Its OUT is line 3's IN:

$\{ \text{OUT}_2 = \{d, c\} \}$

Statement:

TEXT

$d = a + e$

$\{ \text{USE}_2 = \{a, e\} \}$

$\{ \text{DEF}_2 = \{d\} \}$

Thus:

$\{ \text{IN}_2 = \{a, e\} \cup \{d, c\} - \{d\} \}$

$\{ = \{a, e\} \cup \{c\} \}$

$\{ \boxed{\text{IN}_2 = \{a, c, e\}} \}$

49. Line 1

$\text{OUT}_1 = \text{IN}_2 = \{a, c, e\}$

Statement:

TEXT

$a = b + c$

$\text{USE}_1 = \{b, c\}$

$\text{DEF}_1 = \{a\}$

Then:

$\text{IN}_1 = \{b, c\} \cup (\{a, c, e\} - \{a\})$

$= \{b, c\} \cup \{c, e\}$

$\boxed{\text{IN}_1 = \{b, c, e\}}$

50. Final Statement-Level Table

| Line | IN | OUT |
|------|---------------|---------------|
| 1 | $\{b, c, e\}$ | $\{a, c, e\}$ |
| 2 | $\{a, c, e\}$ | $\{d, c\}$ |
| 3 | $\{d, c\}$ | $\{f\}$ |

This is a very typical GATE problem format.

51. One Important Boundary Caveat

If f is not actually needed after line 3, then the block exit OUT might instead be:

$\emptyset$

Then line 3's assignment to f could itself be dead.

So always use the specified exit liveness.

52. GATE-STYLE Example 1

Statement:

TEXT

$x = y + z$

Given:

$\text{OUT} = \{x, w\}$

Find IN.

$\text{USE} = \{y, z\}$

$\text{DEF} = \{x\}$

Thus:

$\text{IN} = \{y, z\} \cup (\{x, w\} - \{x\})$

$\boxed{\{y, z, w\}}$

53. GATE-STYLE Example 2

Statement:

TEXT

`x = 10`

and:

$\text{OUT} = \{x, y\}$

Then:

$\text{USE} = \emptyset$

$\text{DEF} = \{x\}$

$\text{IN} = \{y\}$

So old x is dead before the definition.

54. GATE-STYLE Example 3

Statement:

TEXT

`print(x)`

Suppose:

$\text{OUT} = \{y\}$

Then:

$\text{USE} = \{x\}$

$\text{DEF} = \emptyset$

So:

$\text{IN} = \{x, y\}$

55. GATE-STYLE Example 4 — Branch Merge

Suppose B has two successors:

$\text{IN}[S_1] = \{a, b\}$

$\{ \mid \text{IN}[S_2] = \{b, c\} \}$

Then:

$\{ \mid \text{OUT}[B] = \{a, b\} \cup \{b, c\} \}$

$\{ \boxed{\text{OUT}[B] = \{a, b, c\}} \}$

56. GATE-STYLE Example 5 — Dead Assignment

TEXT

$x = a + b$

Suppose:

$\{ x \notin \text{OUT} \}$

and RHS has no side effects.

Then:

$\{ \boxed{\text{assignment is potentially dead}} \}$

57. GATE-STYLE Example 6 — USE/DEF

Block:

TEXT

1. $x = a + b$
2. $c = x + d$
3. $a = 10$
4. $y = a + e$

Find USE/DEF.

Scan top-down.

Line 1:

Uses a, b before definitions:

$\{ \text{USE} = \{a, b\} \}$

Define x.

Line 2:

x already defined.

d not defined:

$\{ \text{USE} = \{a, b, d\} \}$

Define c.

Line 3:

Define a.

But a was already in USE because its incoming value was used at line 1.

Line 4:

a is now defined, so this later use does not add anything new.

e not defined:

$\backslash[\text{USE}=\{a,b,d,e\} \backslash]$

Definitions:

$\backslash[\boxed{\text{DEF}=\{x,c,a,y\}} \backslash]$

Therefore:

$\backslash[\boxed{\text{USE}=\{a,b,d,e\}} \backslash]$

58. Why a Can Be in Both USE and DEF

In the previous block, a is:

- used before its definition
- later defined

So it belongs to:

$\backslash[\text{USE}[B] \backslash]$

and:

$\backslash[\text{DEF}[B] \backslash]$

This is allowed.

The sets are not necessarily disjoint.

GATE TRAP

A variable cannot belong to both USE and DEF.

'L False.

It can be used before being defined later in the same block.

59. GATE TRAPS

Trap 1

Liveness is forward analysis.

'L False.

$\backslash[\boxed{\text{Backward}} \backslash]$

Trap 2

Liveness is MUST analysis.

'L False.

$\boxed{\text{MAY}}$

Trap 3

Liveness uses intersection at branches.

'L False.

$\boxed{\text{Union}}$

Trap 4

In backward analysis, $\text{OUT}[B]$ comes from predecessors.

'L False.

$\boxed{\text{OUT}[B] \text{ comes from successors}}$

Trap 5

Every variable used anywhere in a block belongs to $\text{USE}[B]$.

'L False.

Only if used before its first relevant definition in the block.

Trap 6

If x is defined in B and live after B , then old x must be live before B .

'L False.

The definition may generate the new x value needed afterward.

Trap 7

If x appears later in the program, its current value is automatically live.

'L False.

An intervening redefinition may kill the current value.

Trap 8

A variable cannot be in both USE and DEF .

'L False.

Trap 9

If destination x is dead, $x=f()$ can always be removed.

'LFalse.

$f()$ may have side effects.

Trap 10

Liveness has nothing to do with register allocation.

'LFalse.

It is fundamental to register allocation.

60. MINI PRACTICE — PART 28

Try first.

Q1

Liveness analysis is:

A. Forward MAY B. Forward MUST C. Backward MAY D. Backward MUST

Q2

For liveness:

$\backslash$ OUT[B] $\backslash$

is computed using:

A. predecessor OUT sets B. successor IN sets C. predecessor IN sets D. DEF only

Q3

Given:

$\backslash$ USE[B] = $\{a, b\}$ $\backslash$

$\backslash$ DEF[B] = $\{x\}$ $\backslash$

$\backslash$ OUT[B] = $\{x, c\}$ $\backslash$

find IN[B].

Q4

Block:

TEXT

1. $x = 10$
2. $y = x + a$

Find USE[B].

Q5

Block:

TEXT

1. $y = x + a$
2. $x = 10$

Find $USE[B]$.

Q6 — Tricky

For assignment:

TEXT

$x = a + b$

if x is not live afterward and RHS has no side effects, what optimization may apply?

SOLUTIONS

Q1

' C — Backward MAY

$\boxed{\text{Backward} + \text{MAY} + \text{Union}}$

Q2

' B

$\boxed{OUT[B] = \bigcup_{S \in \text{Succ}(B)} IN[S]}$

Q3

$IN[B] = USE[B] \cup (OUT[B] - DEF[B])$

$= \{a, b\} \cup (\{x, c\} - \{x\})$

$= \{a, b\} \cup \{c\}$

$\boxed{\{a, b, c\}}$

Q4

TEXT

$x = 10$

defines x before line 2 uses it.

So x is not in block USE .

a is used without prior definition.

$\boxed{\text{USE}[B] = \{a\}}$

Q5

x and a are used in line 1 before either is defined in the block.

Therefore:

$\boxed{\text{USE}[B] = \{x, a\}}$

Q6

$\boxed{\text{text}\{\text{Dead-code / dead-assignment elimination}\}}$

WRITE THIS IN NOTES — PART 28 MASTER SHEET

Definition

A variable is live if:

$\boxed{\text{text}\{\text{its current value may be used later before redefinition}\}}$

Classification

$\boxed{\text{text}\{\text{Liveness} = \text{Backward} + \text{MAY} + \text{Union}\}}$

USE

$\boxed{\text{USE}[B] = \text{text}\{\text{variables used before definition in B}\}}$

DEF

$\boxed{\text{DEF}[B] = \text{text}\{\text{variables defined in B}\}}$

Equations

$\boxed{\text{OUT}[B] = \bigcup_{S \in \text{Succ}(B)} \text{IN}[S]}$

$\boxed{\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] - \text{DEF}[B])}$

Meaning of IN Equation

A variable is live before B if:

TEXT

B needs its old value

OR

it is needed later and B doesn't overwrite it

Fixed Point

Because of loops:

$\boxed{\text{iterate until IN/OUT sets stop changing}}$

Applications

$\boxed{\text{Dead-code elimination}}$

$\boxed{\text{Register allocation}}$

30-SECOND REVISION

Memorize:

TEXT

LIVENESS

= BACKWARD

= MAY

= UNION

Core equations

$\boxed{OUT[B] = \bigcup IN[\text{successors}]}$

$\boxed{IN[B] = USE[B] \cup (OUT[B] - DEF[B])}$

USE

Used before defined in this block.

DEF

Defined in this block.

Why - DEF ?

New definition kills the need for the old incoming value.

Why + USE ?

If the block uses it, it must already be live on entry.

Dead assignment

For:

TEXT

$x = \text{expr}$

if:

$\nexists x \notin \text{OUT}$

and expr has no side effects:

$\boxed{\text{may remove it}}$

Next is the final individual data-flow analysis in your syllabus:

Part 29 — Available Expressions & Common Subexpression Elimination

We will cover:

- what "available" exactly means
- why it is **forward**
- why it is a **MUST** analysis
- **GEN** and **KILL**
- equations

$\text{IN}[B] = \bigcap \text{OUT}[\text{pred}]$

$\text{OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] - \text{KILL}[B])$

- how assignments kill expressions
- fixed-point solving
- applying the result to global CSE

Completed: Parts 1–28/31 | Next: Part 29 — Available Expressions & Common Subexpression Elimination

Reply **NEXT** for **Part 29**.